

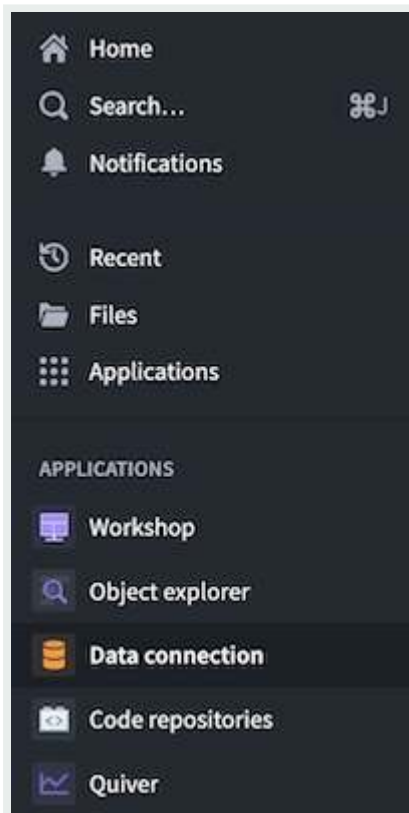
Table of Contents

1. [Data Connection • Overview • Palantir](#)
2. [Data Connection • Overview • Palantir](#)
3. [Data Connection • Core concepts • Palantir](#)
4. [Data Connection • Architecture • Palantir](#)
5. [Data Connection • Connection security • Palantir](#)
6. [Data Connection • Troubleshooting • Palantir](#)
7. [Data Connection • Foundry worker vs. agent worker • Palantir](#)
8. [Data Connection • Initial setup overview • Palantir](#)
9. [Data Connection • Permissions reference • Palantir](#)
10. [Data Connection • FAQ • Palantir](#)
11. [Data Connection • Agents • Set up an agent • Palantir](#)
12. [Data Connection • Agents • Agent configuration reference • Palantir](#)
13. [Data Connection • Agents • Agent worker configuration reference • Palantir](#)
14. [Data Connection • Agents • Agent proxy configuration reference • Palantir](#)
15. [Data Connection • Agents • Troubleshooting reference • Palantir](#)
16. [Data Connection • Sources • Set up a source • Palantir](#)
17. [Data Connection • Sources • Source exploration • Palantir](#)
18. [Data Connection • Syncs • Set up a sync • Palantir](#)
19. [Data Connection • Syncs • Set up a streaming sync • Palantir](#)
20. [Data Connection • Syncs • File-based syncs • Palantir](#)
21. [Data Connection • Syncs • Set up a media set sync • Palantir](#)
22. [Data Connection • Syncs • Optimize JDBC syncs • Palantir](#)
23. [Data Connection • Syncs • Troubleshooting reference • Palantir](#)
24. [Data Connection • Exports • Overview • Palantir](#)
25. [Data Connection • Exports • Export tasks \[Legacy\] • Palantir](#)
26. [Data Connection • Webhooks • Overview • Palantir](#)
27. [Data Connection • Webhooks • Set up a webhook • Palantir](#)
28. [Data Connection • Webhooks • Set up a webhook for the Microsoft Graph API • Palantir](#)
29. [Data Connection • Webhooks • Configuration reference • Palantir](#)
30. [Data Connection • Listeners • Overview • Palantir](#)
31. [Data Connection • Listeners • Listener subdomains • Palantir](#)
32. [Data Connection • HTTPS listeners • Overview • Palantir](#)
33. [Data Connection • HTTPS listeners • Security • Palantir](#)
34. [Data Connection • HTTPS listeners • Event processing • Palantir](#)
35. [Data Connection • HTTPS listeners • Set up a Google Pub/Sub listener • Palantir](#)
36. [Data Connection • HTTPS listeners • Set up a Jira listener • Palantir](#)
37. [Data Connection • HTTPS listeners • Set up a Slack listener • Palantir](#)
38. [Data Connection • HTTPS listeners • Create an AI-powered Slack bot • Palantir](#)
39. [Data Connection • WebSocket listeners • Overview • Palantir](#)

Data Connection

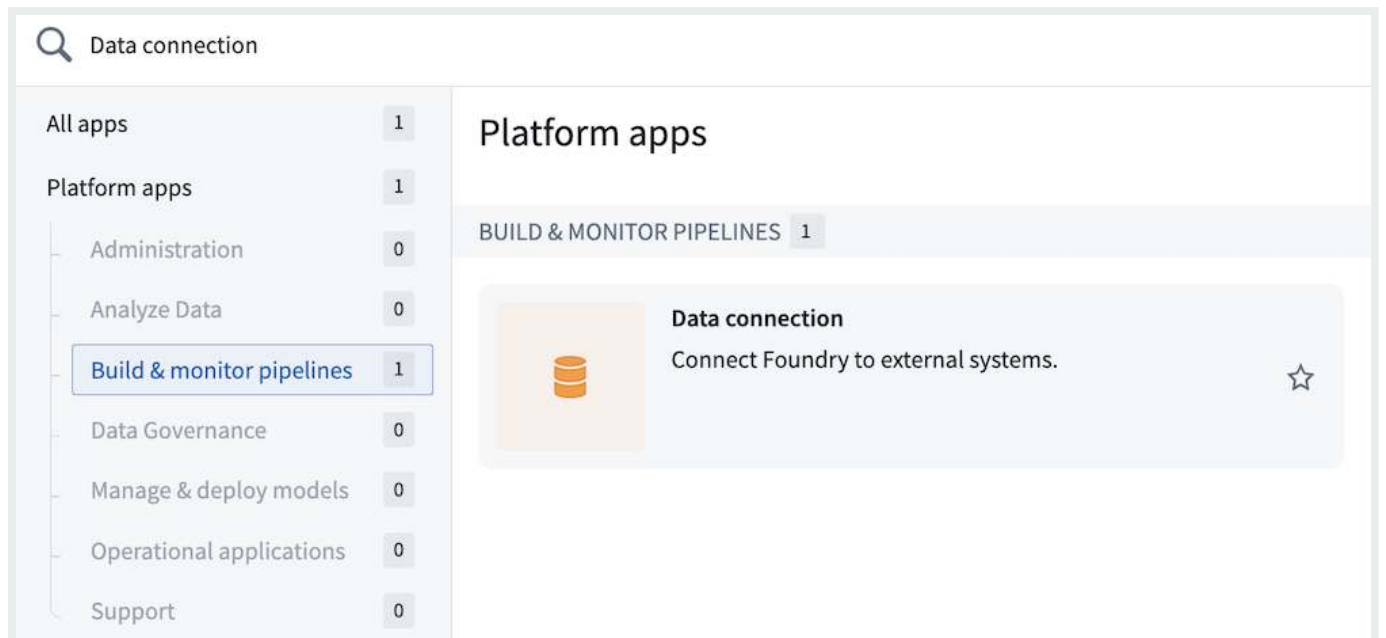
Data Connection is an application used to synchronize data from external systems and other Foundry instances for use in Foundry. Users can use Data Connection to sync data into Foundry for use in the data integration, modeling, and Ontology layers. Additionally, Data Connection enables setting up outbound connections to enable writeback to external systems via webhooks and data exports.

Access the Data Connection application by selecting the icon from the workspace navigation bar.



Y [API Reference ↗](#) Data Connection application by searching in the Applications Portal.

[Send feedback](#)



If you are connecting Foundry to your data for the very first time for your organization, get started by referring to the [initial setup guide](#).

Foundry standardizes the data connection process with the following three principles:

- Robustness
- Extensibility
- Ease of use

Robustness

Often, connecting data between systems is prone to failures that can be difficult to recover from. Issues out of the user's control in the external environment (e.g., poor network connectivity, disk failures, or unresponsive source systems) can affect data syncs, breaking downstream analytics pipelines as well. Incomplete or corrupt data is not only a technical challenge, but also potentially dangerous for the organization if it is used unnoticed or not available for urgent needs.

Foundry proactively addresses these common failure points with automatic retries upon failures, use of simple functions (e.g., filesystem and database syncs) to pull data in small batches, complexity queries from the source systems, and an integrated data health monitoring system. [API Reference ↗](#) alert on critical failures and surface other pipeline health issues.

Combined, these features minimize the risk of incomplete or corrupt

[Send feedback](#)

Foundry is also distinguished by a philosophy that data should be ingested "as-is" from its most raw source, with no external preprocessing. In the absence of external preprocessing, the branched and version-controlled Foundry pipeline becomes the single source for all of the changes that happened to raw data on its journey to Ontology, and any issues that arise on that path can be identified and resolved inside the platform. Data Connection adheres to this design philosophy by supporting both tabular and file-based syncs and deliberately offering minimal options for transforming the data before it arrives in the destination dataset (the starting point of the Foundry pipeline).

Extensibility

Enterprises have a diverse, complex array of systems that add value individually and as an integrated system. Each system has its own requirements for integration and some systems require unique capabilities or features that would make it typically difficult to integrate.

Foundry provides out-of-the-box integration with well-known system types (e.g., relational databases, FTPS, HDFS, S3, SFTP, and local directories) as well as the flexibility to connect and sync data from new system types. In many cases, new systems can reuse an existing plugin or one with just minor changes. Core features (e.g., scheduling and uploading) are standardized so that only the connection itself needs to be adjusted.

Foundry also supports communication between Foundry enrollments via the dedicated [Foundry connector](#). With this connector, one Foundry instance can be treated as a source on another, supporting batch and streaming syncs with incremental options.

[Learn more about the full range of available source types.](#)

Ease of use

Managing data connections between systems can be an involved process that puts a significant burden on administrators who are responsible for every step: syncing, authentication, scheduling and orchestration, and monitoring.

Foundry API Reference ↗ way this complexity with backend services that take on the bulk of the work as users set up and manage pipelines through a simple frontend lowers barrier of entry to a typically complex technical task, making it easier for users to perform data connection.

Send feedback

Palantir Learning portal

Try creating your first data connection with a relevant course on learn.palantir.com [↗](#).

PREVIOUS

← Core concepts / Views

NEXT

Core concepts →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) [↗](#)

[Privacy Statement](#) [↗](#)

[Terms of Use](#) [↗](#)

— [Cookies Settings](#)

[API Reference](#) [↗](#)

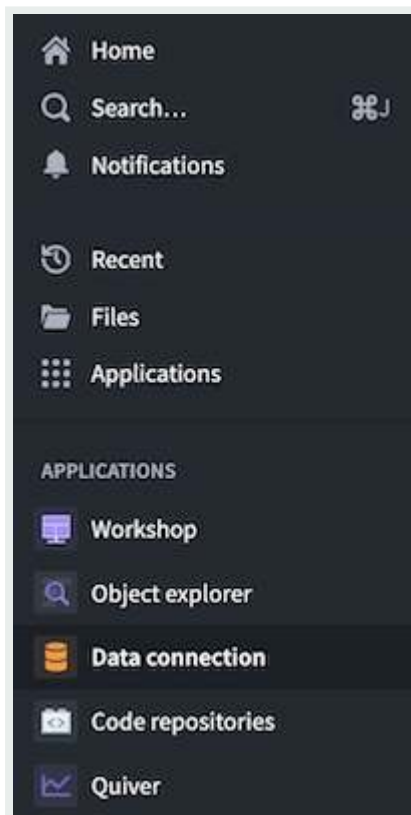
[Send feedback](#)

Data connectivity & integration / Data Connection / Overview

Data Connection

Data Connection is an application used to synchronize data from external systems and other Foundry instances for use in Foundry. Users can use Data Connection to sync data into Foundry for use in the data integration, modeling, and Ontology layers. Additionally, Data Connection enables setting up outbound connections to enable writeback to external systems via webhooks and data exports.

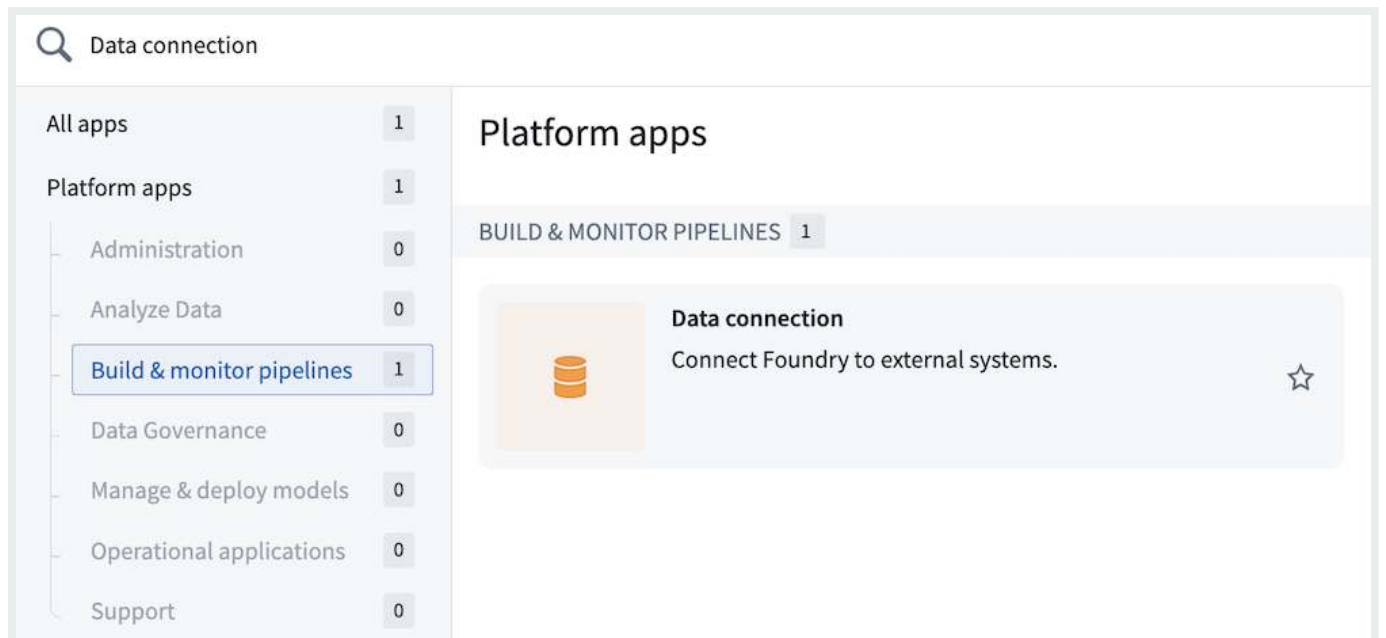
Access the Data Connection application by selecting the icon from the workspace navigation bar.



Y [API Reference ↗](#) Data Connection application by searching in the Applications Portal.



Send feedback



If you are connecting Foundry to your data for the very first time for your organization, get started by referring to the [initial setup guide](#).

Foundry standardizes the data connection process with the following three principles:

- Robustness
- Extensibility
- Ease of use

Robustness

Often, connecting data between systems is prone to failures that can be difficult to recover from. Issues out of the user's control in the external environment (e.g., poor network connectivity, disk failures, or unresponsive source systems) can affect data syncs, breaking downstream analytics pipelines as well. Incomplete or corrupt data is not only a technical challenge, but also potentially dangerous for the organization if it is used unnoticed or not available for urgent needs.

Foundry proactively addresses these common failure points with automatic retries upon failures, use of simple functions (e.g., filesystem and database syncs) to pull data in small batches, complexity queries from the source systems, and an integrated data health monitoring system that sends alerts on critical failures and surface other pipeline health issues.

Combined, these features minimize the risk of incomplete or corrupt data.

[Send feedback](#)

Foundry is also distinguished by a philosophy that data should be ingested "as-is" from its most raw source, with no external preprocessing. In the absence of external preprocessing, the branched and version-controlled Foundry pipeline becomes the single source for all of the changes that happened to raw data on its journey to Ontology, and any issues that arise on that path can be identified and resolved inside the platform. Data Connection adheres to this design philosophy by supporting both tabular and file-based syncs and deliberately offering minimal options for transforming the data before it arrives in the destination dataset (the starting point of the Foundry pipeline).

Extensibility

Enterprises have a diverse, complex array of systems that add value individually and as an integrated system. Each system has its own requirements for integration and some systems require unique capabilities or features that would make it typically difficult to integrate.

Foundry provides out-of-the-box integration with well-known system types (e.g., relational databases, FTPS, HDFS, S3, SFTP, and local directories) as well as the flexibility to connect and sync data from new system types. In many cases, new systems can reuse an existing plugin or one with just minor changes. Core features (e.g., scheduling and uploading) are standardized so that only the connection itself needs to be adjusted.

Foundry also supports communication between Foundry enrollments via the dedicated [Foundry connector](#). With this connector, one Foundry instance can be treated as a source on another, supporting batch and streaming syncs with incremental options.

[Learn more about the full range of available source types.](#)

Ease of use

Managing data connections between systems can be an involved process that puts a significant burden on administrators who are responsible for every step: syncing, authentication, scheduling and orchestration, and monitoring.

Foundry API Reference ↗ way this complexity with backend services that take on the bulk of the work as users set up and manage pipelines through a simple frontend lowers barrier of entry to a typically complex technical task, making it easier for users to perform data connection.

Send feedback

Palantir Learning portal

Try creating your first data connection with a relevant course on learn.palantir.com
[↗](#).

PREVIOUS

← Core concepts / Views

NEXT

Core concepts →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Core concepts

This page describes the core concepts used throughout Data Connection.

Sources

A **Source** represents a single connection to an external system, including any configuration necessary to specify and locate the target system (typically a URL) and the [credentials](#) required to successfully authenticate. Sources must be configured with a particular [worker](#) defining where compute for any [capabilities](#) used with the source will be run. Note that the [agent worker](#) also controls the networking between the Palantir platform and the target system.

A source is set up based on a particular **connector** (also referred to as a **source type**). A [broad range of connectors are available in the Palantir platform](#), designed to support the most common data systems across organizations. Depending on the connector and worker selected, different [capabilities](#) may be available.

For systems without a dedicated connector, the generic connector or [REST API source](#) may be used with code-based connectivity options such as [external transforms](#), [external functions](#), and [compute modules](#).

Credentials

A *credential* is a secret value that is required to access a particular system; that is, a secret value required for authentication. Credentials can be passwords, tokens, API keys, or other secret values. In the Palantir platform, all credentials are encrypted and stored securely. Depending on the worker, secrets may be stored locally on the agent, or directly in the platform.

API Reference ↗

Send feedback

Some sources are able to authenticate without storing any secrets, such as when using [OpenID connect](#), [outbound applications](#), or a [cloud identity](#).

Workers

Worker defines *where* compute for [capabilities](#) is run, while [networking](#) defines *how* target systems are reached. See [architecture diagrams](#) for examples of how worker and networking work together.

Sources run on a [Foundry worker](#). [Agent worker](#) is a legacy option retained for existing customers; see [Foundry worker vs. agent worker](#) for the comparison.

i Not all worker types are available for all source types.

Foundry worker

i Originally, Foundry worker only supported [direct egress policies](#), so sources of that worker type were called "direct connection" sources. Now that these sources also support [agent proxy policies](#), they have been renamed to "Foundry worker" sources to avoid confusion.

Foundry worker sources run compute for their source capabilities in Foundry. These sources benefit from Foundry's containerized and scalable job execution, improved stability, and do not incur the maintenance overhead associated with [agents](#). All sources can be [used in code](#).

Networking is configured through [egress policies](#) and enables [direct connection](#) and connections over an [agent proxy](#).

A [API Reference ↗](#)

[Send feedback](#)

⚠ Legacy

Agent worker is in the [legacy](#) phase of development. We recommend [migrating existing agent worker sources to Foundry worker](#). See [Foundry worker vs. agent worker](#) for the operational reasons behind this recommendation.

Agent worker sources run compute for their source capabilities on a customer-provided host via an [agent](#). Networking is configured manually on the agent host firewall and proxy settings; the host must have network access to the source systems it is connecting to.

Networking and egress policies

Networking defines *how* target systems are reached, while [worker](#) defines *where* [capabilities](#) compute are executed. See [architecture diagrams](#) for examples on how worker and networking are compatible with each other.

For [Foundry worker](#) sources, networking is configured via egress policies. They define at a granular level how each target system can be reached from Foundry, and which egress destinations are permitted.

Sources support two policy types:

- **[Direct connection policies](#):** Used for systems directly accessible from Foundry. For cloud-hosted Foundry instances, this means the public Internet. For privately hosted Foundry, this means the network where Foundry is installed.
- **[Agent proxy policies](#):** Used for accessing external systems hosted in a different network from Foundry. For cloud-hosted Foundry instances, this means external systems hosted on-premise. Requires the set up of an [agent](#). The agent host itself must have its networking configured to reach the target systems. **Agent proxy policies are only available on Foundry enrollments running on Rubix, Palantir's Kubernetes-based infrastructure.**

API Reference ↗

Direct connection and agent proxy policies can be assigned to the same source for use cases requiring both network paths. An example of this would be use

Send feedback

cases needing to retrieve credentials from an internet-hosted system, and using said credentials to authenticate with an on-premise system.

[Learn more about egress policies.](#)

For [agent worker](#) sources, networking is configured directly on the agent host.

Agents

An **agent** is a downloadable program installed within your organizational network and managed from Foundry's [Data Connection](#) interface. Agents have the ability to connect to different external systems within your organizational network and are used to power connections leveraging [agent proxy policies](#) and [agent worker connections](#).

Read more about [how to set up an agent](#).

Capabilities

Sources may support a variety of *capabilities*, where each capability represents some functionality that can run over the source connection. There are a wide range of supported capabilities for bringing data into Foundry, pushing data out of Foundry, virtualizing data stored outside of Foundry, and making interactive requests to other systems.

A summary of available capabilities is included in the following table. For more information about capabilities supported for a specific connector, refer to that connector's documentation page.

Capability	Description
Batch syncs	Sync data from an external source to a dataset .
Iceberg syncs	Sync data from an external source to a Foundry Iceberg table .
Streaming syncs	Sync data from an external message queue to a stream .
API Reference ↗ capture (CDC) syncs	Sync data from a database to a stream with Send feedback

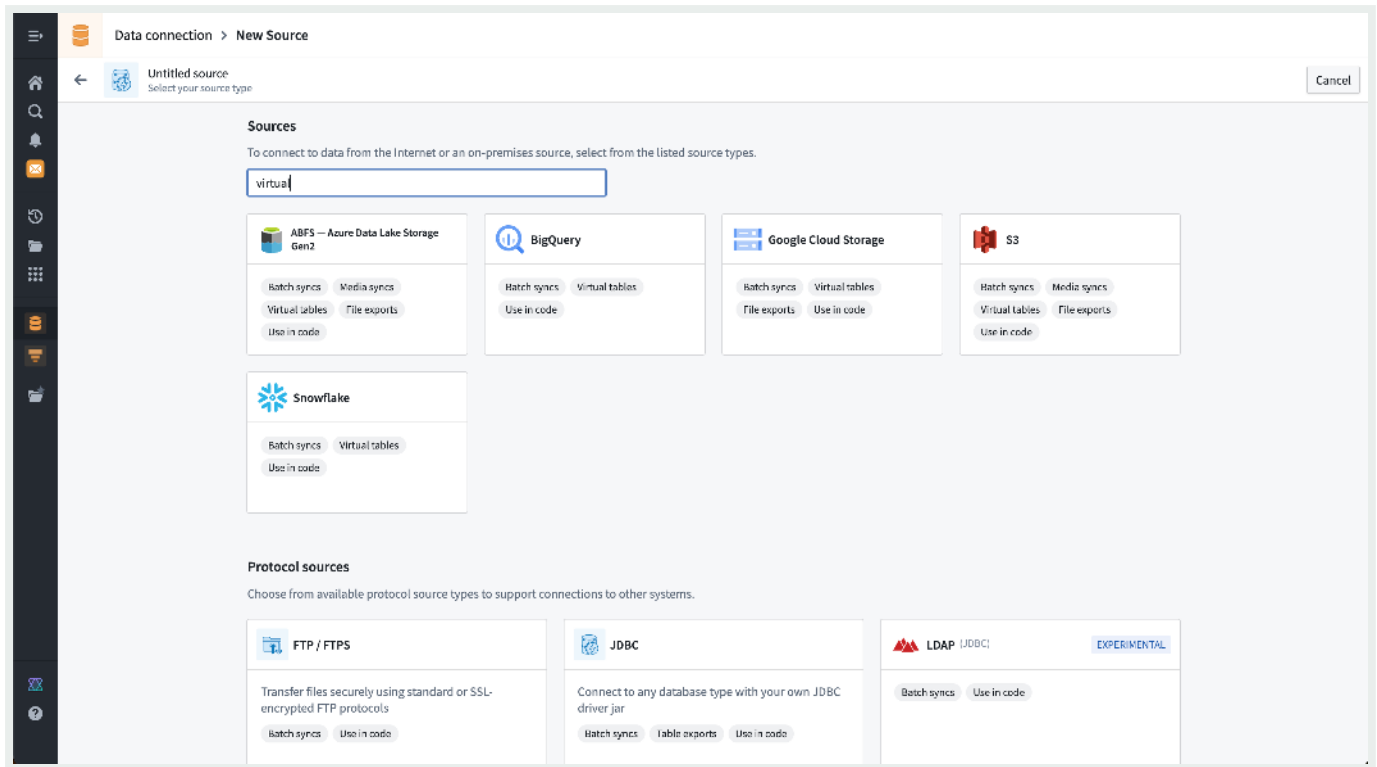
Capability	Description
<u>Media syncs</u>	Sync data from an external source to a <u>media set</u> .
<u>HyperAuto</u>	<u>Sync an entire system automatically.</u>
<u>File exports</u>	Push data as files from a <u>dataset</u> to an external system.
<u>Table exports</u>	Push data with a schema from a <u>dataset</u> to an external database.
<u>Streaming exports</u>	Push data from a <u>stream</u> to an external message queue.
<u>Webhooks</u>	Make structured requests to an external system interactively.
<u>Virtual tables</u>	Register data from an external data warehouse to use as a <u>virtual table</u> .
<u>Virtual media</u>	Register unstructured media from an external system as a <u>media set</u> .
<u>Exploration</u>	Interactively explore the data and schema of an external system before using other capabilities.
<u>Use in code</u>	Use a source in code to extend or customize any functionality not covered by the point-and-click-configurable capabilities listed above. This is only available for <u>Foundry worker</u> sources.

Additional capabilities are being developed, and capability coverage is regularly updated in the documentation for specific connectors.

Supported capabilities for specific connectors are also displayed on the new source page in the Data Connection application. It is possible to search both by connector name and by capability. The example below shows the results of a search for sources that support a "virtual" option.

[API Reference ↗](#)

[Send feedback](#)



Batch syncs

Batch syncs read data from an external system and write it into a Foundry [dataset](#). A batch sync defines what data should be read and which dataset to output into in Foundry. Batch syncs can be configured to sync data incrementally and allow syncing data both with and without a corresponding schema. [Learn how to set up a sync.](#)

Batch syncs are [transactional](#). Each sync run writes data within a single transaction on the output dataset. If a sync fails at any point during execution, the transaction is aborted and no data from that run is committed. This means that a partial failure does not result in partial data; the output dataset remains unchanged from its state before the failed sync. The specific error depends on the cause of the failure. You can inspect the error message and full stack trace for a failed run on the sync's **Runs** tab. For common failure patterns, learn how to [troubleshoot syncs](#).

In general, there are two main types of batch syncs:

- **File batch syncs:** Allows syncing files without a schema directly into a Foundry dataset.

API Reference ↗

can be accessed in downstream transforms; for example, using [file-](#)filesystems and blob stores such as [S3](#), [ABFS](#), [Google Cloud Storage](#), [Sharepoint online](#). [Learn more about file batch syncs.](#)

Send feedback

- **Table batch syncs:** Allows syncing data with a schema into a Foundry dataset. Part of the sync definition in this case also includes how to translate types between the external system schema and the supported Foundry schema options. The most common systems that support table batch syncs are databases and SaaS providers such as [Microsoft SQL Server](#), [Postgres](#), [SAP](#), [Salesforce](#), and [Netsuite](#).

Iceberg syncs [Beta]

Beta

Syncs into Iceberg tables are in the [beta](#) phase of development.

Iceberg syncs read data from an external source and write it directly into a Foundry [Iceberg table](#), rather than a standard [dataset](#). Iceberg syncs support both non-incremental and incremental ingestion, along with several write modes. Iceberg syncs are only supported on the [Foundry worker](#) runtime and are available for a subset of connectors. [Learn more about syncing data to Iceberg tables from Data Connection.](#)

Streaming syncs

Streaming syncs provide the ability to stream data from systems that provide low latency data feeds. Data is delivered into a [streaming dataset](#). Some examples of systems that support streaming syncs include [Kafka](#), [Amazon Kinesis](#), and [Google Pub/Sub](#).

[Learn more about streaming syncs.](#)

Change data capture syncs

Change data capture (CDC) syncs are similar to streaming syncs, with additional changelog metadata automatically propagated to the streaming dataset where data is delivered. This type of sync is normally used for databases that support some form of low-latency

re [API Reference](#) [Learn more about change data capture syncs.](#)

Media syncs

Send feedback

Media syncs allow importing media data into a [media set](#). Media sets provide better tooling than standard datasets for ingesting, transforming, and consuming media data throughout Foundry. When dealing with PDFs, images, videos, and other media, we recommend using media sets over datasets. [Learn more about media syncs.](#)

HyperAuto

HyperAuto is a specialized capability that can dynamically discover the schema of your SAP system and automate syncs, pipelines, and creation of a corresponding ontology within Foundry. HyperAuto is currently only supported for SAP. [Learn more about HyperAuto.](#)

File exports

File exports are the opposite of file batch syncs. When doing a file export, data is taken directly from the underlying files contained within a Foundry dataset, which are written as-is to a filesystem location in the target system. [Learn more about file exports.](#)

Table exports

Table exports are the opposite of table batch syncs. When performing a table export, data is exported as rows from a Foundry dataset with a schema, which are then written to a table in the target system. [Learn more about table exports.](#)

Streaming exports

Streaming exports are the opposite of streaming syncs. When doing a streaming export, data is exported from a Foundry stream, and records are written to the specified streaming queue or topic in the target system. [Learn more about streaming exports.](#)

Webhooks

API Reference ↗

Webhooks represent a request to a source system outside of Foundry. Webhook requests can be flexibly defined in Data Connection to enable a broad range of integrations with external systems. [Learn more about webhooks.](#)

Send feedback

Virtual tables

Virtual tables represent the ability to register tabular data from an external system into a virtual table resource in Foundry.

In addition to registering individual virtual tables, this capability also allows for dynamic discovery and automatic registration of all tables found in an external system.

[Learn more about virtual tables.](#)

Virtual media

Virtual media works similarly to [media syncs](#), allowing media from an external system to be used in a [media set](#) but without copying the data into Foundry. Instead, media files contained in an external system can be registered as virtual media items in a specific media set.

[Learn more about virtual media.](#)

Exploration

The interactive **exploration** capability allows you to see what data is contained in an external system before performing syncs, exports, or other capabilities that interact with that system.

Exploration is most commonly used to check that a connection is working as intended and that the correct permissions and credentials are being used to connect.

Use in code

The ability to use connections from code is intended to allow developers to extend and customize connections from Foundry to other systems. Palantir's general principle is that ~~anything possible in~~ the platform using dedicated connectors and point-and-click

API Reference ↗ should also be achievable by writing custom code. At any point, developers should be able to switch to code-based connectivity for ~~re~~ over the functionality or performance of workflows that perform external

Send feedback

Any connector may be used in code; in most cases, we recommend using either the [REST API source](#) or [generic connector](#) when connecting from code.

[Agent worker](#) connections cannot be used in code. Also, some credential types such as cloud identity, outbound application, and OIDC may not currently be used from code.

Use in code option	Description
External transforms	External transforms allow transforms written in Python to communicate with external systems. External transforms are a code-based alternative for file batch syncs , file exports , table batch syncs , table exports , and media syncs . External transforms may also be used to register data into virtual media sets and virtual tables .
External functions (webhooks)	External functions written in TypeScript support importing a source in order to invoke existing webhooks defined on that source. This allows existing webhook calls to be wrapped in custom typescript logic and error handling.
External functions (direct)	External functions now allow direct calls to external systems using <code>fetch</code> for TypeScript and <code>requests</code> for Python. External functions are a code-based alternative for webhooks .
Compute modules	Compute modules allow for long-running compute and writing connections in arbitrary languages. Compute modules may be used as a code-based alternative for streaming syncs , streaming exports , change data capture syncs , and webhooks .
External models	External models currently do not support importing sources. Instead, you must use network egress policies directly.
API Reference ↗	Code workspaces currently do not support importing sources. Instead, you must use network egress policies directly.
Code workbooks	Code workbooks currently do not support ext

Send feedback

Other concepts

Data connection also includes a variety of other concepts relevant for specific workflows. Some concepts were previously used and are now sunset, but are retained here for reference.

Syncs

Historically, the term **Sync** was used in a generic way to refer to bringing data into Foundry. Syncs are now separated into the more specific capabilities listed above. More details are available for each capability, such as **batch syncs**, **streaming syncs**, **change data capture syncs**, **media syncs**, and so on.

Tasks [Sunset]

Sunset

Tasks are in the [sunset](#) phase of development and will be deprecated at a future date. Full support remains available. We recommend migrating your workflows to [code-based connectivity options](#).

The plugin framework used to implement connectors allows custom extensions called **tasks**. Tasks represent a unit of functionality configured by providing YAML and implemented in Java as part of the Data Connection plugin. Palantir has stopped developing new tasks, and all officially supported capabilities have migrated away from using tasks.

Direct connection

Connections running in Foundry originally only supported direct connections and were referred to as "direct connection". These connections have now been incorporated into the [API Reference ↗](#) concept.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Architecture examples

Foundry worker with direct connection policies

i This type of connection can also be referred to as a "direct connection".

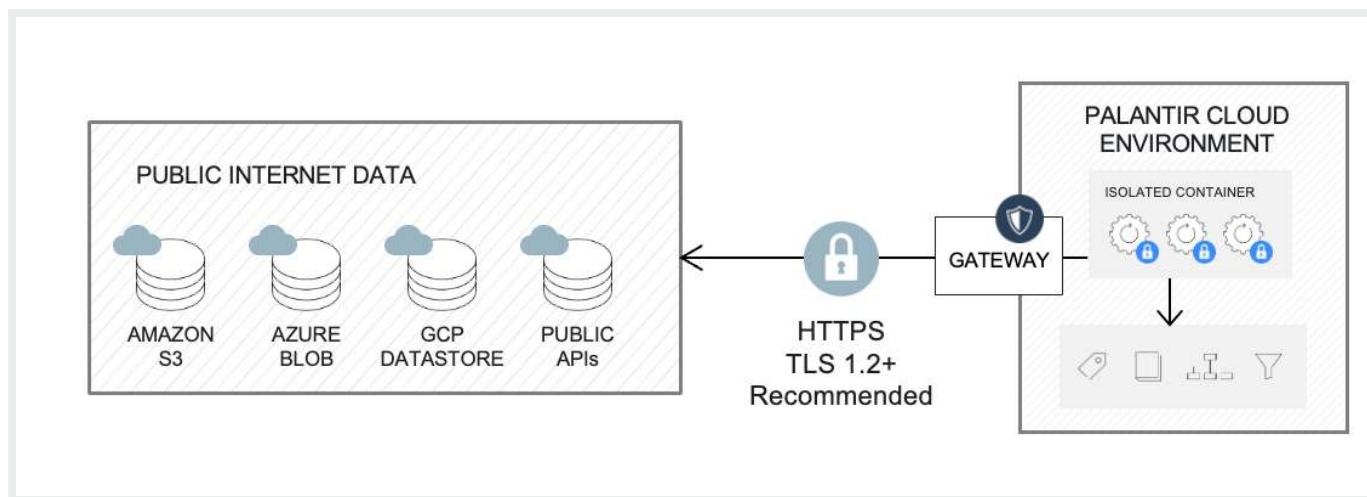
Foundry worker with direct connection policies is the recommended default for accessing external systems that accept inbound traffic from Foundry, such as REST APIs and cloud/SaaS systems.

In this scenario, data connections run in an isolated compute container called a [Foundry worker](#), which handles authenticated encrypted network requests. The external system must accept inbound traffic from Foundry. Network egress from Foundry is managed and administered in-platform via [direct connection egress policies](#).

External system credentials are stored with AES-256-GCM server-side encryption and can only be decrypted by containers triggered by authorized users.

To create a direct connection, follow instructions about [setting up a source](#).

[API Reference ↗](#)[Send feedback](#)



Foundry worker with agent proxy policy

i This type of connection can also be referred to as an "agent proxy" or an "agent thin mode" connection.

Foundry worker with agent proxy policies is the recommended architecture to access external systems hosted on private networks. It requires the use of a [data connection agent](#).

In this scenario, the agent acts as a simple network tunnel without performing any data processing itself.

The agent initiates a network request with Foundry to establish a websocket connection. [Learn more about network controls available on agents.](#)

All data connection and computation capabilities are executed by a [Foundry worker](#), an isolated container with scalable compute resources that processes data and communicates with external systems via the provided websocket.

The ability to use the agent as a proxy is granularly managed and administered in-platform via [agent proxy egress policies](#).

~~Multiple agents can be~~ used to load balance workloads across multiple external systems.

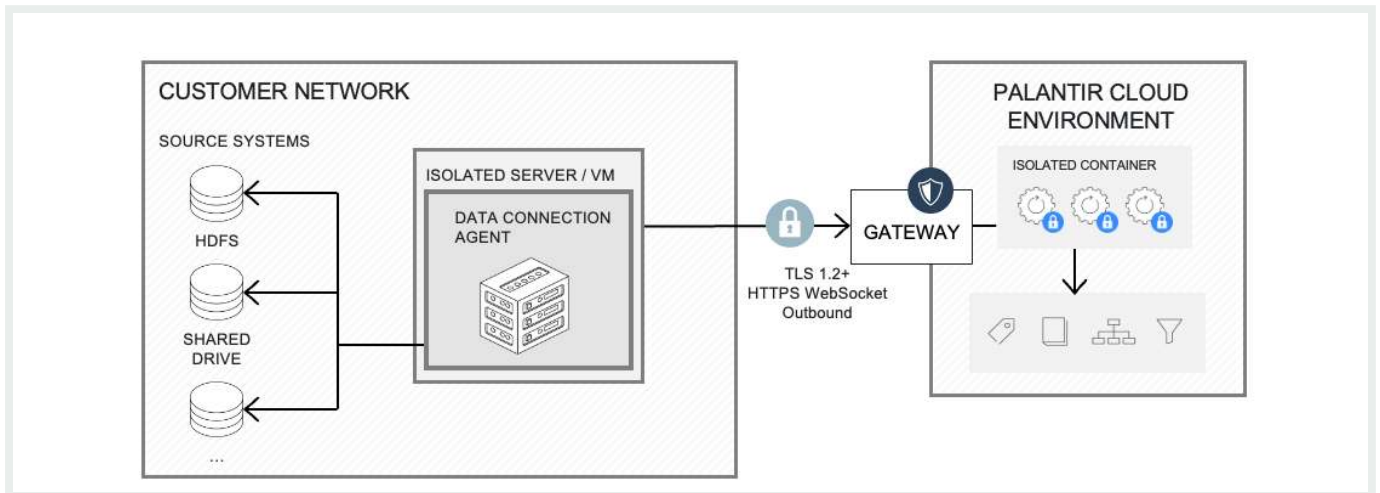
API Reference ↗

Like [direct connections](#), external system credentials are stored using AES-256-GCM side encryption and can only be decrypted by the container triggered

Send feedback

To create an agent-proxy connection, you will need to:

- [Set up an agent.](#)
- Create one or many [agent proxy policies](#).
- [Set up a Foundry worker source](#) using the policies above.



Legacy architecture: Agent worker

⚠ Legacy

Agent worker is in the [legacy](#) phase of development. We recommend [Foundry worker with agent proxy](#) for on-premise and privately hosted systems. See [Foundry worker vs. agent worker](#) for the comparison.

i This type of connection can also be referred to as "agent thick mode".

Agent worker connection is the historical architecture used to access external systems hosted on private networks from Foundry. It requires the use of a [data connection agent](#). [Learn more about agent worker known limitations.](#)

To migrate an existing agent worker source to a Foundry worker, follow the [migration guide](#).

[API Reference](#) ↗

In this scenario, the agent constantly polls Foundry via unidirectional connections secured by HTTPS for new tasks to execute. Once received, the task is

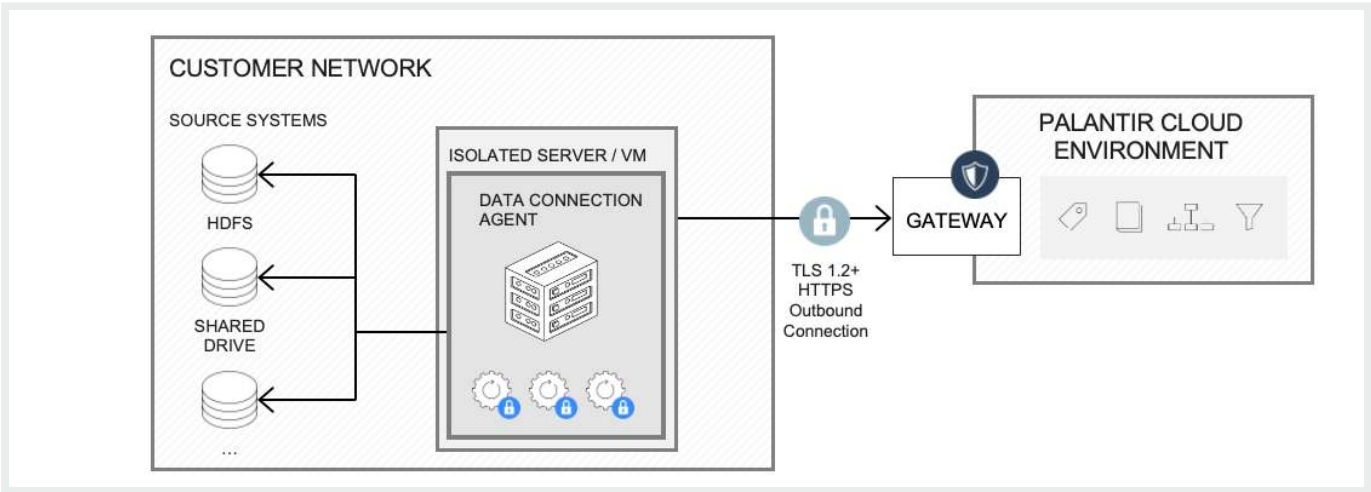
[Send feedback](#)

agent itself, also called [agent worker](#) and results are sent to Foundry over that same unidirectional connection. All the networking configuration required for the agent to be able to communicate with internal source systems is [configured on the agent host itself](#).

External system credentials are stored on the platform using AES-128-GCM encryption with keys stored on the agent. During capability execution, the agent retrieves encrypted credentials from Foundry, decrypts them locally, and uses them for queries. Decrypted credentials are automatically deleted from memory after execution.

To create an agent worker connection, you will need to:

- [Set up an agent](#).
- Configure it to be an [agent worker](#).
- [Set up an agent worker source](#) and assign it that agent.



Summary

Type of connection	Capability execution	Networking	System credentials
Direct connection	In Foundry	Direct from Foundry to source systems.	Stored encrypted in platform with encryption keys in platform.
API Reference ↗	In Foundry	Outbound connection from agent to Foundry to establish	Stored encrypted in Send feedback

Type of connection	Capability execution	Networking	System credentials
		a websocket and proxy traffic.	encryption keys in platform.
Agent worker (legacy).	On agent host	Outbound connection from agent to Foundry to poll tasks. Inbound connection from agent to source systems.	Stored encrypted in platform with encryption keys on agent.

PREVIOUS
← [Core concepts](#)

NEXT
[Connection security](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Connection security

Data Connection establishes outbound TLS connections to external data sources. Not all cipher suites used in TLS connections are supported throughout the platform.

Supported suites

Data Connection supports the following cipher suites for outbound connections, provided they are supported by the underlying Java version. Additional cipher suites may be available depending on your environment's configuration and connection origination point.

i Contact Palantir Support with questions about additional cipher suites that may be available based on your environment's configuration and connection origin point.

TLS 1.3

IANA name	OpenSSL name
TLS_AES_256_GCM_SHA384	TLS_AES_256_GCM_SHA384
TLS_AES_128_GCM_SHA256	TLS_AES_128_GCM_SHA256
TLS_CHACHA20_POLY1305_SHA256	TLS_CHACHA20_POLY1305_SHA256

API Reference ↗



Send feedback

IANA name	OpenSSL name
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384	ECDHE-ECDSA-AES256-GCM-SHA384
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256	ECDHE-ECDSA-AES128-GCM-SHA256
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256	ECDHE-ECDSA-CHACHA20-POLY1305
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384	ECDHE-RSA-AES256-GCM-SHA384
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256	ECDHE-RSA-CHACHA20-POLY1305
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256	ECDHE-RSA-AES128-GCM-SHA256
TLS_DHE_RSA_WITH_AES_256_GCM_SHA384	DHE-RSA-AES256-GCM-SHA384
TLS_DHE_RSA_WITH_CHACHA20_POLY1305_SHA256	DHE-RSA-CHACHA20-POLY1305
TLS_DHE_DSS_WITH_AES_256_GCM_SHA384	DHE-DSS-AES256-GCM-SHA384
TLS_DHE_RSA_WITH_AES_128_GCM_SHA256	DHE-RSA-AES128-GCM-SHA256
TLS_DHE_DSS_WITH_AES_128_GCM_SHA256	DHE-DSS-AES128-GCM-SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384	ECDHE-ECDSA-AES256-SHA384
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384	ECDHE-RSA-AES256-SHA384
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256	ECDHE-ECDSA-AES128-SHA256
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256	ECDHE-RSA-AES128-SHA256
TLS_DHE_RSA_WITH_AES_256_CBC_SHA256	DHE-RSA-AES256-SHA256
TLS_DHE_DSS_WITH_AES_256_CBC_SHA256	DHE-DSS-AES256-SHA256
TLS_DHE_RSA_WITH_AES_128_CBC_SHA256	DHE-RSA-AES128-SHA256
TLS_DHE_DSS_WITH_AES_128_CBC_SHA256	DHE-DSS-AES128-SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA	ECDHE-ECDSA-AES256-SHA
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA	ECDHE-RSA-AES256-SHA
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA	ECDHE-ECDSA-AES128-SHA
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA	ECDHE-RSA-AES128-SHA
TLS_DHE_RSA_WITH_AES_256_CBC_SHA	DHE-RSA-AES256-SHA
TLS_DHE_DSS_WITH_AES_256_CBC_SHA	DHE-DSS-AES256-SHA
TLS_DHE_RSA_WITH_AES_128_CBC_SHA	DHE-RSA-AES128-SHA

API Reference ↗

Send feedback

TLS_DHE_DSS_WITH_AES_128_CBC_SHA

DHE-DSS-AES128-SHA

Verify your external system accepts a supported cipher suite

To verify that your external system accepts a supported cipher suite, run `openssl s_client -connect your-system.example.com:<port> -tls1_3 </dev/null` (or `-tls1_2` for TLS 1.2). The negotiated suite appears on the cipher line of the SSL-Session block in the output. To test a specific suite from the supported list, add `-ciphersuites '<cipher_iana_name>'` for TLS 1.3 or `-cipher '<cipher_openssl_name>'` for TLS 1.2. An `SSLHandshakeException` containing `handshake_failure`, `no cipher match`, or `protocol is disabled or cipher suites are inappropriate` indicates a cipher mismatch, so you should update the cipher configuration on your external system to enable one of the supported suites. You can also run `openssl` against the same network path your source uses from the [source terminal](#).

Vulnerability scan findings on agent hosts

A vulnerability scanner running on an agent host may report findings against files that belong to earlier versions of the agent software or connector plugins. In almost all cases these are inactive files that are no longer loaded or run. The agent uses only the current version of each component, so a finding against a superseded file does not mean the agent is running vulnerable code.

These files are left behind by normal upgrades. When an agent upgrades to a new version, the version it replaces is kept on the host for up to 30 days and then deleted automatically. Security fixes, including for known Common Vulnerabilities and Exposures (CVEs), are delivered as new versions, and agents upgrade to the latest available version automatically during their [upgrade window](#). Configuring an upgrade window for each agent ensures it is running the latest fixes. To learn more about how Palantir identifies, discloses, and patches vulnerabilities, see [Monitoring for vulnerabilities](#).

To resolve a finding without waiting for the automatic 30-day cleanup, you can delete the [API Reference ↗](#) ches, and archived plugin versions yourself. See [Backups and caches](#) for how to identify which versions are still in use.

[Send feedback](#)



It's important to regularly upgrade versions and check vulnerability scans, remediating any issues promptly.

PREVIOUS



Architecture

NEXT



Troubleshooting

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Troubleshooting

When a source is not behaving as expected, use this page to diagnose and resolve the issue. It describes the troubleshooting tools available in Data Connection, where to look first when a connection is not working, and how to resolve common connectivity and certificate problems.

For problems specific to syncs or agents, see [Capability-specific issues](#).

Where to start

Use this list as a first pass when a source or connection is not working.

1. **Confirm the source is reachable.** On the source page, select **Preview** to confirm the connection is established. If it fails, open the [source terminal](#) and run `dig`, `curl`, and `openssl s_client` against your source's hostname and port.
2. **Check egress configuration.** For [Foundry worker](#) sources, confirm that the correct [egress policies](#) are attached and that the addresses, ports, and protocols they allow match what the source connects to. Review the [network egress logs](#).
3. **Check credentials.** Re-enter credentials if you suspect they are stale, especially after a migration.
4. **Check certificates.** `PKIX` or `SSLHandshakeException` errors indicate that the correct certificates are not configured for the source. See [Certificates and TLS](#).
5. **Check OAuth authorization.** If your source uses an [outbound application](#), `HTTP 401: Unauthorized` and similar errors usually indicate that the calling user has not completed the interactive authorization flow, that the outbound application is disabled, or that the API Reference ↗s are insufficient. See [OAuth and outbound applications](#).

Go to AI FDE. For Foundry worker sources, use [Debug with](#) these checks and surface specific fixes.

Send feedback

If your source uses an [agent worker](#), also confirm the agent shows as **Online** in Data Connection and review the [agents troubleshooting reference](#) for status-specific guidance.

Troubleshooting tools

Data Connection provides several built-in tools to help you investigate connectivity, authentication, and data movement issues without leaving the platform.

Debug with AI FDE

For [Foundry worker](#) sources, you can hand off debugging to [AI FDE](#), Palantir's AI-powered forward deployed engineer. AI FDE loads your source and its associated [egress policies](#) into context and uses the same egress logs and source debug tools described on this page to investigate connectivity failures on your behalf.

To start an AI FDE debugging session from a source:

1. Open the source you want to debug in the **Data Connection** application.
2. In the **Connection details** panel, select the dropdown caret next to **Debug**, then select **Debug with AI FDE**.
3. AI FDE opens in a new tab with the source and its egress policies preloaded as context, and proposes a debugging plan that includes inspecting the source configuration, reviewing each egress policy, retrieving network egress logs, and running live connectivity tests.

AI FDE will summarize what is working, what is failing, and suggest specific fixes (for example: missing egress policies, incorrect address or port, TLS configuration issues, or credential problems).

i **Debug with AI FDE** is available only for sources that use a [Foundry worker](#) runtime. AI FDE requires [AIP to be enabled](#) on your enrollment. To learn more about how AI FDE works, what it can do, and how it manages context and permissions, see the [AI](#)

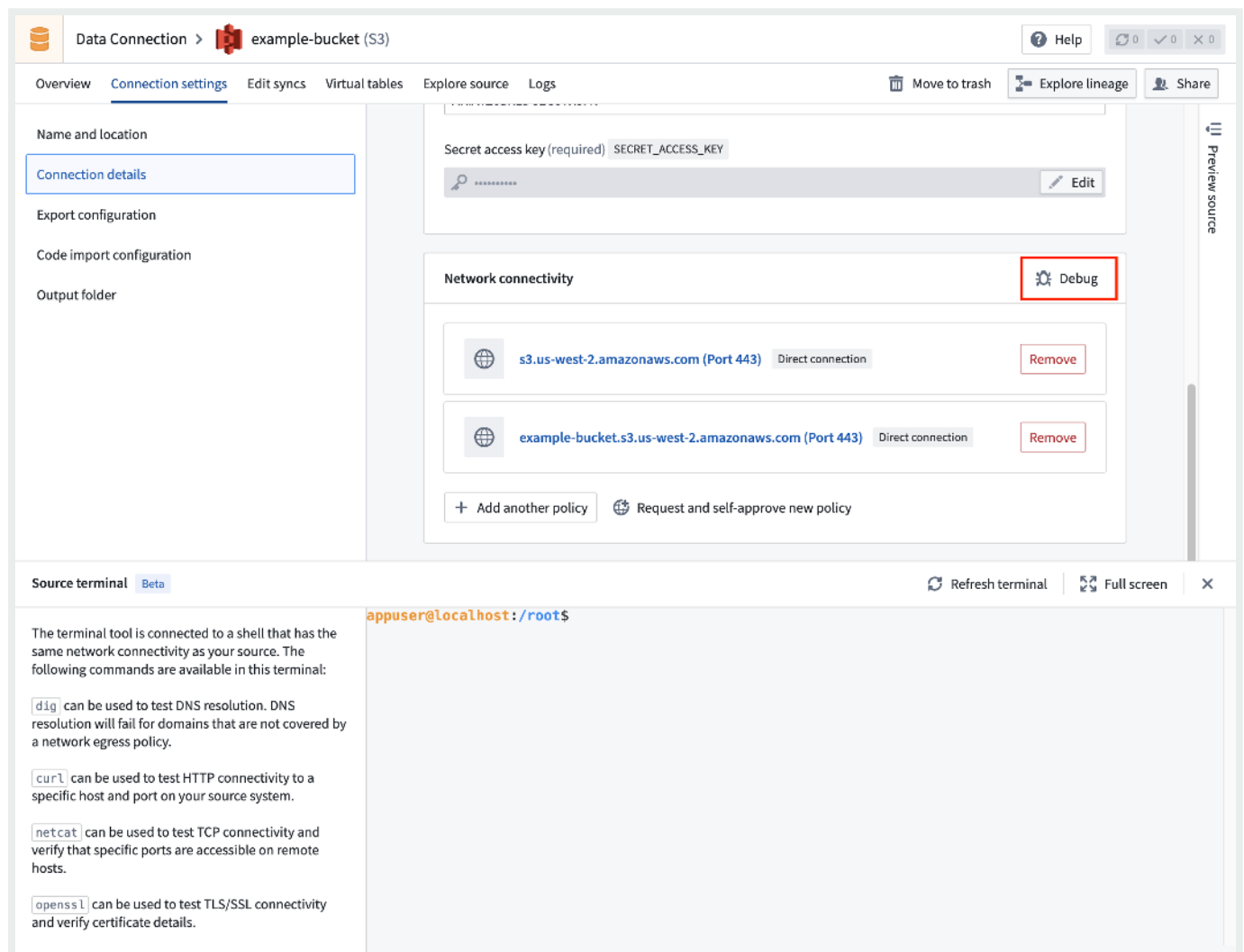
[API Reference](#) ↗

Send feedback

Source terminal

A terminal, using the same connectivity settings as your source, is available to help you test network connectivity with your external systems using commands such as `dig`, `curl`, `netcat`, and `openssl`.

To open the terminal, select **Debug** in the **Network connectivity** panel under **Connection details** on the source page.



dig

The `dig` command tests DNS resolution and can be used to check whether the hostname of your source resolves correctly. For example, to test whether DNS resolves `palantir.com`, run `dig palantir.com`. If you see an `ANSWER SECTION` in the response, DNS resolution succeeded.

[Send feedback](#)

The following is an example of a successful `dig` command:

```

appuser@localhost:/root$ dig palantir.com

; <<>> DiG 9.18.30-0ubuntu0.20.04.2-Ubuntu <<>> palantir.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13561
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;palantir.com.                IN      A

;; ANSWER SECTION:
palantir.com.                 300     IN      A      151.101.129.170
palantir.com.                 300     IN      A      151.101.1.170
palantir.com.                 300     IN      A      151.101.193.170
palantir.com.                 300     IN      A      151.101.65.170

;; Query time: 99 msec
;; SERVER: 10.100.0.53#53(10.100.0.53) (UDP)
;; WHEN: Fri Oct 31 14:56:58 UTC 2025
;; MSG SIZE rcvd: 142

```

curl

The `curl` command tests HTTP connectivity to a specific host and port on the external system. If the port is not specified, it is inferred from the protocol (for example, 80 for HTTP or 443 for HTTPS). For example, to test whether connectivity can be established to `https://palantir.com`, run `curl https://palantir.com`.

The following is an example of a successful `curl` command:

```

appuser@localhost:/root$ curl https://palantir.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
API Reference ↗</TITLE></HEAD><BODY>
The document has moved

```

Send feedback

```
<A HREF="https://www.palantir.com/">here</A>.  
</BODY></HTML>
```

If the `curl` command is hanging, set a timeout (in seconds) using the `--connect-timeout` flag:

```
1 curl --connect-timeout 10 https://palantir.com
```

openssl

The `openssl` command tests SSL/TLS connectivity and verifies certificates for HTTPS connections. For example, to test whether an SSL/TLS connection can be established to `palantir.com` on port `443`, run `openssl s_client -connect palantir.com:443 -servername palantir.com`.

The following is an example of a successful `openssl` command:

```
appuser@localhost:/root$ openssl s_client -connect palantir.com:443 -servername  
CONNECTED(00000003)  
depth=2 C = BE, O = GlobalSign nv-sa, OU = Root CA, CN = GlobalSign Root CA  
verify return:1  
depth=1 C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2  
verify return:1  
depth=0 CN = *.palantir.com  
verify return:1  
---  
Certificate chain  
0 s:CN = *.palantir.com  
i:C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2  
a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256  
v:NotBefore: Jun 17 16:50:15 2024 GMT; NotAfter: Jul 19 16:50:14 2025 GMT  
---  
API Reference ↗ read 3654 bytes and written 442 bytes
```

Send feedback

If the connection is successful, you will see certificate details and a message indicating the SSL handshake completed. If the command hangs or shows certificate verification errors, there may be network or certificate configuration issues.

Certificates configured in the UI are not automatically included in the terminal environment. To use them in the terminal, add them manually, either inline or with `nano` / `vim`:

```
1 echo "-----BEGIN CERTIFICATE-----
2 MIIGXjCCBUagAwIBAgIQASByQ6gv8Z6X7wEqSyBb1DANBgkqhkiG9w0BAQsFADBY
3 ...
4 9lc=
5 -----END CERTIFICATE-----" > server-cert.pem
```

To test with custom certificates, such as self-signed certificates or mTLS, specify the certificate files:

```
1 openssl s_client -connect my-server.example.com:443 \
2 -servername my-server.example.com \
3 -CAfile server-cert.pem \
4 -cert client-cert.pem \
5 -key client-key.pem
```

If these commands succeed but your connection still does not work, some certificates or source credentials are likely configured incorrectly.

Network egress logs and metrics

For [Foundry worker](#) sources, network egress logs and metrics provide visibility into the actual network calls your source is making. These tools can help identify networking issues that may not be apparent from terminal-based debugging alone.

- **Build logs:** From a build page, select **Logs** and apply the **Network egress logs** filter to see

API Reference ↗ is activity for that build.

- **Control Panel:** Navigate to **Control Panel > Network egress**, select a policy and open the **Observability** tab to view aggregated logs per source and usage policy.

Send feedback

Source preview and exploration

Before debugging deeper, confirm whether the source is fundamentally reachable and configured correctly:

- **Preview:** On the source page, select **Preview** in the right panel to confirm that the connection has been successfully established.
- **Explore:** Use the [source exploration](#) view to inspect tables, files, and previews from the source. If exploration succeeds but a specific sync fails, the issue is more likely with the sync configuration than with the source.

Job tracker

To investigate the status of a running or recently completed sync, open the **Job tracker** application and select the sync. From there, you can inspect status, stages, and logs, including the `_data-ingestion.log` for JDBC syncs.

Connectivity and egress

For a connection to succeed, Foundry must be allowed to reach your external system over the network. [Foundry worker](#) sources control this through [egress policies](#), which allowlist the specific hosts, ports, and protocols a source is permitted to connect to. A connection that fails before authentication is usually an egress problem.

To debug a suspected egress issue:

1. Confirm that the correct egress policies are attached to the source, and that the host, port, and protocol they allow match the system you are connecting to.
2. Use the [source terminal](#) to test whether the host is reachable at all, using `dig`, `curl`, and `openssl s_client`.
3. Review the [network egress logs](#) to see the actual outbound calls your source is making and identify which are being blocked.

API Reference ↗

To ~~configure~~ an egress policy, see [Configure egress policies](#).

Send feedback

Certificates and TLS

This section assumes that Foundry acts as the client establishing a connection to an external system that acts as the server. For background on the underlying concepts, see [TLS](#), [mTLS](#), and [SSL](#).

PKIX and SSL exceptions

PKIX exceptions and other `SSLHandshakeException`s occur when Foundry does not have the certificates needed to authenticate with the source. Install the correct certificates:

- **Foundry worker sources:** configure certificates in the source's settings. See [Add certificates](#).
- **Agent worker sources:** see [Data Connection and certificates](#).

For sources that only support the deprecated TLSv1.0 or TLSv1.1 protocols, see [Connect to data sources using the insecure TLSv1.0 and TLSv1.1 protocols](#). Palantir discourages the use of deprecated TLS versions.

Server and client certificates

It is important to understand the difference between a server certificate and a client certificate.

Server certificates (TLS): When the Foundry client establishes a TLS connection with the external system, the external system presents a certificate to prove its identity and to provide the public key needed to establish an encrypted connection. The Foundry client then verifies the external system's identity by validating the certificate chain up to a trusted root Certificate Authority (CA). Most systems trust a well-known list of public CAs by default. You only need to add a server certificate to your source configuration (Foundry's `truststore`) if the external system presents a certificate that is not signed by one of these public CAs, such as a self-signed certificate or one issued by a private or internal CA. Server certificates are public key certificates only, never the private key.

[API Reference ↗](#)

Client certificates (mTLS): A client certificate is used by the Foundry client to prove its identity to the external system. With mTLS, the external system and Foundry each verify the

[Send feedback](#)

other's identity. Client certificates are provided as a certificate and private key pair.

To configure certificates, see [Add certificates](#).

Common certificate errors

[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate in certificate chain

This error indicates that server certificates are missing from the source configuration. In Python environments, the entire [certificate chain](#) is validated by the underlying OpenSSL library when establishing secure connections.

To retrieve the entire certificate chain for an external system, run the following command:

```
1 openssl s_client -connect <external_systems_hostname>:<desired_port> -show
```

- i** This command must be run from a host with network access to the external system. For [Foundry worker](#) sources, run it from the [source terminal](#). For [agent worker](#) sources, run it from the agent host or another system with access to the agent's network.

OAuth and outbound applications

When a source is configured with an [outbound application](#), authentication is delegated to an external OAuth 2.0 provider on behalf of the calling user. The user must complete the [interactive authorization flow](#) at least once before the source can be used from non-interactive contexts, such as scheduled functions or [an automation](#).

[API Reference](#) ↗ with errors

Send feedback

HTTP 401: Unauthorized

If a function or workflow returns `HTTP 401: Unauthorized` despite the source being configured with an outbound application, verify that:

- The user invoking the function has completed the [interactive authorization flow](#) at least once.
- The outbound application is **Enabled** in Control Panel.
- The [scopes](#) configured on the outbound application include the permissions required to access the requested API resource.

Credentials expired and no refresh handler provided

This error is surfaced when calling a source from a Python or TypeScript function. The cached refresh token has expired or was revoked by the external OAuth 2.0 server. The user must invoke the function (or another workflow backed by the same outbound application) from an interactive interface to complete the authorization flow again.

Resolved source credentials are not present on the Source

This error is surfaced when calling a source from a Python or TypeScript function. It indicates that the source does not have an outbound application configured, or that the calling identity could not be resolved to an authorized user. Confirm that the source's domain is configured with **OAuth 2.0** authentication and that the function is invoked on behalf of an authenticated end user rather than a service account.

Capability-specific issues

The guidance above applies to any source. For problems specific to a particular capability, see the dedicated references:

- **Syncs:** Incremental JDBC behavior, intermittent failures and hangs, `REQUEST_ENTITY_TOO_LARGE`, `BadPaddingException`, and unexpected data types: see the [syncs troubleshooting reference](#).
- **Agent workers:** Agent status, Agent Manager and Bootstrapper startup errors, out-of-

API Reference ↗

ns, download issues, and permissions: see the [agents troubleshooting](#)
- **Other common questions:** Scheduling, duplicate rows, schema n

Send feedback

 data types: see the [Data Connection FAQ](#).

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Foundry worker vs. agent worker

Foundry worker vs. agent worker

⚠ Legacy

Agent worker is in the [legacy](#) phase of development. Foundry worker is the recommended way to run all data connection sources. This page explains why, and answers common questions about moving off agent worker.

Context

Agent worker was the original way Data Connection sources ran in Foundry: capabilities executed on a customer-managed [agent](#) host inside the customer network. [Foundry worker](#) is the recommended option for all sources today: capabilities run in Foundry's containerized, scalable job execution environment. Agents now act only as network proxies via [agent proxy egress policies](#); they no longer execute capabilities.

See [architecture diagrams](#) for how each connection type fits together, and [core concepts](#) for definitions.

Benefits of Foundry worker

Scalability and isolation

API Reference ↗

Foundry worker runs each capability in its own container, sized to the capability's needs. It has its own JVM heap, isolated from the agent's shared JVM heap, and a long-running sync n

Send feedback

jobs from running in parallel. Compute scales independently per job, rather than being bounded by the capacity of a single agent host.

Stability and release cadence

Foundry worker runs in Foundry's managed compute environment and is updated continuously alongside the rest of the platform. Agent worker runs on customer-managed hosts and can only upgrade during the agent's [maintenance window](#), typically once a week. Bug fixes and improvements therefore take longer to reach the agent. Agent worker is also exposed to host-level issues (resource exhaustion, OS or JVM problems, scheduled downtime); see [agent troubleshooting](#) for the common failure modes. Foundry worker avoids these by removing the customer-managed runtime from the path.

Access to all capabilities and new development

[Virtual tables](#), [virtual media](#), [using sources in code](#) from [external transforms](#), [external functions](#) and [compute modules](#), and several capabilities added in recent years do not run on agent worker. New capabilities and ongoing feature development target Foundry worker only.

Lower operational burden

Foundry worker removes the need to tune the agent JVM heap, plan agent host capacity, and monitor the agent process at the OS level. When agents remain in the picture, they act only as a network proxy via [agent proxy](#) — a much smaller operational footprint than running compute on the agent.

Cleaner credential lifecycle

Foundry manages source credentials centrally instead of tying them to a specific agent. Reassigning agents to a source no longer requires re-entering secrets. Re-provisioning an agent [API Reference ↗](#) invalidate credentials.

Send feedback

Common questions

Is Foundry worker as secure as agent worker?

Yes. Foundry stores source credentials encrypted at rest and in transit in both models. The only material difference is where the encryption key for source credentials lives: on disk on the agent host (agent worker) or securely managed inside Foundry (Foundry worker). With [agent proxy](#), network connectivity to your source systems remains outbound-only, initiated from the agent inside your network. This matches the agent worker model.

Is Foundry worker as performant as agent worker?

Yes. Containerized jobs do not compete for a single agent's heap and can scale per job.

Does this change my network connectivity model?

No. With [agent proxy](#), the agent still initiates an outbound WebSocket to Foundry, and all traffic to your source systems still originates from the agent inside your network — no new inbound paths into your network, same model as agent worker. What moves is *where capabilities execute*, not *how Foundry reaches your systems*.

What happens to my existing agent worker sources?

Existing agent worker sources will continue to work. Agent worker is legacy, fully supported. When you are ready, migrate using the [guided wizard](#). Note that this migration is reversible for 30 days.

What about agent proxy?

i Agent proxy is not the same as agent worker. Agent proxy is a networking mode for Foundry worker, not a worker type. If "agent" appears in a discussion about a new Foundry worker, [agent proxy](#) is almost always what is meant. See [architecture](#) for the diagrams.

API Reference ↗

Send feedback

Switch source from agent worker to Foundry worker

Most agent-based sources should be migrated to a Foundry worker, either a [direct connection](#) for systems accessible from Foundry, or an [agent-proxy connection](#) for sources hosted on separate networks.

i If you encounter any issues migrating from agent worker to Foundry worker, you may revert to your previous connection settings within 30 days.

Prerequisites

Before starting the migration, ensure the following:

- You have at least one healthy agent assigned to the source that can supply source secrets, certificates, and, if applicable, JDBC drivers.
- You have the required permissions to create egress policies. To create new egress policies, you must have access to the workflow titled `Manage network egress configuration` in [Control Panel](#), which is granted to the `Information Security Officer` role.

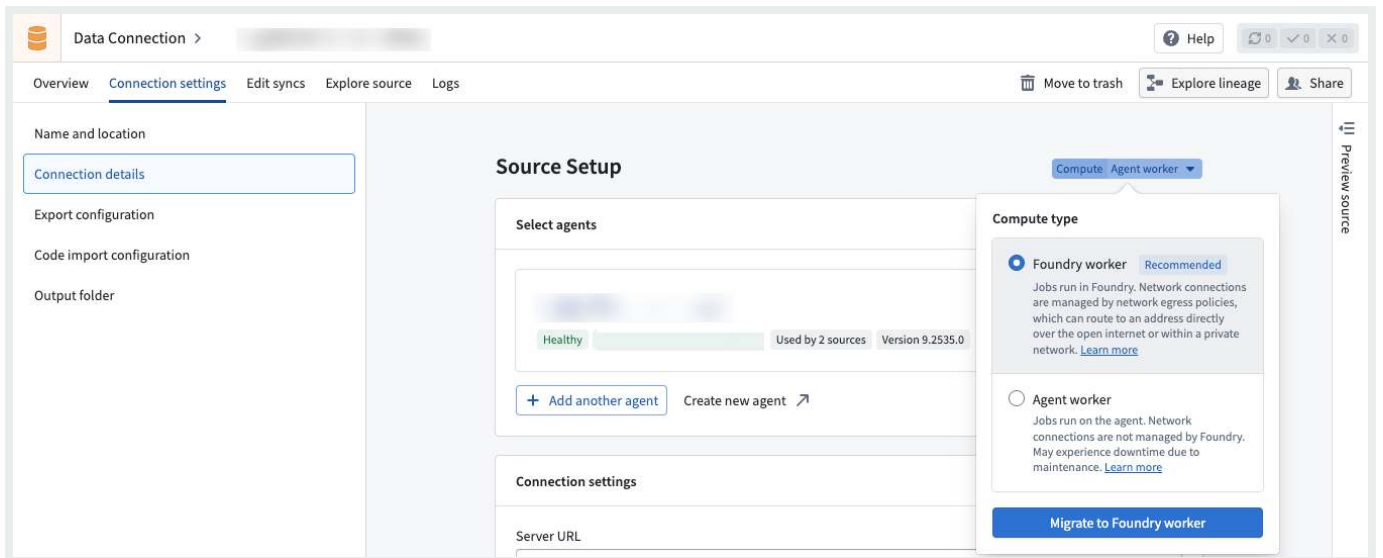
Migration steps

To perform a migration to a Foundry worker, follow the steps below:

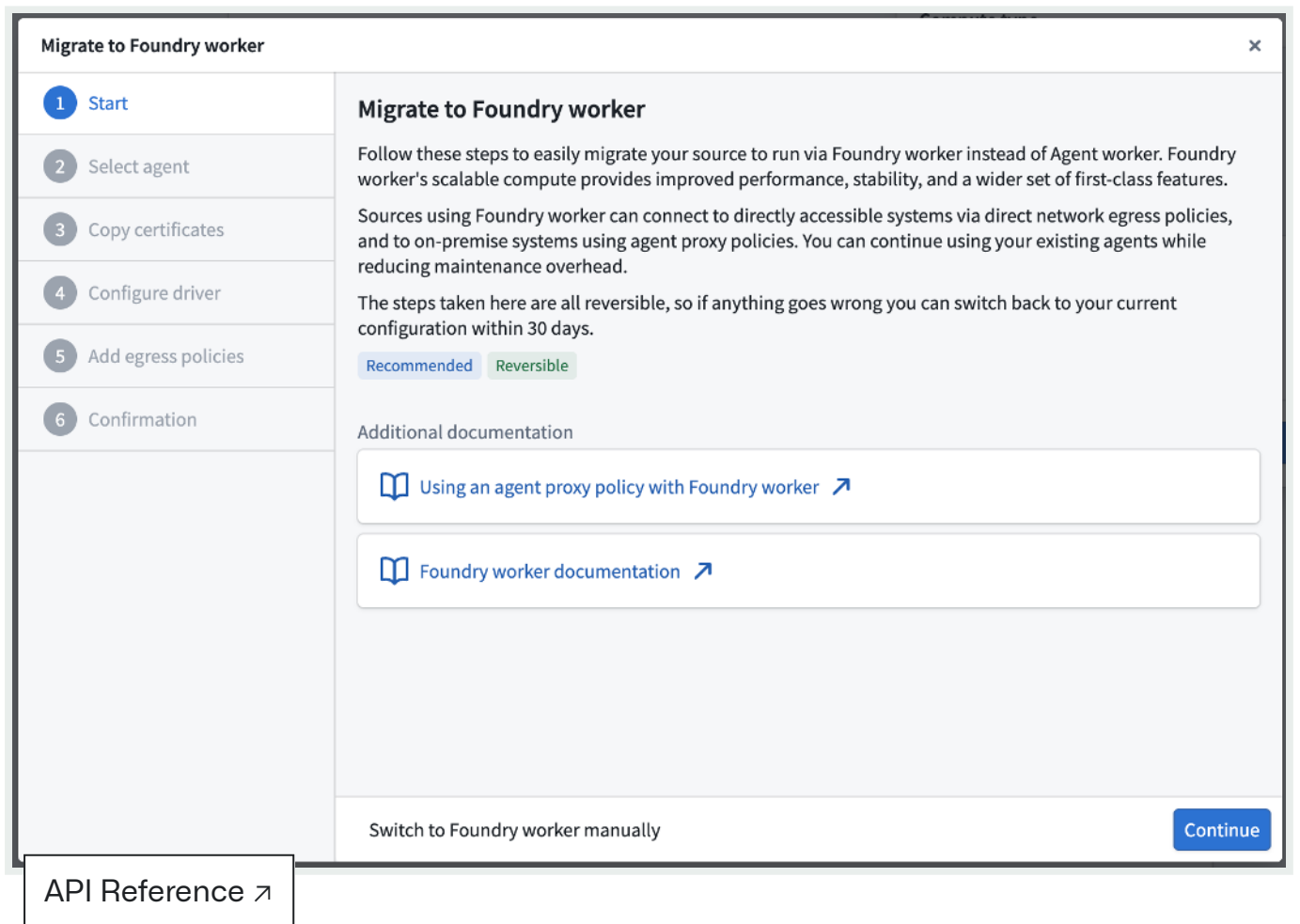
1. Navigate to **Connection settings > Connection details** and select **Compute: Agent worker**. This opens a dialog where you can select **Migrate to Foundry worker**.

[API Reference ↗](#)

[Send feedback](#)



2. Review the migration overview. The walkthrough dialog explains the steps involved and confirms that the process is reversible. Select **Continue** to proceed.



3. Choose a representative agent. Select a healthy agent currently as [Send feedback](#)

The selected agent is used to copy secrets and certificates to Foundry and to suggest

required egress configuration based on the agent's current setup. You must acknowledge that secrets on the selected agent will be saved to Foundry.

Migrate to Foundry worker

1 Start

2 Select agent

3 Copy certificates

4 Configure driver

5 Add egress policies

6 Confirmation

Select a representative agent

Choose from the agents that you currently use to connect to this source. The selected agent will be used for copying secrets to Foundry. We'll also use the configuration of this agent to suggest drivers and required egress for a direct connection.

Healthy

☒ I understand that secrets on the selected agent will be saved to Foundry.

Switch to Foundry worker manually

BackContinue

4. Copy certificates to the source. With Foundry worker, certificates must be applied directly to the source rather than to the agent. Select the certificates from the representative agent that should be transferred to your data source. If no certificates are found on the agent, you can skip this step.

API Reference ↗

Send feedback

Migrate to Foundry worker

1 Start

2 Select agent

3 Copy certificates

4 Configure driver

5 Add egress policies

6 Confirmation

Copy certificates to source

With Foundry worker, certificates must be applied directly to the source rather than to the agent. Select certificates that will be transferred to your data source.

Switch to Foundry worker manually

BackContinue

5. (Optional) Configure a JDBC driver if required by the source type. Select a driver from the representative agent to use with the Foundry worker.

API Reference ↗

Send feedback

Migrate to Foundry worker

1 Start

2 Select agent

3 Copy certificates

4 Configure driver

5 Add egress policies

6 Confirmation

Configure driver

Select drivers from the representative agent for direct connection.

1 JDBC Drivers

Healthy

jar

Switch to Foundry worker manually

Back

Continue

6. Add egress policies. Configure [direct connection](#) or [agent proxy](#) egress policies to define how the Foundry worker should reach your source systems.

[API Reference ↗](#)

[Send feedback](#)

Migrate to Foundry worker

1 Start

2 Select agent

3 Copy certificates

4 Configure driver

5 Add egress policies

6 Confirmation

Add egress policies

+ Add existing policy

Self approve new policy

Suggested network egress policies

These suggested egress policies are required for Foundry to connect to your source.

Agent proxy

Self approve new policy

Switch to Foundry worker manually

Back

Continue

7. Review the confirmation step. Before completing the migration, acknowledge the following:
- All running jobs (syncs, exports, tasks) will be terminated as part of the migration. Scheduled jobs will run in Foundry on the next scheduled run, but unscheduled jobs must be manually restarted.
 - You have 30 days to revert to your previous configuration, if necessary.
- Select **Migrate** to complete the process.

[API Reference ↗](#)

[Send feedback](#)

1

Start

2

Select agent

3

Copy certificates

4

Configure driver

5

Add egress policies

6

Confirmation

Confirmation

Running jobs will be terminated

All running jobs (syncs, exports, tasks) will be terminated as part of the migration. Jobs which do not have schedules will need to be manually restarted. Scheduled jobs will run in Foundry on the next run. If your schedule does not run frequently, you may want to manually trigger jobs now to ensure they are working as expected.

☒ I understand that unscheduled jobs will need to be restarted manually.

Revert to previous configuration

Should you encounter any complex issue, you will have 30 days to revert back to your previous connection settings after migration. After 30 days, your previous setup can not be automatically restored.

☒ I understand I will have 30 days to return to previous setup

Switch to Foundry worker manually

Back

Migrate

Switch manually

If you prefer not to use the guided migration wizard, you can select **Switch to Foundry worker manually** from the migration dialog. Note that when switching manually, credentials must be re-entered, as the automated secret transfer from the agent is not performed.

Revert to agent worker

For 30 days after a migration, you can revert the source to its previous agent worker configuration:

1. Navigate to **Connection settings > Connection details** on the migrated source.

2. In the **API Reference** dropdown, select **Agent worker > Revert**.

3. In the **Revert source to Agent worker** dialog, acknowledge that:

- All connection settings, including credentials, will be reverted to the previous configuration after migration. Any changes made since will be lost.

Send feedback

- Unscheduled jobs (syncs, exports, tasks) will need to be restarted manually. Scheduled jobs will use the pre-migration connection settings on their next scheduled run. Jobs that started successfully after the migration continue running with migrated settings until completion.

4. Select **Revert migration** to confirm.

If you prefer to reconfigure the source on agent worker manually rather than restoring the pre-migration settings, select **Switch to connect via Agent worker manually** from the dialog.

Troubleshooting

This section describes situations that may occur during the migration, as well as suggested resolution steps. As a reminder, the migration is reversible for 30 days.

Could not resolve type id as a subtype of
'com.palantir.magritte.api.Source'

Suggested resolution:

- This happens when a dependency required for the source cannot be found. Ensure that you have configured all required certificates, proxies, and drivers required for a particular source, then retry the migration.

UnknownHostException

Suggested resolution:

- Ensure that correct egress policies are assigned to the source.
- Confirm that Foundry is able to access the endpoint that is throwing the exception.

Driver class not found

See [API Reference ↗](#):

- Confirm that the correct driver is uploaded to the JDBC source.

[Send feedback](#)

PKIX path building failed

Suggested resolution:

- Ensure that correct certificates are added to the source.

PREVIOUS
← Troubleshooting

NEXT
Initial setup overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

[API Reference ↗](#)

Send feedback

Data connectivity & integration / Data Connection / Initial setup overview

Initial setup overview

This guide will walk you through the process of connecting your organization's data to Foundry.

Before starting, it is important to recognize that this first step towards connecting your organizational data to Foundry is fundamentally a **networking concept**. The initial setup is best done by someone familiar with network engineering and aware of the organization's network topology and configurations, such as firewall rules.

Conceptual overview

Connecting data to Foundry requires that three components are configured in the following order:

1. Configure networking connectivity

You must first ensure that there is a valid networking path between Foundry and the external system.

For external systems hosted on the same network from Foundry's network (for cloud-hosted Foundry instances, this is typically systems accessible over the Internet), define [direct connection egress policies](#) to route the traffic. Make sure that the external system allows inbound connections from Foundry.

For external systems hosted on a separate network from Foundry's network (for cloud-hosted Foundry instances, this is typically on-premise systems), an [agent](#) is required. An agent is Palantir software that runs within your organization's network and acts as a secure intermediary between your organization's data sources and your Foundry instance.

API Reference ↗



Send feedback

Make sure the external system allows inbound connections from the agent and that the agent can establish outbound connections to the external system as well as to Foundry.

The agent can then be used to define [agent proxy egress policies](#) to route traffic through the agent when using a [Foundry worker](#).

Agents can also be used as a legacy [agent worker](#) where capabilities execute on the agent host. New sources should use Foundry worker; see [Foundry worker vs. agent worker](#).

Agents can be shared by agent worker sources and sources using agent proxy egress policies, though we recommend always having multiple agents assigned to a given source to [maximize availability](#).

Learn more about various [architecture patterns](#).

2. Source configuration

You must [configure a source](#), or connection, to connect your external system to Foundry before executing any [capability](#). An external system could be, for example, a Postgres database, an S3 bucket, a filesystem on a Linux server, an SAP instance, or a REST API accessible over the Internet.

3. Capability configuration

Once a source is configured to connect to the external system, you must configure the [capability](#) to execute on that source. Capabilities include batch syncs of data, streaming syncs, webhooks, exports, and more.

A [batch sync](#), for example, reads specific data from an external system and ingests it into Foundry. If you have a PostgreSQL database that contains multiple tables, you might configure a sync to ingest one specific table into Foundry. Once a sync has successfully run, the result in Foundry will be a [dataset](#) to use across all of Foundry's data pipelining, model development, and analytical tools.

API Reference ↗

Roles and workflows

Send feedback

Most Foundry users will never need to set up a new agent themselves. Agent setup requires an IT-focused skill set, though the same agent can be reused to support multiple sources and syncs. Some organizations can operate long-term with agents set up during the first week of a Foundry deployment. New agents are only needed to access data that your existing agents cannot access (due to network segmentation or data scale, for example) or to set up an additional agent to allow for [high availability](#).

The table below summarizes the configuration frequency and skill set required for maintaining the resources required for connecting to data:

Resource	Frequency of configuration	Typical user role	Knowledge required
Agent	Rare	IT / Network Engineer	Network and firewall policies; Linux VMs; SSH
Source	Occasional	IT / Network Engineer; Data Engineer	Debugging network access; credential management
Sync	Frequent	Data Engineer; Data Scientist	Writing SQL queries; managing files

High availability

We recommend setting up redundant hardware to establish a high availability (HA) architecture. High availability increases resiliency and allows no-downtime maintenance during operating hours.

Foundry offers HA at the source level, meaning that if a source is assigned to multiple agents, Foundry will dispatch ingestions to one of the healthy agents. We strongly recommend configuring agents in a high availability setup at the start of source creation; adding extra agents to a created source requires re-entering the credentials for that source.

The following best practices are recommended when setting up high availability:

API Reference ↗

nts by pairs, on similar hardware.

- Give each agent in a pair similar names, such as agent-1 and agent-2
- Systematically assign both agents in a pair to every source.

Send feedback

- Configure non-overlapping upgrade windows on both agents in a pair. Upgrade windows should be during business days and provide sufficient soaking time. Doing so ensures that any unexpected issues with an update will be contained to a single agent and can be detected by operators or administrators.

Next steps

To get started, move on to [setting up a source](#).

PREVIOUS

← [Foundry worker vs. agent worker](#)

NEXT

[Permissions reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Permissions reference

All major concepts in Data Connection — agents, sources, and syncs — are managed in Foundry as [resources](#). This enables flexibly organizing these resources across Projects and folders, and allows security primitives such as [markings](#) to be applied to Data Connection resources.

Platform owners should be careful to not over-share Data Connection resources. Users with view and edit permissions on sources and agents should be trusted pipeline developers. The datasets produced by the syncs can be imported to downstream Transforms projects as necessary, thereby granting access to the data without sharing access to the agents or sources.

This page documents how permissions work for all Data Connection resources, including:

- [Agents](#)
- [Sources](#)
- [Syncs](#)
- [Plugins](#)
- [Webhooks](#)
- [External connections from code](#)

Resource permissions

 The permissions listed below are defaults. In some environments, the default behavior may have been changed using [custom roles](#).

API Reference ↗



Send feedback

Agents

Owner

An owner of an agent is usually its creator and gets the following permissions:

- Re-download the agent or regenerate the token for that agent
- Inherits permissions of Editor

Editor

An editor of an agent can:

- Deploy a source to that agent.
- Configure an agent using the Agent Manager (change the plugins and configs, and restart the agent).
- Share the agent with other users.
- Inherit Viewer permissions.
- Delete the agent.

i A user must be the editor of a Project to create an agent in that Project, but must be an *owner* of the Project to administer the agents within that Project. That means that a user may create an agent and then be unable to generate download links or perform other administrative tasks on the agent.

Viewer

A Viewer of an agent can:

- See the agent configuration and status.

Sources

[API Reference ↗](#)

Owner

[Send feedback](#)

By default, the source owner inherits **Editor** permissions.

A source owner can also:

- Modify export configurations if the user configuring exportable Markings has permission to [remove Markings](#) from any Organization or security Markings they add as safe to export to the selected source.
- Allow a source to be imported into code resources.

Editor

An editor of a source can:

- Delete the source.
- Change the assigned agents of the source (additionally requires Edit on every agent being added).
- Update the source configuration (additionally requires Edit on the agents assigned to the source).
- Rename the source.
- Create, edit, and delete syncs for that source.
- Run SQL queries on database sources.
- Share the source with other users.
- Explore and preview sources that support these operations, such as JDBC and Directory sources.
- Inherit Viewer permissions.
- Import a source to a code resource

Warning

Keep in mind that syncs can change the source system if the source credentials allow it (e.g. delete files from a directory or S3 source, or drop data from a database via arbitrary SQL). You should only grant Edit access to the source (or the Project containing it) to users whom you would also grant full access to the account the source uses to access the data.

V

API Reference ↗

- A viewer of a source can view source configuration, including assigned

Send feedback

Syncs

Sync permissions are derived from the relevant source and output dataset.

- **Viewing a sync** requires View on the source and dataset.
- **Editing a sync** requires Edit on the source and dataset.
- **Deleting a sync** requires Edit on the source and dataset.
- **Running a sync** requires Edit on the dataset.

Plugins and JDBC Drivers

Owner

- Inherits Editor permissions.

Editor

- Delete the plugin / driver (the plugin / driver must not be assigned to any agents to successfully delete it).
- Inherits Viewer permissions.

Viewer

- View the plugin / driver.
- Download the plugin / driver.
- Add the plugin to an agent (also requires Editor on the agent).

Webhooks

A Webhook's permissions are inherited entirely from the source with which it is associated.

- **Viewing a Webhook configuration** requires View on the source.

API Reference ↗

Editing a Webhook configuration requires Edit on the source.

- **Deleting a Webhook configuration** requires Edit on the source.

- **Deleting a Webhook** requires Edit on the source.

- **Configuring an Action to use a Webhook** requires Edit on the source.

Send feedback

- **Executing a Webhook** requires Edit on the source. Note that executing a Webhook permission is not required when executing a Webhook through an Action.

By default, only the user who executes a Webhook may view the response using the history tab in the Data Connection application. The `webhooks:read-privileged-data` permission may be added to a custom or default [role](#) to allow users with that role to view the full request and response history for all webhooks that they can access. The standard recommended setup is to add a new role called "Webhook Privileged Data Viewer" with this permission, and ensure that users who need to view full Webhook history have this role on the relevant sources.

External connections from code

External transforms, pipelines, and compute modules

 When a source is imported into a code resource, users with edit access on the repository can write custom code using the connection details configured for that source, including accessing secret values.

This means that the ability to import sources into code and granting `Editor` access to the source should be carefully managed, with only trusted users given access to the source and repository.

- Importing a source in a code resource ([transform](#), [pipeline](#) or [compute module](#)) requires `Editor` permission on the source.
- Triggering a build that imports a source requires `Editor` on the **code resource** (nothing on the source itself).
- Removing an imported source from a code resource requires `Editor` on the source **or** the code resource.

External functions

Refer to the documentation on [external functions permissions](#) for detailed information

related to the [API Reference](#) of using webhooks in a functions repository.

Send feedback

Marking propagation

As noted above, resources can have [Organizations](#) and [Markings](#) applied as an additional level of access control. Organizations and Markings are applied to the source will propagate to the datasets produced from syncs on that source. This means users will not be able to view data in any of the datasets produced from a source, unless they have access to all of the Organizations and Markings applied to that source.

⚠ Foundry-to-Foundry sync exception

When syncing data between Foundry instances using the Foundry connector, Markings and column-based access control (CBAC) policies are not automatically propagated to the destination stack. While CBAC is respected on the source stack — users can only sync data they are permitted to view — these security configurations are not transferred during the sync. You must manually configure appropriate permissions and Markings on the destination stack after the data arrives.

Since it is likely for sources to contain data of varying levels of sensitivity, we recommend that you mark the source with all of the Organizations and Markings that apply to the data available in that source, and then use transforms to clean and unmark the data using the `stop_propagating` and/or `stop_requiring` functionality. To learn more about unmarking, see the guides on [removing Markings](#) and [removing inherited Markings](#).

Note that applying Organizations and Markings to the data after it has been synced should not be considered sufficient; as access to the source is effectively access to the data, the source should be marked with the Organizations and Markings that apply to anything it contains.

Best practices

As a quick reminder, the primary Data Connection resources are agents and sources.

Plugins are deployed to agents to expand functionality.

API Reference ↗

Agent best practices

Send feedback

There are two recommended permission setups for agents: all agents in a single Project per Organization, or each agent in its own Project. The diagram above illustrates the former option; the guidance below describes why you might choose each option.

Option 1: Place all agents in a single Project per Organization.

Placing all agents in a single Project is the cleanest approach, and is recommended if there is one group of users managing all agents. This allows you to control access to the agents at the Project level, and to permission sources and syncs separately, downstream from the agent.

Choose this option only if it is acceptable for every Agent Manager and every source editor to have access to all agents. Remember that to deploy a source on an agent, a user must have Editor permission on that agent.

Option 2: Place each agent in its own Project.

This approach allows for full granularity of agent permissions. It also guarantees that the structure will never drift from the “groupings” at any time during the evolution of the source management. For example, you can safely assign / unassign agents from a source without needing to grant users access to an entire group of agents, and there would be no need to restructure Projects to account for group permission changes in the future.

This approach is recommended if you have multiple groups managing agents, and you would like to permission their access to agents separately.

Warning

Editors of the Project containing the agent will be able to create new sources and syncs that run on that agent. Make sure you do not grant Editor role on the Project to users who should not be able to do this.

Source best practices

[API Reference ↗](#)

Each source should be in its own Project in the [datasource](#) layer. Each sync defined on a source should output to a [dataset](#) within the same Project as the source.

[Send feedback](#)

contains sensitive data such as PII, you can apply a [Marking](#) to the source which will propagate to the output datasets.

Warning

Editors of the Project containing the source will be able to create new syncs and edit existing syncs for that source, as well as create Webhooks and configure them for use elsewhere in Foundry.

Keep in mind that syncs can change the source system if the source credentials allow it; for instance, deleting files from a Directory or S3 source, or dropping data from a database via arbitrary SQL.

You should only grant Edit access to the source (or the Project containing it) to users whom you would also grant full access to the account the source uses to access the data.

Plugins and JDBC drivers

There are two recommended permission setups for plugins and JDBC drivers: all resources in a single Project per Organization, or each JDBC / driver in a Project with one or more agents that require it.

Option 1: Place all plugins and drivers in a single Project per Organization.

This is recommended if there is one group of users managing all agents. This allows Agent Managers to have access to all plugins and drivers that have been uploaded. This approach should only be chosen if there should be equal access to plugins and drivers among all Agent Managers.

The recommended process for implementing this approach is:

1. Create one folder per agent.
2. Migrate the current plugin and driver to the appropriate agent folder.
3. Choose one copy of each plugin and driver to be the authoritative copy of each asset
4. Migrate each agent to use the root copy of the required plugin and driver
5. Deprecate the individual agent folders.

API Reference ↗

Project root.

Send feedback

Option 2: Place each plugin and driver type in its own Project.

This approach allows for certain "sensitive" plugins or drivers to be permissioned separately from others. If this approach is taken, you must ensure to give the required agent owners adequate permissions on each project so they can use them on agents.

PREVIOUS
← Initial setup overview

NEXT
FAQ →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

[API Reference ↗](#)

Send feedback

Data connectivity & integration / Data Connection / FAQ

Data Connection FAQ

The following are some frequently asked questions about Data Connection.

For general information, view our [Data Connection documentation](#).

- [Scheduled build never ran](#)
- [Ingestion led to duplicate rows in a dataset when running `SNAPSHOT` transaction](#)
- [Incremental load run time info](#)
- [Column type is not consistent between database and dataset](#)
- [Status of running query](#)
- [Sync is failing with a schema mismatch](#)
- [Permission error when re-adding a deleted sync job spec](#)
- [Bootvisor status is `Unknown`](#)
- [SSL/TLS issues](#)

Scheduled build never ran

A build was scheduled to run at a given time, but it did not attempt to run.

To troubleshoot, perform the following steps:

1. Is there another running sync for this dataset and branch? Verify that no other job is running at the same time as it is not possible to run two syncs on the same dataset and branch simultaneously.

[API Reference](#) ↗

2. Has the sync been paused? Verify this schedule is not paused on the schedule



page for this dataset. You can access this page via the **Explore** button of the sync or in Dataset Preview's **Manage Schedules** option.

Send feedback

of

3. Was the agent of this sync disabled? Navigate to the agent associated with the source of this sync, and verify the agent is not disabled.

[Return to top](#)

Ingestion led to duplicate rows in a dataset when running a `SNAPSHOT` transaction

A sync ran but led to duplicate rows.

1. When creating a new sync, choose to run an `APPEND` type sync instead of a `Snapshot`.
2. When declaring the incremental settings, if `last_upd_in_appl_ts` is the column that will be unique and always increasing, set that as the column and then select a value that is less than all the other values in this column.
3. After this, no additional action should be required as Data Connection will track the latest value that was synced and only bring in additional newer rows. Newer means greater than previous values, and newer timestamps are greater than older timestamps.

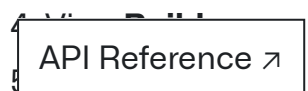
[Return to top](#)

Incremental load run time info

What value was used when I ran my incremental sync?

To troubleshoot, perform the following steps:

1. Go to the dataset.
2. Select **History** near the top of the screen.
3. Select the relevant transaction you are interested in on the left part of the screen.



6. Expand the section for **Custom Metadata** towards the bottom of the page.
7. Review the block for `incrementalMetadata` and verify correctness.

[Send feedback](#)

Column type is not consistent between database and dataset

My type is something different from what appears on my dataset after it synced.

To troubleshoot, perform the following steps:

1. If the column is `TIMESTAMP`, verify if the resultant type is `LONG` in Foundry. If it is `LONG`, you need to parse the type using your data preparation tool of choice (Code Repositories, Preparation, or another application) to `TIMESTAMP`. This is a side effect of many drivers provided by database creators where types are reverted to their safest representations.
2. If the column is `DECIMAL` and has a different precision than the original database, we recommend casting the number to a specific precision and scale in the query on the database itself, or casting the column to `VARCHAR` in the query and re-casting in Foundry.

`TIME` type columns lose sub-second precision between database and dataset

My database includes `TIME` type columns with sub-second precision and values with a non-zero sub-second component, but the value in the dataset only reflects distinctions up to the second component.

For most JDBC-based sources, including PostgreSQL and Microsoft SQL Server, a `TIME` value in the database is reflected in the Foundry output dataset as an `INTEGER` representing the time since midnight rounded down to the nearest full second. As a result, values in the database like `05:00:00.000`, `05:00:00.200`, and `05:00:00.800` will all become `18000000` in the output dataset.

[API Reference](#) ↗

[Send feedback](#)

To preserve sub-second distinctions, consider casting the value to a string in the sync configuration's SQL statement, as in the below Postgres example:

```
1 SELECT record_id, CAST(column_with_time_type AS text) FROM table_name
```

Status of running query

After a query begins running, how do I check its status?

To troubleshoot, perform the following steps:

1. Open the **Job tracker** application and select the running sync.
2. The most granular status of a sync is shown here.
3. If possible, verify the query behavior in the source database.

[Return to top](#)

Sync is failing with a schema mismatch

If the schema of a file or JDBC table changes between incremental **APPEND** transactions, your dataset will start failing with complaints of schema mismatches. Data Connection does infer schema for JDBC extracts, only propagates existing schemas for file-based extracts. In this event, you would have to apply the schema inference again if it is the same. If schemas truly have changed between **APPEND** transactions, a new dataset is needed for the new schema.

To troubleshoot, perform the following steps:

File based

[API Reference ↗](#)

1. If the files are XLSX or CSV tabular data, it may be possible to re-ingest the synced dataset without issue. If this schema matches the previous schema, the appended rows without issue.

[Send feedback](#)

2. If after inferring schema you still get schema errors (either in Dataset Preview or another application), then this new file needs to instead be synced to a new dataset since it represents a fundamentally different view of a table.
3. If the dataset was already appended with the new schema, we recommend reaching out to Palantir Support to revert this transaction. Additionally, the syncs of files to the current dataset need to be paused by going to the sync overview page and pausing any schedule associated with this sync.
4. Subsequent files with new schemas should sync to a different dataset than the original, so we recommend copying the information from the original sync into a new sync, but replacing the target dataset with a different one (annotating in the dataset name the new version).
5. Additionally, it may be best to delete the original sync to avoid any future schema mismatch errors from occurring and corrupting the existing data in Foundry.

JDBC

1. In the case where you expect a schema to change at some point and persist in its new form into the future, it is best to land the original table into a dataset whose name indicates the schema version. (Such as `account_transactions_v1.0`).
2. If a schema changes in the original table before a sync executes:
 - Pause the sync's schedule (if it exists)
3. If a schema changes in the original table after a sync executes:
 - Pause the sync's schedule (if it exists)
 - Contact Palantir Support to revert this transaction which has likely corrupted the target dataset
4. After your sync is paused, and you are ready to move to the new schema, you must first land the new schema into a new dataset:
 - Clone the sync into a new sync and replace the target dataset with a new one (such as `account_transactions_v1.1`). This new dataset can then be unioned with the original to contain the full set of data.
 - If required for your use case, you can delete the original sync after verifying correct behavior in the new sync. This ensures no possibility of landing corrupt data in the old dataset at the cost of decreased transparency on prior loading configuration.

Permission error when re-adding a deleted sync job spec

After deleting a sync's job spec, attempting to re-add it results in a permission error.

To resolve this issue:

Create a new sync that outputs to the same dataset. This establishes a new job spec with the correct permissions while preserving your existing data in the target dataset.

[Return to top](#)

Bootvisor status is Unknown

My Bootvisor is stuck in a `Unknown` state and won't stop / start.



Contact Palantir Support to check your setup before proceeding. Following the below steps will temporarily prevent syncs from running on this agent. Ensure multiple agents for sources or perform these steps during maintenance windows to prevent downtime.

1. Pause syncs on the agent.
2. Wait for all currently running syncs to finish.
3. Stop the agent (this step may fail depending on the current state of the agent).
4. SSH into the agent machine.
5. `kill` all JVM processes related to Data Connection.
6. Start the Bootvisor with `./service/bin/init.sh start`.
7. Start the agent in Data Connection, verifying the agent is not `paused` as well.

[R](#)

[API Reference ↗](#)

[Send feedback](#)

Common TLS/SSL issues

For guidance on TLS, mTLS, and SSL — including whether to add a server or client certificate and how to resolve common certificate errors such as

`CERTIFICATE_VERIFY_FAILED` — see [Certificates and TLS](#).

[Return to top](#)

PREVIOUS
← Permissions reference

NEXT
Agents / Set up an agent →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Set up an agent

An **agent** is a downloadable program installed within your organizational network and managed from Foundry's [Data Connection](#) interface. Agents have the ability to connect to different data sources within your organizational network. They are used to enable **[agent-proxy egress policies](#)** to provide network connectivity to privately-hosted sources, and to enable **[agent worker connections](#)** to read data from those sources and securely ingest to Foundry with a restricted access token.

This guide walks you through the steps required to create an agent. First, complete the following:

1. After logging in to Palantir, navigate to **Data Connection** using the left [sidebar](#).
2. Select the **Agents** tab.
3. Select **New agent** in the upper right corner.

If you do not see the option to create a new agent, you may not have the required role to do so. [Learn more about managing the agent creation workflow in Control Panel.](#)

Once you have the agent running and you want to connect a source to Foundry, you must obtain credentials for the source system that the agent can use to securely read data. Depending on your organization's network setup, you may also need to configure network settings to allow the agent to reach the source system.

Review the sections below to start setting up your agent:

API Reference ↗

Create agent host

Send feedback

For the agent program to successfully run, it must be hosted in a suitable environment (ideally, an environment using Linux as an operating system).

The most commonly used hosting method for Foundry agents is provisioning a Linux virtual machine (VM) in a cloud environment. For example, you could provision a Linux VM in AWS, Azure, or GCP, but you could also host the agent on a Linux server belonging to your organization. Note that while it is possible to host Foundry agents on Windows, this is not recommended by Palantir and should only be used if it is not possible to host in a Linux environment.

i The host you use for the agent should be used exclusively for running a single Foundry agent, not colocated with any other services or processes. Running multiple Foundry agents on the same machine is not supported.

Once you have a suitable location to host your agent, the next step is to ensure the host will meet the necessary hardware and OS requirements for a Foundry agent to work. These requirements include the following:

- **64-bit Linux** or other Linux operating system (recommended RHEL 8 or later, Ubuntu 22.04 or later, or equivalent)
 - Agents run on their own JDK that is compiled for Linux/x86-64. If necessary (for example, when running on AWS Graviton or another ARM-based CPU), it is possible to run an agent on a separate JDK by modifying the value of `javaHome` in `service/bin/launcher-static.yml`.

i We generally do not recommend running agents on a separate JDK, and support for this may not be available in the future.

- **4 CPU** cores
- **16 GB** RAM
- **500GB** free disk space mounted at `/opt` (preferably SSD)

T [API Reference ↗](#) limits are as follows:

- **Core file size:** Hard and soft limit of 0

Send feedback

- **Open files:** Hard and soft limit of 262144
- **Running processes:** Hard and soft limit of 65536
- **Stack size:** Hard and soft limit of 32768 (KB)
- **Max locked memory:** Hard and soft limit of "unlimited"



A server to provision a host

You will need a Linux server in your network to create an agent

Minimum hardware requirements

Cores	4
Memory (RAM)	16 GB
Storage	500 GB
OS	64-bit Linux

Configure agent network access

An agent only makes outbound connections to Foundry. To successfully establish network access, you must allow the following:

- [Egress on the agent host](#)
- [Ingress in Foundry](#)

After allowing egress and ingress, validate that your host can communicate with the Foundry Virtual Private Cloud (VPC), by executing the following command on the agent host:

```
1 curl -s https://<your domain name>/magritte-coordinator/api/ping > /dev/null
```

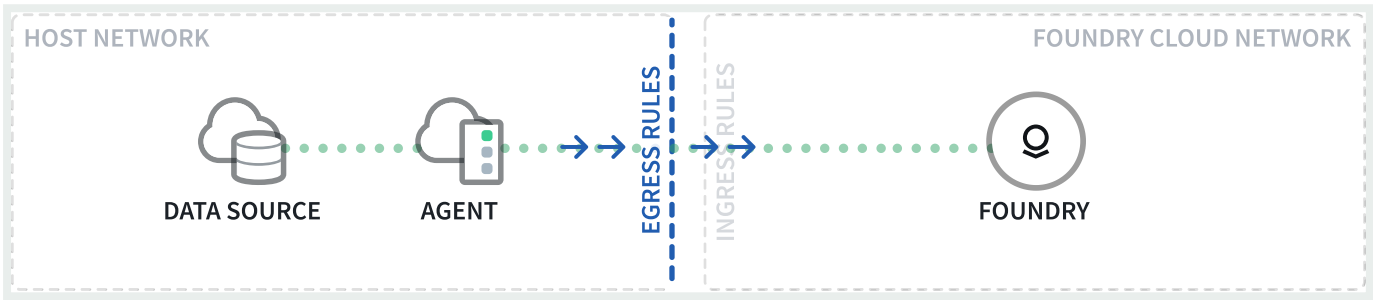
If everything is working as expected, you should see `pass` as an output; `fail` indicates an incomplete test of connectivity to the Foundry VPC.

[API Reference](#)
[↗](#)
[Agent host](#)

Send feedback

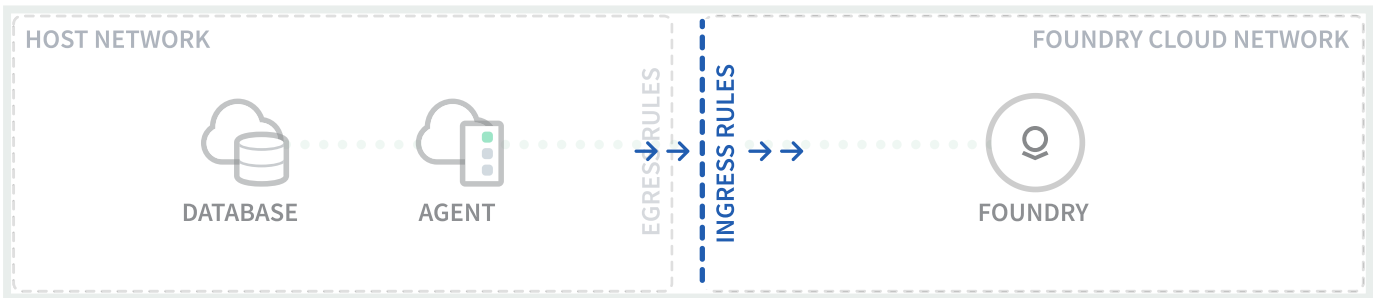
The agent requires network egress to reach the Foundry VPC, which is accessed through the Internet. If your network does not allow egress by default, this may require a specific configuration to allow the outbound connection from your agent (and/or its host) to your Foundry instance, such as opening a firewall or [configuring a proxy](#) for egress.

You can copy Foundry's domain name and port from the **Server Setup** tab in the agent setup workflow in Data Connection to appropriately configure egress network access.



Ingress in Foundry

Foundry must allow inbound traffic from your server's IP. You can manage ingress rules from the [Network ingress](#) page in Control Panel. Your Foundry domain will not be accessible from outside of your approved ingress rules.



Secure an agent host with firewall

We strongly recommend configuring a firewall on the agent host to monitor and restrict network traffic to only destinations that are strictly necessary. Be sure to still allow the agent host to talk to Foundry. The available firewall and monitoring options depend on the operating system you use to run your agent, as well as your organization's security best

p [API Reference ↗](#)

Set up automatic restarts

[Send feedback](#)

If you do not have automatic restarts set up, you will have outages whenever the agent crashes or the agent host restarts.

To set up automatic restarts for an agent manager if it crashes, run the command `${AGENT_MANAGER_DIR}/service/bin/auto_restart.sh` from the agent manager's service directory on the VM or machine terminal as a user with permission to create cron jobs.

If you need to halt the automatic restarts (when upgrading the agent manager, for example), you can do so by running `${AGENT_MANAGER_DIR}/service/bin/auto_restart.sh clear`.

Save agent resource in a Project

Next, you must give your new agent a name and choose a Project in which to save it. In Foundry, an agent is considered a [resource](#) that is saved into a Project to allow for highly configurable permissions.

We recommend creating a new Project in which to store your agent.

Permissions in Foundry are an extensive topic. If you want to learn more, you can refer to these resources:

- [Recommendations for managing agent permissions](#)
- [Guidance on how permissions should be set up end-to-end for data pipeline](#)
- [A full reference of how agent permissions interact with Projects and roles](#)

Download and install the agent

Once you have your hardware provisioned for your agent, the next step is to download the agent software from Foundry and install it on the host. Follow the steps outlined in the in-platform guide on your host to download the package, extract it, and start the agent.

If you need to configure a proxy, more details are available in the [proxy configuration documentation](#).

API Reference ↗

After the agent has started successfully, follow the steps to [configure automatic upgrades](#) to ensure that your agent remains updated.

Send feedback

Next steps

Now that you have created, installed, and started your agent, navigate to the agent page in Data Connection where you can [configure and monitor the agent permissions, health, and connectivity](#).

After your agent is set up, you can move on to [setting up a source](#) to connect your agent with your organization's data sources.

PREVIOUS
← [FAQ](#)

NEXT
[Agent configuration reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Data connectivity & integration / Data Connection / Agents / Agent configuration reference

Agent configuration reference

The following sections describe the various configurations available when creating, monitoring, and managing agents within your network. The following workflows will be explained:

- [Agent permissions](#)
- [Proxy configuration](#)
 - [Agent Manager proxy configuration](#)
 - [Bootstrapper proxy configuration](#)
 - [Source proxy configuration](#)
- [Agent metrics and health monitoring](#)
 - [Metrics](#)
 - ["Time until next expiration" metrics](#)
 - [Health monitoring](#)
- [Automatic upgrade windows](#)
 - [Upgrade windows](#)
 - [Stagger upgrades](#)
 - [Schedule dataset syncs](#)
- [Reinstall an agent](#)
- [Move an agent to a new directory](#)
- [Move an agent to a new host](#)
- [Install on a Windows host](#)
- [Backup and caches](#)
- [Configure host overrides](#)

API Reference ↗

Learn more about [agents](#) and how to [set them up](#) in the Data Connection application



Send feedback

Agent permissions

 Permissions may be managed differently on enrollments provisioned before May 27, 2023. For questions about agent management for these enrollments, contact Palantir Support.

To start managing agent creation in Control Panel, users with **Organization administrator** permissions must be granted an additional workflow called **Change agent creation authentication method**. This workflow permission allows administrators to opt into strict enforcement of role-based agent creation management. This enforcement is done automatically for enrollments provisioned after the above date.

Permission to create agents is administered in [Control Panel](#). To create an agent, you must have either the **Organization administrator** or **Data flows administrator** role assigned to you. It is also possible to create a custom role and assign the **Create agent** workflow to that role. Organization roles are managed on the **Organization permissions** page in Control Panel.

[Learn more about managing roles in Control Panel.](#)

In addition to the Organization-level role, you must be an **Editor** or **Owner** of the Project in which you want to save a newly-created agent.

 After creation, project-based roles are used to control who may view, modify, and delete the agent or assign the agent as the [worker](#) for a specific source. This means that agent creators should ensure that the project where the agent is stored is configured with the correct roles.

Certificates

An **API Reference**  with both Foundry and your internal network. This means that agents need to have the correct certificates in their truststores for these connections to be established.

[Send feedback](#)

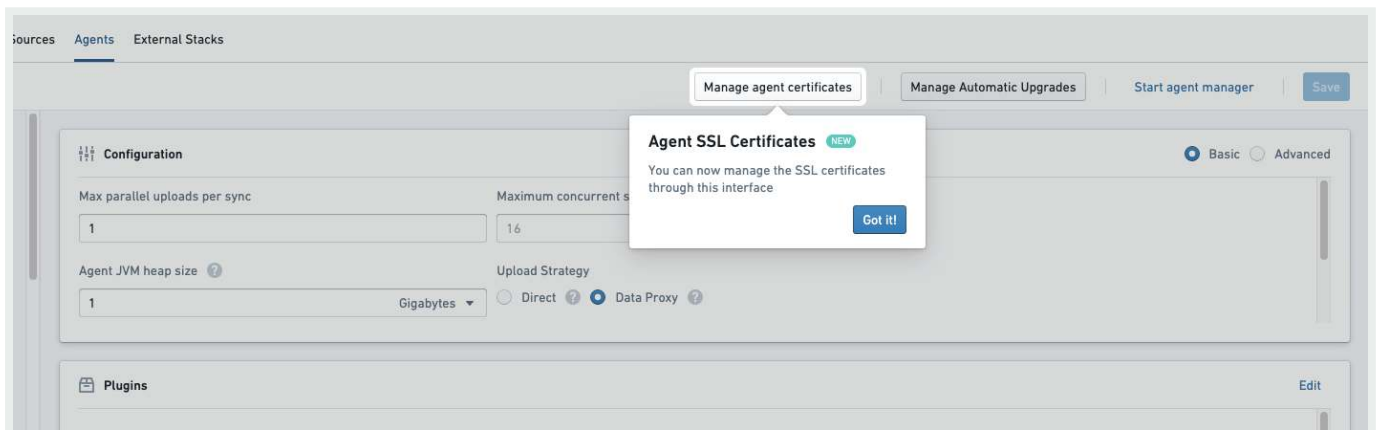
There are two situations that may require additional certificates to be configured on an agent:

- Certificates to allow agent communication with Foundry.
- Certificates to allow agent communication with your systems (agent worker only).

In both cases, certificates may be added from the agent management page in the Data Connection application.

Add certificates to an agent

To add a certificate to an agent, navigate to the agent details page for your agent in Data Connection, then select **Manage agent certificates**.



From here, choose the certificate to be added, along with an alias if desired. Then, add the contents of the certificate. The certificate should be added as a string similar to the below example, including newlines but without a trailing newline character. The certificate shown below is the public certificate for `https://palantir.com`.

Note that you cannot enter a certificate chain; you must enter each certificate separately.

-----BEGIN CERTIFICATE-----

MIIGXjCCBUaqAwIBAqIQASByQ6qv8Z6X7wEqsyBb1DANBqkqhkiG9w0BAQsFADBY

MQswCQYDVQQGEwJCRTEZMBcGA1UEChMQR2xvYmFsU2lnbiBud11zYTEuMCwGA1UE

API Reference = [biBBdGxhcyBSMyBEViBUTFMgQ0EgMjAyNCBRMjAeFw0yNDA2](#)

TA3MTkxNjUwMTRaMBkxFzAVBqNVBAMMDioucGFsYW50aXIu

Y29tMIIBIjANBqkqhkiG9w0BAQEFAAOCAQ8AMIIBCqKCAQEAvBa1UG3JFYaSe

enRnEwdeGUgtal9isJfB1++LxcPwo/DP2dncK+ur7URID0TVW0qu+4vXE2mmC9jz

Send feedback

Kx0o/URrMoz70i6qF6/0yq6Cu0HjZINiAN0ovNBBEPNGbSVD3Xq/eWgI7PNQ8hfI
9BJ/3WVA17oSG3zEXiWP3+CiL3Wm1Gn38o0t4URBMA0hgLqy0oU3ooqYIK8Fz2K/
0xAJvq45z2lonMZFFzj5th05dBBch26mNAac04MvI9mhUrMZtYvGBRZoXrph4EmF
TJDo2UTYiST0Tq6ibNW+NTuv66DrqFvz0pZybNuZsS6VrisYQ4huPN9jVz7RNFhJ
aeJvbQIDAQABo4IDYTCCA10wGQYDVR0RBBIwEII0Ki5wYWxhbnRpci5jb20wDgYD
VR0PAQH/BAQDAgWgMB0GA1UdJQQWMBQGCCsGAQUFBwMBBggrBgEFBQcDAjAdBgNV
HQ4EFgQUuAbgKrz0fIAXR1I/89IpUMN57AgwVwYDVR0gBFAwTjAIBgZngQwBAGew
QgYKKwYBBAGgMgoBAZA0MDIGCCsGAQUFBwIBFiZodHRwczovL3d3dy5nbG9iYWxz
aWduLmNvbS9yZXBvc2l0b3J5LzAMBgNVHRMBAf8EAjAAMIGeBggrBgEFBQcBAQSB
kTCBj jBABggrBgEFBQcwAYY0aHR0cDovL29jc3AuZ2xvYmFsc2lnbi5jb20vY2Ev
Z3NhdGxhc3IzZH0bHNjYTIwMjRzMjBKBggrBgEFBQcwAoY+aHR0cDovL3NlY3Vy
ZS5nbG9iYWxzZWduLmNvbS9jYWNlcnQvZ3NhdGxhc3IzZH0bHNjYTIwMjRzMjBj
cnQwHwYDVR0jBBgwFoAUrw0C0MMbnlj47zdiLecDXZ5BSooW SAYDVR0fBEEwPzA9
oDug0YY3aHR0cDovL2Nybc5nbG9iYWxzZWduLmNvbS9jYS9nc2F0bGFzcjNkdnRs
c2NhMjAAYNHEyLmNybDCCAX0GCisGAQOB1nkCBAIEggFtBIIBaQFnAHUA5tIxY0B3
jMEQQQbXcbn0wdJA9paEhvu6hzId/R43j lAAAAGQJxs/FAAABAMARjBEAiATvomb
hMrUty8Vj703fTBSzY5qDxEMI473IigiGIiXugIgbj4/j24j loGdVoedM3jb6DFw
yAXkuZD3SMHBLsEvP9gAdgDd3Mo0ldfhFgXn1TL6x5/4PRxQ39sA0hQSDgosrLvI
KgAAAZAnGz7qAAAEAwBHMEUCIAHXbm9F2rwyxD36aHoGZRrNdTgg9UDRy5UtHK6D
OrmKAiEajfomH4CGUrKBbwD8pzt9BbC6u6gPPveYiURxFIq//RUAdgB9WR4S4Xgq
exxhZ3xe/fjQh1wUoE6VnrkDL9k0jC55uAAAAZANg0C6AAAEAwBHMEUCIQDD0g9s
KZqzbCu0mNBQMRav6/2HkuLjZSGMxjq/F0R1/wIgKMHBSSNgeVED+LpTcIBYgp1q
SXgbwSizE60D+1Ewol0wDQYJKoZIhvcNAQELBQADggEBAAr/tnc9dtTfwrzcP70k
1+tLKmFRss4/1KQgLY8Tyy45Pag53ikn2n3tSPG10pRTSmfPhPs9/UQRtMf7f2Gk
0bSXDVPpArtFBFDfZug+j22gVSYQr6zgFJu4Y9QD1GGtICqkmTScubfnjwdfTfTv6
5oNY4LbVGp5yctAd800FUXspy+oVGsvv61a1pF0+s/NXleSrqDGL1oWcFW5Uj8GH
jnTM+Lt/HupqZ/ThVSkjM0ug+hB875Yf8mWvadKYBX0Ga2s51cp8CI49FRswziY6
3oXPKfHHQybpIKhGosuSyzY8pL8UoFHNP8gicAV80Vj6Mw+L8gWaAkCR6YnzQIyJ
9lc=

-----END CERTIFICATE-----

 After modifying certificates, the agent must be restarted before changes will take effect. If certificates are added before the initial setup, they will be included in the download link after being added in Data Connection.

API Reference 

For client certificates, refer to our [private key setup](#) documentation.

Send feedback

Certificates to allow agent communication with Foundry

Certificates for connecting to your Foundry instance are included in the default agent install bundle. Generally, this means that no additional certificates will be required for your agent host to communicate with Foundry.

If traffic from your agent to Foundry is intermediated with an explicit or transparent proxy, certificates for the proxy may need to be added to the agent as part of initial setup. Since missing certificates in this scenario will prevent the agent from being managed by Foundry, these certificates must be added to the agent before downloading the initial install bundle.

If there are certificate-related errors on agent startup that prevent the agent from communicating with Foundry, these will appear in a logfile called

```
<bootvisor_directory>/var/service.log .
```

If you encounter an error when attempting to set up your agent for the first time or know in advance that additional certificates will be required, you should:

1. Skip through the agent setup page without attempting to download and install on your agent host.
2. Add necessary certificates from the agent management page as described in the [section above](#), then download and follow the standard installation instructions.

If your additional certificate is for an explicit proxy, [additional configuration may be required](#).

Once an agent successfully connects to Foundry, these certificates should not need to be changed unless your network configuration between the agent host and Foundry changes.

Certificates to allow agent communication with your systems

When used as an [agent worker](#), additional certificates may be required if the agent processes need to communicate with systems using secure protocols like HTTPS, or TLS/SSL-protected protocols. New certificates may need to be added for each new source connection and these certificates should be updated if they expire or are rotated.

API Reference ↗

If required certificates are missing, errors like the following will appear when you use a source capability such as exploration:

Send feedback


```
Wrapped by: javax.net.ssl.SSLHandshakeException: sun.security.validator.ValidatorException: PKIX path building failed: sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target
```

Agents in TLS-inspected environments

If you perform TLS inspection on traffic on the host where your agent is installed, you will need to manually import your root CA to the Agent Manager's truststore before the Agent Manager can connect to Foundry for the first time.

To do this, follow the instructions below:

1. Obtain the public certificate for the root CA used to re-sign inspected traffic. Your network team should be able to provide this. Alternatively, you can use OpenSSL on the agent's host to inspect the certificate chain and find the root certificate: `openssl s_client -showcerts -connect FOUNDRY_URL:443`.
2. Use SSH to access the agent's host and place the certificate somewhere accessible to the user that runs the Agent Manager. One such location may be `<agent-manager-install-directory>/var/security`.
3. Navigate to the Agent Manager's Java installation: `cd <agent-manager-install-directory>/jdk/amazon-corretto*`.
4. Import the certificate to the Agent Manager's truststore: `./bin/keytool -import -alias <CERT_NAME> -file /path/to/certificate.pem -keystore ../../var/security/truststore.jks` (when prompted, the password is `truststore`).
5. Start the Agent Manager. You should see `Connected to Agent Manager, but no connection from agent` on the agent detail page in Data Connection, confirming that the Agent Manager has successfully connected to Foundry.
6. To allow the agent to connect to Foundry, follow the [instructions above](#) to add the same certificate to the agent.

Proxy configuration

[API Reference ↗](#)

When you start your agent for the first time, or when connecting to a remote source, you may need to configure a **proxy** depending on your organization's network.

[Send feedback](#)

proxy may be used to manage communication from the agent to Foundry, or it may be needed to reach data sources within your network.

For your agent to use a proxy, you will need to configure the proxy in both the **Agent** and **Bootstrapper** configurations found in the **Advanced** tab of the **Manage Configuration** window in the agent configuration page within Data Connection. Make sure to provide the hostname *without* `http://` at the beginning.

Agent Manager proxy configuration

The Agent Manager proxy configuration must be added to the `<agent-manager-install-directory>/var/conf/runtime.yml` file on the host where the agent was installed. Below is an example Agent Manager configuration snippet with a `proxy-configuration` block:

```
1 service-discovery:
2   services:
3     magritte-coordinator:
4     ...
5     proxy-configuration:
6       hostAndPort: proxy-host.com:3128
7       credentials: # these are optional
8         username: USERNAME
9         password: PASSWORD
```

This proxy will be used by the Agent Manager to connect back to Foundry's `magritte-coordinator`. It will not be used for connections from the agent to sources.

Bootstrapper proxy configuration

Once you have configured the Agent Manager proxy you can then configure the bootstrapper proxy. To do this, navigate to the agent configuration page within Data Connection, toggle the **Advanced** option in the **Manage Configuration** section, and finally select the **Bootstrapper** tab. Below is an example bootstrapper configuration snippet with a `API Reference` block:

Send feedback

```

1 coordinator:
2   proxyConfiguration:
3     host: HOST
4     port: PORT
5     credentials: # these are optional
6       username: USERNAME
7       password: PASSWORD

```

Once you have updated the configuration, you must save your changes and restart the agent for them to take effect.

This proxy will be used by the Bootstrapper for connecting back to Foundry's `magritte-coordinator`. It will not be used for connections from the agent to sources.

Source proxy configuration

For connecting from an agent to a data source through a proxy, configure the agent's JVM-level proxy on the Bootstrapper from the **Bootstrapper** tab in the **Advanced** section of the **Manage Configuration** page.

Use these JVM flags:

```

1 agent:
2   jvmArguments: >-
3     -Dhttp.proxyHost=<PROXY URL> -Dhttp.proxyPort=<PROXY PORT>
4     -Dhttps.proxyHost=<PROXY URL> -Dhttps.proxyPort=<PROXY PORT>

```

If you do not want to use the configured proxy for specific hosts, add the additional JVM flag `http.nonProxyHosts`. A full proxy configuration may look like the following:

```

1 agent:
2   jvmArguments: >-
3     -Dhttp.proxyHost=proxyhost.com -Dhttp.proxyPort=8000
4     -Dhttps.proxyHost=proxyhost.com -Dhttps.proxyPort=8000
5     -Dhttp.nonProxyHosts=host1.com|*.host2.com

```

API Reference ↗

Send feedback

```
5 -Dhttps.proxyHost=proxyhost.com -Dhttps.proxyPort=8001
6 ...
```

Note that quotes should not be used to encapsulate any of these configured values.

 This configuration affects all outbound network requests from the agent. We recommend using source-specific proxy configuration when available.

Agent metrics and health monitoring

Once you set up an [agent](#) in Data Connection, you can view metrics and monitor health to maintain performance.

Metrics

Navigate to the agent page in Data Connection, then select the **Metrics** tab. The metrics available for your agent include, but may not be limited to, the following:

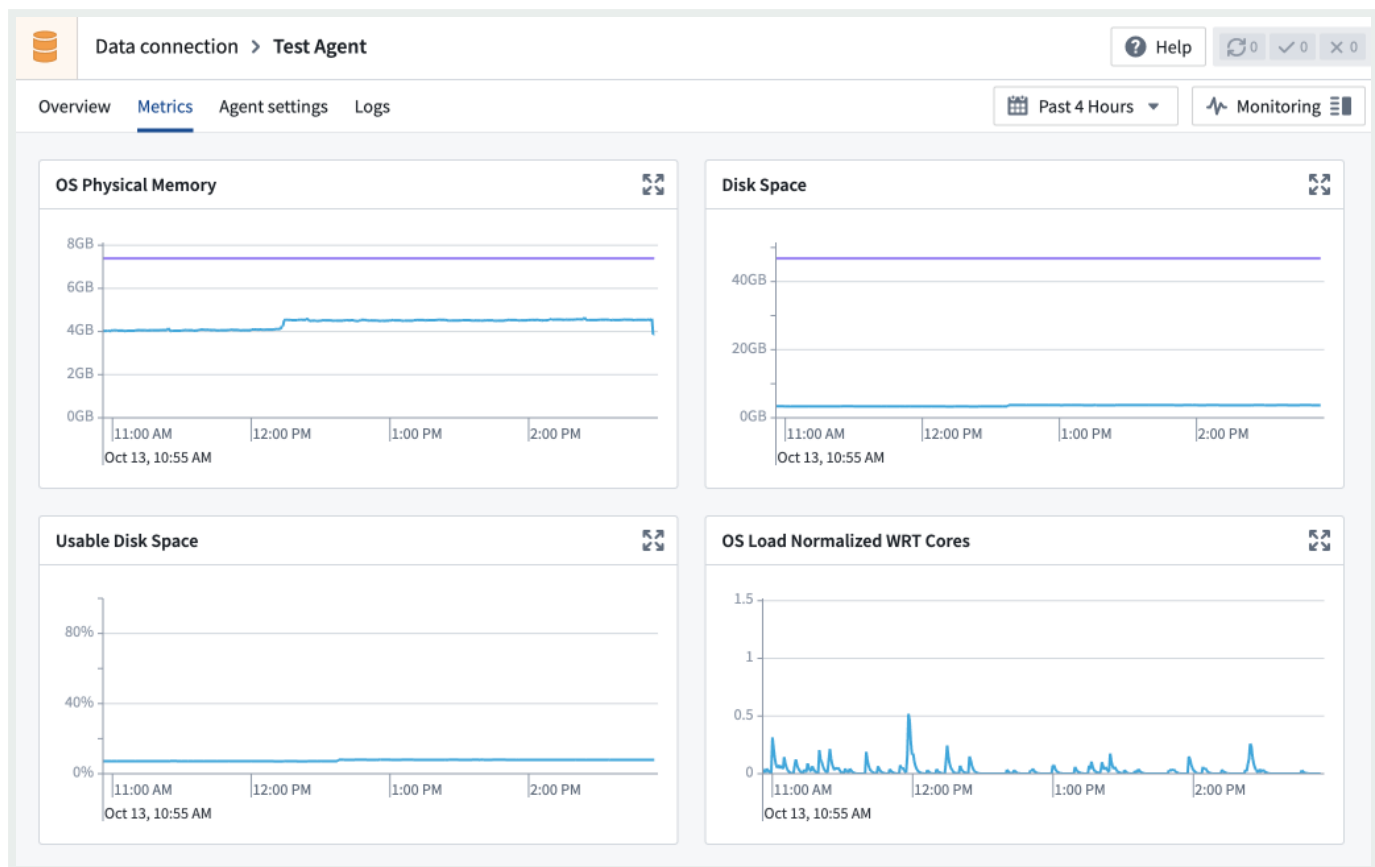
- OS physical memory
- Disk space
- Usable disk space
- OS load normalized WRT cores
- CPU utilization
- Time until next expiration in agent keystore
- Time until next expiration in agent truststore
- Agent heap memory
- Percentage of heap used
- Agent uptime
- Agent threads
- Syncs/tasks upload since last agent restart

API Reference ↗

- Agent syncs/tasks queued
- Syncs/tasks duration

Send feedback

- Agent last heartbeat time
- Agent Manager version stale time
- Agent version stale time



Hover over the metric cards for timestamped details, and select the top right corner of the card to expand a detailed view.

"Time until next expiration" metrics

The **Time until next expiration in agent keystore** and **Time until next expiration in agent truststore** metrics refer to the time until the earliest certificate expiration in the agent keystore and truststore, respectively. For example, if the agent's keystore has two certificates, one that expires in a week and one that expires in a month, the number would be `1w` as that is the closest expiration date.

The agent keystore and truststore include certificates added by users as well as those automatically added by the Agent Manager. Agent Manager certificates are automatically

u [API Reference ↗](#)

If the certificate has already expired, the metric will show `000ms`. If the metric is not stored for the agent, the graph will be empty.

[Send feedback](#)

"Version stale time" metrics

The **Agent Manager version stale time** and **Agent version stale time** metrics refer to how out of date the agent and agent manager are relative to what is available on your environment.

The version out of date metrics are calculated as the number of days between when the agent or agent manager was last updated and the release date of the latest version available. The below example illustrates how these metrics and associated monitors are expected to behave:

Day	Latest released version	Agent / Agent Manager current version	Version stale time metric value	Notes
0	v1.0	v1.0	0	Agent updates to current latest version.
1	v1.0	v1.0	0	
2	v0.1 -> v2.0	v1.0	0 -> 2	Metric jumps to 2 when a new version is released by Palantir.
3	v2.0	v1.0	3	
4	v2.0	v1.0 -> v2.0	4 -> 0	Metric goes back to 0 after a successful update during the maintenance window .

In this example, the agent version stale time metric on the first two days is 0. The metric then jumps to 2 when the new version becomes available, and will then continue to increase until the next agent maintenance window, and finally drop back to 0 once an update completes successfully during a maintenance window.

 If a monitor is set to send alerts when a software version becomes too old, and

version releases are spaced more than the allowed number of days

monitor will start alerting as soon as the new version is available even if

there has been no opportunity for a particular agent to update.

Send feedback

resolve automatically after the next successful update during a maintenance window.

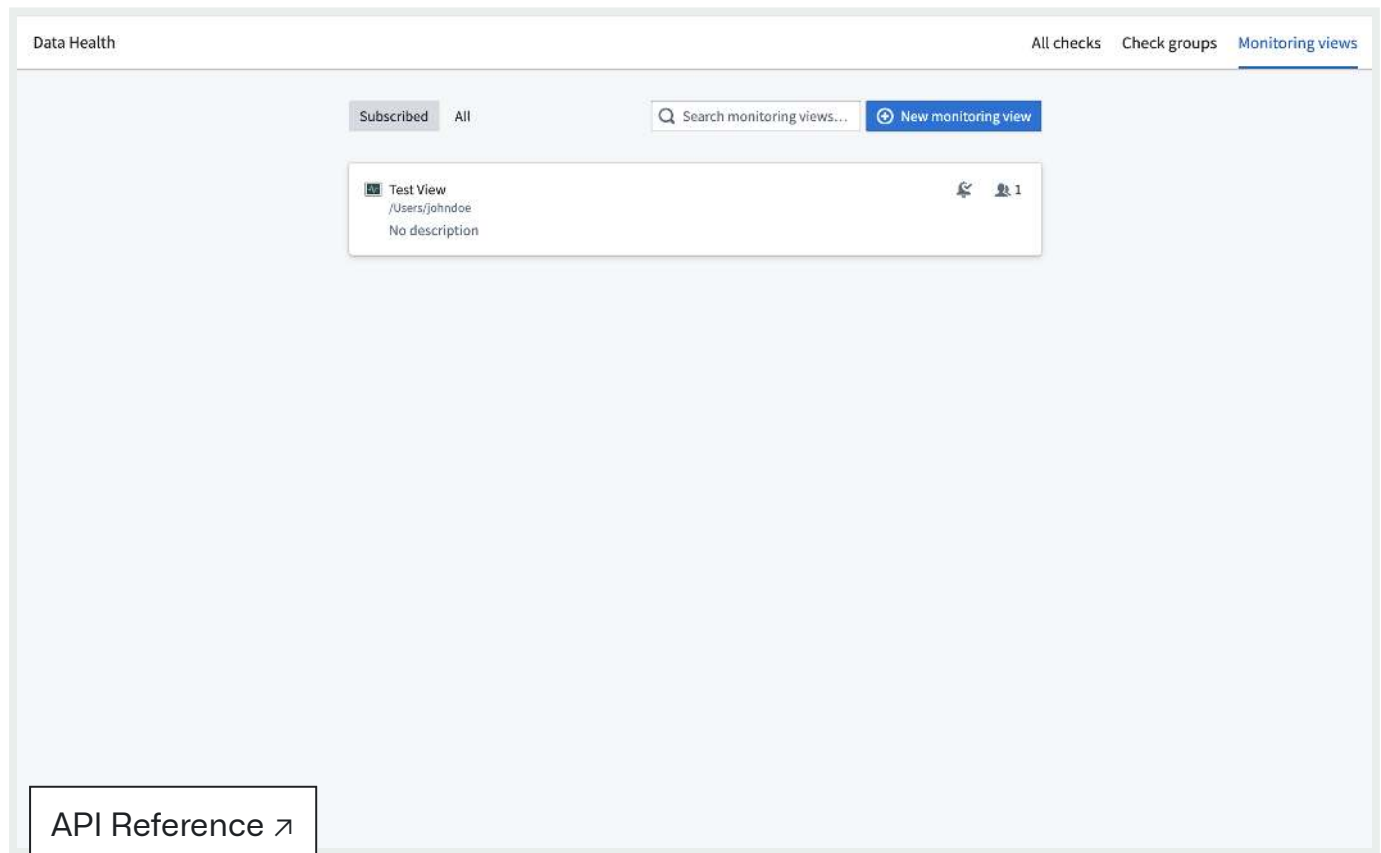
In the example above, if the monitor is set to alert for stale time days `>2`, an alert will be issued on day 3 even though there has been no opportunity for the agent to upgrade to the latest version. The alert will automatically resolve after the successful update on day 4.

Health monitoring

Health monitors allow you to configure alerts of varying severities (high, medium, or low) for any metric when certain conditions or thresholds are met.

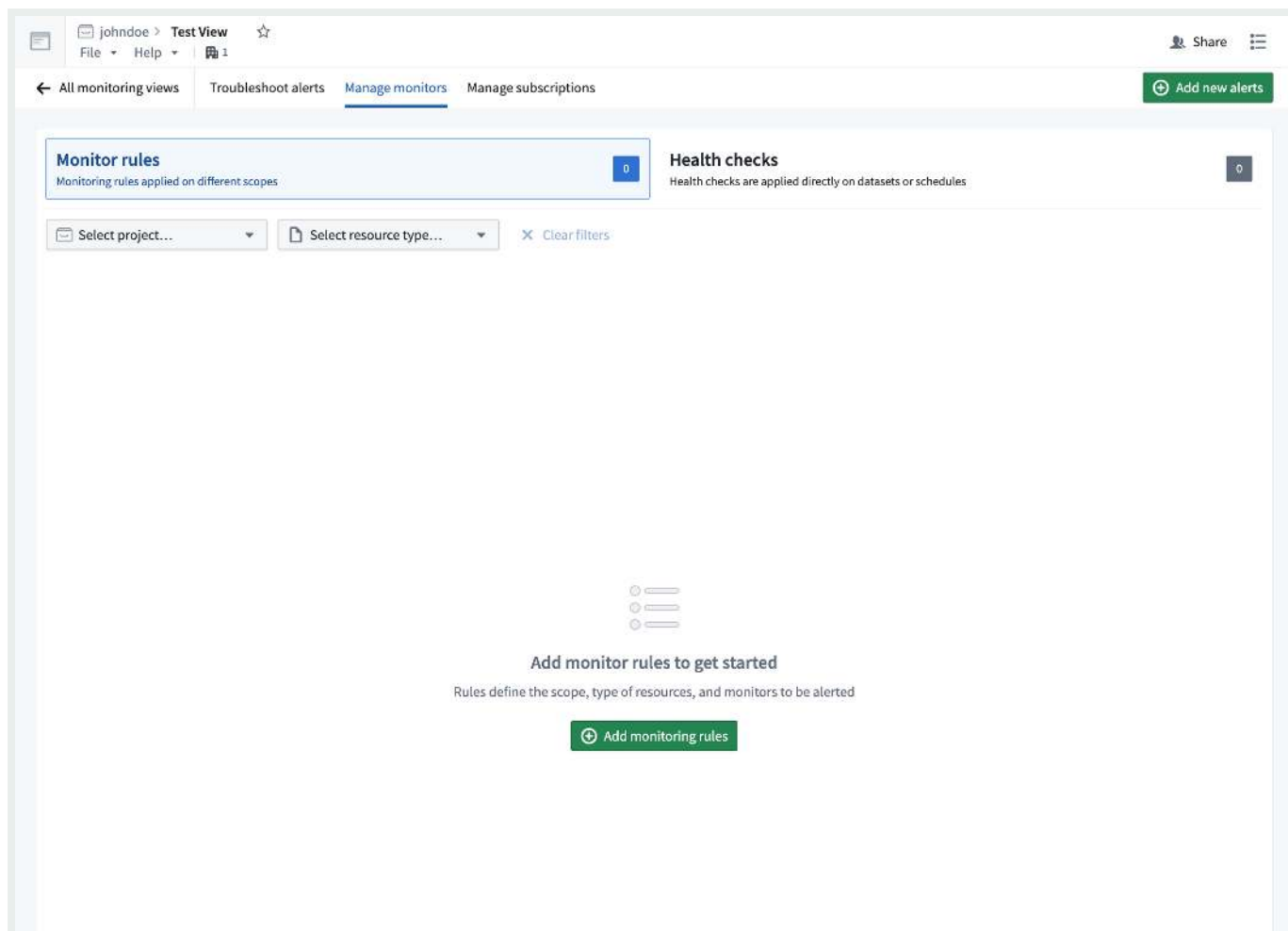
You can monitor an agent's health by [creating a monitoring view](#) in the **Data Health** application. A monitoring view is a collection of monitoring rules that are of particular interest to a subscribed group of users.

You can view existing monitoring views by selecting the **Monitoring views** tab.



After selecting a specific monitoring view, you can configure the health of the agent by selecting **Manage monitors**. From this page, you can create a new monitoring rule.

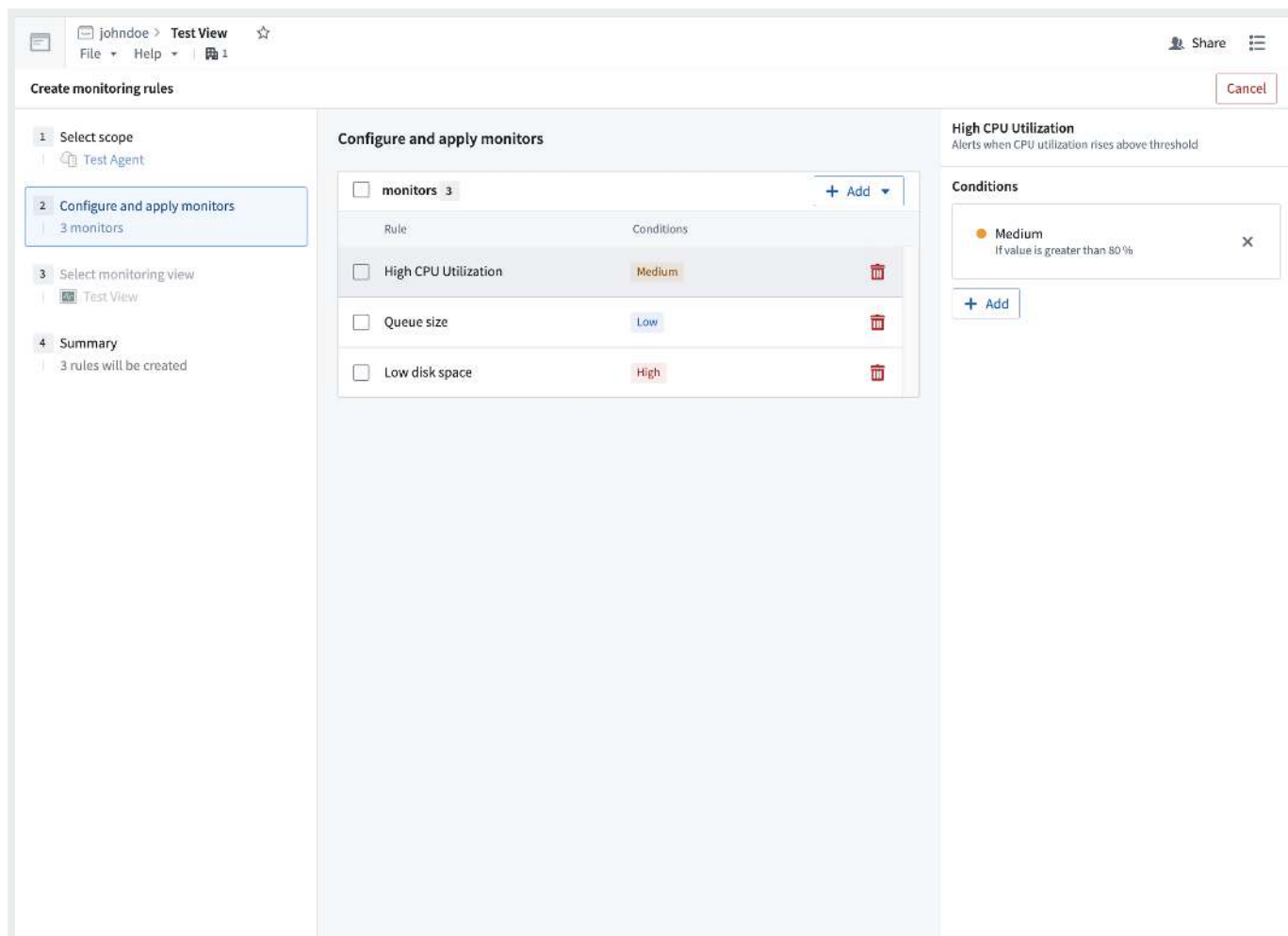
Send feedback



In the **Create monitoring rules** page, you can configure specific rules and alerts of varying severities.

[API Reference ↗](#)

[Send feedback](#)



Learn more about [tracking data health with monitors](#) and [integrating monitors with PagerDuty](#).

Automatic upgrade windows

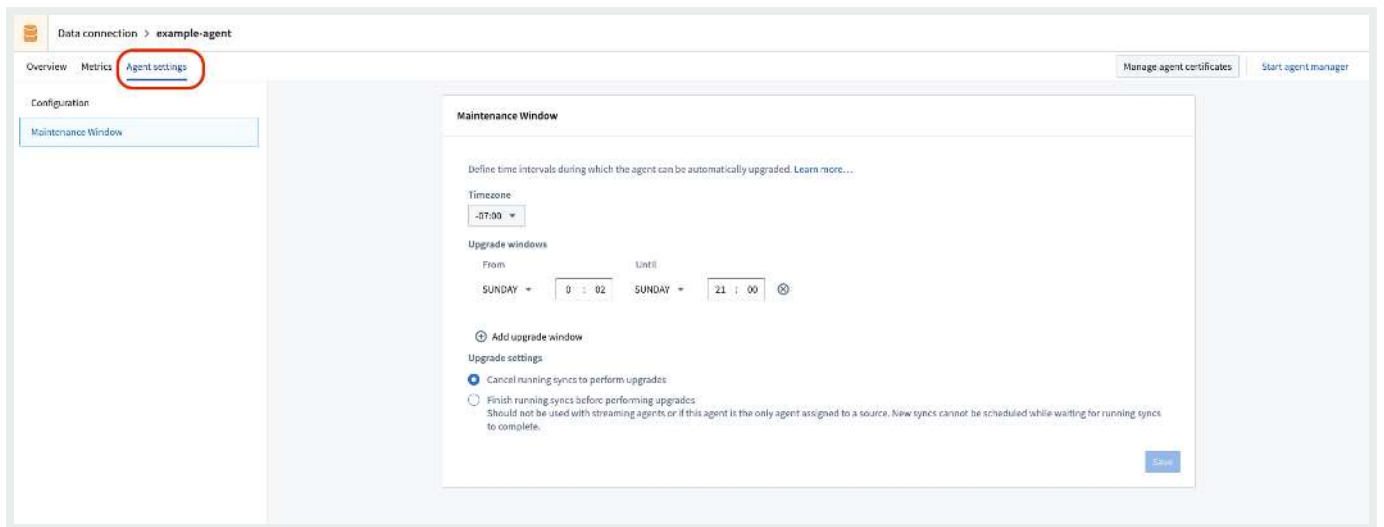
The Data Connection agent service is regularly updated with security, stability, and performance improvements. The best way to ensure that agents receive these important improvements in a timely manner is to configure upgrade windows for each agent in use. The sections below describe what happens during an upgrade window and provide best practice guidelines.

Upgrade windows

An [API Reference ↗](#) upgrade window is a set of time intervals during which it is considered safe for the agent to be temporarily offline. These time intervals recur weekly and

[Send feedback](#)

Maintenance Window page in the **Agent settings** tab for the given agent in the Data Connection application.



The Data Connection coordinator monitors agents and their respective upgrade windows; they will perform automatic upgrades of agents during these upgrade windows when new versions are available.

As part of the upgrade, the agent will be restarted. This will terminate any running jobs and briefly prevent new jobs from running on the agent.

Agent upgrade windows should be at least 60 minutes long. However, the actual upgrade should be relatively short; it should take approximately the same amount of time as a restart of the agent.

Stagger upgrades

To ensure minimal impact to data pipelines, we recommend assigning at least two agents to all Data Connection sources and to stagger upgrade windows for any given set of sources running on those agents. For example, one agent could have an upgrade window defined on Sundays, while the other has an upgrade window scheduled for Wednesdays. This ensures that during any given agent's upgrade window, jobs that are interrupted can be retried on the partner agent, and new jobs can continue to queue and run until the agent being upgraded is fully back online.

[API Reference ↗](#)

Schedule dataset syncs

[Send feedback](#)

When staggered upgrade windows cannot be used, it is important to schedule upgrade windows during periods of low (ideally zero) activity. In this case, dataset syncs should be scheduled such that they finish before the start of the upgrade window or start several minutes after the window (to account for the restart occurring towards the end of the window).

Reinstall an agent

If your agent has not upgraded or has been unhealthy for too long, the easiest solution is to reinstall the agent on the host. Reinstalling an agent is a safe operation and a similar process to initial installation.

The reinstallation process can be started by navigating to the agent overview page and selecting **Reinstall the agent**.

Follow the instructions in the reinstallation wizard as well as the additional steps below to ensure reinstallation is successful:

1. Ensure that the agent is not running before reinstalling. To do so, SSH into the agent host using your computer's command line tool, navigate to the agent folder (`magritte-bootvisor-*`) and stop it by running the following command:

```
1 ./service/bin/init.sh stop
2 ./service/bin/auto_restart.sh clear
```

2. If you are reinstalling the agent in its existing directory, create a backup of the old agent.

```
1 mv magritte-bootvisor-* magritte-bootvisor-*-old
```

3. Copy encryption keys from the old agent to the new agent folder.

API Reference ↗

```
cp -r $OLD_BOOTVISOR_DIR/var/data/source-encryption-key* $NEW_BOOTVISOR_DIR
```

4. Confirm that everything works as expected, and delete the backup `magritte-bootvisor-*-old` to free up disk space.

Send feedback

```
1 rm -r magritte-bootvisor-*old
```



Move an agent to a new directory

Follow the steps below to move an agent to a new installation directory for the same machine.

1. In Data Connection, navigate to the **Syncs** page and ensure that no syncs are currently running.
 - Check if syncs are running by using the filter on the left of the page to view only those that currently have a **Running** status.
 - If syncs are running, either wait for them to complete or cancel them by selecting their status link (for example, **Running 14 minutes ago**), and then choosing **Cancel build** from the build page. Ensure that sync owners are appropriately notified if you need to cancel their syncs.
2. Navigate to the **Agents** page. Select the name of the agent you want to move.
3. In the Configuration panel, select **Advanced**. For each tab of the **Advanced** settings, change all references that use an absolute path. Look for anything that starts with **/** and modify these to what the new path will be.
 - Note: Kerberos settings require an absolute path.
4. Stop the agent in Foundry. To do so, select the dropdown arrow next to **Restart Agent** in the upper right corner of the screen, then select **Stop (Unsafe)**.
 - Note: The **Unsafe** label is meant to warn that stopping the agent will interrupt any running syncs, which is why we took the precautions detailed in Step 1.
5. On the terminal of your machine, SSH into the agent.
6. Change to the administrative user for the agent. Depending on your configurations, this could require entering **-- sudo -su palantir** or **-- sudo -su admin**.
7. In your terminal, navigate to the agent installation directory using **cd**.
8. Stop the agent by running **./service/bin/init.sh stop**.
9. If the agent was previously configured to autostart:
 1. For Linux, run **./service/bin/auto_restart.sh clear**.
10. Wait a few minutes and check that there are no Java processes belonging to the administrative user. If there are, stop them manually.

API Reference ↗

remove any scheduled tasks that were set up while following instructions on [installing on a Windows host](#).

Send feedback

11. Optionally, remove the contents of `./var/data/binaries`, `./var/data/cache`, and `./var/data/processes` folders to reduce the amount of data being transferred.
12. Move the agent to the new directory folder by entering `mv <source directory> <new installation directory>`.
13. Navigate to the new agent installation directory. Restart the agent by running `./service/bin/init.sh start`.
14. If the agent was previously configured to auto-restart, re-enable it by following the [auto-restart setup instructions](#).
15. Wait five minutes, then refresh the agent page in Foundry to ensure that the agent is connected and syncs are successful.

Move an agent to a new host

When moving an agent to a new host, make sure that the new host meets the operating system requirements and has the same operating system and architecture as the previous agent. The new host must also have the same firewall network configurations as the old host. For example, if the agent was previously running on a linux distribution running on an x86_64 architecture, the new host can have any distribution of linux, but must be running on the same architecture. The instructions below will not work to move an agent from Linux to Windows or to an ARM architecture. For these cases, get a new download link from the UI and proceed with a new install.

When reinstalling the agent using a new download link from the UI, existing source credentials will not be able to be decrypted and must be re-entered manually. To preserve these credentials, consider the following steps to migrate the source secrets from the old agent to the new one:

1. Navigate to the directory `<bootvisor dir>/var/data` on the old agent.
2. Copy the files `source-encryption-key` and `source-encryption-key.private` to the same directory (`<bootvisor dir>/var/data`) on the new agent.
 - If these files are lost or overwritten on the old agent before copying them, all source credentials will be lost, and they will need to be manually re-added to the new agent.

If [other files](#) in the file system, ensure they are also moved to the new agent to maintain proper functionality.

[API Reference ↗](#)

Follow the steps below to move an agent to a new host:

[Send feedback](#)

1. Follow steps 1 through 11, excluding step 3 in the instructions for [moving an agent to a new directory](#). This will stop the agent and prepare it to be transferred to a new host.
2. Use [rsync](#) or any other file transfer tool to copy the agent folder from the old host to a location on the new host.
3. Follow previously skipped step 3 from the instructions on [moving an agent to a new directory](#). For each file or folder referenced by an absolute path, copy that resource to a location on the new host and update the configuration to point to the new path.
4. Follow steps 13 through 16 from the [moving an agent to a new directory](#) guide.

Install on a Windows host



Linux hosts are the preferred option for setting up an agent. You should not use a Windows host unless there is no way to procure a Linux host or run Linux in a VM on your Windows host.

Some capabilities may not work on Windows agents, particularly when using a Windows agent as an [agent worker](#). Specifically, [table exports](#) are not supported on Windows agents and will fail to run.

Follow the steps in the [guide to setting up a agent](#), but select Windows as your operating system on the [download and configure agent](#) step. Once you install your agent, you will need to set up a Windows scheduled task that restarts the Agent Manager on crash or machine restart, as follows:

1. Open Windows Task Scheduler as an Administrator.
2. Create a new scheduled task.
3. Under the **General** tab, perform the following:
 1. Set **When running the task, use the following user account** to `SYSTEM`.
 2. Make sure **Run whether the user is logged in or not** is selected.
 3. Make sure **Do not store password...** is *NOT* selected.
 4. Update **Configure for** to the correct operating system for the Windows virtual machine.

5. Under the **Triggers** tab, create two triggers:

[API Reference](#) ↗

6. Under the **Actions** tab, create two triggers:

1. Set up one trigger **on an event** and choose **on startup**.

Send feedback

- Set **Delay task for** to **3 minutes** to ensure no conflicts with windows upgrade processes on startup.
 - Ensure the task is **Enabled**.
2. Set up another trigger on a schedule:
- Set the task to run daily.
 - Check the **Repeat task every** box, and choose **5 minutes** for a duration of **1 day**.
5. Under the **Actions** tab:
1. Add a new **Start a Program** action.
 2. Copy the whole path to the `magritte-bootvisor-win` batch file in the box for **Program/script**.
 - The full path will be similar to
`C:\example\path\to\folder\containing\magritte\agentmagritte-bootvisor-win-{version}\service\bin\magritte-bootvisor-win.bat`.
 3. Copy the whole path to the *enclosing folder* for the batch script into the box for **Start in (optional)**.
 - The full path will be similar to
`C:\example\path\to\folder\containing\magritte\agent\magritte-bootvisor-win-{version}`.
 - Though Windows states this is optional, this step is mandatory as the agent runs commands with relative paths.
 4. There are no arguments, so leave **Add arguments (optional)** blank.
6. Select **OK** to save your task.
7. Attempt to manually run the task by right-clicking on the task and choosing **Run**.
 - This task should enter a **Running** status and remain there indefinitely indicating that the agent manager is running.
8. You should see the agent start up successfully. This can take a minute or two to reflect in the interface in Foundry. If the agent does not start successfully, check the **History** tab of the scheduled task for errors. You can also verify the operation's success by using the Windows command prompt to change directory into the specific folder and then executing the associated .bat file.

Backups and caches

[API Reference ↗](#) for cache files as part of installation, version upgrade, version downgrade, and ongoing usage. This includes earlier versions of the agent and connector plugins. Security and stability fixes are delivered in the [latest](#)

Send feedback

agent has upgraded, the versions it replaced are retained on the host but are no longer loaded or run. Normally, backups and cached files may be safely ignored, and will be automatically deleted after 30 days. If they are using up a higher than desired amount of disk space on the agent host, it is safe to delete them. Perform any deletion outside of a [maintenance window](#).

Backup and cache files will be found in the following sub-directories within the agent installation directory:

Sub-Directory	Usage
/backups	Used to store backups of the agent configuration when performing upgrades.
/var/data/cache	Used to cache data during normal operation of the agent.
/var/data/maven	Local cache of the connector plugin and artifact versions the agent has downloaded during installation and upgrades, stored by version. Only the version currently assigned to the agent is loaded; superseded versions remain

Sub-Directory	Usage
	not run, and they can be removed.
/var/data/processes	Holds a working directory for each agent process, including the connector plugins that process used. Only the currently running process is in use. To keep only the most recent inactive directories (the 10 most recent by default), inactive directories are removed automatically once they age past the retention period (30 days by default) or when host disk space runs low. They can also be deleted manually.
/var/data/processes/<bootstrapper>/var/conf/agent/managed-plugins API Reference ↗	Connector plugins delivered by Foundry, with one .jar per Send feedback highest version

Sub-Directory	Usage
	present is the one in use; superseded versions are moved to the neighboring archive directory and can be removed.

To identify the currently running process directory under `/var/data/processes`, run `service/bin/init.sh status` from within each directory. The directory that reports as running is the one in use; the others are inactive and can be removed.

Configure host overrides

Agents can be configured with custom hostname-to-IP address mappings that override standard DNS resolution, similar to entries in `/etc/hosts` on Unix systems. These host overrides allow the agent to resolve specific hostnames to IP addresses that differ from standard DNS lookups.

How to configure host overrides for an agent

Follow the steps below to add host overrides for an agent directly from the Data Connection application:

1. Navigate to the **Agents** page in Data Connection and select the agent you want to configure.
2. Navigate to the **Agent settings** page and add a host override with the desired hostname and IP address mapping.
3. Save your changes and restart the agent for the configuration to take effect.

[API Reference ↗](#)

[Send feedback](#)

The screenshot shows the 'Manage Configuration' page for an agent. The left sidebar has 'Configuration' selected. The main area is titled 'Manage Configuration' and has tabs for 'Basic' (selected) and 'Advanced'. Under 'JVM heap size', 'Agent process' is set to 1 Gigabytes and 'Explorer process' is set to 1 Gigabytes, with a total of 2.00 GB. Under 'Job configuration', 'Max parallel uploads per sync' is 1 and 'Maximum concurrent jobs' is 16. 'Maximum uploaded file size (bytes)' is 1000000000. The 'Host overrides' section, marked as 'Beta', contains a table with one entry: 'internal-database.local' mapped to '10.0.0.1'. There is an 'Add host override' button below. At the bottom, there is a 'Restart agent after saving' checkbox and a 'Save and restart agent' button.

Host	IP
internal-database.local	10.0.0.1

As an example, if your data source is only accessible at `10.0.0.1` within your private network:

1. Configure a host override that maps an ad-hoc domain (such as `my-private-internal-database.local`) to `10.0.0.1`. The chosen domain name can be anything, although we recommend a descriptive name of the system hosted at that IP.
2. Use this ad-hoc domain instead of `10.0.0.1` in your source and egress policy configuration.

[API Reference ↗](#)

[Send feedback](#)

Data Connection > Private Database (PostgreSQL)

Overview **Connection settings** CDC syncs Edit syncs Explore source Logs Move to trash Explore lineage Share

Name and location

Connection details

Export configuration

Code import configuration

Output folder

Source Setup

Compute Foundry worker

Connection settings

Select host type:

☒ Hostname ☐ IPv4

Hostname: internal-database.local

Port: 5432

Database name: db

Network connectivity

Debug

internal-database.local (Port 5432) Agent proxy Approved Remove

Example policy

+ Add another policy Request

Agents used by this policy

Example Agent

Network connections from within Foundry are initiated using the domain name, while the agent resolves it to the correct private IP address at the network edge.

⚠ Host override configurations affect DNS resolution at the agent level and apply to all connections made by that agent. These mappings take precedence over standard DNS lookups.

When to use host overrides

You should configure host overrides for the following scenarios:

API Reference ↗

1. Source systems in private networks that can only be referenced by IP address:

Network requests initiated within Foundry must use non-private IP addresses to avoid infrastructure collisions. Therefore, when using an [agent proxy egress policy](#) to connect

Send feedback

to a source accessible only through private IP addresses without a resolvable domain name, use host overrides to resolve a placeholder domain to the correct private IPs within the private network.

2. **Local testing:** When testing connections to localhost or other local services during development.

Host overrides require agent version `9.2446.0` or newer. You can verify your agent version from the agent **Overview** page in Data Connection.

PREVIOUS

← Set up an agent

NEXT

Agent worker configuration reference →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Agents / Agent worker configuration reference

Agent worker configuration reference

⚠ Legacy

Agent worker is in the [legacy](#) phase of development, and no additional development is expected. Full support remains available. We recommend using [Foundry worker](#) connections when building a new source or updating existing workflows. See [Foundry worker vs. agent worker](#) for the operational reasons behind this recommendation.

This page describes the configuration options available when using an agent as a [worker](#) and assumes you are already familiar with [data connection agents](#) and their [generic configuration options](#).

To migrate an existing agent worker source to a Foundry worker, see [Switch source from agent worker to Foundry worker](#).

Set up an agent

You must [set up an agent](#) to use the agent worker. When configuring a source, you can select the **Agent worker** option and choose at least one available agent. Capabilities run over that ~~source connection~~ will be executed as jobs on the agent host.

API Reference ↗

Capability execution

Send feedback

When using an agent worker, capabilities are executed by running Java code on the agent host directly. These Java processes may pull data from your systems and push data up to Foundry (as with [batch syncs](#)), or pull data from Foundry and push to your systems (as with [exports](#)), which comes with [known limitations](#).

Networking

All networking for agent worker sources is configured on the agent host. Foundry-side egress policies do not apply: the agent host must have firewalls, DNS, and proxy settings configured to reach the source systems directly. See [Configure agent network access](#) for the full reference.

Memory allocation and usage

Agent memory is one of the key factors determining the performance of capabilities executed using an agent worker.

The primary settings available for agent memory are:

- **JVM heap:** Configured on the agent settings page, and indicates how much memory the agent should allocate on startup. The allocated memory is reserved from the OS and must not exceed the available memory on the host machine. The default JVM heap value is 1 GB.
- **Host memory:** Based on the specification of the machine where the agent is installed. We recommend at least **16 GB** of memory.

Actual memory usage observed on the host will vary based on the workload currently being executed by the agent worker, including other processes running on the same host.

When observing and monitoring memory usage for an agent used as an agent worker, there are two primary metrics:

OS physical memory usage: The actual total memory usage on the agent host, including [API Reference ↗](#); but also any other processes running on the same host. This may go beyond the allocated JVM heap size for the agent process, and up to the total physical memory available.

Send feedback

- **Agent memory usage:** The memory usage by the agent, which will always be smaller than the configured JVM heap size.

For information on monitoring memory usage on agents, review [agent metrics and health monitoring](#).

Load balancing

When using an agent worker, multiple agents may be assigned to a single source connection. An agent is assigned jobs to execute specific [capabilities](#) configured on assigned source connections. Jobs are executed on one of the available agents at the time when the job is started.

Jobs are assigned randomly to one available agent amongst the assigned agents.

Maximum concurrent syncs defaults to `16`, and is configurable under **Agent settings**. The **Maximum concurrent syncs** quota is enforced across all [capabilities](#) and all assigned sources, meaning that any run of any capability on any source uses up one unit of the available concurrent sync quota. This includes [streaming syncs](#), [change data capture syncs](#), [exports](#), [webhooks](#) and legacy data connection [tasks](#). If the assigned agent already has the maximum number of concurrent syncs currently running, then new jobs will be queued until enough running capabilities finish.

In general, we do not recommend using multiple agents as a way to load balance a larger workload than could be successfully run on a single agent. The primary intended use of multiple agents is to allow for agents being taken offline for maintenance. For optimal performance and reliability, we recommend that each agent in a multi-agent setup should be able to handle the full set of [capabilities](#) configured on the assigned source connection(s).

Direct vs. data proxy upload strategy

Agent workers support two options to specify how data from batch syncs should be uploaded to the platform:

[API Reference](#) ↗

Send feedback

Data proxy mode

In data proxy mode, data is uploaded using the public Foundry API using the data proxy service. This leverages the same API gateway that handles Foundry API calls for reading and writing datasets.

Agents configured to use data proxy mode will contain the following in the agent configuration YAML:

```
1 destinations:
2   foundry:
3     uploadStrategy:
4       type: data-proxy
```

Direct mode [Sunset]

i Direct mode is not available on new agents or on enrollments set up after June 2024. Data proxy mode is the default and only option supported for new agents. Existing agents using direct mode will continue to work as long as the host IP addresses where the agent is installed do not change.

In direct mode, data is uploaded directly to the underlying storage buckets in the Foundry data catalog. While providing performance improvements, this is only possible with custom network configuration by Palantir Support, and is not available on our latest cloud infrastructure.

Agents configured to use direct mode will contain the following in the agent configuration YAML:

```
1 destinations:
2   foundry:
3     uploadStrategy:
4       type: direct
```

API Reference ↗

Send feedback

Custom JDBC drivers

See the [JDBC \(custom\) connector documentation](#) for instructions on adding custom JDBC drivers to an agent.

Drivers must be signed by Palantir Support and added directly to the agent to work with the agent worker.

 Contact Palantir Support for assistance signing custom JDBC drivers.

Credentials

One unique aspect of the agent worker is that in-memory credential decryption happens outside of Foundry. When entering credentials in the Data Connection user interface, rather than Foundry encrypting credentials, they are encrypted with the public key of each agent assigned to the source in the browser before being sent to Foundry. The encrypted credentials are then stored in Foundry and can only be decrypted by the respective agent using its private key when needed.

Note that all data in Foundry, including credentials, is encrypted at rest and in transit. Data is only ever decrypted in-memory where access is required.

This means that the following caveats and restrictions apply to credential configuration when using the agent worker:

- If the set of agents assigned to the source changes, credentials must be re-entered in Data Connection.
- If an agent is reprovisioned using a fresh download link, credentials will not be automatically transferred. Encryption keys from the existing agent must be recovered or else all credentials must be re-entered in Data Connection.

More information on moving agents between directories and hosts is covered in the [agent c](#) [API Reference ↗](#) [ce documentation](#), including instructions for retaining encrypted credentials when moving an existing agent directory.

Send feedback

Certificates

Agents communicate with both Foundry and your internal network. This means that agents need to have the correct certificates in their truststores for these connections to be established.

[Review the documentation on adding additional certificates for connecting to specific source systems.](#)

Add a private key

If the system you are connecting to requires mutual TLS (mTLS), you must manually add a private key to the agent.

The default bootstrapper keystore and truststore are regenerated any time the agent is restarted, and any changes made to the default keystore will be overridden on restart. The below instructions explain how to override the default keystore to point at a custom keystore in a different location on the agent host, and how to modify this custom keystore to add your private key.

1. Copy the default bootstrapper keystore and store it in a separate location on the agent host. Run the following commands with the same username that is running the agent on the host. You may choose to name the folder `security` or according to your preferences.

```
1 mkdir /home/<username>/security
2 cp <bootvisor_root>/var/data/processes/<bootstrapper_dir>/var/conf/keyS
```

2. Import the keys from the customer-provided keystore into the copied agent keystore using the Java `keytool` command line tool. If this tool is not already installed, find it in the `bin` directory of the JDK that is bundled with the agent.

```
1 $ keytool -importkeystore -srckeystore <CUSTOM_KEYSTORE.jks> -destkeystore
2 keyStore CUSTOM_KEYSTORE.jks to keyStore.jks...
3 Enter destination keystore password: keystore
4 Enter source keystore password:
```

API Reference ↗

Send feedback

- You can verify that the key/keys were added to the copied keystore using the `keytool -list` command:

```
1 $ keytool -list -keystore /home/<username>/security/keyStore.jks
2 Enter keystore password:
3 Keystore type: jks
4 Keystore provider: SUN
5
6 Your keystore contains 2 entries
7
8 <CUSTOM_KEY>, 15-Dec-2022, PrivateKeyEntry,
9 Certificate fingerprint (SHA-256): A5:B5:2F:1B:39:D3:DA:47:8B:6E:6A
10 server, 15-Dec-2022, PrivateKeyEntry,
11 Certificate fingerprint (SHA-256): DB:82:66:E8:09:43:30:9D:EF:0A:41
```

3. Use the `keytool -keypasswd` command to update the imported key password. The agent keystore requires that both the key and keystore passwords match.

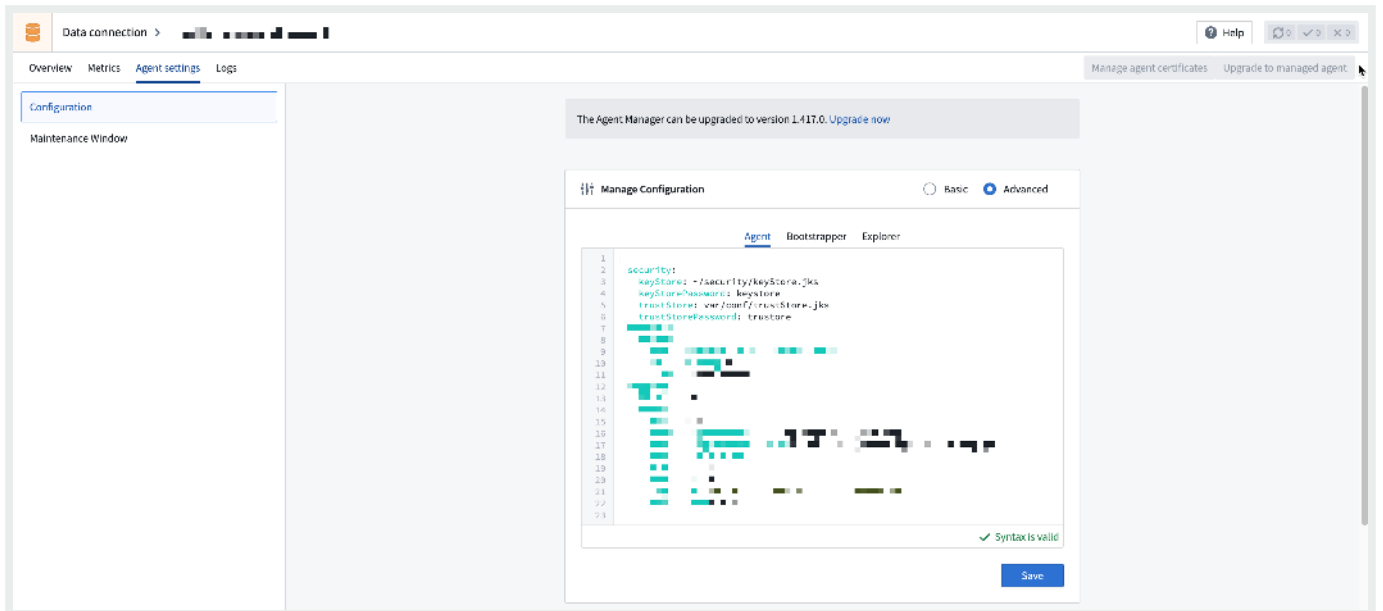
```
1 $ keytool -keypasswd -alias <CUSTOM_KEY> -new keystore -keystore /home/
2 Enter keystore password:
```

4. In [Data Connection](#), navigate to the agent, then open the **Agent settings** tab. In the **Manage Configuration** section, select **Advanced**, choose the **Agent** tab, and update the `keyStore` to point to the newly copied keystore. Then, add `keyStorePassword` and set it to the appropriate value (`keystore` , by default).

```
1 security:
2     keyStore: /home/<username>/security/keyStore.jks
3     keyStorePassword: keystore
4     trustStore: var/conf/trustStore.jks
5     ...
```

[API Reference ↗](#)

[Send feedback](#)



5. Finally, choose the **Explorer** tab and update both the `keyStorePath` and `keyStorePassword`. Save the new configuration.

```
1 security:
2   keyStorePath: /home/<username>/security/keyStore.jks
3   keyStorePassword: keystore
4   trustStorePath: var/conf/trustStore.jks
5   ...
```

6. Restart the agent.

i Note that the field is named `keyStore` when configuring in the **Agent** tab and `keyStorePath` in the **Explorer** tab. No changes are required to the **Bootstrapper** configuration.

Known limitations

Executing capabilities on the agent host directly comes with some downsides, such as:

API Reference [with conflicts on the agent](#): This is particularly relevant for custom JDBC workflows, since custom JARs may conflict with dependencies that ship from Foundry and run on the agent classpath.

Send feedback

- **Contention between jobs running on the agent:** Jobs may arbitrarily use up to the entire memory and disk space allocated to the agent process and may cause other jobs to never start or crash. This is particularly problematic for webhooks run with an agent worker. In this scenario, we strongly recommend a dedicated agent for webhook executions so long-running syncs do not prevent short-running webhooks from executing.
- **Lack of support for some capabilities:** Some capabilities, such as [use in code](#), [virtual tables](#), and virtual media, are incompatible with the agent worker, since these require synchronous connections directly from Foundry and cannot run as jobs on the agent.

PREVIOUS

← [Agent configuration reference](#)

NEXT

[Agent proxy configuration reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Agents / Agent proxy configuration reference

Agent proxy configuration reference

This page describes the configuration options available when using an agent as a network proxy via [agent proxy egress policies](#) on [Foundry worker](#) connections and assumes you are already familiar with [data connection agents](#) and their [generic configuration options](#).

Agents used as a proxy provide a network path for services in Foundry to access systems within a separate network that are not directly accessible over the Foundry network. When Foundry is hosted in the cloud, a proxy is required to access systems which are not accessible over the Internet. When Foundry is hosted on a private network, a proxy is required to access systems in other networks, which may include the public Internet if otherwise not accessible.

Agent proxies allow applications in Foundry to operate as if they are connecting to a source directly over the Foundry network, and they can be configured with multiple agents to allow for alternating maintenance windows that prevent downtime. Follow the guide below to configure and manage an agent used as a proxy.

Use an agent as a proxy with a Data Connection source

You must first [set up an agent](#) before using it as a proxy. An existing agent already used for [agent worker](#) sources can be reused without additional configuration.

Once an agent has been set up, create an [agent proxy egress policy](#) using that agent. You can then create a data connection source and assign that policy to it.

API Reference ↗



Send feedback

Security controls specific to agent proxy

To ensure that connections over the proxy only have access to required resources, a number of security controls are available.

Security controls are either *mandatory* and cannot be disabled, or *optional* and are not enabled by default.

The following security control configurations are available:

Security control	Required?	Description
Source configuration enforcement	Yes	The agent proxy always restricts traffic to only the hostname(s) and port(s) configured on the assigned source.
Agent allowlist (Foundry)	Optional	Configure an allowlist in Foundry to filter traffic before it reaches the agent. Only agent resource owners can modify this setting.
Agent allowlist (local file)	Optional	Create an allowlist on the agent host for the local agent process to enforce within your network. Only users with SSH access to the agent host can modify this setting.
Agent host firewall	Optional	Set up host-level firewall controls to limit agents to communicating only with intended target systems (strongly recommended). These firewalls work independently of Foundry and add an extra security layer.

Source configuration enforcement

The hostname and port in the URL defined on the source restrict access to only that hostname and port when connecting to the agent proxy. Attempts to connect to any other hostname or port will result in an HTTP 403 (Unauthorized) response code from the proxy.

API Reference ↗

This prevents unauthorized connections when importing and using the connection in code.

Send feedback

If only this security control is used, ensure that users who are able to assign an agent to a particular source are also trusted to connect to any system that is reachable from the agent host.

Agent allowlist (Foundry)

Configure an allowlist in Foundry to restrict connections to specific IP addresses or CIDR blocks for a specific agent, regardless of assigned sources.

To configure this, navigate to **Agent settings**, then toggle the **Advanced** option in the **Manage Configuration** section. In the YAML file, add a block for `agentProxyConfiguration` at the same indentation level as the `security` block.

List the CIDR blocks and ports where you want to allow agent connections. For example:

```
1 agentProxyConfiguration:
2   allowListedCidrs:
3     - cidrBlock: '192.168.1.1/32'
4       port: 7000
5       endPort: 9000
6     - cidrBlock: '192.168.2.2/24'
7       port: 443
```

Agent allowlist (local file)

The same configuration used for an [agent allowlist in Foundry](#) can also be set in a file on the agent host itself, preventing users in Foundry from editing the configuration.

To configure an agent allowlist locally, follow the steps below:

1. SSH into the agent's host.
2. Create a folder named `agentProxyConfig` in the same directory that contains your `source-<version>` (the root folder).

API Reference ↗

Inside the `agentProxyConfig` folder, create a file named

`agentProxyConfig.yml`.

Send feedback

4. The `agentProxyConfig.yml` file and `agentProxyConfig` folder *must* be created with the root user, and the file permissions must be set to prevent the agent from writing to this file or folder. The agent will *not* run the agent proxy feature if the `agentProxyConfig.yml` file exists and is writeable, or if one of the parent directories is writeable by the agent.
5. The content of this file is the same as the configuration available in the agent settings interface.

For example:

```
1 allowListedCidrs:
2   - cidrBlock: '192.168.1.1/32'
3     port: 7000
4     endPoint: 9000
5   - cidrBlock: '192.168.2.2/24'
6     port: 443
```

Agent host firewall

As for any agent, we recommend setting a firewall on the host to restrict connectivity to only what is necessary.

Connection pool configuration

The agent proxy maintains a pool of connections for handling requests. You can configure the connection pool size to optimize performance based on your workload.

To configure the connection pool, navigate to **Agent settings**, then toggle the **Advanced** option in the **Manage Configuration** section. In the YAML file, add the connection pool settings within the `agentProxyConfiguration` block:

```
1 agentProxyConfiguration:
  connections: 250
  idleConnections: 100
```

API Reference ↗

Send feedback

Option	Default	Description
maxConnections	300	The maximum number of connections the agent proxy can open.
coreConnections	20	The minimum number of connections maintained in the pool.

i Some connectors, such as [Databricks](#), may open a large number of connections at the same time. If you observe connection pool exhaustion errors on your agent when using agent proxy policies, consider increasing the `maxConnections` and `coreConnections` values.

PREVIOUS
← [Agent worker configuration reference](#)

→ [Troubleshooting reference](#) NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Agents / Troubleshooting reference

Troubleshooting reference

i The Agent Manager is referred to as a "bootvisor" on the server where it is installed.

This page contains information on [how to configure agent logs](#), describes several [common issues with agent configuration](#), and provides debugging guidance. The steps described must be taken after SSHing into the host where the agent has been installed. Before exploring additional troubleshooting topics, we recommend first checking `./var/diagnostic/launch.yml` to confirm the agent successfully connected to Foundry. If the connection was unsuccessful, follow the instructions described in the field `enhancedMessage`.

[Common issues with agent configuration](#)

- [Agent and agent manager shows "offline" status but returns "running" on the agent host](#)
- [Agent manager shows "offline" status](#)
- [Bootstrapper shows "never reported" status](#)
- [Agent shows "online" but is not responding to restarts](#)
- [Agent status shows "Unhealthy"](#)
- [Unable to download agent package](#)
- [Unable to administer an agent](#)
- [Unable to create an agent due to PERMISSION_DENIED error](#)

[Agent configuration reference](#)

API Reference ↗

[Sources using the insecure TLSv1.0 and TLSv1.1 protocols](#)



[View agent logs](#)

Send feedback

- [What happens to cached files when the host where the agent is installed crashes?](#)

Common issues with agent configuration

Agent and Agent Manager shows "offline" status but returns "running" on the agent host

- The first step is to check that Foundry is reachable from the host where the agent is installed. To do this, run `curl -s https://<your domain name>/magritte-coordinator/api/ping > /dev/null && echo pass || echo fail` from the host where the agent is installed.
 - If everything is working, you should see `pass` as output. In which case you should:
 - Determine if a proxy is required to reach Foundry and if so, check whether the agent has been configured to use it (instructions on how to configure an agent to use a proxy can be found on the [proxy configuration](#) page). You can verify whether a proxy is being used by running `echo $http_proxy` on the command line of a Unix-based machine.
 - If you don't think a proxy is required or you have already configured one, contact your Palantir representative.
 - If Foundry is unreachable from the host, you might see an error such as: `curl: (6) Could not resolve host: ...`. In this instance, it is likely there is something blocking the connection (e.g. a firewall or a proxy), and you should contact your Palantir representative.

Agent manager shows "offline" status

- Check the contents of the `<agent-manager-install-location>/var/log/startup.log` file.
 - If you see the following error: `Caused by: java.net.BindException: {} Address already in use`, it means there is a process already running on the port to which the Agent Manager is trying to bind.
 - To resolve this, you should first ascertain to which port the Agent Manager is trying to bind. This can be done by checking the contents of the `<agent-manager-directory>/var/conf/install.yml` file and looking for a `port` parameter (e.g. `port: 1234` - here 1234 is the port). Note if there is no port parameter defined, Agent Manager will use the default port 7032.

API Reference ↗

Now the port to which the Agent Manager is trying to bind, you should identify the process that is already running on it. This can be done by running the following command: `ps aux | grep $(lsof -i:<PORT> | awk 'NR>1 {print`

Send feedback

`$2}' |sort -n |uniq)` where `<PORT>` is the port to which the Agent Manager is trying to bind.

- If the response returned by the above command contains:
`com.palantir.magritte.bootvisor.BootvisorApplication` it means another Agent Manager is already running.
- In this case you should determine if this is intentional; if so, you will need to change the port in the configuration to de-conflict the two Agent Managers by following the steps below. Otherwise, you'll need to determine which specific Agent Manager install you want to use on this host, stop any others that are running, and start up only the one you intend to use going forward.
- To fix the `BindException` error, you will need to find a new port for the Agent Manager, that isn't currently being used.
 - Port numbers should be between 1025 and 65536 (port numbers 0 to 1024 are reserved for privileged services and designated as well-known ports).
 - You can check if a process is already running on a port by executing the following command: `lsof -i :<PORT>` where `<PORT>` is the chosen port number.
- Once you have found an available port, you will need to add (or update) the `port` parameter in the configuration stored at `<agent-manager-directory>/var/conf/install.yml`
- Below is an example **Agent Manager** configuration snippet with the port set to `7032` :

```
1 ...  
2 port: 7032  
3 auto-start-agent: true
```

- Once you have saved the above configuration, restart the Agent Manager by running `<agent-manager-root>/service/bin/init.sh stop && <agent-manager-root>/service/bin/init.sh start`.

Bootstrapper shows "never reported" status

- Check the contents of the `<agent-manager-directory>/var/data/processes/<latest-bootstrapper-directory>/var/log/startup.log` file.

◦ If you see the following error: `Caused by: java.net.BindException: {} Address`

[API Reference ↗](#) `use`, it means there is a process already running on the port to which the Bootstrapper is trying to bind.

[Send feedback](#)

- In order to resolve this, you should first ascertain to which port the Bootstrapper is trying to bind. This can be done by navigating to the agent overview page within the Data Connection application. From there, you will need to select the "advanced" configuration button and finally click the "Bootstrapper" tab. The port to which the Bootstrapper will try to bind is defined under the `port` parameter (for example, `port: 1234` - here 1234 is the port). Note the default port for the Bootstrapper is 7002.
- Once you know the port to which the Bootstrapper is trying to bind, you should identify the process that is already running on it. This can be achieved by running the following command: `ps aux | grep $(lsof -i:$PORT | awk 'NR>1 {print $2}' | sort -n | uniq)` where `$PORT` is the port to which the Bootstrapper is trying to bind.
 - If the response returned by the above command contains `com.palantir.magritte.bootstrapper.MagritteBootstrapperApplication` it means another Bootstrapper is already running.
 - In this case, you should determine if this is intentional; if so, you will need to change the port in the configuration to de-conflict the two Bootstrappers by following the steps below. Otherwise, you'll need to determine which specific Bootstrapper install you want to use on this host, stop any others that are running, and start up only the one you intend to use going forward.
- To fix the `BindException` error, you will need to find a new port for the Bootstrapper, that isn't currently being used.
 - Port numbers should be between 1025 and 65536 (port numbers 0 to 1024 are reserved for privileged services and designated as well-known ports).
 - You can check if a process is already running on a port by executing the following command: `lsof -i :<PORT>` where `<PORT>` is the chosen port number.
- Once you have found an available port, you will need to set the `port` parameter in the Bootstrapper's configuration. This can be done by navigating to the agent overview page in the Data Connection application. From there select the advanced configuration button and finally navigate to the "Bootstrapper" tab.
- Below is an example **Bootstrapper** configuration snippet with the port set to `7002`:

```

1 server:
2   adminConnectors:
    port: 7002 #This is the port value

```

API Reference ↗

Send feedback

- Once you have updated the configuration, you will need to save your changes and restart the agent for them to take effect.

Agent shows "online" but is not responding to restarts

More often than not, this is caused by another "ghost" instance of the agent running that you need to find and shut down.

To find and terminate old processes, follow the steps below:

1. Stop the Agent Manager by running: `<agent-manager-install-location>/service/bin/init.sh stop`.
2. Delete the `<agent-manager-install-location>/var/data/processes/index.json` file.
3. Run `for folder in $(ls -d <agent-manager-root>/var/data/processes/*/); do $folder/service/bin/init.sh stop; done` to shut down the old processes.
4. Return to Data Connection and check the agent is no longer reporting (takes 2-3 minutes).
5. Start the Agent Manager (`<agent-manager-install-location>/service/bin/init.sh start`).

i Manually starting agents on the host where they are installed (as opposed to through Data Connection) can lead to the creation of "ghost" processes.

Agent status shows "Unhealthy"

Often when the agent process shows as "unhealthy" it is because it has crashed or been shut down by either the operating system or another piece of software such as an antivirus.

There are multiple reasons why the operating system might have shut down the process, but the most common one is because the operating system does not have enough memory to run it, which is referred to as being OOM (Out Of Memory) killed.

To check if any of the agent or Explorer subprocesses were OOM killed by the operating system, use the following command: `grep "exited with return code 137" -r <agent-manager-directory> --include=*.log`. This will search all the log files within the Agent Manager directory for entries containing 'exited with return code 137' (which signifies a process was OOM killed).

Send feedback

The following is an example output produced by the above command and shows the agent subprocess is being OOM killed: `./var/data/processes/bootstrapper~`
`<>/var/log/magritte-bootstrapper.log:ERROR [timestamp]`
`com.palantir.magritte.bootstrapper.ProcessMonitor: magritte-agent exited with return code 137`. If you see an output similar to this, you should follow the steps below on tuning heap sizes.

You can also check the operating system logs for OOM kill entries by running the following command: `dmesg -T | egrep -i 'killed process'`. This command will search the kernel ring buffer for 'killed process' log entries, which indicates a process was OOM killed.

Actual log entries of OOM killed processes will look like the following:

- `[timestamp] Out of memory: Killed process 9423 (java) total-vm:2928192kB, anon-rss:108604kB, file-rss:0kB, shmem-rss:0kB, UID:0 pgtables:1232kB oom_score_adj:0`
- The above log line shows the process killed had a PID 9423 (note: your log messages may vary depending on Linux distribution and system configuration).
- In this scenario, you should try to verify whether the process being killed is related to your agent. The easiest way to do this is by aligning time stamps, i.e., if an entry's timestamp ties in with the time your agent became unhealthy it is likely the two are correlated. Note any entries that don't contain `(java)` can be ignored as they are not related to your agent.

Tuning heap sizes

Before you change any heap allocations, you should first:

- Calculate how much memory the host has available.
- To see how much memory the host has available, you can run `free -h`. On a 6 GB system, the output might look something like this:

1	total	used	free	shared	buff/cache	⌵
2 Mem:	5.8Gi	961Mi	2.8Gi	9.0Mi	2.1Gi	
	1.0Gi	0B	1.0Gi			

API Reference ↗

Send feedback

In the output produced by the `free` command, the `available` column shows how much memory can be used for starting new applications. To determine how much memory can be allocated to the agent, we recommend that you stop the agent and run `free -h` while the system is under normal to high load. The available value will tell you the maximum amount of memory you can devote to all agent processes combined. We recommend that you leave a buffer of approximately 2 - 4GB, if possible, to account for other processes on the system needing more memory, as well as off-heap memory usage by the agent processes. Note that not all versions of `free` show the available column, so you may need to check the documentation for the version on your system to find the equivalent information.

Determine how much memory is assigned to each of the following subprocesses: Agent Manager, Bootstrapper, agent, and Explorer.

In order to find out how much memory is assigned to the agent and Explorer subprocesses, you should navigate to the agent configuration page within Data Connection, choose the advanced configuration button, and select the "Bootstrapper" tab. From there you will see each of the subprocesses have their own configuration block; within each block you should see a `jvmHeapSize` parameter which defines how much memory is allocated to the associated processes.

By default, the Bootstrapper subprocess is assigned `512mb` of memory. This can be confirmed by first navigating to the `<agent-manager-directory>/var/data/processes/` directory; from there you will need to run `ls -lrt` to find the most recently created `bootstrapper~<uuid>` directory. Once in the most recently created `bootstrapper~<uuid>` directory, you can inspect the contents of the `./var/conf/launcher-custom.yml` file. Here, the `Xmx` value is the amount of memory assigned to the Bootstrapper.

By default, the Agent Manager subprocess is also assigned `512mb` of memory. This can be confirmed by inspecting the contents of the file `<agent-manager-directory>/var/conf/launcher-custom.yml`. Here, the `Xmx` value is the amount of memory assigned to the Agent Manager.

i Agents installed on Windows machines do not use the `launcher-custom.yml` files and thus, by default, Java will allocate both the Agent Manager and Bootstrapper

of the total memory available to the system. To fix this you will need to manually set the Agent Manager and Bootstrapper heap sizes manually, which can be done by following the steps below:

[API Reference ↗](#)

[Send feedback](#)

1. Make sure you have killed all the agent processes, namely: (Agent Manager, Bootstrapper, agent, and Explorer).
2. Set JAVA_HOME: `setx -m JAVA_HOME "{BOOTVISOR_INSTALL_DIR}\jdk\{JDK_VERSION}-win_x64\"`
3. Set the Agent Manager heap size: `setx -m MAGRITTE_BOOTVISOR_WIN_OPTS "--Xmx512M -Xms512M"`
4. Set the Bootstrapper heap size: `setx -m MAGRITTE_BOOTSTRAPPER_OPTS "--Xmx512M -Xms512M"`
5. Close the command prompt and open a fresh one. This is required for the settings above to take effect.
6. Start the Agent Manager: `.\service\bin\magritte-bootvisor-win`

Once you have determined how much memory the host has available and how much memory is assigned to each of the above subprocesses, you should then decide whether to: decrease the amount of memory allocated to the above processes or increase the amount of memory available to the host.

Whether or not you can safely decrease the amount of memory used by the agent processes will depend on your agent settings (for example, the maximum number of concurrent syncs and file upload parallelism), the types of data being synced, and the typical load on the agent. Decreasing the heap size makes it less likely that the OS will kill the process but more likely that the java process will run out of heap space. You may need to test different values to find what works. Contact your Palantir representative if you need assistance tuning this value.

To decrease the amount of memory allocated to one (or multiple) of the subprocesses, do the following:

1. Decide on how much memory should be allocated to each of the aforementioned subprocesses.
 - *Note: We do not recommend reducing the heap sizes below the defaults which are listed below.*
2. Next, navigate to the agent within Data Connection, choose the advanced configuration button, and select the **Bootstrapper** tab.
3. [API Reference ↗](#) the `jvmHeapSize` parameter for each of the individual subprocesses.
4. Below is an example **Bootstrapper configuration snippet** with the `jvmHeapSize` set to 3gb:

Send feedback

```
1 agent:
2     ....
3     jvmHeapSize: 3g #This is jvm heap size value
```

5. Once you have updated the configuration, you will need to save your changes and restart the agent for them to take effect.

Default heap allocations

By default an agent requires ~3gb of memory, allocated as follows:

- 1gb for the agent subprocess
- 1gb for the Explorer subprocess
- 512mb for the Bootstrapper subprocess
- 512mb for the agent Manager subprocess

Java processes also use some amount of off-heap memory; thus, we recommend you ensure there is at least $\geq 4\text{gb}$ left free for them.

Unable to download agent package

There are two main causes of failed agent downloads: network connections and expired links.

If you can connect to Foundry but are getting an invalid `tar.gz` file or an error message on the download, you may have an expired or invalidated link.

- *Expired links*: Download links expire after ten minutes.
- *Invalidated links*: Download links are protected with a one-time download secret. Pasting agent download links in applications such as Microsoft Teams can invalidate the link because those applications will attempt to scan the link to see if it can be previewed; this scan invalidates the one-time download secret. If you have an invalid link, try regenerating the link in the UI and retyping the two secret words instead of copying the whole link

[API Reference ↗](#)

If you encounter a `disconnected unexpectedly` error during the agent download, try forcing `curl` to use HTTP 1.0 by adding the `--http1.0` flag to your command.

[Send feedback](#)

This can resolve protocol negotiation issues that may occur in certain network environments.

Unable to administer an agent

A user must be an editor of a Project to create an agent in that Project, but must be an owner of the Project to administer the agents within that Project. That means that a user may create an agent and then be unable to generate download links or perform other administrative tasks on the agent. For more on agent permissions, review the guidance in our [permissions reference](#) documentation.

Unable to create an agent due to PERMISSION_DENIED error

If you receive a `PERMISSION_DENIED` error when attempting to create an agent, verify the following:

- **Organization-level role:** You must have the `Organization administrator`, `Data flows administrator`, or a custom role with the `Create agent` workflow assigned to you. Organization roles are managed on the **Organization permissions** page in Control Panel.
- **Project-level role:** You must be an `Editor` or `Owner` of the project in which you want to save the newly created agent.

If you only have `Editor` permissions on an existing project but lack the required organization-level role, try creating the agent in a different project where you have the appropriate permissions, or request the necessary role from your administrator.

For more details on agent permissions, review the guidance in our [permissions reference](#) documentation.

Agent configuration reference

Connect to data sources using the insecure TLSv1.0 and TLSv1.1 protocols

[API Reference ↗](#)

TLSv1.0 and TLSv1.1 are not supported by Palantir as they are outdated and insecure protocols. Amazon Corretto builds of the OpenJDK used by Data Cor

[Send feedback](#)

explicitly disable TLSv1.0 and TLSv1.1 by default under the `jdk.tls.disabledAlgorithms` security property in the `java.security` file.

Attempts to connect to a data sources system exclusively supporting TLSv1.0 and TLSv1.1 will fail with various errors including `Error: The server selected protocol version TLS10 is not accepted by client preferences`.

 We actively discourage the usage of deprecated versions of TLS. Palantir is not responsible for security risks associated with its usage.

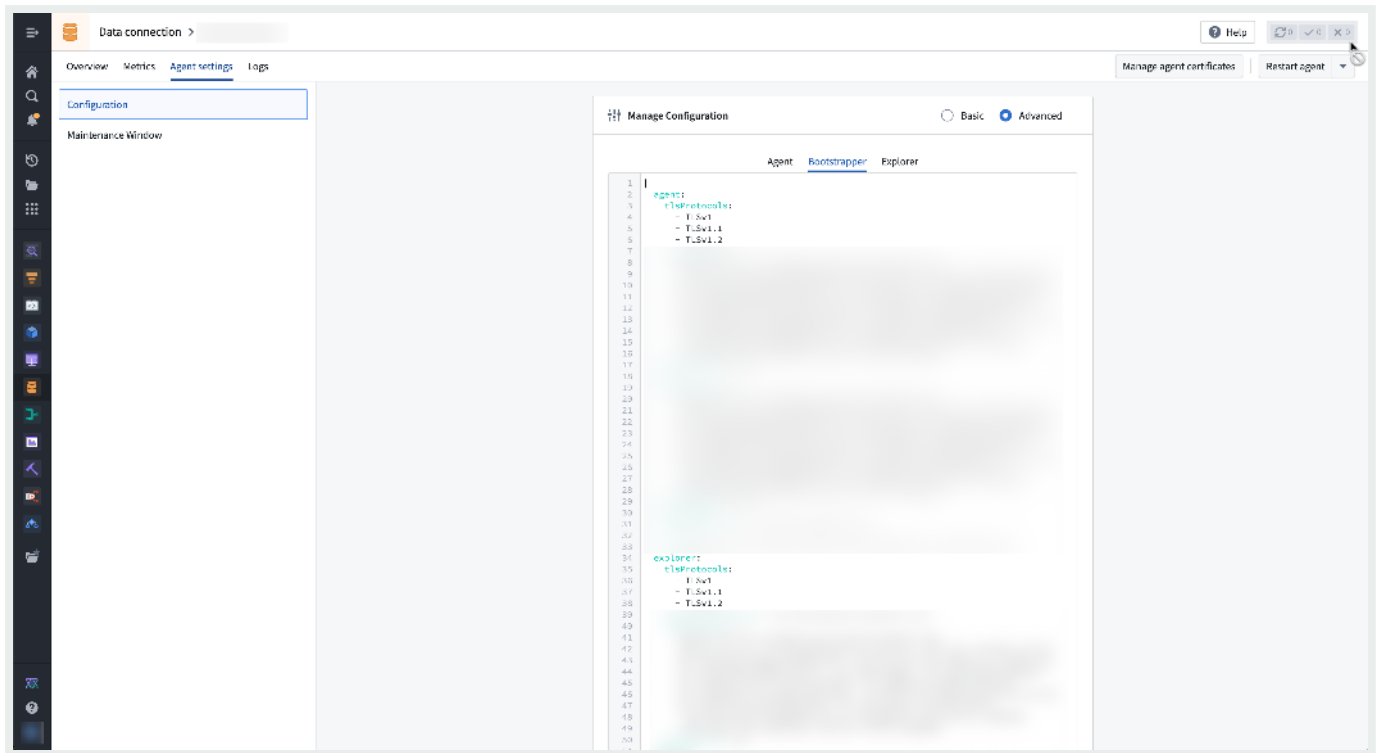
If there is a critical need to temporarily support TLSv1.0 and TLSv1.1, perform the following steps:

1. From the agent overview page, navigate to **Agent settings** and select **Advanced** in the **Manage Configuration** section. Then, select the `Bootstrapper` tab.
2. Add `tlsProtocols` entries to both the `agent` and `explorer` configuration blocks followed by the protocols you want to enable. Be sure to also include TLSv1.2 so any sources using it will not break. For example:

```
1 agent:
2   tlsProtocols:
3     - TLSv1
4     - TLSv1.1
5     - TLSv1.2
6   ...
7 explorer:
8   tlsProtocols:
9     - TLSv1
10    - TLSv1.1
11    - TLSv1.2
12  ...
```

[API Reference ↗](#)

[Send feedback](#)



3. Select **Restart agent**.

With this configuration, the agent will continue to allow TLSv1.0 and TLSv1.1 across agent upgrades and restarts. Once the datasource has moved to new TLS versions, revert all changes made to the advanced agent configuration.

Configure agent logs

To adjust the log storage settings for an agent on its host machine, follow the steps below:

1. In Data Connection, navigate to the **Agents** page. Select the name of the agent you want to configure.
2. In the Configuration panel, select **Advanced**.
3. The configuration options for logging can be found under the **Logging** block. Here, you can configure limits on when to start discarding logs, if and how to archive logs, and other settings.
 - Note that the configuration should consider the allocated agent host machine resources, your preference of log level granularity, and your preference of log retention. For more information and guidance, consult the [Dropwizard configuration](#)

API Reference ↗

4. Restart the agent in Foundry by selecting **Restart Agent** in the upper-right corner of the screen.

Send feedback

Your new configuration should now be in effect.

How long has my agent been down (unavailable)?

There are a number of reasons your agent could be unavailable; for instance, the agent may be restarting or the underlying hardware running the agent could be offline or restarting.

There are two ways to determine when the agent first became unavailable:

- After selecting your agent in the Data Connection UI, you can see a visual representation of metrics related to uptime and availability in the **Metrics** tab.
- In the **Overview** section of the Data Connection UI, you can see the status of your agent, as well as the date and time the agent's status was last reported.

What happens to cached files when the host where the agent is installed crashes?

The files will remain on disk until the Bootvisor cleans up old process folders (30 days or 10 old folders triggers a clean up). These files are encrypted and the keys to decrypt them only existed in the memory of processes that died.

← PREVIOUS
[Agent proxy configuration reference](#)

NEXT →
[Sources / Set up a source](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Set up a source

A Foundry source represents a connection between Foundry and an external system. Some example Foundry source types include a Postgres database, an S3 bucket, a filesystem on a Linux server, an SAP instance, or a REST API over the Internet.

At a high level, below are the steps required to connect a source to Foundry. Note that Step 1 through Step 3 may require you to change or validate configurations within your existing architecture:

1. [Ensure there is a valid network connection](#) between the source and Foundry.
2. [Provision valid credentials](#) for Foundry to authenticate against the source.
3. For legacy [agent worker](#) sources only: ensure the agent has the [appropriate drivers](#) to access the external system.
4. Finally, [create the source in Data Connection](#).

Once you have this source connection established, you can configure [syncs](#) to bring specific sets of data into Foundry. Syncs can be entirely configured through the Data Connection UI, so the source setup is the final task that may require configurations to be updated in your organization's environment before you can access your data within Foundry.

Configure network access

To connect Foundry to an external system, first validate network paths:

- External systems hosted in the same network as and accepting inbound connections

API Reference ↗

requires a single valid network connection from Foundry to the external system. For cloud-hosted instances of Foundry, this is typically the case for cloud-based or SaaS services.

Send feedback

- External systems hosted in a separate network from Foundry must use an [agent](#) with two valid network paths: (1) from the agent host to the external system and (2) from the agent host to Foundry. For cloud-hosted instances of Foundry, a separate network usually means an on-premise network.

(Optional) Set up an agent and configure agent connectivity

You will need to [set up an agent](#) if the external system you are connecting to is hosted on a separate network from Foundry's network.

With [Foundry worker and agent proxy policies](#), the agent is used as a networking proxy only and compute runs in Foundry. Legacy [agent worker sources](#) use the agent for both networking and compute; see [Foundry worker vs. agent worker](#).

Ensure there is a connection between an agent set up within your network and the external system. The agent acts as a single point of validated entry to Foundry from your network and will handle the process of reading and sending data on to the Foundry instance. For each new system, you will only need to confirm there is a valid connection between the agent and the new system.

i You will not need to establish direct network egress from the external system to Foundry, as traffic only flows from the agent to Foundry and from the agent to the external system. [Learn more about the architecture of data connection.](#)

The steps required to establish this connection will vary depending on your organizational network settings. Regardless of your specific setup, the goal is for the agent to have the ability to connect to the external system. This could involve the configuration of egress settings on the agent host, ingress settings on the external system, firewall rules across the network, proxy settings on the agent, adding source system [certificates](#) to the agent truststore, and so on.

If you need to configure proxy settings for the agent to reach the external system, you can do this through [Data Connection](#).

[API Reference ↗](#)

Configure a network policy

[Send feedback](#)

⚠ You must have the **Information security officer** role on your [enrollment](#) to configure network egress. If you do not have permissions to configure egress, contact your Palantir administrator to request access.

You can find the **Information security officer** role in the **Enrollment permissions** section of [Control Panel](#). A user must have the **Enrollment administrator** role to view this section.

[Foundry worker](#) sources additionally require network egress policies to route the traffic.

To configure a network policy, navigate to the **Network egress** section in Control Panel. Pick a [direct connection policy](#) if the external system you are connecting to is hosted in the same network than Foundry. Pick an [agent proxy egress policy](#) if the source you are connecting to is in a separate network from Foundry.

If you are unable to view the **Network egress** section, contact your Palantir administrator to set up the network policy.

Add network egress policy

1 Configure egress

2 Configure permissions

Policies must be applied to sources in Data Connection to take effect.

Description

Describe the type of traffic and/or destination that this policy will allow.

Export to cloud storage provider

Address

Specify the address that this policy will allow.

CIDR

192.168.0.0/16

Port number

Specify the port number or range that this policy will allow.

Single port

443

☐ Make this a global policy

Not recommended

Global policies are applied to all user workflows, including code workbooks, code transforms, and modeling live deployments. This should only be enabled for highly trusted destinations.

Next

API Reference ↗

Send feedback

Provision credentials

For most cases, Foundry will require authorized credentials (such as a username and password) to access external systems. We recommend using a dedicated service account with credentials scoped specifically for the required access in Foundry.

Provision a service account for the source following any internal guidelines and processes that your organization has for establishing service accounts. Note the credentials before proceeding to the next step.

Create the source in Data Connection

Once the above steps are done, you can proceed with creating the source in Data Connection:

- After logging in, navigate to **Data Connection** using the [sidebar](#).
- Select the **Sources** tab.
- Select **New source** in the top-right.
- Select the [source type](#) corresponding to your data source.
- Select **Continue** in the bottom right. The source is configured to run on a [Foundry worker](#). For existing [agent worker](#) sources, see the [agent worker configuration reference](#).

Save the source in a Project

Next, name your source and choose a [Project](#) to place it in. We generally recommend creating a new Project for each source, as this provides the most natural way to permission datasets derived from this source.

You can read more about [source permission best practices](#) or consult the [full guidance for how to structure data pipelines end-to-end in Foundry](#).

Select **Create source and continue** in the bottom-right.

API Reference ↗

Choose your network policy [Foundry worker]

Send feedback

On the next setup page, select the network policy you [configured earlier](#) by choosing **Use existing policy** and searching for the policy name.

Network Connectivity

[Switch to connect via Agent](#)

Select a policy

A network egress policy is required to connect to sources outside Foundry's secure network. Select an existing policy if one already exists, otherwise create a new policy. [Learn more](#)

+ Use existing policy

 Create new policy

Add egress policies

mydomain

mydomain.com (Port 443)

Select

mydomain.com (Port 443)

Select

mydomain.com (Port 44)

Select

mydomain.com (Port 443)

No description

Select

API Reference ↗

Cancel

Select

Send feedback

Configure source

Add details about how to connect to your source. These will depend on the [source type](#) you are using and typically consist of basic credentials such as connection URLs, cloud provider regions, and so on.

(Optional) Add JDBC drivers

JDBC sources may require you to upload [JDBC drivers](#), then specifying which Java class from the driver should be used.

(Optional) Add certificates

External systems might require certificates to ensure the connection can be trusted. This would occur for the following:

- Systems using TLS with self-signed certificates, for which you will need to add **server certificates**.
- Systems using mTLS which require the Foundry client to prove its identity with a **client certificate**.

To understand whether to add server or client certificates, see [Server and client certificates](#).

For Foundry worker connections, add certificates on to the source itself using the following steps:

1. Expand the **Certificates** card to view the certificates selected on the source.
2. Select **Add certificate** (or **Create new certificate** first if you have not configured the certificate).
3. Select the client certificates and server certificate bundles to attach to the source.

Review the [egress certificate configuration documentation](#) for more information.

API Reference ↗

Configure server certificates (legacy) or **Configure client certificates and legacy** at the bottom of the **Connection details** page under **More options** to configure sources you have created with legacy certificates.

Send feedback



Learn how to [configure certificates on an agent when using agent worker connections.](#)

Add credentials

Add the credentials you [provisioned](#) previously to allow the source to connect to your data.

Save and continue

Select **Save** in the bottom-right to complete setting up your source. Once your source is fully set up, you can proceed to [set up a sync](#) to bring data into Foundry.

Troubleshooting

To confirm the connection has been established, select **Preview** in the right panel of the source page.

If the source does not work as expected, see [Troubleshooting](#) for the available debugging tools — including the [source terminal](#) and [network egress logs](#) — and guidance on connectivity and certificate issues. For sync-specific problems, see the [syncs troubleshooting reference](#); if you are using an agent worker, see the [agents troubleshooting reference](#).

PREVIOUS



Agents / Troubleshooting reference

NEXT



Source exploration

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference](#)

[Privacy Statement](#)

[Terms of Use](#)

Send feedback

Source exploration

When setting up syncs from a source, you may want to explore the source and the data it contains to preview syncs before they bring data into Foundry. Depending on the source type, Data Connection allows you to preview data from a sync via the **Explore** page.

To access the **Explore** page, follow the steps below:

1. Navigate to the **Sources** page and click on the name of the source you want to configure.
2. Select the **Explore** link from the **Explore and Create Syncs** component of the Sync page.

Overview Connection settings Edit syncs **Explore source**

1 You are previewing table resources on the remote system that have not been synced to Foundry yet. Start by selecting table resources from the resource list and then create a sync to import the data into Foundry.

Previewing source

These table resources are a preview of what can be synced into Foundry.

Filter by name...

AccountHistory Previewing 20 rows 9 columns

id	Deleted	AccountId	CreatedBy	CreatedDate	Field	Data Type	Old Value
0117F00000v2QAS	false	00117F00000UCvUQAW	00517F00000JUN0QAM	2020-07-24 08:55:13.0	Account_Status__c	Dynamic Enum	...
0117F00000v3QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-07-24 06:05:35.0	Account_Status__c	Dynamic Enum	...
0117F00000v4QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-07-24 06:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v5QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v6QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v7QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v8QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v9QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v10QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v11QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v12QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v13QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v14QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v15QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v16QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v17QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v18QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v19QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...
0117F00000v20QAK	false	00117F00000JID7QAN	00517F00000JUN0QAM	2020-11-16 12:59:00.0	Account_Status__c	Dynamic Enum	...

Tables to sync into Foundry

To complete synchronizing the data into Foundry, create a Sync.

Create sync for 2 datasets →

Run syncs after creating

New syncs will be placed in: /Palantir/Connections

AccountHistory 9 columns

User 1 column

AccountPartner 13 columns

Account 13 columns

1 API Reference ↗

2

3

4

Send feedback

1. **Left panel:** Find and add tables and views from the source system to the graph. Use the free text search helper to find specific tables, or browse the tree to find resources.
2. **Graph:** Explore tables and views and the relationships between them. You can also create a sync from the graph by right-clicking on the table. When selecting a table with a relation, the foreign key will be highlighted within the expandable column list and above the link.
 - Note that the graph is not always available for table-based data exploration. For example, a graph would not appear for exploration of a table-based REST API model as there are no clear relations between objects.
3. **Table details:** Preview a sample of the selected table.
4. **Right panel:** Review chosen tables, and create syncs for the tables.

File-based data

1. **Left panel:** Explore and select the directory containing contents to sync into Foundry.
2. **Preview:** See a preview of the files that will be synced. A filter can be added to control which files get imported into Foundry.
3. **Right panel:** Review chosen tables, and create syncs for the tables.

PREVIOUS
← Set up a source

NEXT
Syncs / Set up a sync →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Set up a batch sync

The **batch sync** capability enables syncing data from an external system into a Foundry [dataset](#). Batch sync is the most widely supported capability and is available on almost all [connectors](#). Batch syncs allow syncing tabular data with a schema as well as raw files without a schema.

Creating a batch sync will also create a new Foundry [dataset](#) where synced data will be written. Once the sync is configured, you can either manually run it or set up a [schedule](#) to trigger the [build](#) that will read data from the external system and write it to the output dataset.

Follow the steps below to set up a batch sync. This setup guide assumes you have already successfully [configured a source connection](#) that supports the **batch sync** capability.

Create a new batch sync

First, navigate to your source connection in the Data Connection application, then select **New batch sync** from the overview page. If this is a newly configured source, you should see the available capabilities as shown below, and select **Create** next to the batch sync option.

[API Reference ↗](#)[Send feedback](#)

Explore source capabilities

Explore and try out the full range of capabilities available on this source connection



Batch syncs

Imports are tools used to synchronize data from a variety of sources like databases, file systems, or other data systems into Foundry datasets. They can be customized to read only the necessary data and can be set to incrementally sync data.

Create



File exports

File export is the schema-less process of transferring data from Foundry to external systems in batches. It's beneficial for synchronizing cleaned and transformed data with warehouses or data lakes. It's also helpful for exporting inference results or recording operational data.

Create



Virtual tables

Virtual tables enable you to query tables in supported data platforms without needing to store the data in a Foundry dataset first. They serve as pointers to tables in an external source system, abstracting the source system and storage formats.

Create



Media syncs

Media imports are created to handle large-scale, unstructured data such as audio, images, and PDFs. Data Connection supports media synchronization from sources like Azure Blob Filesystem (ABFS), Amazon S3, and external transforms.

Create



Code Repositories

Import this source into code repositories to write custom code for syncing and exporting data or use virtual tables in transforms.

Enable

If your connector supports [source exploration](#), you can also select **Explore and create syncs** to explore your data source and begin creating syncs directly from the exploration view. Refer to the [source exploration](#) documentation for details.

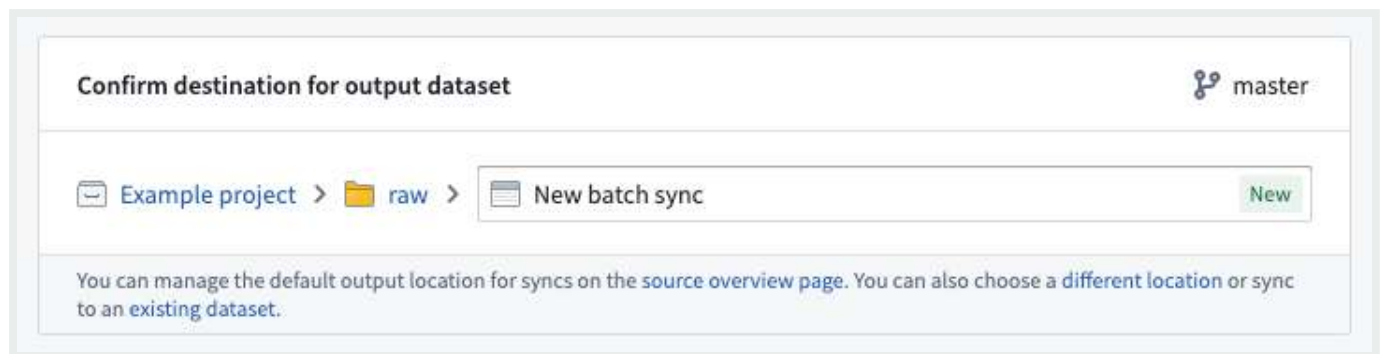
Specify output location

The output location defines where the dataset of synced data will be created and will determine who has permission to access the resulting data, based on [Project-level](#) permissions. A default output folder may be specified for the source, which can be overridden for each sync if desired.

API Reference ↗ Best practice when creating a sync dataset is to save it alongside the connector. This enables the pattern of uniformly permissioning all da

Send feedback

connector, which is helpful when creating data pipelines. Learn more about the [recommended Project structure for data pipelines](#).



i Syncing to an existing dataset is supported but not recommended, since syncs may overwrite any data already in the selected dataset.

Configure batch sync

On the same page as the destination, you will see various settings for configuring your batch sync.

Depending on the source, different options may be available. The two most common types of batch syncs are:

- [File batch syncs](#), where files are synced to a dataset without a schema.
- [Table batch syncs](#), where a table with a schema is synced.

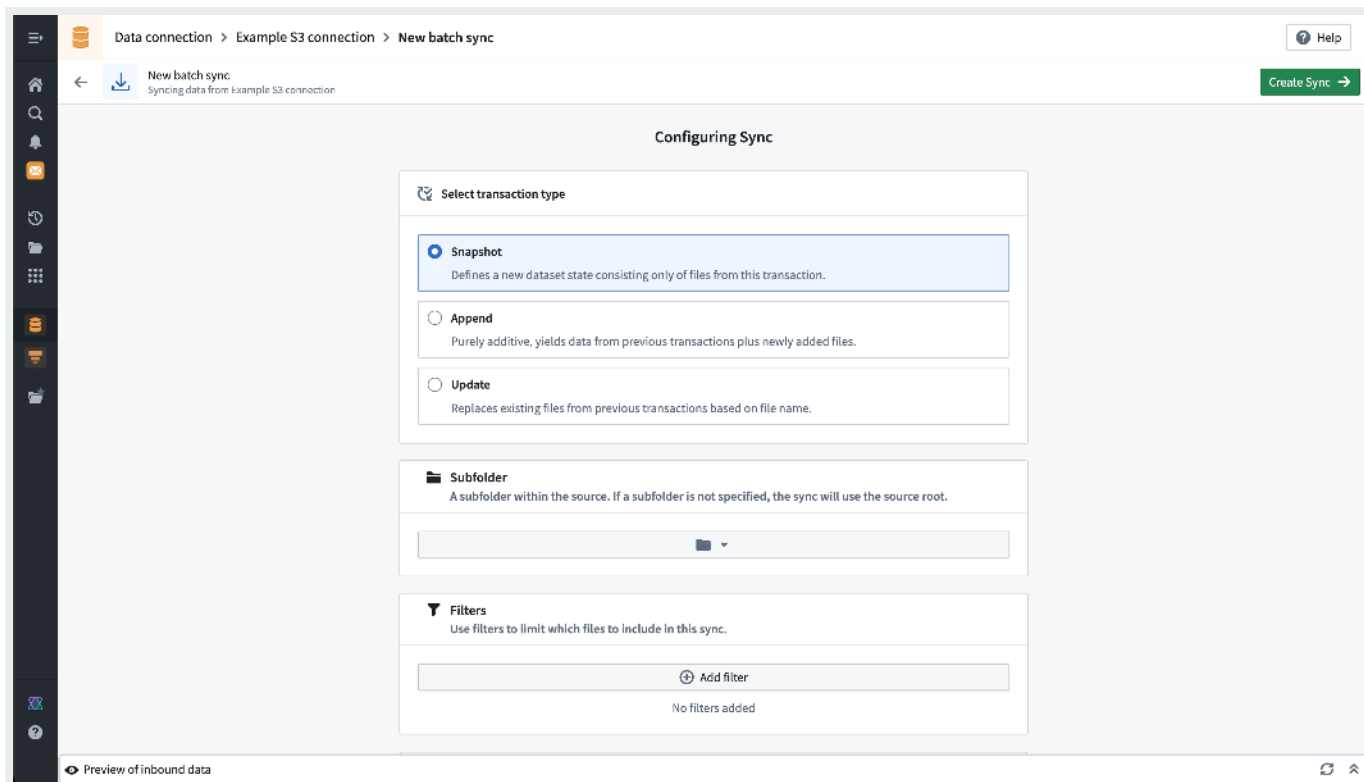
Most systems support either file or table batch syncs, but some systems may support both.

File batch sync example

The below example shows the configuration for a file batch sync from S3 that does a **SNAPSHOT** update on each build. You may optionally specify a subdirectory and filters to narrow down the selection of files to be synced to the output dataset. Our example does not specify a subdirectory or filters, meaning all files found under the root directory that was chosen when setting up the source connection will be synced.

[API Reference ↗](#)

[Send feedback](#)



Additional settings for file batch syncs are described in the reference documentation for [file batch syncs](#), including detailed documentation for the available [filters](#).

Table batch sync example

This example shows the configuration for a table batch sync from [Microsoft SQL Server](#). A query defines which data will be pulled from the target system. In this case, the incremental batch sync setting is also enabled, which allows data to be incrementally updated based on a monotonically increasing column.

[API Reference ↗](#)

[Send feedback](#)

i Although you cannot visualize SQL stored procedures on the [Explore source](#) tab, you can run a SQL stored procedure by running the `EXEC` command followed by the corresponding procedure in the **SQL Query** field.

Additional options

A number of other options are available for most batch syncs and may vary depending on the connector. Some examples of widely available configuration options for batch syncs are listed below:

- The [transaction type](#) determines whether ingested data overwrites previously ingested data (`SNAPSHOT`) or whether it is added incrementally (`APPEND`). Learn more about [incremental syncs](#).
- [Schedules](#) allow you to configure how often data should be synced using Foundry's build system. We recommend setting up a schedule for your newly created sync. Learn more about [scheduling best practices](#).

API Reference ↗ shows you to restrict when a sync is allowed to run, regardless of the configured schedule.

- Maximum duration allows you to automatically cancel syncs that exceed a specified time limit. All syncs [running on a Foundry worker](#) will be cancelled automatically if they

[Send feedback](#)

run longer than approximately 48 hours.

The following options are available for table batch syncs only:

- Timestamp without timezone settings allow you to customize how timestamp data without a timezone is handled when syncing into Foundry. By default, timestamps without timezones are synced as strings, but you may choose to sync as a `timestamp` with a manually specified timezone, or as a `long`.
- Allow schema changes. This setting allows you to prevent batch syncs from running if there is a schema change in the external system. By default, schema changes are not allowed.

Preview your sync output

Before proceeding, you can run a preview of the data that will be synced based on the settings you have configured. You should use this to verify that your sync is configured as expected.

- For file batch syncs, the preview will show a list of files.
- For table batch syncs, the preview will show the selected table results, limited to the first 20 rows.

Below, we show an example preview for an S3 file batch sync, with a filter to a subfolder called `csv_files`:

[API Reference ↗](#)

[Send feedback](#)

Send feedback

 Schedule

Run this import

On a schedule ▾

Schedule Details

Run every

1

Day ▾

At

12 : 00

AM ▴ ▾

London +01:00 ▾

Use the [Data Lineage application](#) to set up a [schedule](#) for multiple syncs at the same time.

i Schedules should not be configured from both Data Connection and Data Lineage for the same batch sync. Schedules configured from Data Lineage should always use the **Force build** option when building Data Connection syncs.

Next steps

In this setup guide, you learned how to create a batch sync to bring data from a connector into a Foundry dataset. Here are some additional resources we recommend:

- Learn more about other capabilities including [change data capture syncs](#), [exports](#), and [media set syncs](#).
- Refer to [Building pipelines](#) to learn about transforming datasets in Foundry.
- Browse the [Source types reference](#) to learn more about the configuration options for each connector type.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Syncs / Set up a streaming sync

Set up a streaming sync

A **sync** is a task that reads specific data from a source and ingests it into Foundry. For example, if you have a relational database source that contains multiple tables, you might configure a sync to ingest a specific table into Foundry.

A streaming sync is similar to a non-streaming (i.e. batch or incremental) [sync](#) but with some differences. The primary difference is that a batch or incremental sync runs periodically while a streaming sync runs consistently to pull data into Foundry with as little latency as possible.

Below, we will discuss the steps required to create a sync :

1. [Define the data](#) to sync from the source.
2. [Define a location](#) in Foundry to send the data.
3. [Configure](#) the streaming sync.
4. [Run](#) the streaming sync.

For this tutorial, we will use a [Kafka](#) source to set up the sync.

Part 1. Define data

First, decide which data you would like to sync into Foundry. Select your [streaming source](#) in Data Connection, then select the available action in the top right corner:

API Reference ↗

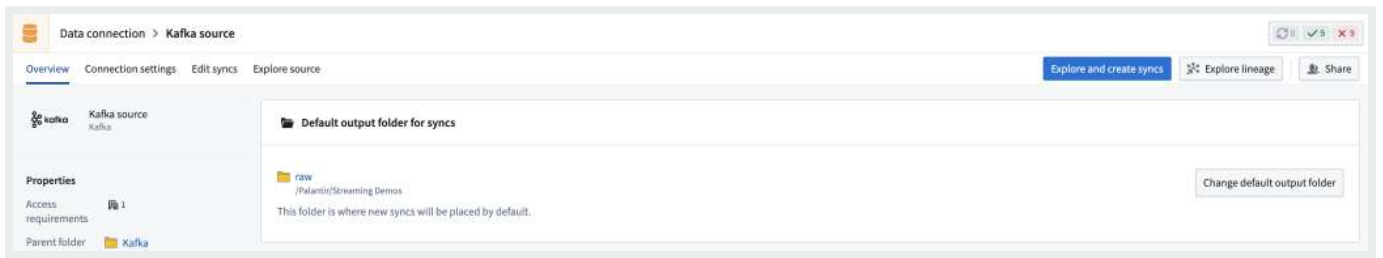
☰

🔍

Streaming syncs: This option appears if your source type supports [source exploration](#), allowing you to explore your data source while creating a sync.

Batch sync: This option appears if your source type does not support [source exploration](#).

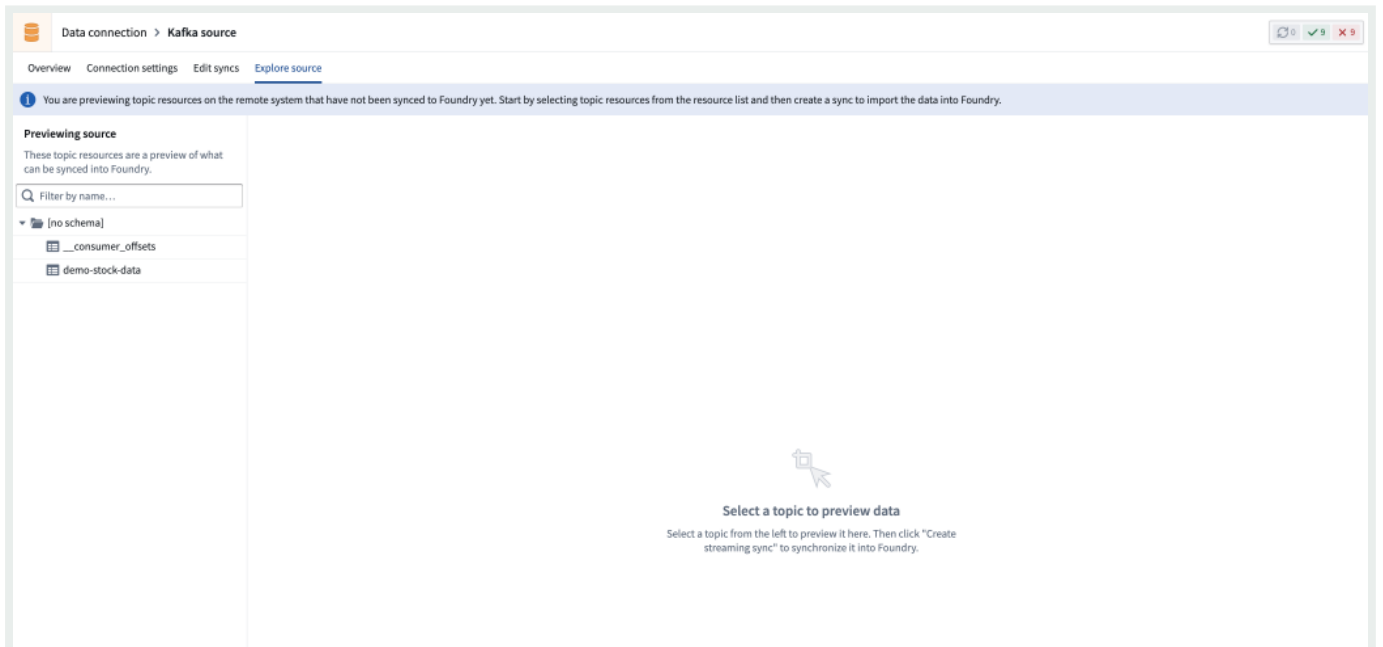
Send feedback



Explore and create syncs

If your source type supports source exploration, you will land on the **Explore source** page in Data Connection that shows data available to sync. The exploration view interface depends on the source type you are using. For example, a Kafka source exploration allows you to see the [topics](#) present on the Kafka broker and preview the data contained in those topics.

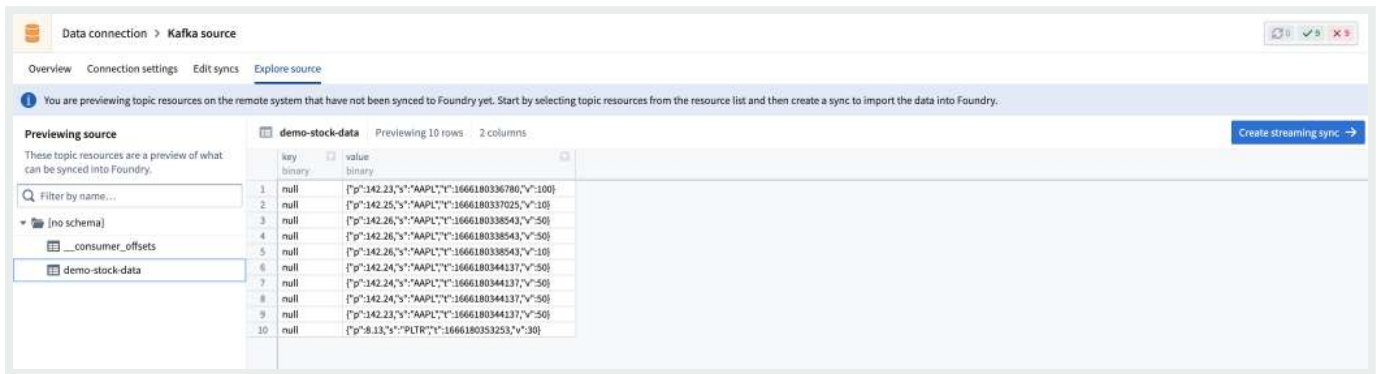
From the Kafka exploration view, you can view existing topics in the list to the left of the page.



Selecting a topic will let you preview a sample of data from that topic.

API Reference ↗

Send feedback



Part 2. Define the sync location

Next, you need to decide where to save your synced dataset in Foundry. The location of your dataset will determine who has permission to access the resulting dataset, based on [Project-level](#) permissions.

We recommend saving a synced dataset next to its source in a Project, allowing them to have the same permissions; matching dataset and source permissions are helpful when creating data pipelines. [Learn more about the recommended Project structure for data pipelines.](#)

Once you choose your sync location, click **Create streaming sync** in the upper right corner.

Part 3. Configure the streaming sync

Now, you will land on the **Sync creation** page in Data Connection where you can define source-specific and core streaming configurations for your sync.

- **Source-specific:** Located at the top of the configuration page, these options depend on your source type and configures the parameters passed to the specific source to which you are connecting.
- **Core streaming:** Located below the source-specific configuration, these options are shared across all streaming syncs. Core configurations include the throughput, schema, and sync location.

API Reference ↗

Send feedback

Data connection > Kafka source > demo-stock-data

demo-stock-data
Syncing data from Kafka source

Create Sync

Extract from...

Topic
demo-stock-data

Start reading from...

☐ Future records

☒ Earliest record

Confirm destination for output streaming dataset

The output of your sync will be saved to: Streaming Demos > Kafka > New demo-stock-data

By default, the output dataset of this sync is placed into the same subfolder as the selected source. You can also choose a different location or sync to an existing dataset.

Select throughput...

Normal
Suitable for up to 5MB/s of throughput

- Row order guaranteed on arrival
- Lowest compute cost
- Suitable for up to 5MB/s of throughput

Recommended

Very high
Best for data over 5MB/s of throughput

- Order not guaranteed
- Higher compute cost
- Best for data over 5MB/s of throughput

10MB/s 15 20 25 30 35 40 45 50

Next, select the [throughput](#) for your stream. The throughput determines the number of partitions that will be created. Selecting a larger number of partitions allows for higher throughput. Selecting a **Normal** throughput will allow up to 5 MB/s for that stream.

Then specify the schema of the input data, by default this is inferred from source, but it can be overwritten if necessary.

[API Reference ↗](#)

[Send feedback](#)

Data connection > Kafka source > demo-stock-data

demo-stock-data
Syncing data from Kafka source

Create Sync →

dataset.

Select throughput...

Normal

Suitable for up to 5MB/s of throughput

- Row order guaranteed on arrival
- Lowest compute cost
- Suitable for up to 5MB/s of throughput

Recommended

Very high

Best for data over 5MB/s of throughput

- Order not guaranteed
- Higher compute cost
- Best for data over 5MB/s of throughput

10MB/s 15 20 25 30 35 40 45 50

Define schema...

Schema inferred
The schema was inferred from a sample of the data

Override inferred schema

Name	Type	Nullable?
value	01 Binary	☐
Add a column...	99 String	☐

SCHEMA JSON

```
{
  "type": "BINARY",
  "nullable": false,
  "name": "value",
  "customMetadata": {}
}
```

Once you configure your sync, select **Create Sync** on the top right.

Now that your sync is created, you will be taken to the **Overview** tab.

Part 4. Run the sync

Now, you are ready to run the sync. Select the **Overview** tab to view a summary of your new sync, including the output dataset, location, and available actions.

Click **Start** to begin running the sync of data from the external stream into Foundry.

[API Reference ↗](#)

[Send feedback](#)

Data connection > Kafka source > demo-stock-data

Overview Configuration

**demo-stock-data**
streaming-kafka-ingest-task

Properties

Output dataset  demo-stock-data

Location  Kafka
/Palantir/Streaming Demos/Docs

Streaming sync
RID ri.magritte..streaming-extract.f0759f 

Source  Kafka source

Actions

 Start

 Delete

To view the stream data, navigate to the stream you configured while creating the sync to view the stream preview page. You should see records flowing from the Kafka topic in the stream.

Streaming De... > Docs > Kafka > demo-stock-data

File Help 1

Actions   9  9 Share 

Live Preview Stream Details Metrics

demo-stock-data 

About Columns Schedules

Enter description...

Location /Palantir/Streaming Demos/Docs/Kafka/de...

Type Streaming dataset (Streaming)

Table size One column

Last archived 2 minutes ago

Updated via demo-stock-data

Show more

Stream

Status  Running Up since a few seconds ago

Throughput Unavailable for raw streams

Tags

Add tags

Health Checks

View details

There are no health checks on this dataset.

Inputs  Explore data lineage

 Kafka source

API Reference ↗

	value
	Binary
1	[{"p":170.8,"s":"SNOW","t":1666185045003,"v":1}]
2	[{"p":141.88,"s":"AAPL","t":1666185043518,"v":5}]
3	[{"p":141.88,"s":"AAPL","t":1666185043518,"v":1}]
4	[{"p":141.88,"s":"AAPL","t":1666185043517,"v":50}]
5	[{"p":101.28,"s":"GOOG","t":1666185043387,"v":34}]
6	[{"p":141.88,"s":"AAPL","t":1666185043299,"v":1}]
7	[{"p":141.85,"s":"AAPL","t":1666185042872,"v":50}]
8	[{"p":8.04,"s":"PLTR","t":1666185042293,"v":20}]
9	[{"p":101.22,"s":"GOOG","t":1666185041778,"v":1000}]
10	[{"p":101.31,"s":"GOOG","t":1666185040069,"v":8}]
11	[{"p":141.89,"s":"AAPL","t":1666185039863,"v":1}]
12	[{"p":8.04,"s":"PLTR","t":1666185039788,"v":100}]
13	[{"p":141.85,"s":"AAPL","t":1666185039630,"v":50}]
14	[{"p":141.85,"s":"AAPL","t":1666185039630,"v":68}]
15	[{"p":141.85,"s":"AAPL","t":1666185039630,"v":15}]
16	[{"p":141.85,"s":"AAPL","t":1666185039433,"v":1}]
17	[{"p":101.265,"s":"GOOG","t":1666185038287,"v":1}]
18	[{"p":141.84,"s":"AAPL","t":1666185037041,"v":1}]
19	[{"p":8.04,"s":"PLTR","t":1666185036967,"v":900}]
20	[{"p":8.04,"s":"PLTR","t":1666185036967,"v":100}]
21	[{"p":170.8,"s":"SNOW","t":1666185033555,"v":19}]
22	[{"p":171,"s":"SNOW","t":1666185033555,"v":1}]
23	[{"p":141.82,"s":"AAPL","t":1666185033541,"v":100}]
24	[{"p":8.04,"s":"PLTR","t":1666185033002,"v":900}]
25	[{"p":8.04,"s":"PLTR","t":1666185033002,"v":100}]
26	[{"p":141.81,"s":"AAPL","t":1666185031597,"v":100}]
27	[{"p":8.04,"s":"PLTR","t":1666185031319,"v":100}]
28	[{"p":8.04,"s":"PLTR","t":1666185031318,"v":20}]
29	[{"p":8.04,"s":"PLTR","t":1666185031318,"v":1}]
30	[{"p":8.04,"s":"PLTR","t":1666185031318,"v":1}]
31	[{"p":8.04,"s":"PLTR","t":1666185031318,"v":1250}]
32	[{"p":8.04,"s":"PLTR","t":1666185031318,"v":100}]
33	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":394}]
34	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":200}]
35	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":800}]
36	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":6}]
37	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":200}]
38	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":240}]
39	[{"p":8.04,"s":"PLTR","t":1666185031317,"v":100}]

Send feedback

Next steps

Now that you have successfully run a sync, learn how to [debug a failing stream](#), [push data into a stream](#) with push-based ingestions, or [integrate your stream with the Ontology](#).

PREVIOUS
← Set up a sync

NEXT
File-based syncs →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

File-based syncs

After creating a file-based sync using [exploration](#), you can update the configuration in the **Configurations** tab of the sync page.

Conceptual file-based ingestion modes

While Foundry file-based syncs offer low-level settings for greater flexibility and configuration, most use cases will follow a known mode. The following table documents known modes and the low-level settings required to achieve the desired behavior, as well as settings that could be contradictory with those modes.

Batch mirror with **SNAPSHOT** (default)

- **Transaction type:** **SNAPSHOT**
- **Filters:** **None**

Each run will [ingest all files](#) nested in the external system's subdirectory, including files ingested in previous runs, and commit a **SNAPSHOT** transaction to the output dataset containing exactly those files. The output Foundry [dataset view](#) will contain a single **SNAPSHOT** transaction containing all files.

Contradictory settings

- **Filters:** **Exclude files already synced**

API Reference ↗ [trailing window ingestion mode](#).

☰

🔍

Limit number of files

Send feedback

- Results in the output Foundry dataset view containing only a non-deterministic subset of the desired files if the limit is lower than the total number of available files.
- **Filters:** At least N files
 - If there are not N nested files in the specified subfolder of the external system, this setting will yield an empty transaction and result in 0 files being ingested. Otherwise, this setting has no effect.

Incremental mirror with APPEND

- **Transaction type:** APPEND
- **Filters:** Exclude files already synced

The output [dataset view](#) will contain a collection of [APPEND](#) transactions, which in aggregate contain all nested files ever present during any job run from the specified subfolder. Each run will ingest [all files that have not yet been ingested](#), keyed by file path name, and commit an [APPEND](#) transaction to the output dataset.

Contradictory settings

- **Filters:** Exclude files already synced with the [Last modified date](#) or [File size](#) option
 - These options would attempt to incorrectly re-ingest existing files, keyed by file path name, in an [APPEND](#) transaction when their [Last modified date](#) or [File size](#) change, respectively. To allow updates to existing files, review the [incremental with UPDATE](#) ingestion mode.

Incremental mirror with UPDATE

- **Transaction type:** UPDATE
- **Filters:** Exclude files already synced
- One or both of:
 - **Filters:** Exclude files already synced with the [Last modified date](#) option

[API Reference ↗](#) Exclude files already synced with the [File size](#) option

The output [dataset view](#) will contain a collection of [UPDATE](#) transactions that contain the latest version of all nested files ever present during any job run from the

Send feedback

specified subfolder. Each run will ingest all files that have not yet been ingested or have since changed, keyed by file path name, and commit an `UPDATE` transaction to the output dataset.

Caveat

Only use this mode if modifications to existing files are a non-negotiable behavior of the external system. While ingestion is incremental in the sense that only files that are new or changed are ingested in a given run, downstream pipelines cannot run incrementally, as the output dataset (input to the downstream pipelines) is not [append-only](#).

Trailing window with `SNAPSHOT`

- **Transaction type:** `SNAPSHOT`
- **Filters:** `Exclude files already synced`

The output [dataset view](#) will contain a single `SNAPSHOT` transaction containing only files **that were never present in any previous job run**. Each run will ingest all files that have not yet been ingested, keyed by file path name, and commit a `SNAPSHOT` transaction to the output dataset, containing exactly those files.

This mode is useful when only "recent" files (files that were created in the external system between the second-to-last and last run) are relevant to downstream pipelines and operations. Files ingested in previous runs will not be visible in the output dataset view.

Contradictory settings

- **Filters:** `Limit number of files`
 - When the number of files created in the external system during a given window exceeds the specified limit, a non-deterministic subset of those files will be ingested, and the remainder will be deferred to a subsequent window. This number may grow rapidly over time, destroying the "recency" intended in the output dataset.

API Reference ↗ to specify the subfolder and optional regex, in addition to filters that limit the file types desired in the output. Such filters include `Last modified < timestamp` to exclude outdated files or `Path does not match` to exclude files with a certain extension, such as `.sh` executable files.

Send feedback

Only the `Exclude files already synced`, `At least N files`, and `Limit number of files` filters are tightly coupled to the desired sync mode and might interfere with it.

Sync failure behavior

As with all [batch syncs](#), file-based syncs are transactional. If a sync fails at any point, the transaction is aborted and none of the files from that run are committed to the dataset.

For example, if a sync is configured to upload one million files and a network error occurs after uploading all but one file, the entire transaction is aborted. None of the uploaded files are committed and the dataset view is not modified.

Because of this behavior, syncs that process a large number of files in a single run are more vulnerable to transient failures. To reduce the impact of sync failures, consider using [incremental APPEND syncs](#) with a file limit filter so that each run processes a smaller batch of files. This way, a failure only requires re-syncing the current batch rather than all files.

Configure file-based syncs

Configuration options for file-based syncs include the following:

Parameter	Required?	Default	Description
Subfolder	Yes		Specify the location of files within the connector that will be synced into Foundry.
Filters	No		Apply filters to limit the files synced into Foundry.
Transformers	No		Apply transformers to data before it is synced into Foundry.
API Reference ↗ strategies [Legacy]	No		Enable to delete files and/or empty parent directories Send feedback sync. Requires write permission on the

Parameter	Required?	Default	Description
			source filesystem. Completion strategies are read-only and cannot be configured on new syncs.

 Syncs will include all nested files and folders from the specified subfolder.

Filters

Filters allow you to filter source files before they are imported into Foundry. The supported filter types are:

- **Exclude files already synced:** Only sync files that were added or modified in size or date since the last sync.

 `Exclude files already synced` has known scale limitations, as it requires scanning all files on every sync run. For syncs ingesting a large number of files, we recommend splitting the sync into a `SNAPSHOT` sync for historical data and an `INCREMENTAL` sync with `Exclude files already synced`, so that the filter is only applied to a smaller subset of recent files.

- **Path matches:** Only sync files with a path (relative to the root of the connector) that matches the regular expression.
- **Path does not match:** Only sync files with a path (relative to the root of the connector) that does not match the regular expression.
- **Last modified after:** Only sync files that have been modified after a specified date and time.
- **File size is between:** Only sync files with a size between the specified minimum and maximum byte value.
- **Any file has path matching:** If any file has a relative path matching the regular expression, sync all files in the subfolder that are not otherwise filtered.
- **At least N files:** Sync all filtered files only if there are at least N files remaining.

API Reference ↗

- **Limit number of files:** Limit the number of files to keep per transaction to increase the reliability of incremental syncs.

Send feedback

Transformers

Transformers allow you to perform basic file transformations (compression or decryption, for example) before uploading to Foundry. During a sync, the files chosen for ingest will be modified per the chosen transformer.

 Rather than using Data Connection transformers, we recommend performing data transformations in Foundry with [Pipeline Builder](#) and [Code Repositories](#) to benefit from provenance and branching.

The following transformers are supported in Data Connection:

- Compress with Gzip
- Concatenate multiple files
 - Join multiple files into a single file.
- Rename files
 - Replace all occurrences of a given filename substring with a new substring.
 - Drop the directory path from the filename by replacing `^(.*)/` with `/`.
- Decrypt with PGP
 - Decrypt files that have been encrypted with PGP encryption.
 - Requires that the agent system has PGP keys configured.
 - Unavailable for syncs running on a Foundry worker.
- Append timestamp to filenames
 - Add a timestamp in a custom format to the filename of each file ingested.

Completion strategies

 Completion strategies are in the [legacy](#) phase of development, and no additional development is expected. Completion strategies are read-only on existing syncs and cannot be configured on new syncs. We recommend using a downstream [ORM](#) as an alternative.

API Reference ↗

Send feedback

Completion strategies are a mechanism to attempt deleting files and empty parent directories after a successful batch sync of those files into a Foundry dataset. Completion strategies are read-only and cannot be configured on new syncs. Existing syncs with completion strategies will continue to function, but the configuration cannot be modified.

This workflow may be useful when data is synced from an intermediate staging area, such as a file storage system, rather than directly from the external system of record. This workflow is not recommended. Foundry should be connected directly to the external system of record. In cases where Foundry is the system of record, data should be pushed directly to Foundry rather than an intermediate staging area.

Limitations of completion strategies and alternatives

Completion strategies are subject to several important limitations and caveats. These limitations and potential mitigations or alternatives are described below.

Completion strategy support

Completion strategies are in the [legacy](#) phase of development, and no additional development is expected. Completion strategies are read-only on existing syncs and cannot be configured on new syncs. Completion strategies were only available for sources using an [agent worker](#). We recommend implementing the functionality provided by completion strategies as a downstream transform instead.

As an example, assume you have a [direct connection](#) to an S3 bucket containing the files `foo.txt` and `bar.txt`. You want to use a file batch sync to copy them to a dataset, and then delete the files from S3. Instead of using completion strategies, you should do the following:

- Configure a batch sync without any completion strategies and schedule it to run.
- Write a downstream [external transform](#) job which is scheduled to run when the sync output dataset is updated, taking the synced data as an input.
- In that external transform, write Python transforms code to iterate through the files that have appeared in the synced dataset, and make calls to S3 to delete those files from the bucket

[API Reference ↗](#)

Note that this approach is retryable if any deletion calls fail, and guarantees that data is successfully committed to Foundry before attempting to perform any deletions. This approach is also compatible with incremental file batch syncs.

[Send feedback](#)

Completion strategies are best effort

For existing syncs using completion strategies, note that completion strategies are best effort. This means that they do not guarantee that data will be effectively removed. The following are some situations that may cause completion strategies to fail:

1. Completion strategies will not be retried if the agent worker crashes or is restarted after the batch sync commits data to Foundry, but before the completion strategies run.
2. If the credentials used to connect do not have write permissions, the batch sync may successfully read data and commit to Foundry, but fail to perform the deletions specified by the completion strategy.

In general, we recommend using an alternative to completion strategies wherever possible. Custom completion strategies are no longer supported.

For directory sources specifically, if you require guaranteed file deletion after ingestion, consider using an [external transform to delete files via SSH](#) instead.

Optimize file-based syncs

Warning

This guide is recommended for users setting up a new sync or troubleshooting a slow or unreliable sync. If your sync is already working reliably, you do not need to take any action.

Syncing a large number of files into a single dataset can be slow and vulnerable to transient failures. As described in [sync failure behavior](#), a failure at any point aborts the entire transaction, which means larger syncs risk more lost work.

If the dataset grows over time, the time to sync the data as a `SNAPSHOT` increases because `SNAPSHOT` transactions sync all data into Foundry. Instead, use syncs configured with `transaction_type APPEND` to import your data incrementally. Since you will be syncing smaller chunks of data, you will create an effective checkpoint; a sync failure will result in a minimal amount of duplicated work rather than requiring a full sync. Additionally, your dataset syncs will run faster as you no longer need to sync all data for every sync.

[API Reference](#) ↗

[Send feedback](#)

Configure incremental APPEND syncs

APPEND transactions require additional configuration to run successfully.

By default, files synced into Foundry are not filtered. However, APPEND syncs require filters to prevent the same files from being imported. We recommend using the Exclude files already synced and Limit number of files filters to control how many files get imported into Foundry in a single sync. Finally, [schedule your sync](#) to remain up to date with your source system.

Data connectivity & integration / Data Connection / Syncs / Set up a media set sync

Media set syncs

This page discusses how to set up a media set source and sync into Foundry via Data Connection.

The following source types support media syncs:

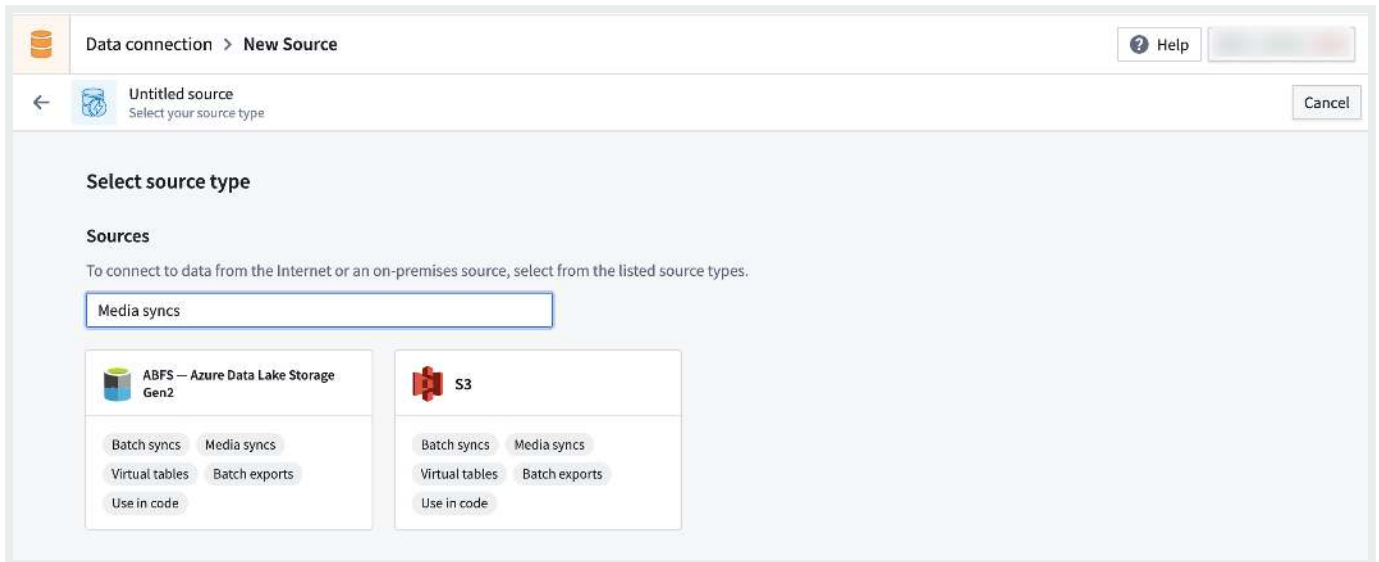
- [Amazon S3](#)
- [OneLake and Azure Blob Filesystem \(ABFS\)](#)

A growing list of sources support media syncs. However, if your desired file-based source is not yet supported, you can ingest your files within a dataset and convert them into media sets via [Python transforms](#). For example, to ingest files from SharePoint Online into a media set, you can use the [SharePoint Online connector](#) to sync files into a Foundry dataset, then create a Python transform that reads those files and writes them into a media set. The [SMB connector](#) documentation includes a similar example.

Set up a media set source and sync

1. Find a supported source by navigating to the **Source** page via **+ New Source**. Then, search for **Media Sync** to find all supported sources.

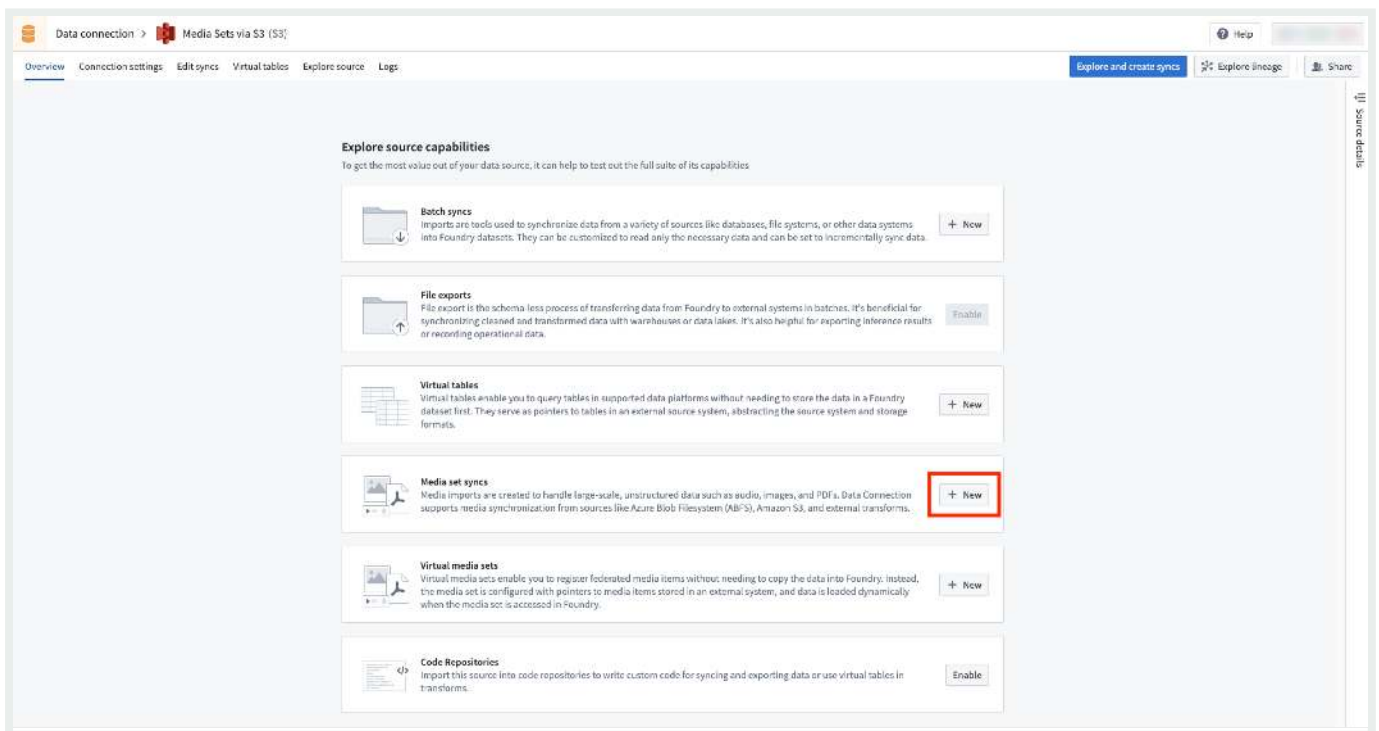
[API Reference ↗](#)[Send feedback](#)



2. Ensure you have permissions to import any necessary network policies and then set up supported source using the appropriate instructions below:

- [Amazon S3](#)
- [OneLake and Azure Blob Filesystem \(ABFS\)](#)

3. In the **Overview** page of the source, find the **Media set syncs** section to create a media set sync.



4. [API Reference](#) set sync by selecting the desired media file types. See [supported media set schemas](#).

[Send feedback](#)

Data Connection > Media Sets via S3 > NYT PDFs

NYT PDFs
Syncing data from Media Sets via S3

Save media set syn

Media set creation

Sync configuration

Media set creation

Confirm destination for output media set

master

media ... > r. > NYT PDFs

New

You can manage the default output location for syncs on the [source overview page](#). You can also choose a [different location](#).

Image
PNG, BMP, JP2K, JPG, NITF, TII

Document
PDF

Audio
WAV, FLAC, MP3, MP4, NISTSPI

Video
MP4, MOV, TS, MKV

DICOM
DCM

Spreadsheet
XLSX

Multimodal
No format constraints

5. Create the desired build schedule for your media sync ingest. You can edit the schedule after the initial configuration.

Data connection > Media Sets via S3 > NYT PDFs

NYT PDFs
Syncing data from Media Sets via S3

Save media set syn

Media set creation

Sync configuration

Retention policy

Media items will only be accessible for the configured retention window, after which they will be permanently deleted. [Learn more](#).

☐

Sync configuration

Schedule

☒
Run this media sync automatically on a schedule

Run every

1

Day

At

12 : 00

AM

New York (EDT) -04:00

6. Select the relevant subfolder within your source. If your media files are at the root path, [API Reference](#) to add a subfolder configuration.

Send feedback

 **Subfolder**

A subfolder within the source. If a subfolder is not specified, the sync will use the source root.

 ▼

7. Set up your sync filters. Available sync filters include **Exclude files already synced**, **Path matches**, **File size limit**, and **Ignore items not matching schema**.

 **Filters**

Exclude files already synced

If enabled, do not re-ingest files with a path that already exists in the media set.

☐

Path matches

Only sync files whose path (relative to the root of the source) matches the regular expression.

☐

File size limit

If specified, only upload items whose size does not exceed this limit.

☐

Ignore items not matching schema

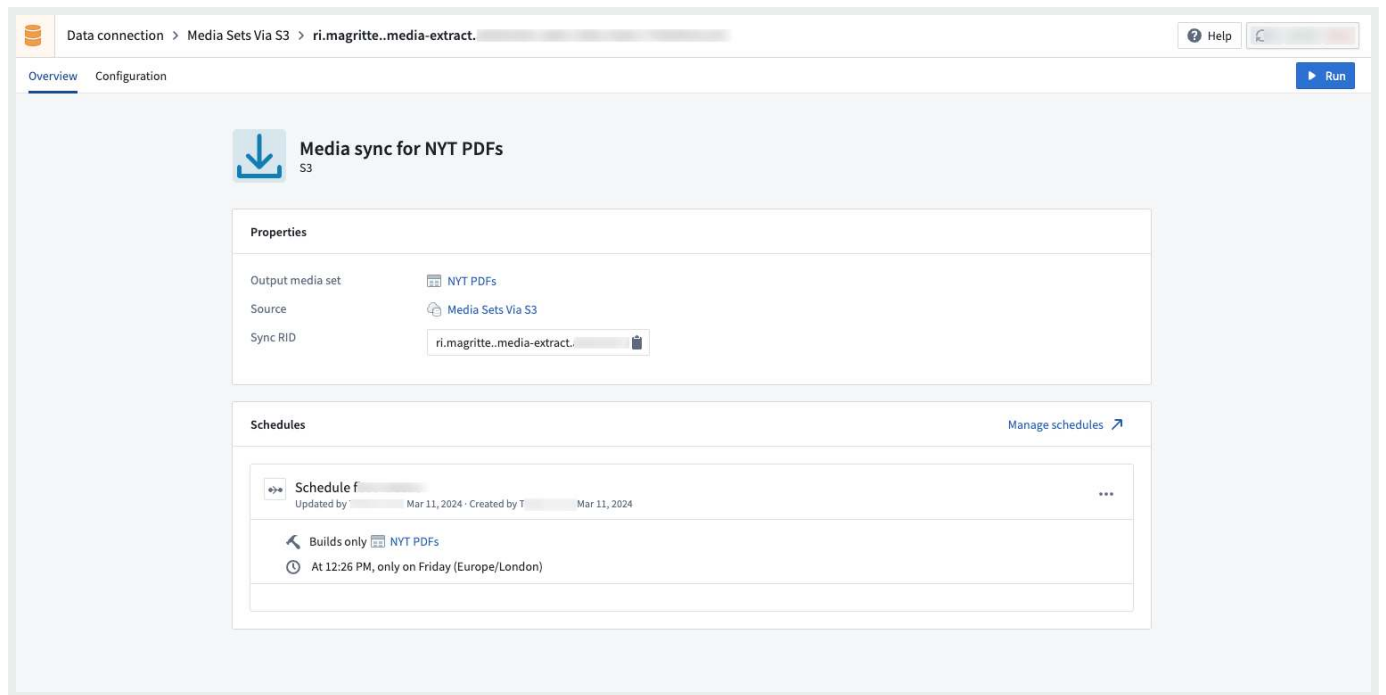
If true, any items which do not match the schema will simply be ignored. If false, items not matching the schema will cause the sync to fail.

☐

8. Choose **Save media set sync** when you have selected your initial configuration.
9. Select **Run** to trigger your first sync and view your media sync.

[API Reference ↗](#)

[Send feedback](#)



Once you have set up your media set sync, learn how to leverage your media set with [transforms in Pipeline Builder](#).

PREVIOUS
← [File-based syncs](#)

NEXT
[Optimize JDBC syncs](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Optimize JDBC syncs

This guide provides tips to improve the speed and reliability of JDBC syncs.

⚠ Warning

If your sync is already working reliably, there is no need to take the actions described below. If you are setting up a new sync or your sync takes too long to complete or does not complete reliably, we recommend following this guide.

There are two primary methods for speeding up JDBC syncs. We recommend starting by making your sync incremental and only moving on to parallelizing the SQL query if the incremental sync is insufficient:

- [Make your sync incremental](#)
- [Parallelize the SQL query](#)

Incremental syncs

By default, batch syncs will sync all matching rows from the target table. *Incremental syncs*, by contrast, maintain state about the most recent sync and thus can be used to ingest only *new* matching rows from the target. This can improve sync performance dramatically for tables with a large number of rows. Incremental syncs work by adding data as an [APPEND transaction](#) to the synced dataset.

By [API Reference ↗](#) configuration for an incremental batch sync:



Send feedback

SQL Query

1 SELECT * FROM purchases WHERE timestamp > ?

[Learn how to optimize this sync](#)

Incremental

Disable

Incremental syncs are stateful syncs enabling append-style transactions from the same table without duplication of data. You must provide a monotonically increasing column such as a timestamp or id and an initial value for this column. An incremental sync will import rows where the value is greater than the largest already imported.

timestamp

< January 1970 >

Su	Mo	Tu	We	Th	Fr	Sa
28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

1 : 00 : 00 . 001

Perform the following steps to set up an incremental batch sync:

1. Navigate to the configuration page for the sync you want to convert, and ensure that the preview is working.
2. Set the transaction type to `APPEND`. This is necessary to avoid overwriting rows from previous syncs.
3. Select **Enable** in the **Incremental** box. Ensure that the preview has successfully run on this sync. With a working preview, the **Incremental** box will expand to allow you to configure the initial incremental state for the sync.
4. Configure the sync's incremental state. This state consists of an **incremental column** and an **initial value**, which can be configured in the user interface. Keep the following

API Reference ↗

Considerations in mind when setting these values:

- the **incremental column** must be *strictly increasing* between syncs. If your rows are *immutable* (i.e., existing rows cannot be updated in place), any incrementing column (e.g., an auto-incrementing ID, or a timestamp indicating when

Send feedback

the row was added) will be sufficient. If your rows are *mutable* (i.e., your table allows existing rows to be updated, as opposed to only allowing new rows to be inserted), you'll need a column that increases with every mutation of the data (e.g. an `update_time` column).

- To avoid ingesting rows more than once, the **initial value** of the incremental column must be greater than that of *all* rows synced in previous runs. For example, if the most recent `SNAPSHOT` sync brought in rows with values of an integer `id` column ranging up to `1999`, you could set the initial value to `2000`.

i When you ingest an updated version of an existing row, the Foundry dataset will still include previous versions of the row (remember, we're using the `APPEND` transaction type). If you want only the *latest* version of each row, you will need to use another tool in Foundry, such as Transforms, to clean the data. Refer to the guidance on [incremental pipelines](#) to learn more.

5. Finally, update the query to use the wildcard symbol `?`. Exactly how you include the wildcard in the query depends on your query logic; see below for a simple example, and note the following:

- In the first incremental run, this wildcard will be replaced by the **initial value** we specified in the previous step.
- In any subsequent run, the wildcard will be replaced with the *maximum* synced value of the **incremental column** from the previous run.

i As mentioned above, the incremental state interface only works if a preview of the sync has run successfully. This means that if you are creating an incremental sync from scratch or duplicating an existing incremental sync, you will need to run a preview **without the wildcard `?` operator in your query**.

Example

Suppose you are ingesting a table called `employees`, with transaction type set to `SNAPSHOT`.
a [API Reference ↗](#) Example SQL query:

Send feedback

```
1 SELECT
2   *
3 FROM
4   employees
```

At time `T1`, the table looks as follows:

id	name	surname	update_time	insert_time
1	Jane	Smith	1478862205	1478862205
2	Erika	Mustermann	1478862246	1478862246

And suppose this table is mutable, so that at a later time, `T2`, it looks like this:

id	name	surname	update_time	insert_time
1	Jane	Doe	1478862452	1478862205
2	Erika	Mustermann	1478862246	1478862246
3	Juan	Perez	1478862438	1478862438

We want to convert this sync to be incremental, so we update the transaction type to `APPEND`.

What should we use as the incremental column? It's important to note that **neither the `id` nor `insert_time` columns are appropriate to use as the incremental column** because they will miss updates, like the change in the `surname` column of the `Jane` row. Instead, we should use `update_time` as the incremental column.

What we choose for the initial value depends on whether or not we've previously synced rows from this table. Supposing that we ran a `SNAPSHOT` sync at time `T1` and have already synced rows with values of `update_time` as high as `1478862246`; we should use `1478862247` as our initial value to avoid duplicates. If we never synced any rows from this table, we could use `0` (or `01/01/1970` if setting a date) as the initial value.

For API Reference ↗ the SQL query to

Send feedback

```
1 SELECT
2     *
3 FROM
4     employees
5 WHERE
6     update_time > ?
```

The conversion is now complete. Note that after running the sync incrementally, we will have multiple `Jane` rows in our dataset (one for each update). As mentioned previously, we'll have to handle these duplicates in our downstream logic—in Contour or Transforms, for example.

If you run into issues with incremental JDBC syncs, [this section](#) of the troubleshooting guide may be helpful.

Parallelize the SQL query

⚠ Warning

Because the parallel feature runs separate queries against the target database, carefully consider the case of live-updating tables being treated differently by slightly differently-timed queries.

The parallel feature allows you to easily split the SQL query into multiple smaller queries that will be executed in parallel by the agent.

In order to achieve this behavior you need to change your SQL statement to this structure:

```
1 SELECT
2     /* FORCED_PARALLELISM_COLUMN({{column}}), FORCED_PARALLELISM_SIZE({{size}})
3     column1,
4     column2
5     ...
6     {{table_name}}
7 WHERE
```

API Reference ↗

Send feedback

```
8      {{condition}}
9      /* ALREADY_HAS_WHERE_CLAUSE(TRUE) */
```

The key parts of the query are:

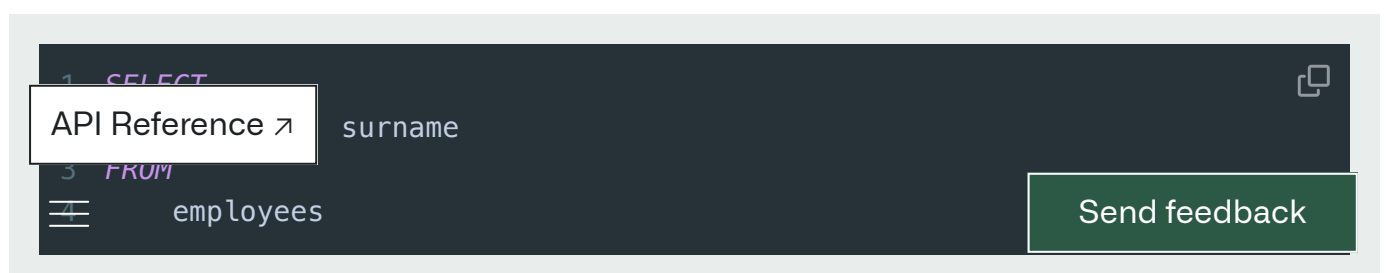
- `FORCED_PARALLELISM_COLUMN({{column}})`
 - This specifies the column on which the table will be divided.
 - It should be a numeric column (or a column expression that yields a numeric column) with a distribution as *even as possible*.
- `FORCED_PARALLELISM_SIZE({{size}})`
 - Specifies the degree of parallelism, e.g. `4` would result in five simultaneous queries: four which split up the values for the specified parallelism column, plus a query for NULL values in the parallelism column.
- `ALREADY_HAS_WHERE_CLAUSE(TRUE)`
 - This specifies if there is already a `WHERE` clause or if one needs to be generated. If this is `FALSE`, `WHERE column%size = X` will be added to each of the generated queries. If this is `TRUE`, this condition will instead be appended with an `AND`.

Example

Suppose you are syncing a table called `employees` that contains the following data:

id	name	surname
1	Jane	Smith
2	Erika	Mustermann
3	Juan	Perez
NULL	Mary	Watts

The basic query will look like this:



This will execute a single query in the database and attempt to retrieve all records from the table.

To leverage the parallel mechanism the query can be changed to the following:

```
1 SELECT
2     /* FORCED_PARALLELISM_COLUMN(id), FORCED_PARALLELISM_SIZE(2) */
3     id, name, surname
4 FROM
5     employees
6 /* ALREADY_HAS_WHERE_CLAUSE(FALSE) */
```

This will execute the following three queries in parallel:

```
1 SELECT
2     id, name, surname
3 FROM
4     employees
5 WHERE
6     id % 2 = 1
```

Extracting:

id	name	surname
1	Jane	Smith
3	Juan	Perez

and

```
1 SELECT
2     id, name, surname
3 FROM
```

API Reference ↗

```
6     id % 2 = 0
```

Send feedback

Extracting:

id	name	surname
2	Erika	Mustermann

and

```
1 SELECT
2     id, name, surname
3 FROM
4     employees
5 WHERE
6     id % 2 IS NULL
```

Extracting:

id	name	surname
NULL	Mary	Watts

Parallelisms with a WHERE clause that contains an OR condition

When using parallelism with a WHERE clause that contains an OR condition, you should wrap conditions in parentheses to indicate how the conditions should be evaluated. For instance, examine the sync provided below:

```
1 SELECT /* FORCED_PARALLELISM_COLUMN(col1), FORCED_PARALLELISM_SIZE(32) */
2 col1,
3 col2
4 FROM tbl
5 WHERE
6 condition1 = TRUE OR condition2 = TRUE
7 AS WHERE_CLAUSE(TRUE) */
```

API Reference ↗

Send feedback

This example sync will be transformed to the following:

```
1 condition1 = TRUE OR condition2 = TRUE AND col1 % X = 0
```



However, that statement may be logically interpreted as `condition1 = TRUE OR (condition2 = TRUE AND col1 % X = 0)`, rather than the desired `(condition1 = TRUE OR condition2 = TRUE) AND col1 % X = 0`. You can ensure the intended interpretation by wrapping the entire WHERE clause in parentheses. For the example above, this would mean:

```
1 SELECT /* FORCED_PARALLELISM_COLUMN(col1), FORCED_PARALLELISM_SIZE(32) */  
2 col1,  
3 col2  
4 FROM tbl  
5 WHERE  
6 (condition1 = TRUE OR condition2 = TRUE)  
7 /* ALREADY_HAS_WHERE_CLAUSE(TRUE) */
```

PREVIOUS

← Set up a media set sync

NEXT

Troubleshooting reference →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Syncs / Troubleshooting reference

Troubleshooting reference

This page describes several common issues with Syncs with steps to debug:

- [PKIX and SSL Exceptions](#)
- [Egress Proxy Issues](#)
- [Incremental JDBC Syncs](#)
- [Intermittent sync failures or hangs](#)
- [A sync started failing](#)
- [Unexpected data was synced](#)

PKIX and SSL exceptions

For certificate and TLS troubleshooting, including `PKIX` and `SSLHandshakeException` errors, see [Certificates and TLS](#).

If your sync fails with the error `Response 421 received. Server closed connection`, this suggests you may be connecting with an unsupported SSL protocol / port combination. An example includes implicit FTPS over port 991, which is an outdated and unsupported standard. Explicit SSL over port 21 is the preferred method in this case.

Egress proxy issues

API Reference ↗



Send feedback

If your sync is an FTP/S sync, ensure that you are not using an egress proxy load balancer. FTP is a stateful protocol, so using a load balancer can cause the sync to fail if sequential requests don't originate from the same IP.

Note that due to the nature of load balancing, failures will be non-deterministic; syncs and previews may sometimes succeed, even with the load-balancing proxy in place.

Additionally, HTTP proxies do not support active mode transfers for FTP and FTPS. Active mode requires the FTP server to connect back to the client, which is not possible through an HTTP proxy. When using HTTP proxies with FTP/FTPS connections, only passive mode transfers are supported. Ensure your FTP source is configured to use passive mode when connecting through an HTTP proxy.

S3 syncs

If your sync or exploration is failing with the error

`com.amazonaws.services.s3.model.AmazonS3Exception:Forbidden (Service: Amazon S3; Status Code: 403; Error Code: 403 Forbidden; Request ID: null; S3 Extended Request ID: null)`, this means that the command is hitting an error on going through the egress proxy. If you receive this error, you should check whether any of the following scenarios are applicable:

- The Region field is empty. This is required when the S3 bucket you are connecting to is in a different region than the proxy.
- STS is unreachable due to not being allowlisted.
- The S3 URL is unreachable due to not being allowlisted.
- The STS credentials are invalid or you are unable to assume the IAM role.
- The sync needs to use VPC instead of proxy; to resolve the S3 endpoint, address(es) must be excluded from the proxy by adding the following configurations:
 - To the S3 source proxyConfiguration block, add:

```
1 host: <address of deployment gateway or egress NLB>
2 port: 8888
3 protocol: http
4 proxyHosts: <bucket>.s3.<region>.amazonaws.com,s3.<region>.amazonaws.com
```

API Reference ↗

Send feedback

e.g: Allowlisting all VPC buckets would involve a config addition of:

```
1 clientConfiguration:
2     proxyConfiguration:
3         host: <color>-egress-proxy.palantirfoundry.com
4         port: 8888
5         protocol: http
6         nonProxyHosts: *.s3.<region>.amazonaws.com, s3.<region>.amaz
```

If your sync or exploration is failing with the error `Could not list objects in bucket`, check whether both the region and signing region are configured. Setting both values can cause this error; configure only the signing region to resolve the issue.

Incremental JDBC sync issues

To see the exact query that ran against your source system, refer to `_data-ingestion.log`.

If your sync is an incremental sync, ensure you have provided a monotonically increasing column (e.g. timestamp or id) and an initial value for this column.

Once you've chosen the incremental column, you need to make sure you have added the `?` operator to the `SQL` query in the sync configuration page (the `?` is replaced with the 'incremental' value and only a single `?` may be used). For example, `SELECT * FROM table WHERE id > ?`.

Missing rows and rows not being updated

If you believe there are rows missing from your synced dataset or that previously synced rows aren't being properly updated, check the following:

- If new rows are added to the existing dataset, and the value stored in the incremental column for the new row(s) is less than the current cursor, the new rows will not be synced.
 - For example, if you're using `ID` as your monotonically increasing column and the last `ID` value synced in the last sync was 10, and then you add a row with `ID` 5, that row

[API Reference ↗](#)

- If you have rows that have not been synced due to the above, you can resync rows by either:

[Send feedback](#)

- Performing a snapshot sync rather than an incremental sync, or
- Adjusting your query to target the missing rows, and run it as a one-off.

Duplicate rows

If you believe existing rows are being re-synced, check the following:

- If existing rows in the source database are updated and the incremental column for those row(s) is changed such that it becomes greater than the current cursor, those rows will be re-synced, and thus duplicated. For example, if the column you are using as the incremental column is a timestamp, representing when the row was inserted or last updated, and you update a row between dataset syncs, that updated row will be re-synced.
 - If you have duplicate rows present due to the above, you can remove them by either:
 - Performing a snapshot sync rather than an incremental sync, or
 - Using a downstream transform that removes duplicate rows.
- If the data type of your chosen incremental column is a timestamp and it uses sub-millisecond precision, duplicate rows will be re-synced. This is because currently incremental JDBC syncs only serialize timestamp values up to millisecond precision, and the incremental value is then always rounded down to the nearest millisecond. Therefore, rows with microsecond and/or nanosecond precision will always be re-synced because the comparison against the current (rounded-down) incremental value is always "positive".
- If you have duplicate rows present due to the above you will need to cast the incremental column to either a `LONG` or a `STRING` (in ISO-8601 format).

NullPointerException thrown on incremental sync

If a `NullPointerException` is thrown on your incremental sync, this may indicate that the SQL query is retrieving rows from the database that would cause the incremental column to contain null values.

- For example, take the query `SELECT * FROM table WHERE col > ? OR timestamp > 1`, where `col` is the incremental column being used for the sync. The use of `OR` means that `col` does not guarantee that `col` only contains non-null values. If a null value for any row, then the sync will fail upon Data Connection attempting to update the incremental state for the sync since the current state value is null. To avoid this, you should use `col > ? AND timestamp > 1` to ensure that the synced null value and throw an error.

API Reference ↗

Send feedback

- To remediate situations like these, either choose a different incremental column or ensure that no null values can be synced for the current incremental column. For the query above, we could avoid the errors with a rewrite like `SELECT * FROM table WHERE (col > ? OR timestamp > 1) AND col IS NOT NULL`.

i If you wish to change the incremental column used for your sync, we recommend that you create a new sync.

Intermittent sync failures or hangs

Check if your agent is running out of resources

On the agent host, in the `<bootvisor-directory>/var/data/processes` directory, run `ls -lrt` to find the most recently created `bootstrapper~<uuid>` directory.

- `cd` into that directory and navigate to `/var/log/`.
- Review the contents of the `magritte-agent-output.log`.

If you see the error `OutOfMemory Exception`, it means that the agent cannot handle the workload being assigned to it.

- To fix this, you may need to increase the "agent heap size" parameter, which can be done in the agent overview page. However, before doing so we recommend you read the instructions for [Tuning Heap Sizes](#) in the agent troubleshooting reference guide.
- If you cannot increase the agent's heap size, you may need to reduce the "Maximum concurrent syncs" parameter. This can also be done in the agent overview page.

Hanging syncs

Below are some common causes of hanging syncs and their associated fixes:

All syncs: Hanging during the fetching stage

API Reference ↗

If your sync is hanging during the fetching stage, check if the source is operational:

Send feedback

- To check whether the source is available, try to connect and interact with the source (without using your agent or other Foundry products). If you are able to connect successfully and queries run as expected, contact your Palantir representative for further assistance.
- If you find that you are unable to connect to the source or that responses to queries sent to the source are slow, it is likely the source is either experiencing higher than normal volumes of traffic or is down. To mitigate the impact of busy sources, we suggest doing the following:
 - Break up your sync in to smaller syncs.
 - Use incremental syncs (if applicable).
 - Schedule your syncs at times when you know the source won't be busy.

JDBC syncs: Hanging during the fetching stage

If your sync is taking longer than expected to complete the fetching stage, it could be because the agent is making a large number of network and database calls. In order to tune the number of network and database calls made during a sync, you can alter the `Fetch Size` parameter:

- The `Fetch Size` parameter is located within the "advanced options" section of the source configuration and defines the number of rows fetched during each database round trip for a given query. Therefore:
 - Decreasing the `Fetch Size` parameter will result in fewer rows being returned per call to the database, and more calls will be required. However, this means the agent will use less memory as fewer rows will be stored in the agent's heap at a given time.
 - Increasing the `Fetch Size` parameter will result in more rows being returned per call to the database, and fewer calls will be required. However, this means the agent will use more memory as a larger number of rows will be stored in the agent's heap at a given time.
 - We recommend starting with `Fetch Size : 500` and tuning accordingly.

JDBC syncs: Hanging during the upload stage

If your sync is taking a long time to upload files or fails during the upload stage, you could be overloading a network link. In this case we suggest tuning the `Max file size` parameter:

API Reference ↗

- The `Max file size` parameter is located within the "advanced options" section of the source configuration and defines the maximum size (in bytes or rows) of files which are uploaded to Foundry. Therefore:

Send feedback

- Decreasing the `Max file size` parameter can increase pressure on the network as smaller files are uploaded more frequently; if a file upload fails, the cost of re-uploading is less.
- Increasing the `Max file size` parameter will require less total bandwidth, but such uploads are more likely to fail.
- We recommend `Max file size`: 120mb.

FTP / SFTP / Directory / syncs: Hanging during the fetching stage

The most common reason why file-based syncs hang during the fetching stage is because the agent is crawling a large file system.

- In order to avoid long crawl times, ensure you have specified the subfolder to crawl within the source configuration page.
 - Note: Any regex filters will run on the path of the file relative to the source's root directory.
- If a subfolder is not specified, the sync will crawl the source root.

i Syncs that crawl a filesystem will do two complete crawls of the filesystem (unless configured otherwise). This is to ensure the sync does not upload files which are currently being written to or altered in any way.

Large file-based syncs are vulnerable to transient failures

File-based syncs are transactional. If a sync fails at any point, the transaction is aborted and none of the files from that run are committed to the dataset. For example, if a sync is configured to upload one million files and a network error occurs after uploading all but one file, the entire transaction is aborted. None of the uploaded files are committed and the dataset view is not modified.

Due to this transactional behavior, syncs that process a large number of files in a single run are more vulnerable to transient network issues or source system problems. To reduce the impact of sync failures:

[API Reference ↗](#)

Use incremental `APPEND` syncs instead of `SNAPSHOT` syncs.

- Apply a file limit filter so that each run processes a smaller batch of files.

[Send feedback](#)

This way, a failure only requires re-syncing the current batch rather than all files.

For more details, see [sync failure behavior](#) and [optimize file-based syncs](#).

If your sync fails with the `REQUEST_ENTITY_T00_LARGE` error

Downloading, processing, and uploading large files is error-prone and slow.

`REQUEST_ENTITY_T00_LARGE` service exceptions occur if an individual file exceeds the maximum size configured for the agent's upload destination. For the `data-proxy` upload strategy, this is set to 100GB by default.

Overriding the limit is **not** recommended; if possible, find a way to access this data as a collection of smaller files. However, if you wish to override this limit as a temporary workaround, use the following steps:

1. Within Data Connection, navigate to your agent and select the **Advanced** configuration tab.
2. Select the "Agent" tab.
3. Under the destinations block, include the following to increase the limit to 150Gb:

```
1 uploadStrategy:
2   type: data-proxy
3   maximumUploadedFileSizeBytes: 161061273600
```

A sync started failing

If your sync fails with the `BadPaddingException` error

`BadPaddingException` exceptions occur because the source credential encryption key stored within the agent is not what was expected. This commonly happens when an agent manager is manually upgraded, but the old `/var/data` directory is not copied to the new install location.

To solve this is to re-enter the credentials for each of the sources using the [API Reference ↗](#) affected agent.

Send feedback

Unexpected data was synced

If timestamp columns in your JDBC sync show up as Long columns in Foundry

When rows are synced from a JDBC source and they contain timestamp columns, those timestamp columns will be cast to long columns in Foundry. This behavior exists for backwards compatibility reasons.

To fix the data type for these columns, we recommend using a [Python Transform](#) environment to perform this cleaning. Here is an example code snippet that casts the column `"mytimestamp"` back into timestamp form:

```
1 df = df.withColumn("mytimestamp", (F.col("mytimestamp") / 1000).cast("time"))
```

PREVIOUS

← [Optimize JDBC syncs](#)

NEXT

[Exports / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Exports

Data Connection supports exporting [datasets](#) and [streams](#) from Foundry to external systems. This can be useful for a variety of purposes:

- Data cleaned and transformed in Foundry using data pipelines can be synced to systems such as data warehouses or data lakes. This pattern is described in greater detail in "[How Palantir Foundry Fuels Your Data Platform](#)".
- Inference results from machine learning models created using [batch deployments](#) can be exported to other systems to enable operationalizing ML projects across the organization.
- Operational data captured from end users in the [Ontology](#) can be written to other systems for analysis outside of Foundry.

⚠ Data Connection exports are not yet supported for all source types. Review the individual source pages listed in the [source type overview](#) documentation to check availability. Each source page will list either [file export](#), [streaming export](#), [table export](#), or [legacy export task](#) support if export capabilities are available. For example, the [Amazon S3](#) source type supports [file exports](#) while the [BigQuery](#) source requires export tasks.

Some sources that do not yet support the file, streaming, or table export capability may support [legacy export task configuration](#). Export tasks are no longer recommended for source types that implement the updated export capability.

⚠ Updated export behavior

API Reference ↗

From June 2025, exports have been marked as **failed** if there are no new files or rows to be exported during a build. From June 2025 onward, exports with new files or rows to be exported will be marked as **success**. This change may impact

Send feedback

workflows that depend on the original failure behavior; users will need to adjust affected workflows accordingly.

Export types

The behavior of exports will depend on the type of data the destination system can accept.

Export type	Summary
File export	Used for exporting to systems without a schema. Raw files from a dataset are copied on a schedule and written to the target system.
Streaming export	Used for exporting continuously to a system that supports event streaming. Records from a Foundry stream are pushed to the target system continuously.
Table export	Used for exporting to systems with a schema. Rows of a dataset are inserted on a schedule to the target system according to the selected export mode .

File exports

An example of a system supporting file exports is [S3](#).

File exports write files from the selected Foundry dataset to the configured destination. By default, only files that were modified since the last successfully exported transaction on the upstream dataset will be written. This means that if a file was not updated in a given transaction, it will not be re-exported on the next scheduled or manual run of any downstream exports.

If the entire dataset must be re-exported, set up a new export on the same source or ensure your upstream transformation overwrites all files that must be exported.

By default, if a file already exists in the destination, export jobs will overwrite that file with the new data. This behavior may vary by source type. To avoid accidentally overwriting data stored in the destination system, we recommend creating a dedicated [export folder](#) in which to land exported data from Foundry.

[API Reference](#) ↗

Send feedback

Streaming exports

Exports to streaming destinations will stream records to the destination as long as the export job is running in Foundry. If the job is stopped and restarted, it will resume streaming records from where it left off.

The following source types support streaming exports:

- [Amazon Kinesis](#)
- Amazon SQS
- [Google Pub/Sub](#)
- [Kafka](#)
- [PostgreSQL](#)
- Solace

Streaming export replay behavior

When configuring an export to a streaming destination, you must specify the desired behavior for when the stream is replayed in Foundry. Normally, streams are [replayed](#) after making breaking changes to the processing logic. In this case, previously processed records must be re-processed using the new logic in one of the following ways:

- **Export replayed records:** Re-export all records when the stream is replayed. Note that any previously exported records will be exported again, and the external system must be configured to handle duplicates.
- **Do not export replayed records:** Pause exports until the replayed offset matches the most recent offset in the export job. Because offsets are not guaranteed to match across replayed streams, this option often results in some number of dropped records that will never be exported.

Table exports

API Reference ↗

1, table exports are generally available but may not be available for all source types. When available, table exports are recommended over [export tasks](#). To check if a source type supports table exports, visit the new source type page.

Send feedback

Connection and look for the `Table exports` tag, or visit the documentation page for the relevant source type.

Table exports allow exports to systems that contain tabular data with a known schema, where the external target schema matches the schema of a Foundry dataset you wish to export. Table exports will execute the system-specific `INSERT` statements or equivalent API calls to write rows into the target table according to the selected table export mode as described below.

Table export modes

The selected **export mode** specifies how data will be exported during a table export. The following export modes are available for all table exports and are also explained on the export setup page in Data Connection.

Table export mode	Description
Efficiently mirror dataset to external table (<i>recommended</i>)	The external table will always match what you see in the Foundry dataset.
	This is achieved by always incrementally exporting any unexported transactions from the current dataset view and truncating the external table when there is a <code>SNAPSHOT</code> transaction on the Foundry dataset being exported.
	This mode does not support <code>UPDATE</code> and <code>DELETE</code> transactions in the dataset being exported.
Full dataset without truncation	Always export a snapshot of the entire view of the Foundry dataset, without truncating the external table first. Note: This option will almost always result in duplicates in the external table. This option can be useful when external systems consume and remove rows after each run or if you want to re-export already exported rows.
API Reference ↗ truncation	This option will truncate (drop) the target table, and then export a snapshot of the full current dataset view.
This will result in the external table always mirroring the	

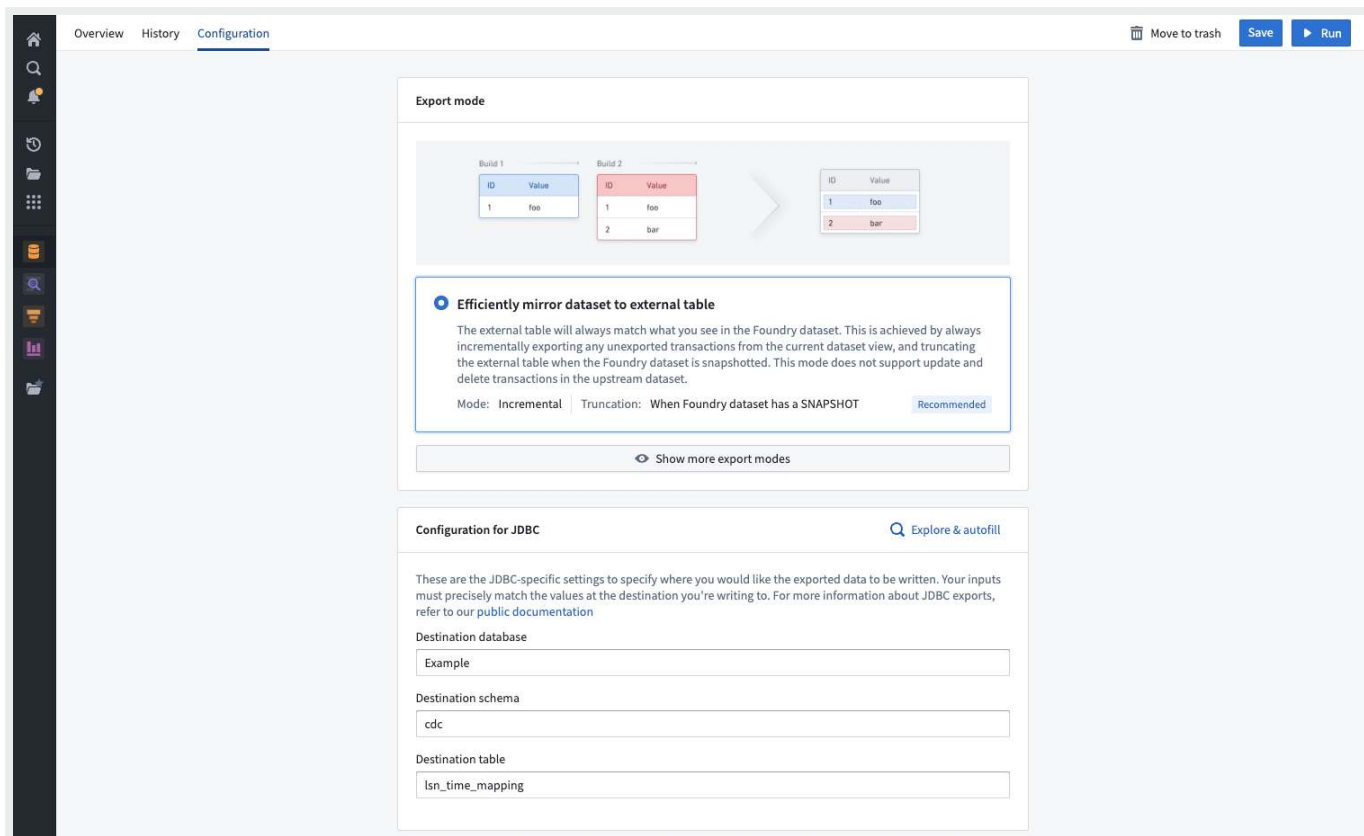
Send feedback

Table export mode	Description
	Foundry dataset but is less efficient than the incremental option to "efficiently mirror to external table". This option may be useful if edits to the external table happened between exports and you wish to always overwrite them.
Export incrementally	Exports <i>only</i> unexported transactions from the current view without truncating the target table. This mode does not support <code>UPDATE</code> and <code>DELETE</code> transactions in the dataset being exported. This option mirrors the dataset with <code>APPEND</code> transactions only to avoid exporting the same data again. This mode may produce duplicate records in the target table if the upstream dataset has a <code>SNAPSHOT</code> transaction.
Export incrementally with truncation	This option truncates (drops) the target table, then exports only transactions from the current view that have not previously been exported. This mode is useful when treating the target table like a message queue (for example, when another task picks up rows from the table and writes them elsewhere). Note: This mode does not support <code>UPDATE</code> and <code>DELETE</code> transactions in the upstream dataset.
Export incrementally and fail if not <code>APPEND</code>	Exports only unexported transactions from the current view, failing if there is a <code>SNAPSHOT</code>, <code>UPDATE</code>, or <code>DELETE</code> transaction (after the first run). This option mirrors the Foundry dataset as long as it only contains <code>APPEND</code> transactions. It protects against potentially re-exporting all data in case there is a snapshot in the Foundry dataset and guarantees there will never be duplicate data exported to the target table.

[API Reference ↗](#)

Export modes are not available for legacy export tasks.

[Send feedback](#)



Special considerations for table exports

- The dataset you export from Foundry must have a 1:1 match with the external source, including exact column names (case-sensitive) and data types. For example, exporting `COLUMN_ABC` of type `LONG` to a destination where `COLUMN_ABC` is of type `STRING` will fail at runtime.
- When setting up a table export, ensure that `DATABASE`, `SCHEMA`, and `TABLE` fields are filled out if required by your source. Missing any of these fields will cause the export to fail at runtime. Similarly, including unnecessary fields can also result in failure. For highest reliability, use explore source and autofill to select the destination in your source.
- When using an [agent worker](#) with table exports, the agent must be running on a Linux host. Windows agents are not supported for running table exports.
- The underlying files for a dataset being exported to a tabular destination must be in Parquet format. CSV and other file types that support a schema will fail to export.
- The destination table must already exist in the source system; it will not be automatically created by Foundry.
- When using an export mode that does truncation, the credentials entered in the Foundry [API Reference](#) must have permissions to truncate the table in the external system.
- `Array`, `Map`, and `Struct` types are not supported for exports. If the dataset you are exporting contains a column with type `Array`, `Map`, or `Struct`, then

[Send feedback](#)

SQL dialects in table exports

Table exports are currently supported for custom JDBC source types. Though Foundry provides dedicated connectors for specific systems such as Oracle, the table exports source capability will not appear as an option unless the connection is established via the [custom JDBC connection](#).

When performing table exports to a custom JDBC source for which you've provided your own driver JAR file, you must confirm that Palantir's INSERT statement syntax for exporting data is compatible with the SQL dialect that your system will accept.

Below, you can find examples of the INSERT statements used by Palantir for exports to [custom JDBC sources](#). These statements cannot currently be customized. If your system does not accept the statements used when running exports, you can fall back on code-based connectivity options to write to your system.

You must also ensure that the types used in your database are compatible with the supported types listed below.

Supported JDBC types

Supported JDBC source types include:

- DECIMAL
- BIGINT
- BINARY
- BOOLEAN
- SMALLINT
- DATE
- DOUBLE
- FLOAT
- INTEGER
- VARCHAR
- TIMESTAMP

[API Reference ↗](#)

Unsupported JDBC types

Unsupported JDBC source types include:

[Send feedback](#)

- `ARRAY`
- `MAP`
- `STRUCT`
- Other types not listed as supported above (such as less-common types like `GEOGRAPHY`)

If your dataset contains columns of unsupported types (such as `ARRAY`, `MAP`, or `STRUCT`), the export will fail.

INSERT statement examples

The examples below provide more information about the syntax used by table exports. If your source system does not support INSERT statements in this format, the INSERT statements will fail at runtime.

Assume that a `target_table` exists within our source system, with the same schema as the corresponding dataset in Foundry. Both the names and datatypes of the columns must match between the external source and Foundry. Below is a mock example of a dataset in Foundry:

column1	column2	column3
value1	value2	value3
value4	value5	value6
value7	value8	value9

The `INSERT` statement for a single row follows the general format below:

```
INSERT INTO target_table (column1, column2, column3) VALUES ('value1', 'value2',
```

Depending on source type, an insert statement will be applied against one or multiple rows of data. For example, Snowflake supports `batch inserts` and thus permits multiple rows to be written at once.

API Reference ↗ `target_table (column1, column2, column3) VALUES ('value1', 'value2',`

Send feedback

By contrast, a source like Apache Hive has limited support for `batch inserts` and will undergo an export via three subsequent insert statements.

```
INSERT INTO target_table (column1, column2, column3) VALUES ('value1', 'value2',  
INSERT INTO target_table (column1, column2, column3) VALUES ('value4', 'value5',  
INSERT INTO target_table (column1, column2, column3) VALUES ('value7', 'value8',
```

TRUNCATE TABLE statement example

Export options like `Full dataset with truncation` will first truncate the target table at the external source, then run an insert statement. The actual code to accomplish this would look like the example below:

```
TRUNCATE TABLE table_name;  
INSERT INTO target_table (column1, column2, column3) VALUES ('value1', 'value2',
```

If a more specific dialect is needed, we recommend one of Foundry's [code-based connectivity options](#) such as [external transforms](#).

Multi-threaded table exports

Some source types support multi-threaded table exports, which can improve export performance. When using multi-threaded exports, Foundry creates a temporary staging table in the destination system to maintain atomicity. Data is written to this staging table in parallel, then copied to the final destination table when the export completes. The staging table is automatically deleted after the export finishes.

When using multi-threaded exports, credentials must have `CREATE`, `READ`, `UPDATE`, and `DELETE` permissions on tables in the destination system.

Prepare data for export

[API Reference ↗](#)

In `Full dataset with truncation`, you cannot support any data transformations executed as part of the export job. This means that the dataset or stream you select to export should

[Send feedback](#)

desired format, including filters, renamed or repartitioned files, and any other transformations of the data.

[Pipeline Builder](#) and [Code Repositories](#) are Foundry's tools for building data transformation pipelines; both applications should provide the complete tooling you need to prepare your data for export, including job scalability, monitoring, version-control, and the flexibility to write arbitrary logic as necessary.

Controlling output file structure

Foundry datasets containing structured data store their underlying files under a `spark/` directory by default. If you need files to land in a specific directory structure when exported - for example, organized by date partition - you can use a [Python transform](#) to write the dataset with the desired layout before exporting.

The following Python transform uses the [filesystem API](#) to copy files directly to a date-based path:

```
1 from transforms.api import transform, Input, Output
2 import datetime
3 import shutil
4
5 @transform(
6     output_dataset=Output("/path/to/output/dataset"),
7     input_dataset=Input("/path/to/input/dataset"),
8 )
9 def compute(input_dataset, output_dataset):
10     dt = datetime.date.today().strftime("%Y%m%d")
11     for file_status in input_dataset.filesystem().ls():
12         relative_path = file_status.path[len("spark/"):] if file_status.p
13         with input_dataset.filesystem().open(file_status.path, "rb") as i
14             with output_dataset.filesystem().open(f"data_dt={dt}/{relativ
15                 shutil.copyfileobj(in_f, out_f)
```

With this transform, the exported dataset will contain files at paths such as

[API Reference](#) `art-00000-xxx.parquet` instead of `spark/part-00000-xxx.parquet`.

Configure the file export to use this transformed dataset as the dataset to export. After the first build, select **Apply schema** on the dataset to make it viewable as

Foundry.

Send feedback

i It is possible to allow Base64 decoding of stream records when exporting to Kafka. For more information on Kafka exports, review the full [Kafka connector documentation](#).

Set up an export

Enable exports for source

To export data, you must enable exports in the **Connection settings** section of the source to which you are exporting. Source types that support exports will show an **Export configuration** tab to the left of the screen. A Foundry user with the **Information Security Officer** role should navigate to this tab and toggle on the option to **Enable exports to this source**.

The **Information Security Officer** is a default role in Foundry; users can be granted the **Information Security Officer** role in [Control Panel](#) under **Enrollment permissions**.

After enabling exports, you must provide the set of [markings](#) that may be exported to this source. If markings and organizations are not added in the export configuration, data with those markings or organizations will fail to export to this source. To add an exportable marking, a user must be both an **Information Security Officer** and have unmarking permission on the markings or organizations they wish to allow to be exported to this source.

As an example, you may have a dataset with a **Sensitive** marking in the **Palantir** organization. To export this dataset, you must add both the **Sensitive** marking and the **Palantir** organization to the set of **Exportable markings** for this source.

Exportable markings also control which resource metadata can appear in [Slack notifications from monitoring views](#).

[API Reference ↗](#)

[Send feedback](#)

⚠ The credentials entered in the Data Connection source configuration must have write access on the table in the external system. For example, [S3](#) requires the `"s3:PutObject"` permission. Other [file exports](#) may require specific permissions to create directories if the target path does not already exist. [Table exports](#) may require additional permissions [when using an export mode that does truncation](#). Review the individual source pages listed in the [source type overview](#) documentation for source-specific considerations.

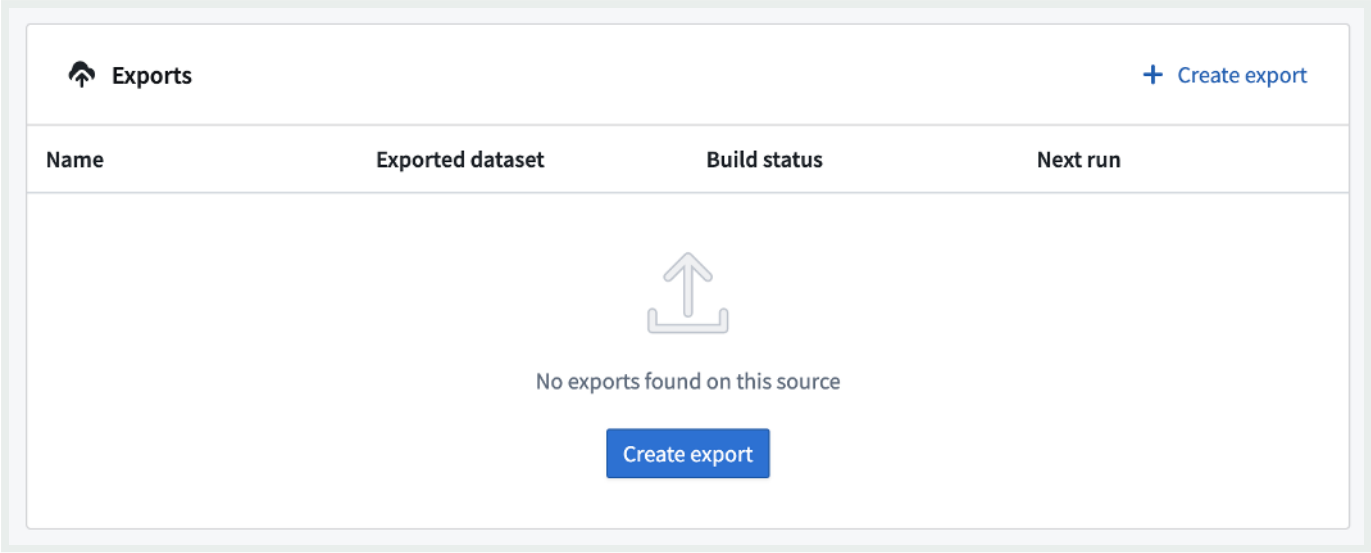
Create a new export

To create an export, first navigate to the **Overview** page of the source to which you want to export.

If this is the first export you are setting up for the given source, you will see an empty table and a button to create an export.

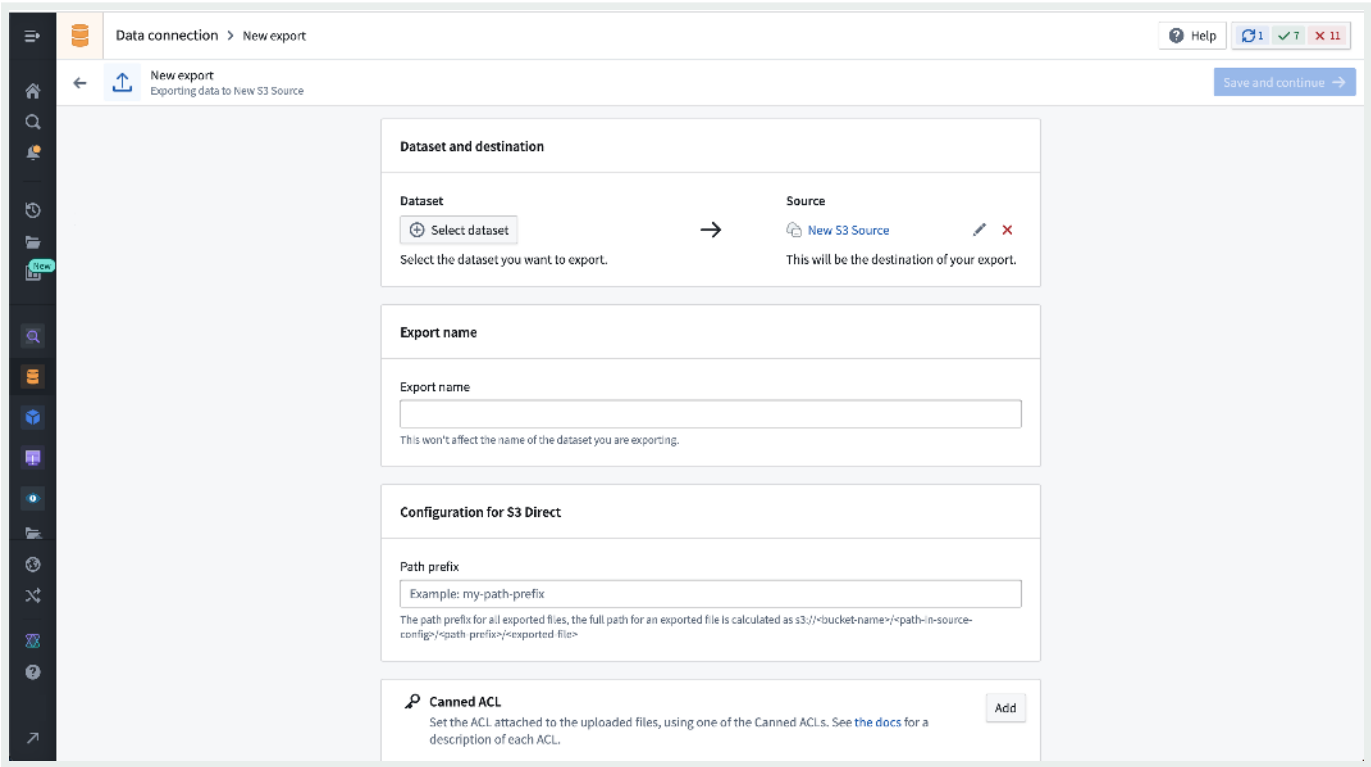
[API Reference ↗](#)

[Send feedback](#)



Select **Create export**, then select the dataset or stream to export and any source-specific export configuration options. These options will vary by source connector and are explained on the corresponding [source type](#) pages in our documentation. If multiple branches exist on the exported dataset, only data on the master branch will be exported.

The example below shows the export configuration interface for the [S3](#) connector:



After you finish the export, you will land on the export management page where you can do the following:

- Manually run the export.

Send feedback

- Set a schedule for the export.
- View export history.
- Modify configuration options.

 Streaming exports use a **start/stop** button instead of a **run** button. If a schedule is configured on a streaming export, it will behave similarly to schedules on other streams; if the stream is stopped when the schedule triggers, it will be automatically restarted. If the stream is not stopped when the schedule triggers, it will continue to run.

 Some source export options may not be editable after initial setup. If immutable options must be changed, you must delete and re-create the export.

Scheduling exports

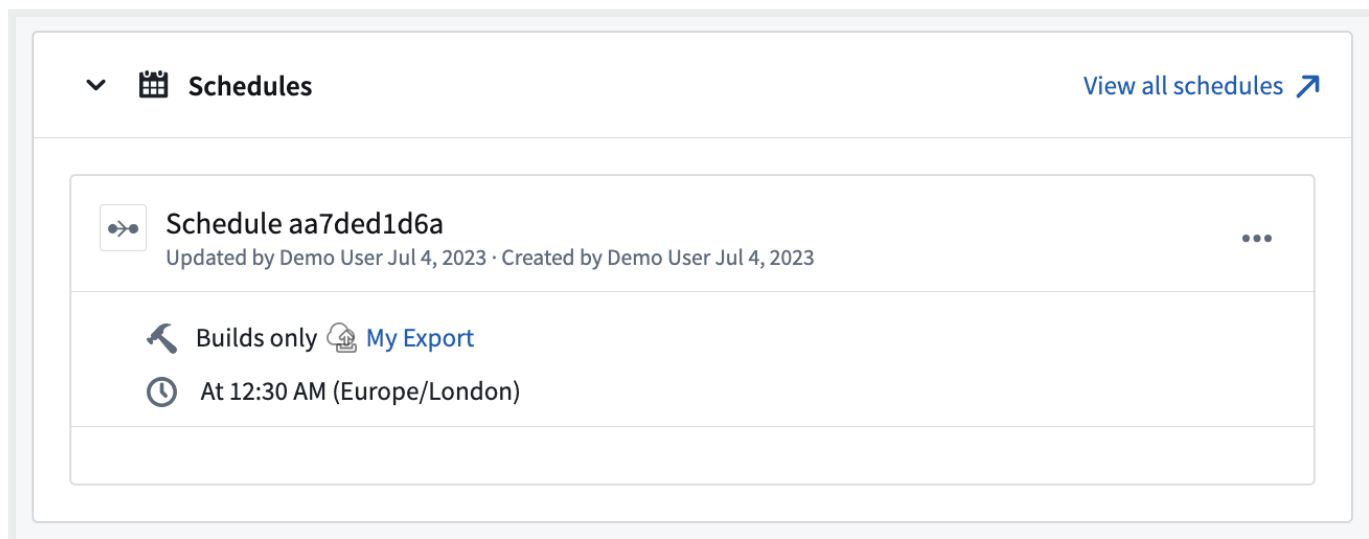
Exports should be scheduled to run regularly, exporting recent data to the external destination. Streaming exports do not need to be scheduled since they should simply be started or stopped.

To schedule an export, navigate to the **Overview** page for the export. Then, select **Add schedule** to open the export in [Data Lineage](#). From there, select **Create new schedule** to the right of your screen and configure as you would for any other job. Learn more about [available scheduling options](#).

View any schedules that trigger a specific export on the **Overview** page for that export, as shown below:

[API Reference ↗](#)

[Send feedback](#)



Export history

Similar to [syncs](#), exports run as jobs using the Foundry [build system](#). The **History** view of an export shows the history of the jobs associated with it. Each job may be opened and viewed in the **Builds** application by selecting **View build report** to the upper right of the **Job details** section.

For streaming exports, the export history will also display if the streaming export job is currently running or stopped.

Exports vs. export tasks

Table exports are intended to fully replace the now sunsetted [export tasks](#) that will eventually be deprecated according to the published [product lifecycle](#). While much of the functionality offered by export tasks is available in the new export options, there are some differences in the features and configurations.

Migrating from export tasks to table exports

Migrations from export tasks to new export options must be performed manually. Configure your [API Reference](#) options documented below; once the new export process is working as expected, manually delete the export task that was previously used.

Send feedback

If a feature available in export tasks has no equivalent option in new exports, the recommended migration is to switch to using [external transforms](#). External transforms provide a flexible alternative for executing custom logic that interacts with an external system, and are intended to be used when the UI configuration options for source capabilities do not cover the required functionality.

Feature comparison between exports and export tasks

Export task option	Supported in new exports?	Details
<code>parallelize: <boolean></code>	Depends on the source type	This may be required for a per-source support for it documented specific export if available
<code>preSql: <sql statements></code>	Not supported	If this is needed for a use case, use transforms .
<code>stagingSql: <sql statements></code>	Not supported	If this is needed for a use case, use transforms .
<code>afterSql: <sql statements></code>	Not supported	If this is needed for a use case, use transforms .
<code>manualTransactionManagement: <boolean></code>	Not supported	If this is needed for a use case, use transforms .
<code>transactionIsolation: READ_COMMITTED</code> API Reference ↗	Not configurable	This may be a source configuration option, and it is applied when performing exports. See the documentation for more details.

Send feedback

Export task option	Supported in new exports?	Details
		this configuration option is available
datasetRid: <dataset rid>	Supported	The dataset on which you wish to export
branchId: <branch-id>	Not configurable	Data is always exported from the master branch.
table: database: mydb # Optional schema: public # Optional table: mytable	Supported	These options are specific to table exports on the JDBC source
writeDirectly: <boolean>	Not configurable	New exports support the option equivalent to writeDirectly
copyMode: <insert directCopy>	Not configurable	New exports support the option equivalent to copyMode: insert
batchSize: <integer>	Depends on the source type	Batch size is supported for exports on source but not supported for other source types.
writeMode: <ErrorIfExists Append Overwrite AppendIfPossible>	Supported	Together, these are similar to export modes documented in all combinations valid for exports and the combinations have been normalized into the six modes
incrementalType: <snapshot incremental>		

API Reference ↗

Send feedback

Export task option	Supported in new exports?	Details
exporterThreads: <integer>	Not supported	If you need granular control over the number of threads, use exportTransforms .
quoteIdentifiers: <boolean>	Not supported	This option is relevant when exporting tables in the target system, which is not supported in new exports. The tables must exist before a new export can be configured.
exportTransactionIsolation: READ_UNCOMMITTED	Not configurable	As with legacy exports, the transaction isolation level is READ_COMMITTED. If you need a transaction isolation level of READ_UNCOMMITTED, the export will be taken from a legacy source configuration that is available and properly configured.

[← PREVIOUS](#)
Syncs / Troubleshooting reference

[NEXT →](#)
Export tasks [Legacy]

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

Send feedback

Data connectivity & integration / Data Connection / Exports / Export tasks
[Legacy]

Export tasks (legacy)

⚠ We generally do not recommend using export tasks to write data back to external sources. However, export tasks may be available and supported for some source types based on your Foundry enrollment.

The following export task documentation is intended for users of export tasks who have not yet transitioned to our [recommended export workflow](#).

Overview

Export tasks allow you to export data from Foundry to various external data sources. The configuration for an export task consists of two parts:

- **Source:** Defines how the agent connects to the data source. Should be configured the same way as for dataset syncs.
- **Task configuration:** Defines the type of export, input datasets, and parameters for the export.

An export task is triggered when a job is started on the task output dataset.

Known export task limitations

API Reference ↗

Export tasks are not integrated with markings and export controls. Data exported via export tasks do not require unmarking permission on the exported dataset or stream.

Export tasks are not optimized for performance. Exporting large queries can result in long-running jobs or jobs that fail to complete.

Send feedback

- Export tasks do not have a user interface for configuration and instead must be configured using YAML to provide the required configuration options. Not all export task options are documented for self-service use; in some cases, export tasks can only be configured through support from Palantir.
- Export tasks only work on [agent-worker sources](#).

Getting started

To configure an export task:

1. Configure a source in Data Connection to connect to the target system where you are exporting.
2. In the Foundry file system, navigate to the folder containing the source. Right-click on the resource and select **Create Data Connection Task**.
3. In the Task Editor UI:
 - Keep the source named `inputSource`
 - Add an input dataset named `inputDataset`
 - Create or choose an output dataset named `outputDataset`
 - Configure the YAML for your specific export type

 The input source **must** be named `inputSource` and the output dataset **must** be named `outputDataset`. Failure to use these specific names may result in task errors.

[API Reference ↗](#)

[Send feedback](#)

Data Connection
Syncs
Sources
Agents
Webhooks

Configure Task
Save

Sources

postgres /Project

Add entry

Inputs

dataset_to_be_exported /Project

master

Add entry

Outputs
 master

Type: foundry

my_export /Project

Add entry

```

1 type: export-jdbc-task
2 exporterType: parquet
3 datasets:
4   - writeDirectly: true
5     writeMode: Overwrite
6     table:
7       schema: public
8       table: some_table
9

```

✓ Syntax is valid

Supported export types

Data Connection export tasks support writing to a wide range of common enterprise systems:

- [Amazon S3](#)
- [OneLake and Azure Blob Filesystem \(ABFS\)](#)
- [HDFS](#)
- [JDBC-compatible systems](#), including:

API Reference ↗ databases

- [PostgreSQL](#)
- [Microsoft SQL Server](#)

Send feedback

- MySQL
- Data warehouses
 - Teradata
 - [Snowflake](#)
 - Vertica
- File systems, including network file systems [mounted on the intermediary agent](#)
- [SFTP](#)

Common configuration options

Incremental exports

All export types support the `incrementalType` parameter, which controls how data is exported over time. For JDBC exports, this parameter is specified per dataset rather than globally.

When set to **snapshot** (the default value), the export task exports all files visible in the current view of the input dataset. This means every export will include the complete dataset, regardless of what was previously exported.

When set to **incremental**, the export behavior changes to optimize for efficiency. The first export behaves like a snapshot, exporting all available data. On subsequent exports, only new transactions added since the last export will be included, provided the initial exported transaction is still present in the dataset. If the initial transaction is no longer available (for example, due to a dataset rebuild), the system automatically falls back to a full snapshot export.

Example for file-based exports:

```
1 incrementalType: incremental
```



API Reference ↗

Path rewriting

Send feedback

File-based exports support the `rewritePaths` configuration option for customizing file names and paths during export. This field accepts a map of regular expressions and substitution templates:

```
1 rewritePaths:
2   "^spark/(.*)": "$1" # Removes the spark/ prefix
```

The substitution templates support several dynamic replacement patterns:

- regex capture groups such as `$1`, `$2`, and so on to reference matched portions of the original path.
- timestamp patterns like `${dt:yyyy-MM-dd}`, which use Java's `DateTimeFormatter` syntax to insert the current date and time.
- Foundry transaction ID using `${transaction}`
- Foundry dataset ID using `${dataset}`

← PREVIOUS
[Overview](#)

NEXT →
[Webhooks / Overview](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Webhooks

You can use Data Connection to configure **webhooks** to connect Foundry to systems and workflows outside of Foundry.

i This section contains information on *outbound* webhooks (Foundry making requests to another system). If you need to receive *inbound* webhooks (another system sending requests to Foundry), consider using [listeners](#) instead.

Each webhook provides a way to make a request to an external system outside of Foundry. For example, you could create a webhook that performs an HTTP request to an external server when a user selects a button in a Foundry application, connecting that application to existing workflows and source systems.

Each webhook is associated with a single [source](#) in Data Connection. The source stores the credentials necessary for connecting to the external system. Depending on the type of source the webhook is associated with, certain task types are available for use. For example, when using [REST](#), you can flexibly configure an HTTP call that should be made to an external service.

Webhooks can be configured flexibly to accept specific inputs and capture outputs from external system requests. Additionally, you can set time, concurrency, and rate limits on webhook executions. For detailed configuration options, see the [webhooks reference](#).

Review the following documentation to learn more about webhooks:

API Reference ↗

the [webhooks reference](#) to learn more about configuration options.

Send feedback

- Review the [actions documentation](#) to learn about how webhooks can be configured for end-user applications.
- [Call webhooks from external functions](#) for writing custom code to interact with external systems.
- [Publish a webhook as a function](#) so it can be used in Workshop, Ontology SDK (OSDK), functions, and actions.

PREVIOUS

← Exports / Export tasks [Legacy]

NEXT

Set up a webhook →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Webhooks / Set up a webhook

Set up a webhook

This tutorial guides you through how to set up webhooks to send HTTP requests to external systems.

Prerequisites

This tutorial will walk through creating a REST API source and a webhook associated with that source. If you have already created a source, you can skip ahead to the [Create a webhook](#) step.

⚠ If you are trying to connect to a system that is not accessible from Foundry's network, we recommend using a [Foundry worker source with agent proxy egress policies](#). If the system can only be referenced by IP address and not hostname, use an [agent proxy runtime](#).

Using an [agent worker](#) source for webhooks comes with additional complexity to configure and manage. In particular, we recommend provisioning a dedicated agent for webhooks running on an agent worker. Agents execute tasks in the order received; if bulk syncs and webhooks are executed on the same agent, short-running webhook tasks may end up queued behind long-running bulk syncs. [Learn more about agent workers and their configuration.](#)

Ensure that you have created a [Project](#) to manage permissions for the source you will create in this tutorial. If you have not yet created a Project, learn how in the [Project documentation](#).

API Reference ↗



Send feedback

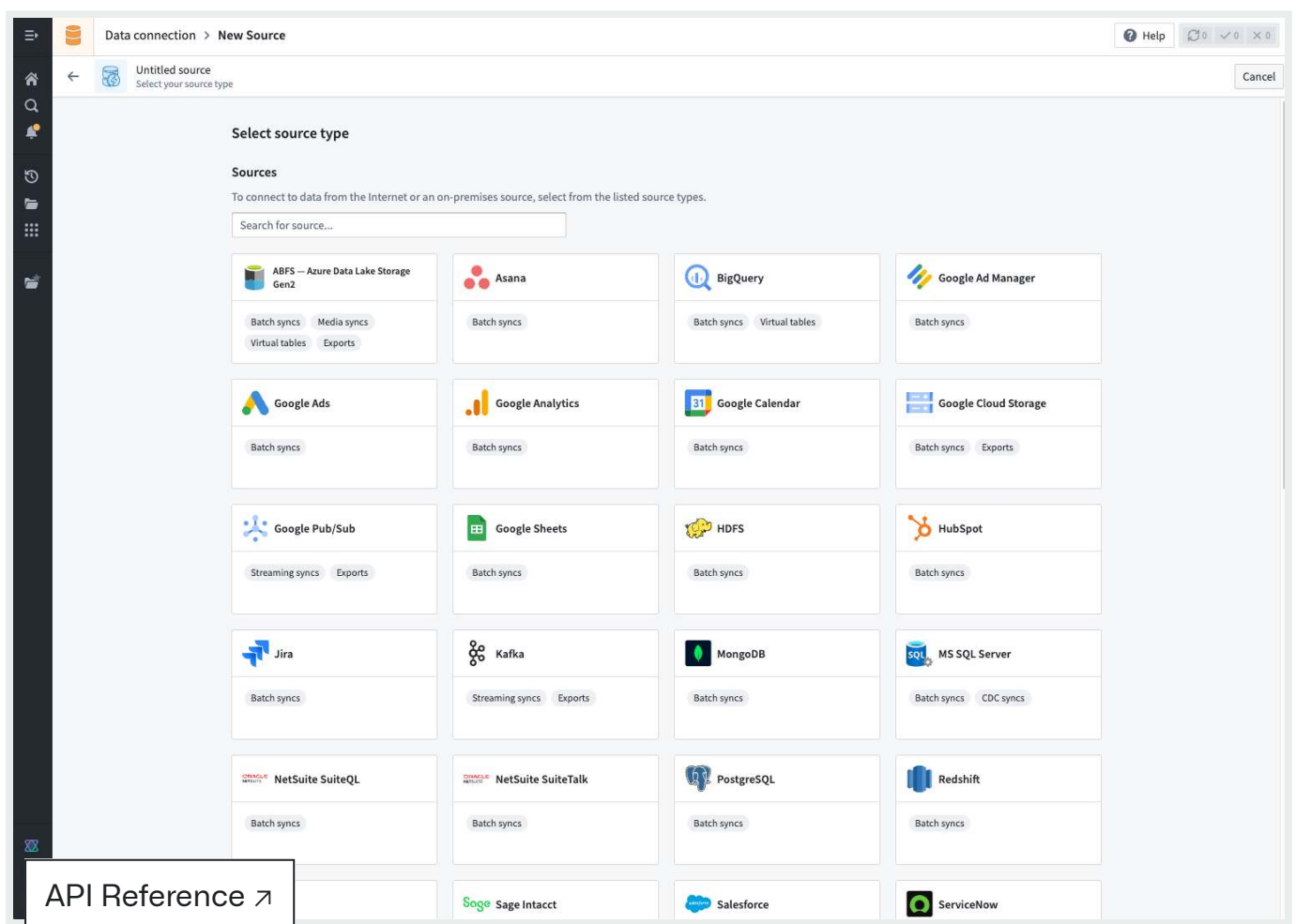
Tutorial

This tutorial assumes you will be using a REST API for a system available over the open Internet.

i Some sources, including SAP, legacy Salesforce, and legacy `magritte-rest-v2` use a more complex task configuration. For more information on configuring webhook tasks, see the [webhooks reference](#).

Create a source

Start by navigating to **Data Connection** and selecting the **Sources** tab. Then, select **New source** and **REST API**.



Use the source editor to fill in configuration details for the REST API source type. [Send feedback](#)
More details on the REST API source type can be found in the [source type](#)

[reference](#).

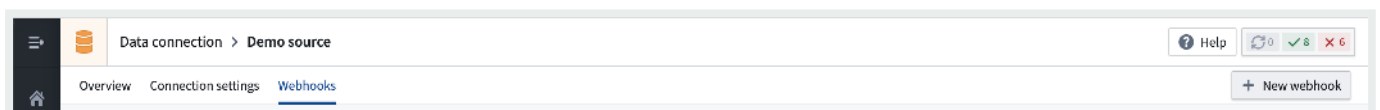
The source is meant to contain the minimal set of secrets and connection details required to establish a connection. When configuring individual webhooks using this source, you will have an opportunity to add additional request details, including the relative path, query parameters, headers, and body content.

Other source types

Some other source types also support webhooks. For more details, review the [reference page](#) for the specific source type.

Create a webhook

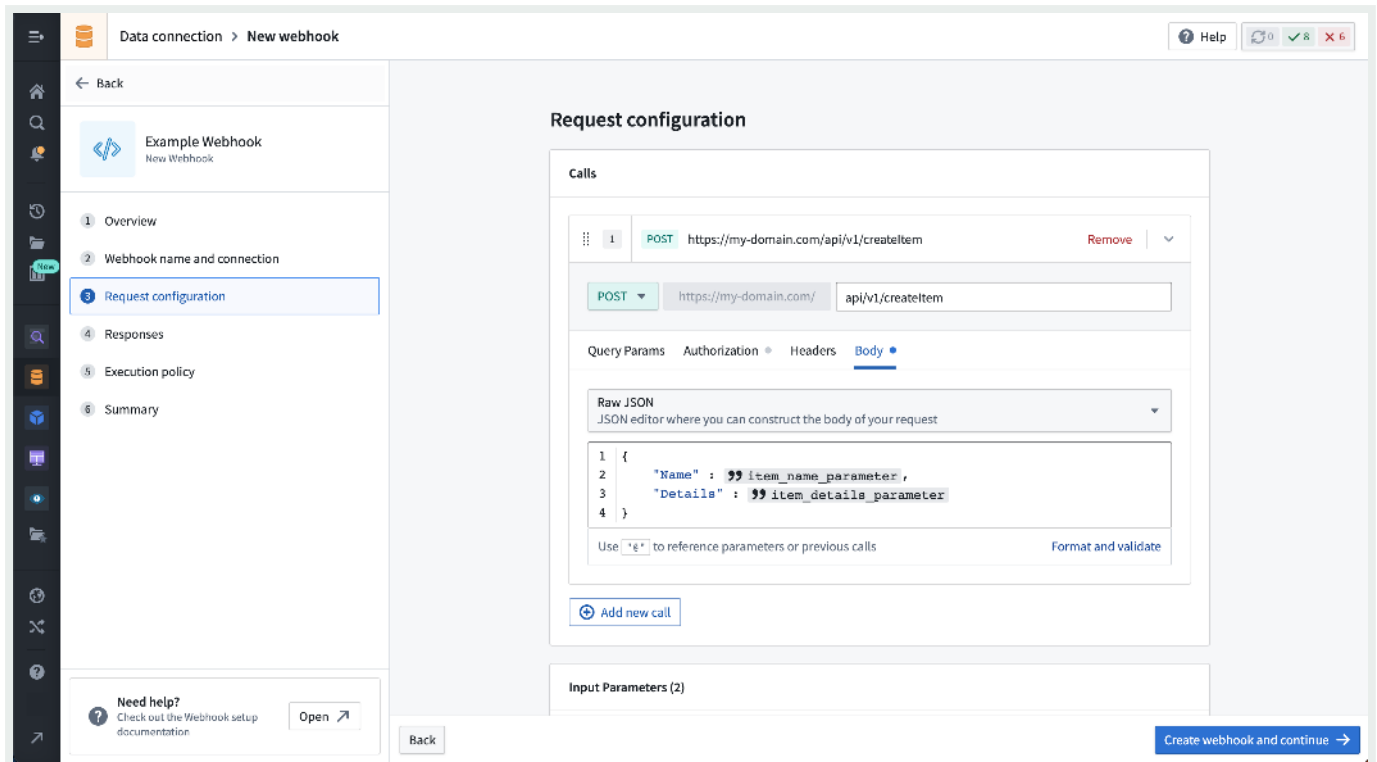
Once the source has been created, select the **Webhooks** tab and select **New webhook**.



Follow the steps in the **New webhook wizard** to create a webhook that makes an external request. The example below illustrates a `POST` request to the `/api/v1/createItem` endpoint on our example domain. The body is constructed as Raw JSON and, in this case, is configured to accept two string input parameters.

[API Reference ↗](#)

[Send feedback](#)



Duplicate a webhook

If you are looking to create a new webhook that has a similar configuration to an existing webhook (for example, two similar endpoints on the same API), you may choose to duplicate the existing webhook. To duplicate a webhook, open the **Actions** menu from the **Webhooks** tab of the source overview or use the **Actions** button on the webhook page, then select **Duplicate**.

Configuration options

1. Enter a name for the webhook, and add a description if desired.
2. Optionally, toggle on **Function configuration** to publish the webhook as a function. Provide an Ontology and API name for your function. See the [Webhook functions reference](#) for details.
3. Select the source you created in the previous step of the tutorial. If you landed in the wizard directly from a source page, the source will be pre-selected.
4. Add any input parameters that you want to use in the external request.
5. Populate any required parts of the request configuration. You can reference input parameters in the path, query params, headers, and body configuration sections.

API Reference ↗

6. Optionally, set any **Limits** you want to apply to the webhook. Refer to the [Limits reference](#) for details.

Send feedback

e

7. Select **Create** to complete creating the webhook.

Direct import from cURL commands

Alternatively, you can also create a request Call by directly importing a cURL command. To do this, choose the **Import from cURL** option and put your cURL command in the TextArea. If your cURL uses the supported options and relates to a domain already in Source, selecting **Import cURL** will create a new Call.

Test the webhook

After saving, you will be able to run a test request to see if your configuration is correct. This may be done at any time by using the **Test Connection** side panel.

After a test request is made, you may use the response to parse output parameters. For more information on output parameters, see the [webhooks reference page](#).

Next steps

In this tutorial, you learned how to create a Source and create a Webhook associated with that Source. Here are some resources we recommend to learn more:

- View the [Action types tutorial](#) to learn how to configure a Webhook for use in end-user applications.
- Read the [Webhooks reference](#) to see the full details about options available for configuring Webhooks.
- Follow the step-by-step guide to [set up a webhook for the Microsoft Graph API](#) for a concrete example of integrating with Microsoft Cloud services.

PREVIOUS

← Overview

NEXT

Set up a webhook for the Microsoft Graph API →

API Reference ↗

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Webhooks / Set up a webhook for the Microsoft Graph API

Set up a Webhook for the Microsoft Graph API

This guide shows step-by-step how to configure a webhook for the Microsoft Graph API to access Microsoft Cloud service resources from Foundry.

[Learn more about the Microsoft Graph API ↗.](#)

Prerequisites

Prior to configuration, you must:

- Sign in to your Microsoft account by typing `login.microsoftonline.com` into your browser to authenticate and generate an access token you can use to create the webhook.
- Configure [network egress policies](#) for both `login.microsoftonline.com` and `graph.microsoft.com` to allow outbound connections from Foundry.

Instructions

1. [Create a REST API source](#) for your webhook.
2. Include `login.microsoftonline.com` and `graph.microsoft.com` as the source's

API Reference ↗

any **Authentication** restraints.

3. Set any necessary **Additional secrets**, such as the `TenantId`, `ClientId` and

`Secret` that you will use to authenticate against `login.mic`

Send feedback

You will find these after you [register an application in Microsoft Entra](#) ↗.

4. Add the network egress policies you created above in the **Network connectivity** section before choosing **Save and continue**.

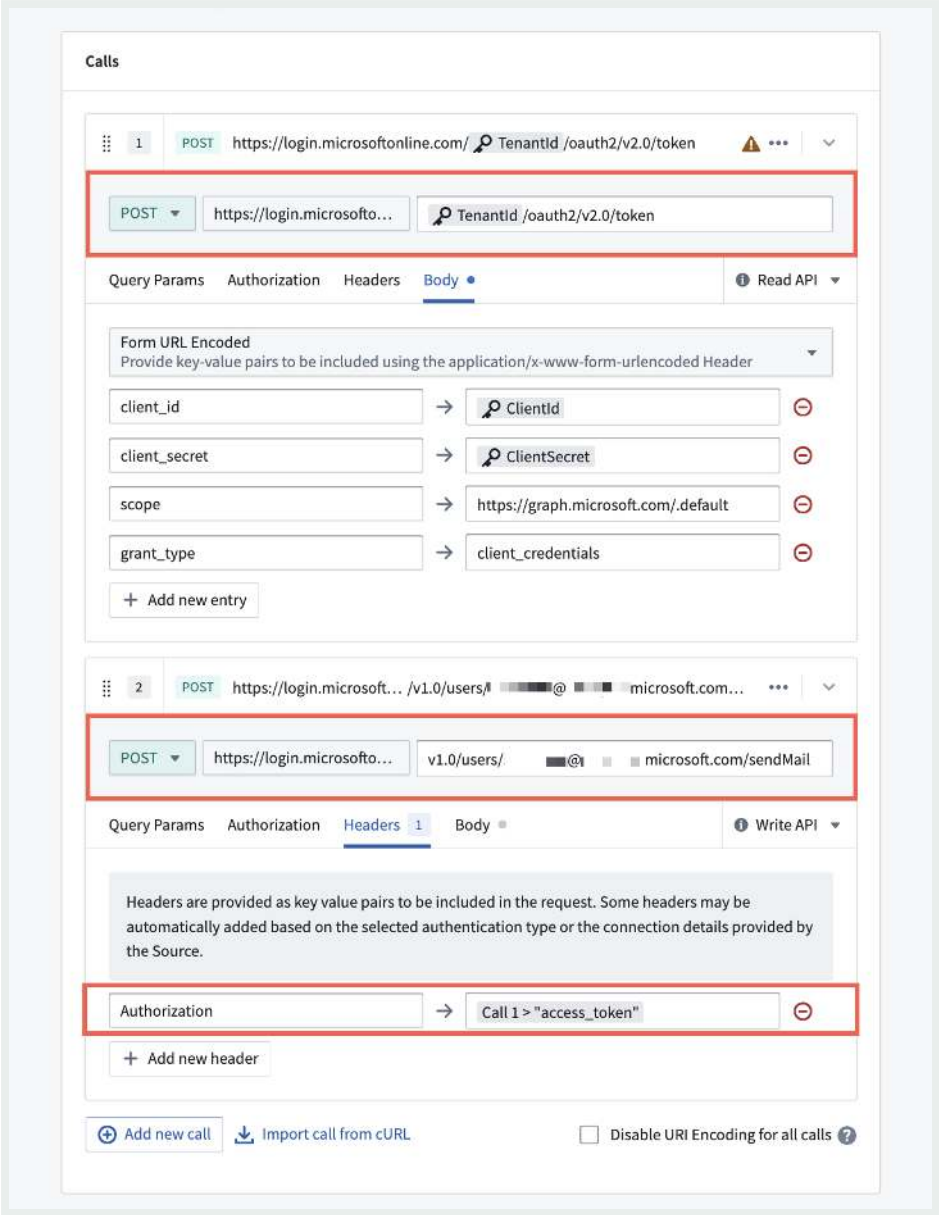
The screenshot displays the 'Source Setup' configuration page. It is divided into three main sections: 'Domains', 'Additional secrets', and 'Network connectivity'. In the 'Domains' section, two domains are listed: 'https://login.microsoftonline.com' and 'https://graph.microsoft.com', both with 'No auth' status. Below them, the 'Domain base URL' is set to 'https' and 'graph.microsoft.com', and the 'Authentication' mode is set to 'None'. The 'Additional secrets' section contains four entries: 'additionalSecretTenantId', 'additionalSecretClientId', 'additionalSecretClientSecret', and 'additionalSecretMailboxAddress', each with a corresponding secret value and an 'Edit' button. The 'Network connectivity' section shows two policies: 'login.microsoftonline.com (Port 443)' and 'graph.microsoft.com (Port 443)', both with 'Direct connection' and 'Approved' status. At the bottom, there are buttons for 'Add new domain', 'Add Secret', 'Add another policy', and 'Request and self-approve new policy'.

After you configure your REST API source, you will next configure your webhook to make two **POST** requests that:

1. Login using the `login.microsoftonline.com` credentials created on the source to get a short-lived access token.
2. Make an API call to `graph.microsoft.com` using the access token in the response from the first call as the bearer token in the call's authentication header.

API Reference ↗

Send feedback



[Learn more about configuring Webhooks in Data Connection.](#)

PREVIOUS
← Set up a webhook

Configuration reference →
NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

Send feedback

Data connectivity & integration / Data Connection / Webhooks / Configuration reference

Configuration reference

This page documents the detailed configuration options and permissions associated with [Webhooks](#) in Data Connection.

Configuring inputs and outputs

Each webhook can be configured flexibly to map **input parameters** into the request that will be made to the external system. Then, the response from the system can be captured as **output parameters** that can then be used elsewhere in Palantir Foundry.

Input parameters

Input parameters represent the inputs that can be passed in when a user executes the Webhook. Typically, a Webhook is configured for use in [Action types](#), where you can specify a mapping between [Action parameters](#) and [Webhook input parameters](#).

There are many data types and constraints available for Webhook input parameters:

- **Boolean** parameters can be `true` or `false`.
- **Integer**, **Long**, and **Double** parameters represent numeric values.
- **String** parameters represent text values, and can be constrained to only allow specific values.
- **Date** and **Timestamp** parameters represent time-based data.

API Reference ↗

In addition to these basic types, several collection types are available:

- **List** parameters represent an ordered collection of inputs of a specific type.

Send feedback

- **Record** parameters enable passing in key/value pairs, and can be constrained to require specific keys with values of a specific type.
- **Optional** parameters represent inputs that may or may not be present.

i If you want to use [Functions on Objects](#) to map from Action parameters to webhook inputs, you can also conditionally not fire the webhook at all if your function to map inputs returns `undefined`. For example, `WebhookInput | undefined`."

Finally, an **Attachment** type may be used to pass a file that has been uploaded in an Action form. Note that this feature is not supported for agent worker sources.

Output parameters

Output parameters allow you to capture data returned from the external system for use elsewhere in Foundry. For example, when you create a new record in an external system, the system may return an ID for the new record. By capturing this new ID in an output parameter, you can propagate it to Actions and immediately write it into a Foundry object.

An example configuration for a single output parameter called `unique_id` is shown below in the Webhook setup wizard:

Output Parameters (1)

i Output parameters allow you to capture data returned from the external system for use elsewhere in Foundry. For example, when you create a new record in an external system, the system may return an ID for the new record. By capturing this new ID in an output parameter, you can propagate it to Actions and immediately write it into a Foundry object.

PARAMETER ID ?

Extracts the "unique_id" field from the webhook response json so that it can be used in Actions.

✕

ASSIGN VALUE FROM WEBHOOK RESPONSE

☐ Whole response

☒ Extract field from returned JSON object ?

EXPECTED VALUE TYPE ?

A String

▼

☐ Only allow specific values

API Reference ↗

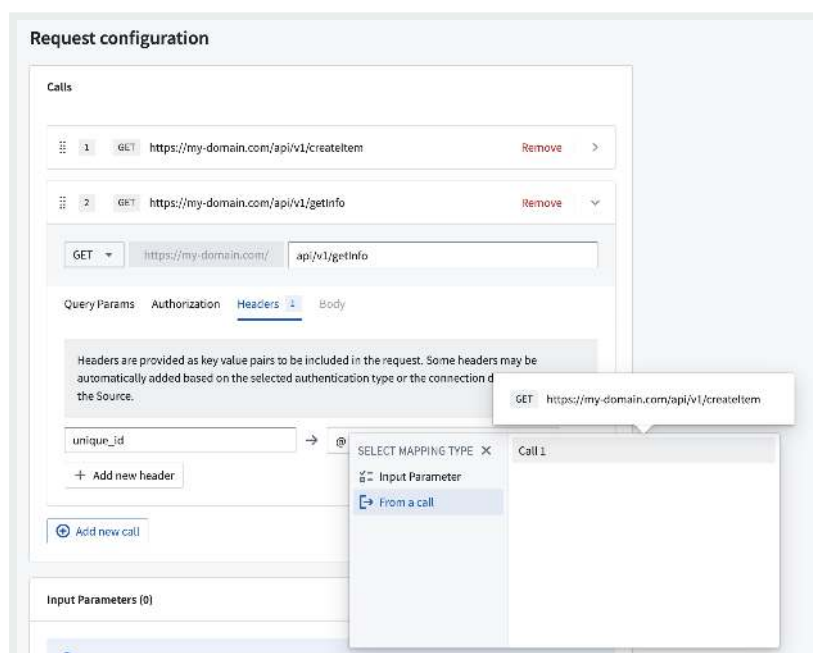
[Add parameter](#)

Send feedback

Alternatively, output parameters can also be directly accessed in the Webhook configuration step. An example is shown below where a webhook call uses a value from the response of the previous call. The first webhook call returns a response that looks like this:

```
{
  "results": {
    "unique_id": "ID4567"
  }
}
```

Configure the second webhook call. In the **Headers** tab, you can create inline references by typing @ that opens a menu where you can reference values from the response of the previous call.



Choose **From a call** and select the previous call that has the response to parse. There are three options available to extract the necessary value from the response:

- **Whole response:** Extract the whole response as string.

API Reference Extract the required value using the keys in the response.
Array Extract the required value from array response using index positions (zero-indexed).

Send feedback

As in the example below, select **Extract by key** and configure the keys: `results` followed by `unique_id` using the **Add nested key** option.

SELECT MAPPING TYPE X

☒ Input Parameter

☒ From a call

← Call 1

☐ Whole response

☒ Extract by key
Extract value by key from dictionary response.

☐ Extract by index
Extract value by index in array response (zero-indexed).

Key for value from dictionary

results X

↓

Key for value from dictionary

unique_id X

↓

Add nested key Add another index

Add

Select **Add** and you will be able to see the reference displayed in the header value field.

2 GET https://my-domain.com/api/v1/getInfo Remove

GET https://my-domain.com/ api/v1/getInfo

Query Params Authorization Headers 1 Body

Headers are provided as key value pairs to be included in the request. Some headers may be automatically added based on the selected authentication type or the connection details provided by the Source.

unique_id → Call 1 > "results" > "unique_id" ⊖

+ Add new header

The output parameters that are captured depend on the task type you are using.

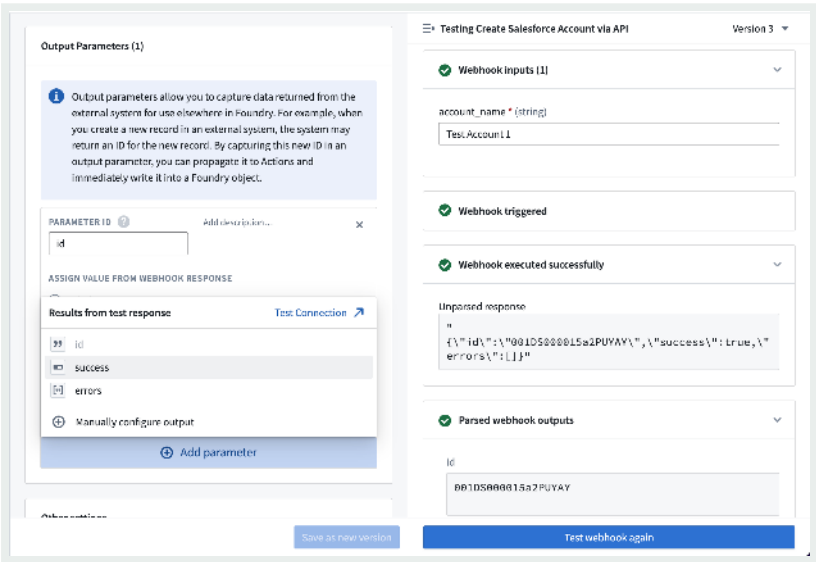
[Learn more about how to capture output parameters from a REST Webhook and use these API Reference ↗](#)

For convenience, the result of a Webhook task is automatically converted to the appropriate output parameter type in certain cases:

Send feedback

- If a **String** output parameter is configured and the Webhook task result is not a string, then the result will be converted to a JSON string.
- If a **Record** output parameter is configured and the Webhook task result is a string, then the Webhooks service will attempt to parse the string into JSON.
- If a **Double** output parameter is configured, then Webhook task results which are Integers or Longs will be converted automatically.

Output parameters may be configured using a response received from a request made via the "Test Connection" side panel. Once a successful test request has been made, suggested outputs parsed from the response will be automatically displayed when adding a new output parameter.



Task body

Some webhooks are implemented as Tasks. For these webhooks, the **Task body** represents the data that will be sent to the Task when the webhook is executed.

The structure of what each Task expects depends on the specific Task type that will be listed, if relevant, under the entry for that [webhook type](#). A Task body should be defined using [Handlebars](#) syntax. Within the Task body, you can refer to any of the input parameters [API Reference](#) have defined for the webhook as documented above.

Send feedback

Webhook types

Each type of webhook supports different configuration options. This section documents the options available for various webhook types, and provides basic examples to help you get started.

REST API

The most common use of webhooks is to make HTTP requests to REST APIs. After configuring a REST API source, you can use the request builder interface to construct the request you wish to make.

Available options for the REST API source include:

Option	Required	Description
Method	Yes	The HTTP method for the request.
Relative path	No	Specify the relative path for the request endpoint. This should be relative to one of the domains configured in the REST API source.
Query Params	No	Provide key-value pairs for any query params to be included in the request. Some query params may be included at runtime based on the source configuration. Those will be displayed here as read-only.
Authorization	No	Authorization details are based on the source configuration. Any edits should be done by navigating back to the source.
Headers	No	Provide key-value pairs for any headers to be included in the request. Some query params may be included at runtime based on the source configuration. Those will be displayed here as read-only.
Body	No	For requests that accept a body, you may include a body from the available types. These types include Raw JSON , Form Data , Form URL Encoded , binary File , XML , Plain Text , and HTML .

API Reference ↗

Multiple requests

Send feedback

A single webhook may contain multiple requests. Requests may be chained together, with response values from a previous call referenced in subsequent calls.

REST API (legacy)

⚠ The legacy REST API options documented here and using the `magritte-rest-v2` plugin are for historical reference only. New workflows should be implemented using the REST API source.

After setting up a `magritte-rest-v2` Source, you can create REST Webhooks using the `generic-rest-webhook-task`.

For a REST Webhook, the structure of the Task body must be an array of `calls`, each representing a REST request that will be made to the external system.

i The only supported type of `call` for webhooks is `magritte-rest-call`.

Below is an example of a basic Task body template for a REST Webhook, representing a single HTTP call:

```
1 {
2   "calls": [
3     {
4       "type": "magritte-rest-call",
5       "method": "POST",
6       "requestMimeType": "application/json",
7       "path": "your/request/path",
8       "body": { "text": "{{json message}}" }
9     }
10  ]
11 }
```

[API Reference ↗](#)

This task body template will make a POST request to the `your/request/path` endpoint with a request body of `{ text: <message> }`, where `<message>` is a String the Webhook.

[Send feedback](#)

Extracting output parameters

There are two main ways to extract [output parameters](#) from a REST Webhook: 1) capturing top-level fields from a JSON response by name, 2) defining JSON extractors for more customized extraction logic, and explicitly listing which fields you want to output.

You may also choose to extract the full response JSON as a string output. This provides additional flexibility to traverse the response when performing subsequent edits or notification rendering using functions.

i The REST plugin supports a wide variety of extractor types including JSON, XML, HTML, HTTP Status, and more. Webhooks require an external endpoint that returns JSON, so only the `json` extractor may be used in a webhook task configuration.

For capturing top-level fields, suppose you've configured a REST call which returns the following response:

```
1 {  
2   "id": "c52fd6e4-6eb5-4da1-8908-4845e51c801b"  
3 }
```

You can define output parameters with the same IDs as keys in your JSON response to capture them. In this example, you can add a string output parameter with a parameter ID of `id`, and this field will be captured from the response.

If you need to capture a nested field from the response, you can extract values by specifying `extractors`. Extracted values can be also used in subsequent calls to chain calls together. If you need to capture the full response, you can extract the entire response by specifying `extractors` with the root path as target: `"result": "/"`.

Below is an example of a Task body template that makes two requests: one to a GET endpoint to retrieve some data, then one to a POST endpoint using data from the previous

[API Reference ↗](#)

[Send feedback](#)

```

1  {
2      "calls": [
3          {
4              "type": "magritte-rest-call",
5              "method": "GET",
6              "path": "path/to/fetchData",
7              "extractor": [
8                  {
9                      "type": "json",
10                     "assign": {
11                         "request_output": "/json/path/to/output"
12                     }
13                 }
14             ]
15         },
16         {
17             "type": "magritte-rest-call",
18             "method": "POST",
19             "path": "your/request/path",
20             "body": { "text": {%request_output%} },
21             "extractor": [
22                 {
23                     "type": "json",
24                     "assign": {
25                         "result": "/json/path/to/result"
26                     }
27                 }
28             ]
29         }
30     ],
31     "output": ["result"]
32 }

```

The first call makes a `GET` request to the endpoint at `path/to/fetchData`, then extracts data from the JSON Path of `/json/path/to/output` into a state variable called `request_output`. Then, the `request_output` state is used in the body of the second call. From the second call, we extract another field from the JSON response into a state variable called `result`. Finally, the `"output"` field in the configuration defines which extracted fields should be returned as output parameters.

[API Reference ↗](#)

Other options

[Send feedback](#)

In addition to specifying the `calls` and `output`, there are a few additional configuration options available for REST Webhook Tasks:

- If you are making multiple calls in a REST Webhook, only one call is allowed to use an unsafe HTTP method. An unsafe call is one which may modify state in an external system. By default, only `GET`, `OPTIONS`, and `HEAD` requests are considered safe. You can override the safety of a different type of call (such as `POST` or `PUT`) by specifying `"isHttpMethodSafe": true` as a field of the call. This can be useful if you are making multiple requests where one of the earlier requests is a `POST`, and you are using its response in a subsequent request.
- You can specify an array of `retryable-status-codes` which represents HTTP status codes that can be retried.
 - For example, you could configure the Webhook to retry the external request when it receives a 503 response from the server. This option defaults to an empty list.
- You can specify an array of `external-system-not-changed-status-codes` indicating that the server you are connecting to did not change any data despite a failing request.
 - This option defaults to all status codes from 400 to 431.
 - When a Webhook is executed and fails, the indication of whether the external system may have been changed is captured to enable debugging of write failures.

Salesforce

 We recommend using the REST API source to configure webhooks that interact with Salesforce. The legacy task-based webhook options described below are for historical reference only. Legacy Salesforce sources should be [migrated](#) to use the new configuration options.

The following task types are available for Salesforce webhooks:

- `create-record-salesforce-webhook-task`: Creates a Salesforce record of the given type.
- `update-record-salesforce-webhook-task`: Updates a Salesforce record of the given type.
- `delete-record-salesforce-webhook-task`: Deletes a Salesforce record.
- `composite-salesforce-webhook-task`: Modifies Salesforce record using a [Salesforce composite request](#).

API Reference ↗

Send feedback

Below are examples of the Task body for each Task type.

create-record-salesforce-webhook-task

This Task type corresponds to [this Salesforce API ↗](#).

```
1 {
2   "record-type-name": "Account",
3   "data": {
4     "Name": {{json name}},
5     "Industry": {{json industry}},
6     "BillingCountry": {{json country}}
7   }
8 }
```

update-record-salesforce-webhook-task

This Task type corresponds to [this Salesforce API ↗](#).

```
1 {
2   "record-type-name": "Account",
3   "record-id": {{json record-id}},
4   "data": {
5     "Name": {{json name}},
6     "Industry": {{json industry}},
7     "BillingCountry": {{json country}}
8   }
9 }
```

delete-record-salesforce-webhook-task

This Task type corresponds to [this Salesforce API ↗](#).

API Reference ↗

```
2   "record-type-name": "Account",
```

Send feedback

```
3   "record-id": {{json record-id}}
4 }
```

composite-salesforce-webhook-task

Information about the `collateSubrequests` option is available in the [Salesforce documentation](#).

In the example below, we use `"@{createAccount.id}"` to refer to the ID of the record created in the first subrequest. Learn more about dependent requests in the [Salesforce documentation](#).

```
1 {
2   "request": {
3     "collateSubrequests": true,
4     "compositeRequest": {
5       "createAccount": {
6         "type": "createRecord",
7         "createRecord": {
8           "recordTypeName": "Account",
9           "data": {
10            "Name": {{json name}}
11          }
12        }
13      },
14      "updateId": {
15        "type": "updateRecord",
16        "updateRecord": {
17          "recordTypeName": "Account",
18          "data": {
19            "Industry": {{json industry}}
20          },
21          "recordId": "@{createAccount.id}"
22        }
23      }
24    }
25  }
26 }
```

[API Reference](#)

[Send feedback](#)

For SAP-specific webhook configuration, see the webhooks section of the [SAP ERP](#) connector documentation. We recommend running SAP webhooks on a [Foundry worker](#) in Foundry-managed cloud compute.

Limits

For each Webhook, you can set three types of limits that constrain how the Webhook can be executed: time limits, concurrency limits, and rate limits.

A **time limit** allows you to set the maximum duration a Webhook should execute for. This enables you to ensure that end-user applications remain responsive and show a timeout error when the external system to which you are connecting is taking too long to respond. The default timeout is 20 seconds if no value is provided. The maximum allowed timeout value is 180 seconds.

A **concurrency limit** specifies the maximum number of Webhook executions that run at a single time. This can be defined to avoid overwhelming an external system with too many concurrent requests.

A **rate limit** restricts how many times a Webhook can be executed within a time window that you specify. If you want to guarantee that a Webhook will only execute a specific number of times every second, minute, hour, or day, you can enable this type of limit.

Webhook functions

Beta

Publishing a webhook as a function is in the [beta](#) phase of development. Functions published from webhooks cannot be packaged in Marketplace products.

Publishing a webhook as a function turns it into a reusable function for other applications. To publish, toggle on **Function configuration** when creating a webhook, or publish it directly from the [API Reference](#)  **configuration** section on an existing webhook. All webhook versions created after function publishing is enabled will also be published as functions. This operation is irreversible and the function published from the webhook will be the only version available.

[Send feedback](#)

Webhook functions can be invoked from the same surfaces as other Ontology functions:

- [Workshop](#) modules
- [TypeScript v2 functions](#) and [Python functions](#)
- The [Ontology SDK \(OSDK\)](#).
- [Function-backed actions](#)
- Machinery and [Automate](#)

Permissions

Permissions to create, configure, and delete webhooks extend from the permissions on the associated [source](#).

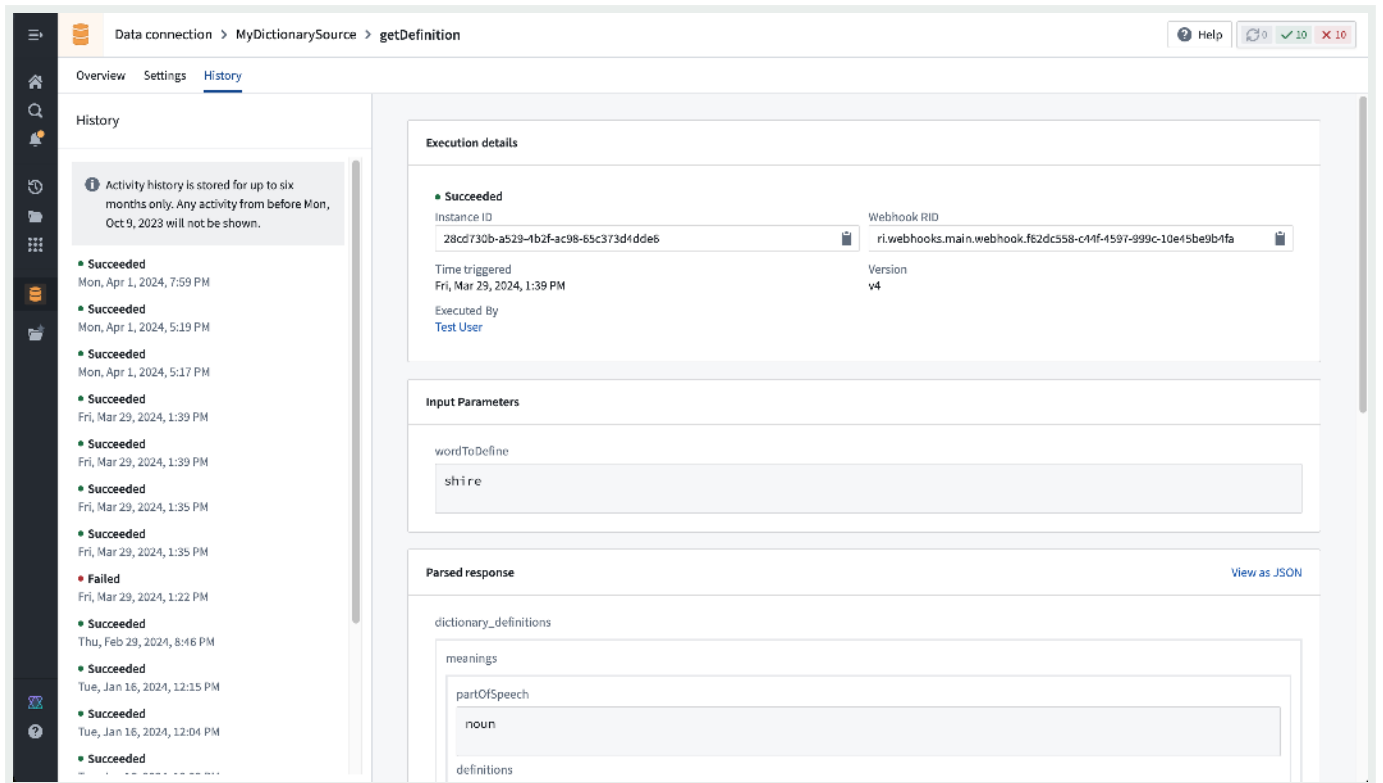
Learn more about [Data Connection permissions](#) and see [more details about Webhook permissions](#).

Webhook history

The webhook history displays a timeline of when webhooks were triggered. The history metadata will always include the user ID who triggered the webhook, a timestamp, and the success or failure status of the execution, as shown below:

API Reference ↗

Send feedback



By default, inputs passed to the webhook and the full response will only be visible to the user who called the webhook. This protects any sensitive data passed in or returned from the webhook call. The `webhooks:read-privileged-data` permission will allow access to the full history, and is not granted to any users by default. A custom role with this permission is required to access the full history for a webhook.

Other options

Store responses

By default, webhook responses will be stored and displayed in the history view for six months. The full response will only be visible to the user who executed the webhook and any admin with the `webhooks:read-privileged-data` permission.

This option may be disabled entirely for a webhook that is known to return sensitive information that should not be stored in the webhook history.

[API Reference ↗](#)

OAuth 2.0 with Webhooks

[Send feedback](#)

Authorization code grant

Palantir Webhooks support calling endpoints using an OAuth 2.0 authorization code grant flow. This requires using an [outbound application](#) to define the interaction with the OAuth 2.0 server. Once configured, an outbound application may be used as the authentication for a REST API Webhook and will prompt individual users to authenticate with the OAuth server when attempting to execute the Webhook.

Client credentials grant

Webhooks may be used to perform an OAuth flow using client credentials. The client credentials grant flow normally uses a long-lived secret to retrieve a short-lived access token that may be used to perform actions against the target system. REST API Webhooks support chaining multiple API calls in a row and can be used to perform the client credentials handshake followed by the desired request as a single execution.

The following walkthrough explains how to configure a client credentials grant flow against a notional example system.

To proceed, you need the following information:

- The **OAuth 2.0 server domain** and path to the **token endpoint**. The token endpoint is used to retrieve a valid token for making a request to the **resource domain**.
 - In many cases, the OAuth 2.0 server domain and resource domain are identical. For example, this is the case for the Foundry API when authenticating using a [third-party application](#).
- The **resource domain** and path to the desired endpoint you wish to call using an Authorization header containing the bearer token retrieved from the **OAuth 2.0 server**.
- The **client ID** for the OAuth 2.0 server.
- The **client secret** for the OAuth 2.0 server.



Not all OAuth 2.0 servers adhere precisely to the OAuth 2.0 standard. The Webhooks request builder interface is intended to be flexible enough to

[API Reference](#) ↗

any non-standard configuration you may need to do to successfully authenticate. We recommend reviewing the documentation for connecting to before attempting to follow this tutorial.

[Send feedback](#)

Create a REST API source

First, choose to create a **+ New source** in the Data Connection application, and select **REST API** as the source type. Learn more about the [REST API source type](#).

When configuring the source, add both the **OAuth 2.0 server domain** and the **resource domain** as shown below. Do not select any authentication option, since we will be manually performing the OAuth 2.0 handshake directly within the sequence of webhook calls.

In the section for **Additional secrets**, add a new secret and enter the **ClientSecret** that will be included when making requests to the token endpoint. We will reference this value when constructing our Webhook call; when entered here, the `ClientSecret` will be encrypted and never exposed, even to other editors or owners of the source in Foundry.

The screenshot shows the 'Data connection > Client Credentials Webhook Example' configuration page. The left sidebar has tabs for 'Overview', 'Connection settings' (selected), and 'Webhooks'. Under 'Connection settings', there are sections for 'Name and location', 'Connection details' (selected), and 'Code import configuration'. The main area is titled 'Source Setup' and contains two sections: 'Domains' and 'Additional secrets'. The 'Domains' section lists two domains: 'https://oauth2-server-domain.com' and 'https://resource-domain.com', both with 'No auth' status and 'Remove' buttons. Below them is an 'Add new domain' button. The 'Additional secrets' section has a warning icon and text: 'Define additional encrypted secrets that can be referenced in webhook configurations. Removing secrets can break existing configuration.' It includes a 'ClientSecret (required)' field with a masked value and an 'Edit' button, and a 'New secret name...' input field with a plus icon. At the bottom are 'Back' and 'Save' buttons.

Build a Webhook that performs the client credentials handshake

Once the source is created, select the option to create a new Webhook.

The Webhook will consist of two chained calls:

API Reference ↗

- An initial call to the **token endpoint** to retrieve a valid bearer token

Send feedback

- A subsequent call that includes the parsed bearer token in a request to the desired endpoint on the **resource domain**.

An example of how to configure the first call is shown below:

The screenshot displays the 'Calls' configuration window. At the top, a list of calls shows '1' with a 'POST' method and the URL 'https://oauth2-server-domain.com/token'. Below this, the configuration for the selected call is shown. The 'POST' method is selected in a dropdown, and the URL is partially visible as 'https://oauth2-server-do...'. The 'token' endpoint is entered in the path field. The 'Body' tab is selected, showing 'Form URL Encoded' as the body type. Below this, three key-value pairs are listed: 'grant_type' pointing to 'client_credentials', 'client_id' pointing to '0c6541a4-e306-4d72-a58d-17ccbd7a9ae6', and 'client_secret' pointing to 'ClientSecret'. A '+ Add new entry' button is at the bottom of the list. At the bottom of the configuration block, there are buttons for '+ Add new call' and 'Import call from cURL'. On the right side of the configuration block, there is a 'Read API' dropdown menu.

i The parameters shown above in the call to the **token endpoint** are standard for many systems that use OAuth 2.0. However, the names of the fields may vary, and other fields may be required. Consult the documentation of the system you are connecting to and construct a request that is compatible with the token endpoint provided by that system.

Often, the request type will need to be `POST`, which defaults to being a "write API" call. In fact, it is usually safe to perform repeated calls to the token endpoint, and

[API Reference ↗](#)

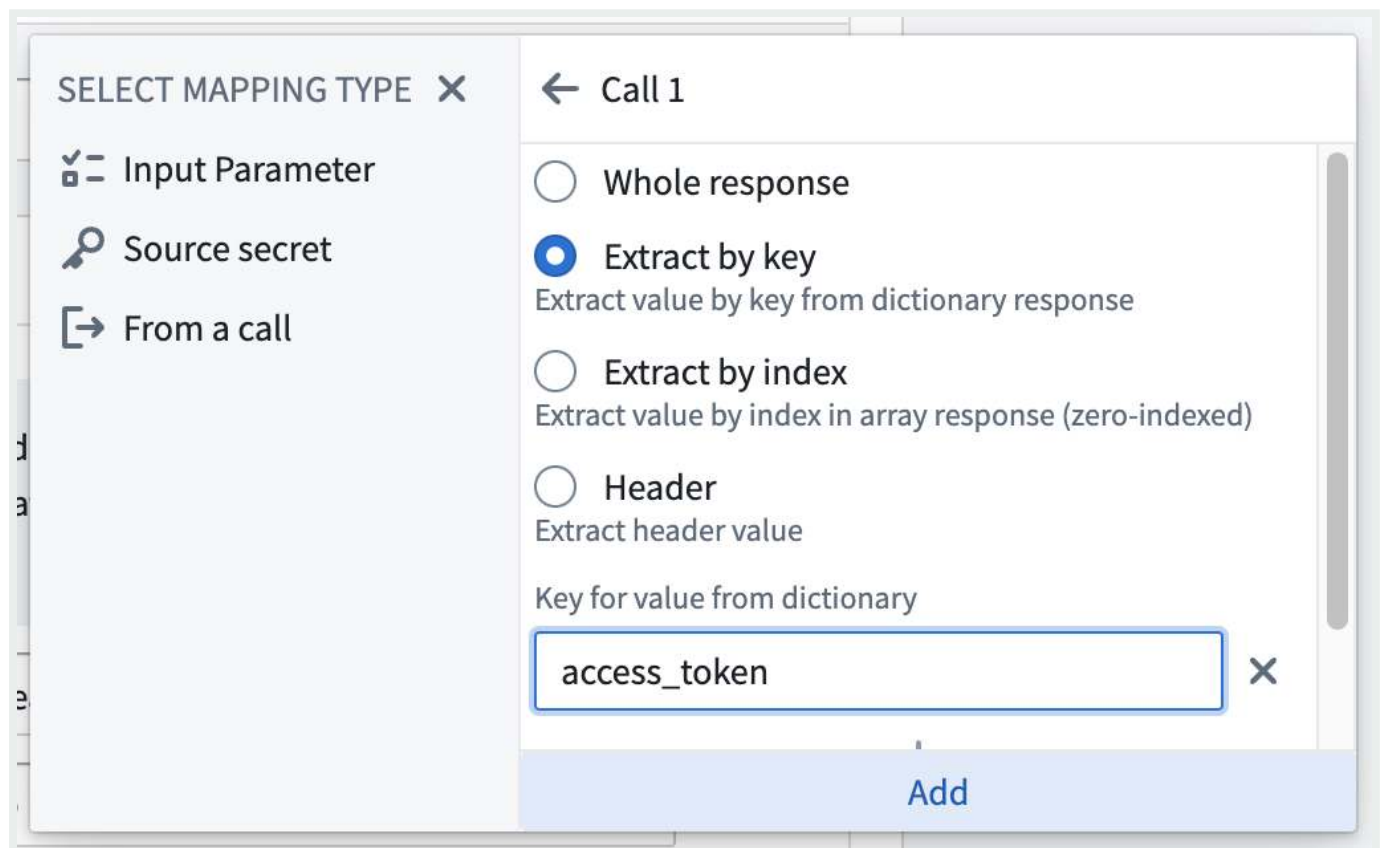
this call as a "read API" call instead using the dropdown selector on the right side of the configuration block for that call. In general,

[Send feedback](#)

perform multiple write API calls are not allowed, since transactionality across multiple requests cannot be guaranteed.

In the second call, build the desired request using the available configuration options. To inject the bearer token into an authorization header, go to the **Headers** section of the call configuration and add a header as shown in the example below . You can use `@` to reference values from the previous call. Most OAuth 2.0 servers will return a JSON parameter called `access_token` or similar. Choose the option to add a value **From a call**, then select **Extract by key** and enter the value of the key you wish to extract from the response to the first call you made.

An example of how to select the bearer token from the first call is shown in the screenshot below:



Once configured, your completed Webhook with two chained calls should look similar to this example:

[API Reference ↗](#)

[Send feedback](#)

Data connection > New webhook

Client Credentials Webhook Example
New Webhook

- Overview
- Webhook name and connection
- Request configuration**
- Responses
- Storage and retention
- Execution policy
- Summary

Need help? Check out the Webhook setup documentation [Open](#)

Back

Create webhook and continue →

Form URL Encoded
Provide key-value pairs to be included using the application/x-www-form-urlencoded Header

grant_type → client_credentials

client_id → 0c6541a4-e306-4d72-a58d-17ccbd7a9ae6

client_secret → ClientSecret

+ Add new entry

2 GET https://resource-domain.com/read-data

GET https://resource-domain... read-data

Query Params Authorization **Headers** 1 Body

Headers are provided as key value pairs to be included in the request. Some headers may be automatically added based on the selected authentication type or the connection details provided by the Source.

Authorization → Bearer Call 1 > "access_token"

+ Add new header

+ Add new call + Import call from cURL

Finally, we highly recommended disabling storing full responses from the first call of the Webhook. If responses are stored, the bearer tokens contained in them may be visible to other users with permission to view full Webhook history. An automatic prompt will usually appear on the **Storage and retention** page of the Webhook configuration to disable storing history for requests to a token endpoint, as shown below:

Data connection > New webhook

Client Credentials Webhook Example
Version 1

- Overview
- Webhook name and connection
- Request configuration
- Responses
- Storage and retention**
- Execution policy
- Summary

Need help? Check out the Webhook setup documentation [Open](#)

Back

Test Webhook

Storage and retention

Basic information about each webhook execution is always recorded and stored for **6 months**. This includes time of the execution, input values, user who triggered the webhook, and error details from failed requests.

In addition, the **full response** will also be stored, unless opted out, including:

- Output parameters' values,
- Response body for every successful request.

If you do not want to record the full response for any call, opt out below.

Storage and retention

1 POST https://oauth2-server-domain.com/token Record full response ☒

2 GET https://resource-domain.com/read-data Record full response ☒

This call might contain secrets in its full response. You may want to opt-out of recording full responses for this sensitive call.

API Reference ↗

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

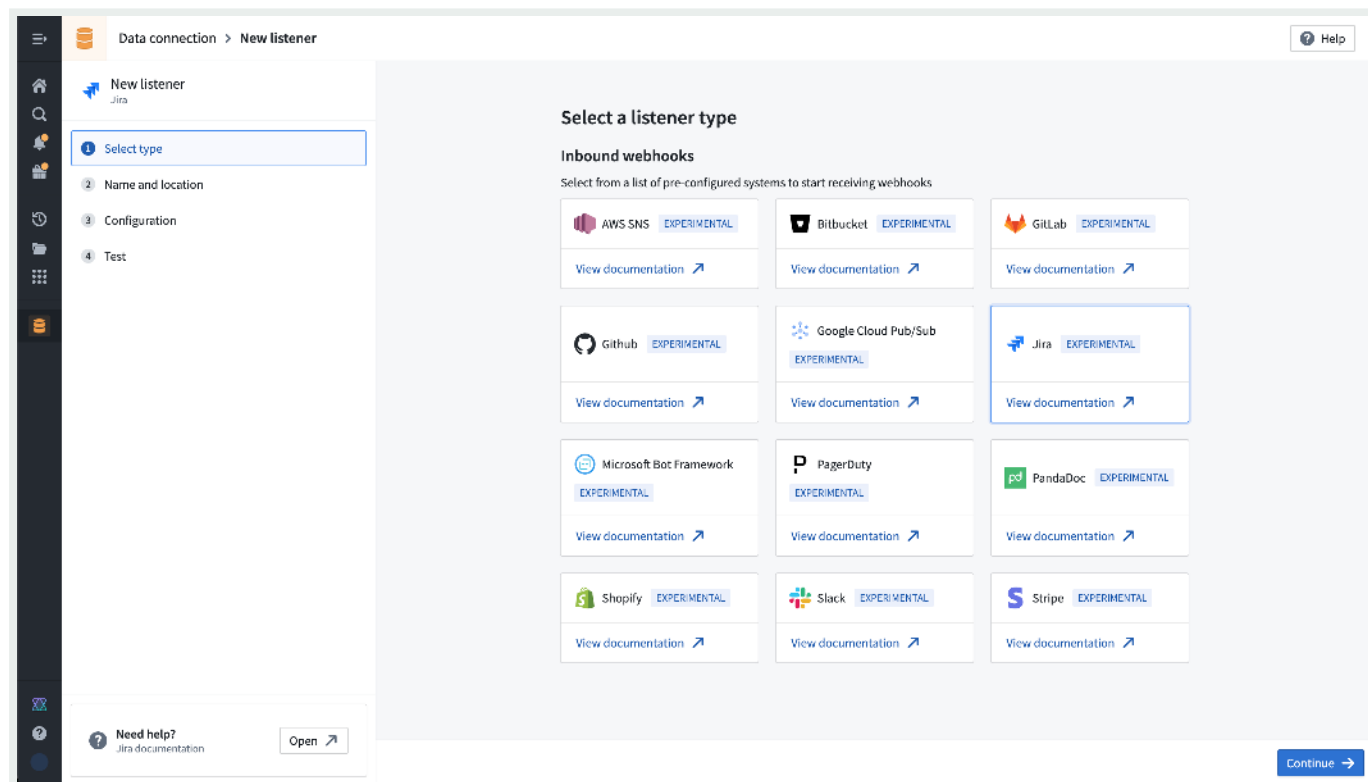
[API Reference ↗](#)

[Send feedback](#)

Listeners

Listeners enable the Palantir platform to receive events from other systems that do not support OAuth 2.0 authentication directly or cannot provide a configurable payload compatible with standard Foundry API endpoints.

To accept inbound connections from these systems, data connection listeners provision a URL endpoint, implement the specific message signing or other verification schemes for specific external systems, and allow a simple and low-latency mechanism to receive data feeds into Foundry.

[API Reference ↗](#)

Types of listeners

[Send feedback](#)

Foundry supports three types of listeners, each designed for different integration scenarios and processing approaches:

Listener type	Status	Connection type	Output destination
HTTPS listeners	GA	Request-response	Streams
WebSocket listeners	Experimental	Persistent bidirectional	Compute modules
Email listeners	Beta	Inbound email	Media sets

HTTPS listeners

HTTPS listeners receive webhook requests from external systems through HTTPS endpoints. Events are written to a [stream](#), enabling processing through [Automate](#), [streaming pipelines](#), or batch processing using the stream's backing dataset.

[Learn more about HTTPS listeners.](#)

WebSocket listeners [Experimental]

Experimental

WebSocket listeners are an experimental capability that may not be available in your enrollment. To enable this capability, contact Palantir Support.

WebSocket listeners are primarily designed for real-time audio and telephony workflows, enabling bidirectional communication between Foundry and external services such as Twilio, Genesys, and Azure Communication Services. Inbound WebSocket connections are processed directly by a [compute module](#) where you can host a custom WebSocket server.

[Learn more about WebSocket listeners.](#)

API Reference ↗

Email listeners

Send feedback

Beta

Email listeners are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to request access to email listeners.

Email listeners allow Foundry to receive inbound emails at dedicated email addresses for downstream processing. Each listener has a unique email address scoped to your enrollment, and applies sender allowlisting, email authentication, and attachment validation before forwarding content.

[Learn more about email listeners.](#)

When to use listeners

Listeners are one of many options to ingest data into the Palantir platform. Before setting up a listener, evaluate whether your use case meets the criteria below for inbound connections or for external systems that cannot be customized.

For inbound connections

Listeners can be used solely for inbound connections when an external system establishes the connection into Foundry. To make requests from Foundry to external systems, use [sources](#). No-code capabilities are also available, such as [syncs](#) and [outbound webhooks](#). Custom code-based data connections are also available with [external transforms](#), [functions](#), [compute modules](#), and more.

When the external system cannot be customized

Listeners should only be used when the external system cannot be customized to connect to Foundry. For example, many webhook and WebSocket providers have bespoke payload schemas. [API Reference](#) ↗ properly authenticate with Foundry, instead relying on other authentication and verification protocols.

Send feedback

For cases where you can customize the external system, such as when the system allows you to write custom integration code, use the [public API](#) instead. With the public API, you can push to streams, upload to datasets, or directly interact with your ontology.

All product names, logos, and brands mentioned are trademarks of their respective owners. All company, product, and service names used in this document are for identification purposes only.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

Data connectivity & integration / Data Connection / Listeners / Listener subdomains

Listener subdomains

HTTPS and WebSocket listeners can be mounted to dedicated subdomains allowing for granular ingress control, comprehensive governance workflows, and isolation of less secure endpoints from the environment's primary enrollment domains. All requests to the mounted listeners will then be required to be made over that subdomain.

Listeners can only be mounted to a single subdomain, but a subdomain may be shared by many mounted listeners.

Listener subdomains availability

Subdomains for listeners are not available self-service in every Foundry enrollment. For use in FedRAMP and on-prem enrollments, contact Palantir Support.

Creating a listener subdomain

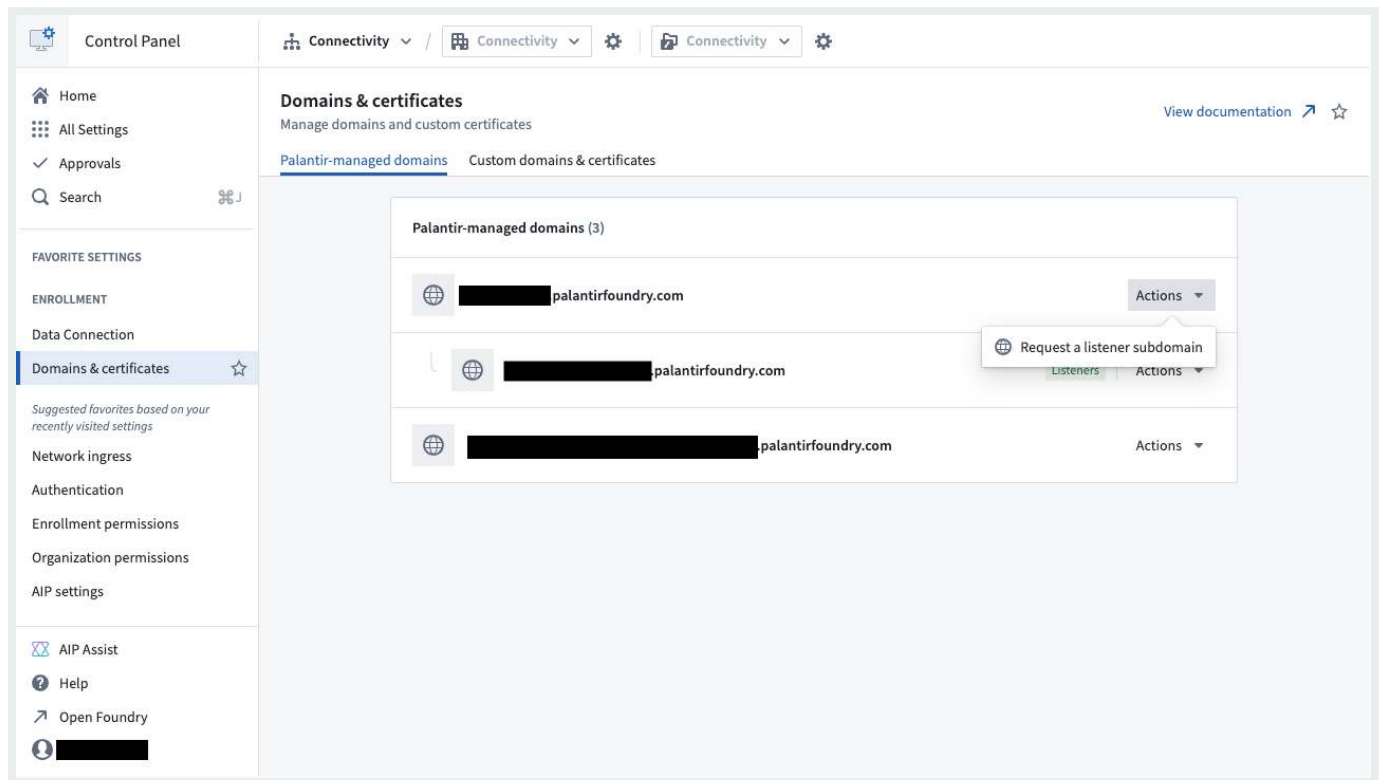
Before mounting a listener to a subdomain, you need to create the subdomain in Control Panel.

Navigate to **Control Panel > Domains & certificates**, find the domain that you would like to create a new subdomain for, and select **Request a listener subdomain**. Once requested, the new subdomain will need to be approved by a user with the **Information Security Officer** role for the enrollment.

[API Reference ↗](#)



[Send feedback](#)



There is a limit of three listener subdomains per enrollment. Contact Palantir support if more are needed.

Ingress allowlisting

Listener subdomains can be configured in one of two modes: custom ingress or inherited ingress.

Custom ingress

A subdomain with custom ingress will have a separate ingress configuration from its parent domain. For example, your enrollment may allow ingress from only your corporate IP addresses. However, listener subdomains can be configured to allow ingress from entire countries or specific IP ranges that you otherwise do not want to allow to access the rest of your enrollment.

Configuring appropriately sized ingress allowlists for specific use cases enables you to restrict access to listener subdomains in instances where listeners are using nonstandard authentication or authorization protocols.

[API Reference ↗](#)

[Send feedback](#)

Some example scenarios of ingress configurations for listener subdomains might include:

- Adding country-wide ingress in the regions that the external system is hosted in when they do not publish any specific list of IP addresses, or if their published list changes frequently.
- Configuring a small IP range (smaller than the primary enrollment ingress allow list) to allow requests to a listener with only basic authorization or header secret verification available.

Once the subdomain is created, you can manage ingress in **Control Panel > Network ingress**. [Learn more about ingress configuration.](#)

Inherited ingress

In some situations, the ingress allowlist configured for the primary domain is sufficient for usage with listeners. In these cases, you can create subdomains to inherit the ingress allowlist configuration from the parent domain. Any changes to the ingress configuration of the parent domain will be reflected automatically by the subdomain.

Once created, the subdomain cannot be reconfigured with custom ingress.

Using a listener subdomain

1. Navigate to the **Listeners** tab in Data Connection and select a listener.
2. In the **Configure connection** step of the listener settings wizard, select a listener subdomain.
3. After requesting a subdomain for your listener, an Approvals request will be created, which an administrator will need to approve before the listener becomes accessible.

API Reference ↗

Send feedback

Data Connection > Listeners > New listener

Help

New listener

Stack

- Select type
- Name and location
- Configure connection
- Test

Need help? [Stack documentation](#) [Open](#)

Back

Configure connection

Stack (HTTPS) [View documentation](#)

Set up endpoint [Set up endpoint](#)

Configure how the external system will connect to your listener

Subdomain

Select

- ██████████.palantirfoundry.com [Select](#)
- ██████████.palantirfoundry.com [Select](#)
- ██████████.palantirfoundry.com [Select](#)

IP address ranges

No IP address ranges

Allowed countries

United States

[Create a new subdomain in Control Panel](#)

Network ingress

Ensure that you have an adequate range of IPs allowlisted on your subdomain to receive requests from the external system

Endpoints [Rotate endpoints...](#)

No endpoints available

Continue

When the mount is approved, the listener will be able to process requests over the given endpoints (after the listener is started, if it is not already running).

Data Connection > Listeners > New listener

Help

New listener

Stack

- Select type
- Name and location
- Configure connection
- Test

Need help? [Stack documentation](#) [Open](#)

Back

Configure connection

Stack (HTTPS) [View documentation](#)

Set up endpoint [Set up endpoint](#)

Configure how the external system will connect to your listener

Subdomain [Approved](#)

██████████.palantirfoundry.com [Edit](#)

The listener will only be reachable by the external system over this subdomain

[Why do I need a subdomain?](#)

Network ingress

Ensure that you have an adequate range of IPs allowlisted on your subdomain to receive requests from the external system [Manage ingress](#)

Endpoints [Rotate endpoints...](#)

URL	Status	Actions
https://██████████.palantirfoundry.com/inbound-message-listener/message/cb27c7fb-aa1f-4b25-98b5-34117da416db	Active	Add

Configure security [Configure security](#)

Verify incoming connections and requests using the provided parameters

Continue

API Reference [API Reference](#) [subdomain for a listener](#)

Send feedback

If you need to change the subdomain that a listener is using, you can select a new one from the **Configure connection** step. This is a destructive action that will cause downtime if the listener is being actively used.

The listener will immediately stop processing requests over the old subdomain, and will not be able to process any further requests until the new subdomain mount is approved. At that point, any usages of the old endpoints will need to be swapped over to the endpoints with the new subdomain.

Migration to subdomains

For listeners created before subdomains were available in an enrollment, a zero-downtime migration path is available. After creating a new listener subdomain, navigate to the **Configure connection** step of the listener settings wizard and follow the provided instructions.

The screenshot shows the 'Configure connection' step of the Twilio Listener settings wizard. The left sidebar contains 'Overview' and 'Settings' tabs, with 'Settings' selected. Under 'Settings', there are sections for 'Name and location', 'Configure connection' (highlighted with a 'Migration required' warning), and 'Test'. The main content area is titled 'Configure connection' and shows 'Twilio (HTTPS)' as the selected provider. Below this, a 'Set up endpoint' section provides instructions on how to migrate the listener to a subdomain for improved security and governance. The instructions include a list of four steps: 1. Request to mount the listener to a subdomain (you may need to create a new subdomain first), 2. Wait for your request to be approved by an Information Security Officer, 3. Swap any usages of your listener's endpoint to use the new endpoint with the subdomain (find this in the 'Configure' step), and 4. Disable 'Legacy administrator approval' below, which will disallow further requests over your primary domain to this listener. Below the instructions, there is a 'Subdomain' section with a dropdown menu set to 'Select' and a note that the listener will only be reachable by the external system over this subdomain. There is also a 'Why do I need a subdomain?' link. The 'Legacy administrator approval' toggle is currently turned on. The 'Network ingress' section has a note to ensure an adequate range of IPs is allowed on the subdomain. At the bottom, there is a 'Need help?' link to Twilio documentation, a 'Back' button, and a 'Continue' button.

Endpoint rotation

API Reference ↗

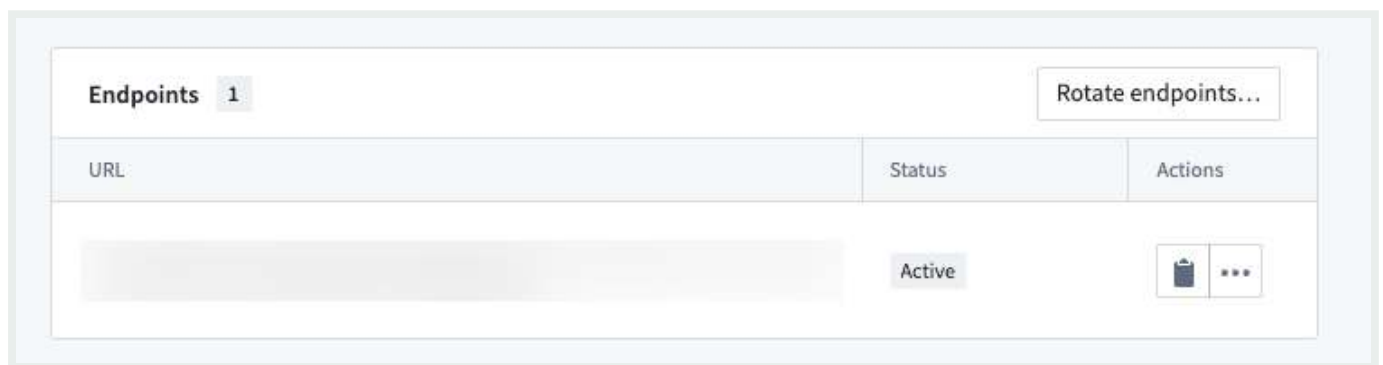
If the listener's endpoint is compromised, it can be migrated to a new endpoint with minimal downtime.

Send feedback

The steps to migrate endpoint usage are as follows:

1. Generate a new endpoint, and add an expiration date for the old endpoint. You should now have two usable endpoints.
2. Replace any usage of your old endpoint with the new endpoint.
3. Delete the old endpoint.

To support this process, listeners provide an endpoint rotation mechanism. To rotate your endpoint, navigate to the **Configuration** tab of your listener's settings and locate the **Rotate endpoints** option. Note that you can only have a maximum of two endpoints at a time, and a maximum of one active endpoint. An active endpoint is an endpoint without a set expiration date.



When rotating your endpoint, you can choose to set an expiration date for the endpoint for zero-downtime rotations, or to delete it immediately. When an endpoint expires, it will no longer be able to process events.

[API Reference ↗](#)

[Send feedback](#)

All product names, logos, and brands mentioned are trademarks of their respective owners. All company, product, and service names used in this document are for identification purposes only.

← PREVIOUS
Overview

NEXT →
HTTPS listeners / Overview

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

HTTPS listeners

HTTPS listeners receive inbound webhook requests from external systems through HTTPS endpoints. Events are written to a [stream](#), enabling processing through [Automate](#), [streaming pipelines](#), or batch processing using the stream's backing dataset.

When to use HTTPS listeners

You can use HTTPS listeners in the following situations:

- The external system supports outbound webhooks.
- You need to receive event notifications when something changes in the external system.
- The external system cannot authenticate with standard OAuth 2.0 or Foundry APIs.

Configure a listener

Navigate to **Data Connection > Listeners** to connect the Palantir platform to external systems and workflows. For information on securely configuring listeners and incoming data, refer to the [HTTPS listener security](#) documentation.

You can explore a listener's streaming dataset in Data Lineage to better understand data transformations and flows.

Throughput and limitations

[API Reference ↗](#)

Throughput event streams should be pushed to HTTPS listeners. Outgoing event streams are rate-limited at approximately 100 requests per second, so integrations requiring

[Send feedback](#)

substantially higher throughput should use [streaming syncs](#) or the [public streaming API endpoints](#) when possible.

If your use case requires a higher rate limit than what is currently provided, and listeners are the only appropriate integration option, contact Palantir Support to discuss increasing this limit.

Individual event and request payloads are limited to 1 MB in size, as this is the maximum row size in the output streaming dataset. Foundry rejects events that exceed this limit.

Supported HTTPS listeners

You can either configure a custom, basic authentication listener, or one of the following listeners:

Listener, linked to external documentation	Platform setup guide
Airtable ↗	
Apollo ↗	
AWS SNS ↗	
Azure Event Grid ↗	
Bitbucket ↗	
Cisco Meraki ↗	
Dialpad ↗	
FedEx ↗	
GitLab ↗	
GitHub ↗	
Google Cloud Pub/Sub ↗	Guide
ION Factory OS ↗	
Jira ↗ and Jira Cloud Automation ↗	Guide
API Reference ↗ Meta ↗	
Microsoft Bot Framework ↗	Send feedback

[PagerDuty ↗](#)[PandaDoc ↗](#)[project44 ↗](#)[RevelocPlus](#)[ShipStation ↗](#)[Shopify ↗](#)[Skydio ↗](#)[Slack ↗](#)[Guide](#)[Stripe ↗](#)[Twilio ↗](#)[Twilio SendGrid ↗](#)[Zendesk ↗](#)

All product names, logos, and brands mentioned are trademarks of their respective owners. All company, product, and service names used in this document are for identification purposes only.

HTTPS listener security

HTTPS listeners differ from standard Foundry data ingestion, so ensure that you understand these security paradigms before enabling your connections.

Request authorization

Request interfaces for HTTPS listeners are defined by external systems, so they do not conform to standard Foundry authentication or authorization mechanisms. Instead, listeners implement the security protocols laid out by those external systems, which vary widely.

Palantir makes no guarantees about the suitability or effectiveness of these external system protocols. You are responsible for ensuring that you understand which guarantees each protocol does or does not provide for the incoming requests and data.

The specific protocols implemented for each listener can be found in the **Configuration** step of the listener setup wizard, as well as the external system's documentation.

Configure connection

 Twilio (HTTPS) [View documentation](#)

Protocol security measures

The following security measures are specified by the external system and implemented by the listener:

- Basic authentication credentials are required and validated with each message

shes using the shared secret key and these signatures are validated for each inbound event
mps are validated

[API Reference](#)

[Send feedback](#)

Redaction and data security

A minimal set of redactions is sometimes performed on incoming data. It is important to note that these redaction mechanisms are best effort, and Palantir cannot guarantee that sensitive data, such as tokens, will be completely redacted from request bodies.

For HTTPS listeners it is essential to *secure both your listener and the output stream*. This includes restricting access to both by placing them in a restricted project, as well as applying [markings](#) on the listener when necessary.

Subdomains

HTTPS listeners can be mounted to dedicated subdomains, allowing for granular ingress control, comprehensive governance workflows, and isolation of less secure endpoints from the environment's primary enrollment domains. [Learn more about listener subdomains](#).

Endpoint rotation

If the listener's endpoint is compromised, it can be rotated to a new endpoint. [Learn more about endpoint rotation](#).

← PREVIOUS
Overview

NEXT →
Event processing

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / HTTPS listeners / Event processing

Event processing with HTTPS listeners

HTTPS listeners write events to a [stream](#). You can locate this stream by navigating to your listener's **Overview** page in Data Connection. Once your data resides in a stream, several processing options are available.

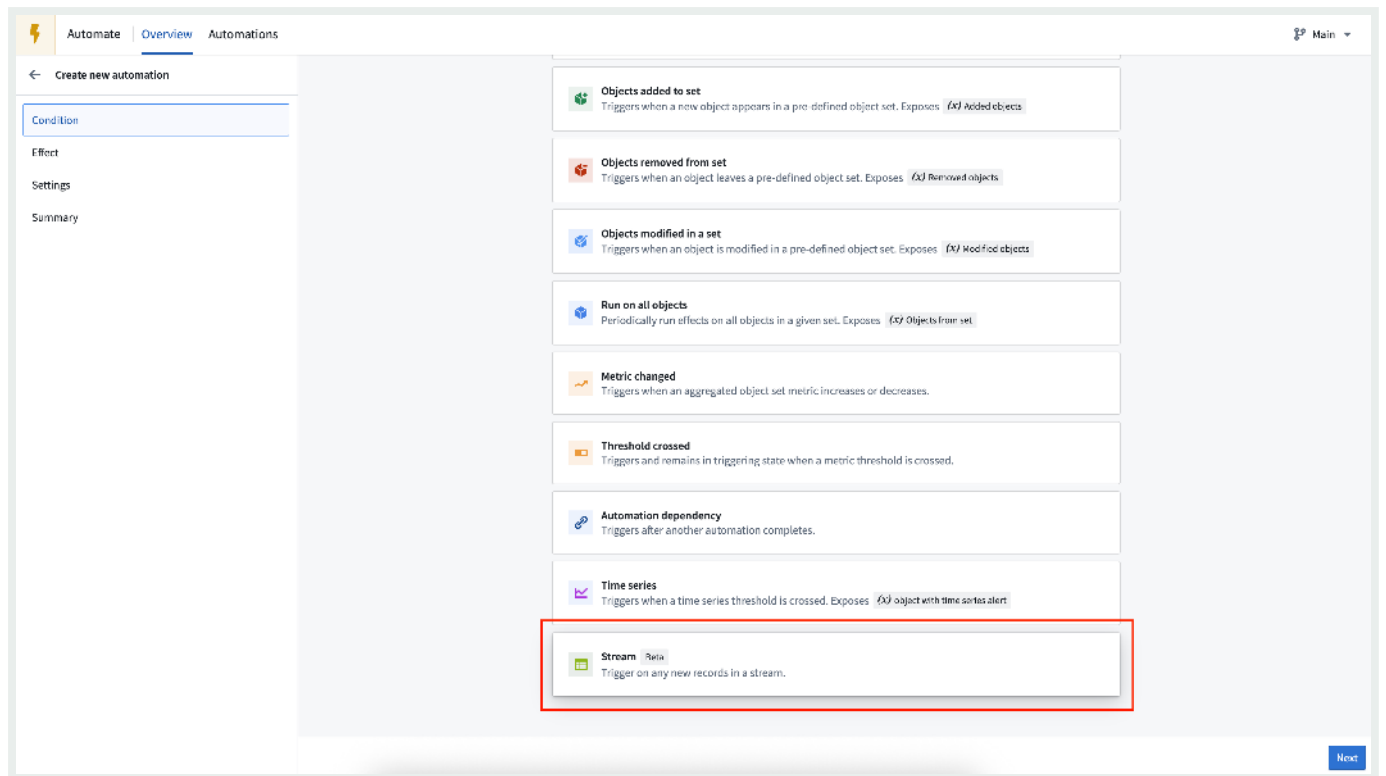
Stream processing with Automate

Streams can be directly processed in [Automate](#), enabling you to execute an action or function for each inbound event. You can create objects in your ontology, run AIP logic, or use [sources](#) in functions to interact with external systems, such as writing data back to the system that sent the event.

API Reference ↗



Send feedback



Processing streaming events with Automate is appropriate when:

- The event stream is not high throughput
- Your event processing is stateless
- Latency of a few seconds is acceptable
- At-least-once processing is sufficient

This approach is suitable for most listener integrations, providing low-cost, low-maintenance event processing workflows.

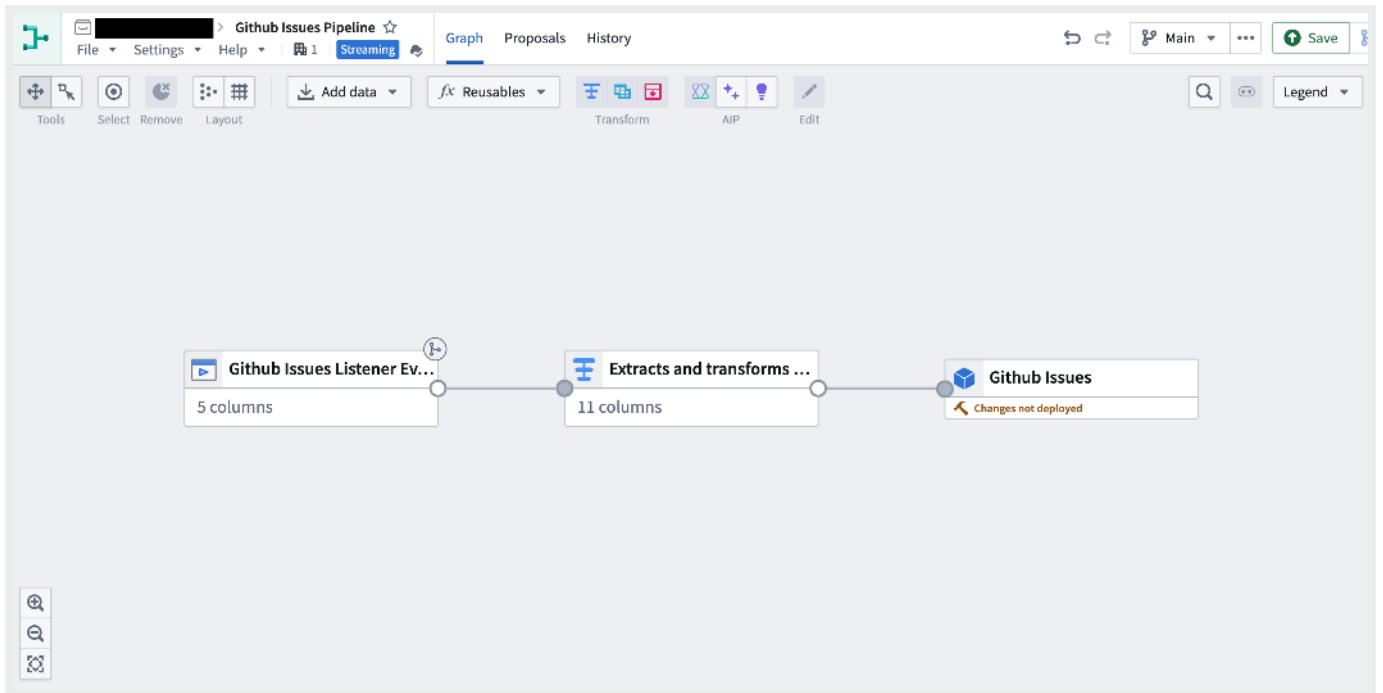
You can learn more about processing listener events with Automate in the [guide to creating an AI-powered chatbot with listeners](#).

Streaming pipelines

Listener event streams can be processed with [streaming pipelines](#) as a lower-latency alternative. These pipelines support high-throughput streams, [stateful event processing](#), and (optionally) exactly-once event processing. Streaming pipelines can additionally [leverage](#) powerful real-time event-processing workflows.

[API Reference](#) ↗

[Send feedback](#)



Batch pipelines

Every few minutes, the listener event stream will [archive into a backing dataset](#). This dataset can be used like any other dataset in the platform, allowing you to build [data pipelines](#) and [back your ontology](#).

The screenshot shows a data table interface for a 'Github Event Stream' dataset. The table has columns: id, timestamp, payload, and queryParams. The first few rows of data are visible. A sidebar on the left shows dataset details: 'Updated: May 1, 2025, 10:49 AM by Foundry', 'Created: Apr 29, 2025, 10:38 AM by Foundry', 'Location: [redacted]', 'Type: Streaming dataset (Streaming)', 'RID: [redacted]', 'Size: 5 columns, 6 files, 52.5KB', and 'Calculate row count'.

id	timestamp	payload	queryParams
f2fec333-c8ca-4dc7-b23c-55a3c89b7b2	2025-04-29T17:04:27.351Z	{"zon":"Responsive is better than empty string	{"x-github-event":"ping","x-github-
a1cc953d-4d59-4b9d-b990-6a31029d08f	2025-04-29T17:04:38.093Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
fa3d7f06-6834-4e37-88a6-54760a44a0f	2025-04-29T17:11:10.735Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
fa97c1b6-3569-4727-82fe-ca7508f268f	2025-04-29T17:11:39.816Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
68f2914d-77a4-4e85-ba38-7852c4f55f5	2025-04-29T17:11:14.744Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
ba3a68ec-2462-411f-b2d6-c965c384df4	2025-04-29T17:11:15.276Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
f90f713a-f020-433a-a60f-4f15d0ddfe5	2025-04-29T17:11:14.958Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
1a5308a2-f047-4f49-a441-91a40775d0f	2025-04-29T17:11:28.596Z	{"action":"closed","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
2242335c-f574-4c88-94d3-893bb39142f	2025-04-29T17:15:29.362Z	{"zon":"Find your words, they are empty string	{"x-github-event":"ping","x-github-
196988a2-b4e3-405a-bdfe-c5355f0920f	2025-04-29T17:16:18.311Z	{"action":"added","member":{"login":"empty string	{"x-github-event":"member","x-github-
f961dbba-1a62-46ec-82ac-6af90c9287f	2025-04-29T17:30:22.154Z	{"action":"added","member":{"login":"empty string	{"x-github-event":"member","x-github-
4524f57e-4925-4627-ac7e-c9b97892b6f	2025-04-29T17:41:24.368Z	{"ref":"refs/heads/main","before":"empty string	{"x-github-event":"push","x-github-
4a0c3a21-7db4-431b-b3ba-bcf40e108a4	2025-04-29T18:18:03.193Z	{"action":"labeled","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
f278a30a-5069-418f-9a06-74c852e15f1	2025-04-29T18:18:04.184Z	{"action":"labeled","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
9a0b0d8d-ed22-421c-abe1-092f389bc9f	2025-04-29T18:19:33.495Z	{"action":"labeled","issue":{"url":"empty string	{"x-github-event":"issues","x-github-
52837f1c-2dc5-4a3b-b8d6-44d8566a0cf	2025-04-29T18:18:35.405Z	{"action":"opened","issue":{"url":"empty string	{"x-github-event":"issues","x-github-

For use cases that require historical analysis or that are not real-time, batch pipelines should be used.

← [API Reference](#) →

← [Set up a Google Pub/Sub listener](#) →

[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / HTTPS listeners / Set up a Google Pub/Sub listener

Set up a Google Cloud Pub/Sub listener

This guide shows step-by-step how to configure a listener for Google Pub/Sub to get a real-time feed of events from a Pub/Sub topic to a Foundry streaming dataset.

Google Pub/Sub can be used in conjunction with various Google Cloud services to route events to Foundry. For example, it can be used for streaming emails from a Gmail account. [Review guidance provided by Google for streaming from Gmail ↗](#).

[Learn more about Google Pub/Sub. ↗](#)

Prerequisites

Prior to configuration, ensure:

- Your enrollment's ingress policy has been appropriately configured to accept Google Pub/Sub requests. You can either allow-list the [IPs of your Google Cloud region ↗](#), or the countries. [Learn how to Configure ingress](#). Note that the provider may alter the IPs anytime after the listener has been implemented and you would need to update ingress appropriately.
- You have access to Pub/Sub in the Google Cloud Console.

API Reference ↗



Send feedback

1. Create a Pub/Sub topic if you do not already have one. This is the stream of events that will be pushed to your Foundry Listener.

Google Cloud | My First Project | Search (/) for resources, docs, products and more

Pub/Sub / Topics / Create topic

Pub/Sub

- Topics
- Subscriptions
- Snapshots
- Schemas

Pub/Sub Lite

- Lite reservations
- Lite topics
- Lite Subscriptions

Release notes

Create topic

Topic ID *
myTopic

Topic name: projects/[redacted]/topics/myTopic

- ☐ Add a default subscription
- ☐ Use a schema
- ☐ Enable ingestion
- ☐ Enable message retention
- ☐ Export message data to BigQuery
- ☐ Backup message data to Cloud Storage

Encryption

- ☒ Google-managed encryption key
Keys owned by Google
- ☐ Cloud KMS key
Keys owned by customers

CREATE

2. Navigate to the **Listeners** tab in Data Connection. Create a Foundry Pub/Sub listener by selecting Google Cloud Pub/Sub from the listener type menu. This step will not finish setting up the listener, but is required to generate the listener URL which is then used in step 3.

Data connection > New listener

New listener
Google Cloud Pub/Sub

1. Select type
2. Name and location
3. Configuration
4. Test

Configuration

Endpoints 1

https://[redacted].com/inbound-message-listener/message/[redacted]

Google Cloud Pub/Sub Configuration

Service account email

The email address of the service account associated with the topic subscription.

Audience claim

If specified, the audience claim of the token of each request will be verified to match the provided value.

Shared secret

The shared secret appended to the query parameters of the URL in Google Cloud (e.g. "token=YOUR_SECRET"). If specified, the shared secret will be checked for each incoming request.

API Reference ↗

documentation

Open ↗

Back

Send feedback

3. Create a subscription for your Google Pub/Sub topic from step 1.

- a. Select **Push** as the subscription type.
- b. Copy the URL generated from step 2 into the **URL** field for your push subscription.
- c. Tick the box to enable **Authentication**. [Review Pub/Sub authentication information on external documentation.](#) ↗
 - Select a service account to use for authentication.
 - To use a shared secret, enter your shared secret into the URL field as a query parameter after the listener endpoint URL. Example: `?token=<YOUR_TOKEN>`
- d. Save your subscription.

The screenshot shows the Google Cloud console interface for adding a new subscription to a topic. The left sidebar shows the navigation menu with 'Topics' selected under 'Pub/Sub'. The main panel is titled 'Add subscription to topic' and contains the following fields and options:

- Subscription ID ***: A text input field.
- Subscription name**: A text input field with a placeholder value.
- Topic name**: A text input field with a placeholder value.
- Delivery type**: A section with three radio button options:
 - ☐ Pull
 - ☒ Push
 - ☐ Write to BigQuery: A variant of the push operation. Select this option if you want Pub/Sub to deliver messages directly to an existing BigQuery table. [Learn more](#)
 - ☐ Write to Cloud Storage: A variant of the push operation. Select this option if you want Pub/Sub to deliver messages directly to an existing Cloud Storage bucket. [Learn more](#)
- Endpoint URL ***: A text input field with a placeholder value.
- ☒ **Enable authentication** [Learn more](#)
- Service account**: A dropdown menu showing 'foundry-user@...iam.gserviceaccount.com'.
- Audience (optional)**: A text input field with the value 'connectivity-pub-sub-demo'.
- ☐ **Enable payload unwrapping**: Remove all metadata from messages. Only delivers the raw, decoded message data. [Learn more](#)

4. Copy the **service account email address**, optional **audience claim**, and optional **shared secret** into the listener configuration as shown below then **Continue**.

API Reference ↗

Send feedback

Data connection > New listener

New listener
Google Cloud Pub/Sub

- Select type
- Name and location
- Configuration**
- Test

Need help?
Google Cloud Pub/Sub documentation

Open

Back

Continue

Configuration

Endpoints 1

https://...:com/inbound-message-listener/message/

Google Cloud Pub/Sub Configuration

Service account email

foundry-user@...iam.gserviceaccount.com

The email address of the service account associated with the topic subscription.

Audience claim

connectivity-pub-sub-demo

If specified, the audience claim of the token of each request will be verified to match the provided value.

Shared secret

[redacted]

The shared secret appended to the query parameters of the URL in Google Cloud (e.g. "token=YOUR_SECRET"). If specified, the shared secret will be checked for each incoming request.

- Administrator approval is now required from the Information Security Officer. Review the toggle description.
- Select **Start** on the listener **Test** page to turn on your listener and start accepting events.
- Send a test message to your topic, and see it appear in the listener test interface.

Google Cloud My First Project

Pub/Sub / Topics / Topic: myTopic

Pub/Sub

- Topics
- Subscriptions
- Snapshots
- Schemas

Pub/Sub Lite

- Lite reservations
- Lite topics
- Lite Subscriptions

myTopic

Topic name

SUBSCRIPTIONS

SNAPSHOTS

Test if your subscription can receive a message

Step 1

Manually publish a message containing

PUBLISH MESSAGE

Manually publish

Step 2

Select a Cloud Pub/Sub subscription to

Select a Cloud Pub/Sub subscription to

Publish message

Topic name

projects/.../topics/myTopic

Publish count

You can publish the given message once or multiple times in an interval. This can be useful for getting messages in new subscriptions and testing. For a more robust way to publish messages multiple times, consider using Cloud Scheduler.

Number of messages *

1

Enter an amount between 1 and 100.

Message interval (seconds)

1

How long to wait before publishing the next message

Message body

The message that you want to publish to this topic. Either message or attribute will be required to publish.

Message *

Hello world

Message size should not exceed 10 MB.

Message attributes

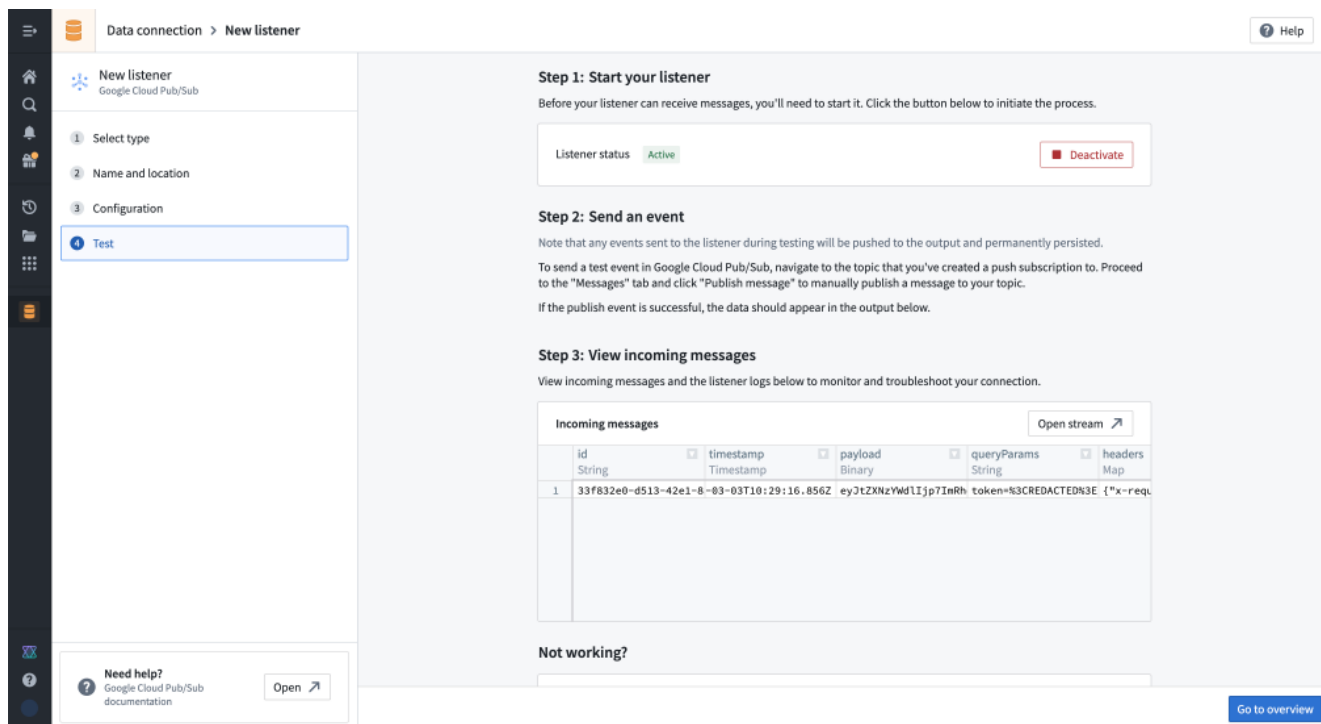
At most 100 attributes per message, attribute key should not start with 'goog' and should not exceed 256 bytes, attribute value should not exceed 1024 bytes.

PUBLISH

CANCEL

[API Reference](#)

[Send feedback](#)



i The payload arrives base64 encoded. You can decode it in a streaming pipeline using the base64 decode transformation board in order to get a string representation of the message.

All screenshots of Google Cloud Pub/Sub™ are provided for reference purposes only and are the property of Google LLC.

PREVIOUS
← Event processing

NEXT
Set up a Jira listener →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / HTTPS listeners / Set up a Jira listener

Set up a Jira listener

This guide shows step-by-step how to configure a listener for Jira Cloud, to get a real-time feed of events from Jira to a Foundry streaming dataset.

[Learn more about Jira.](#) ↗

Prerequisites

Prior to configuration, ensure:

- Your enrollment's ingress policy has been appropriately configured. [Learn how to Configure ingress.](#) Review [Jira documentation pertaining to IP addresses and domains](#) ↗.
- You must have your own instance of Jira with administrator access.

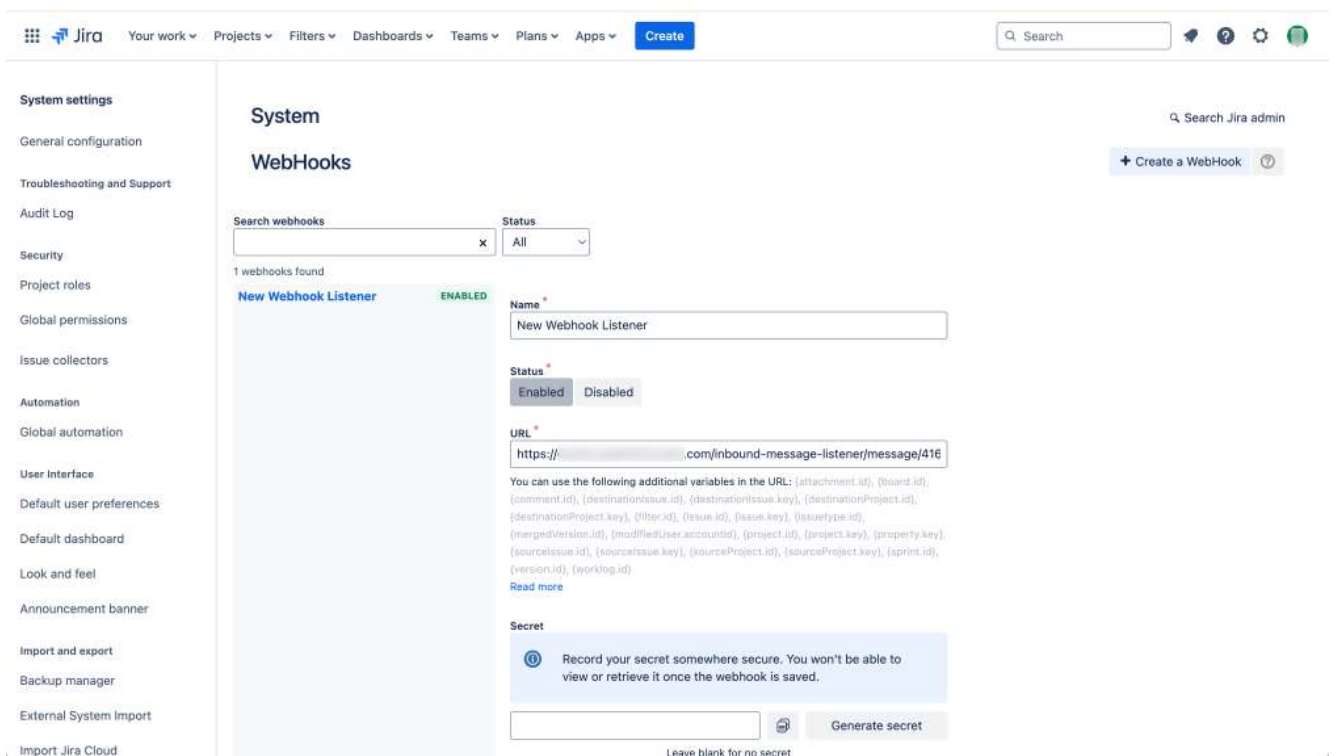
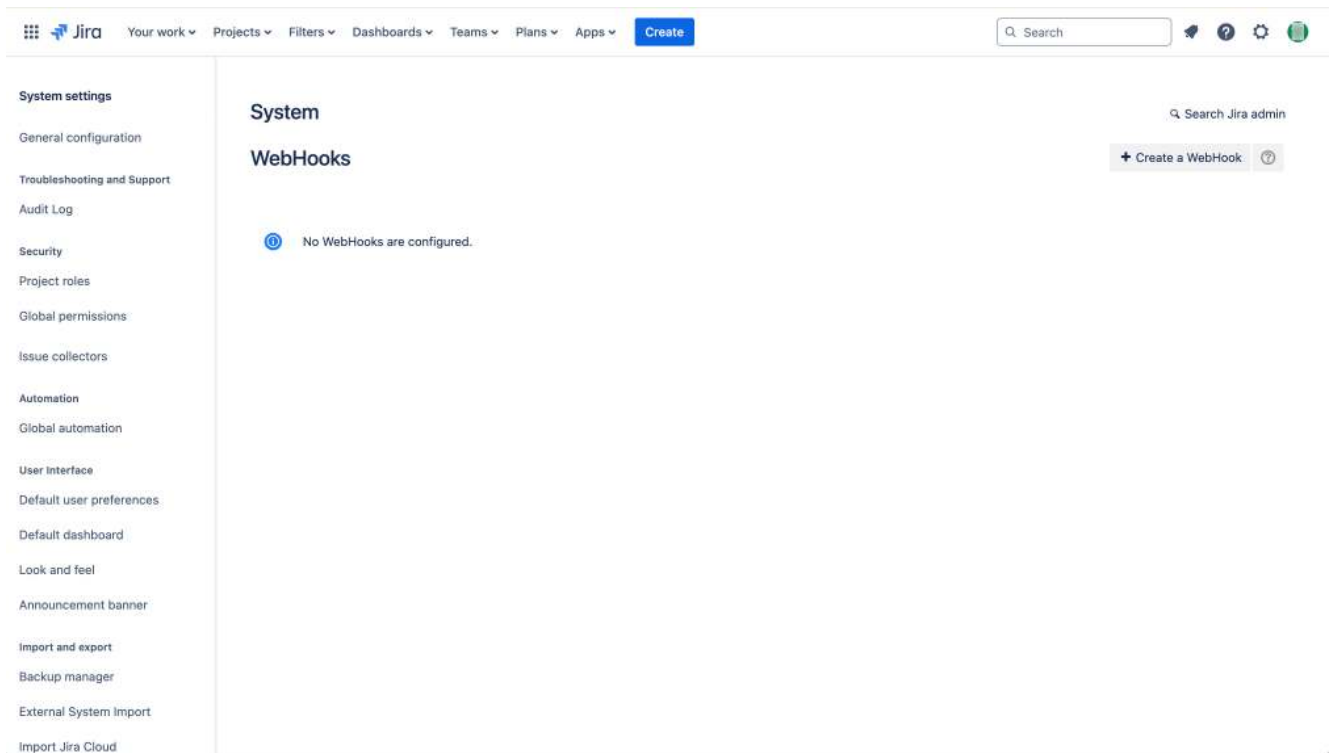
Instructions

1. Create a WebHook in Jira. This can be done in the Jira admin panel by navigating to `https://<your jira domain>/plugins/servlet/webhooks#`. [Visit public Jira documentation on webhooks](#) ↗.

API Reference ↗



Send feedback



2. Create a Jira listener in the Palantir platform. This will generate the listener URL that you need to copy and paste into the Jira **URL** field when creating a WebHook.
 - a. You should also generate a message signing secret in Jira, and copy to the message signing secret in the Foundry listener configuration. You can also set up without a signing

API Reference ↗

Send feedback

Data connection > New listener

New listener

1 Select type

2 Name and location

3 Configuration

4 Test

Need help? Jira documentation Open

Back

Cancel Save and continue

Configuration

Endpoints 1

https://...com/inbound-message-listener/message,

Message signing verification

Message signing secret

3. Choose the set of Jira events that should be sent to your listener and then select **Create**. The screenshotted example shows subscription for the issue created, updated, and deleted events:

Jira Your work Projects Filters Dashboards Teams Plans Apps Create

Search webhooks x Status All

1 webhooks found

New Webhook Listener ENABLED

WebHook 'New Webhook Listener' has been successfully created.

New Webhook Listener ENABLED

Last updated: 25/Feb/25 8:58 PM by Titan Medical

URL https://.../inbound-message-listener/message/

Secret Set

Events Issue related events

JQL: All Issues

Issue: created, updated, deleted

Exclude body No

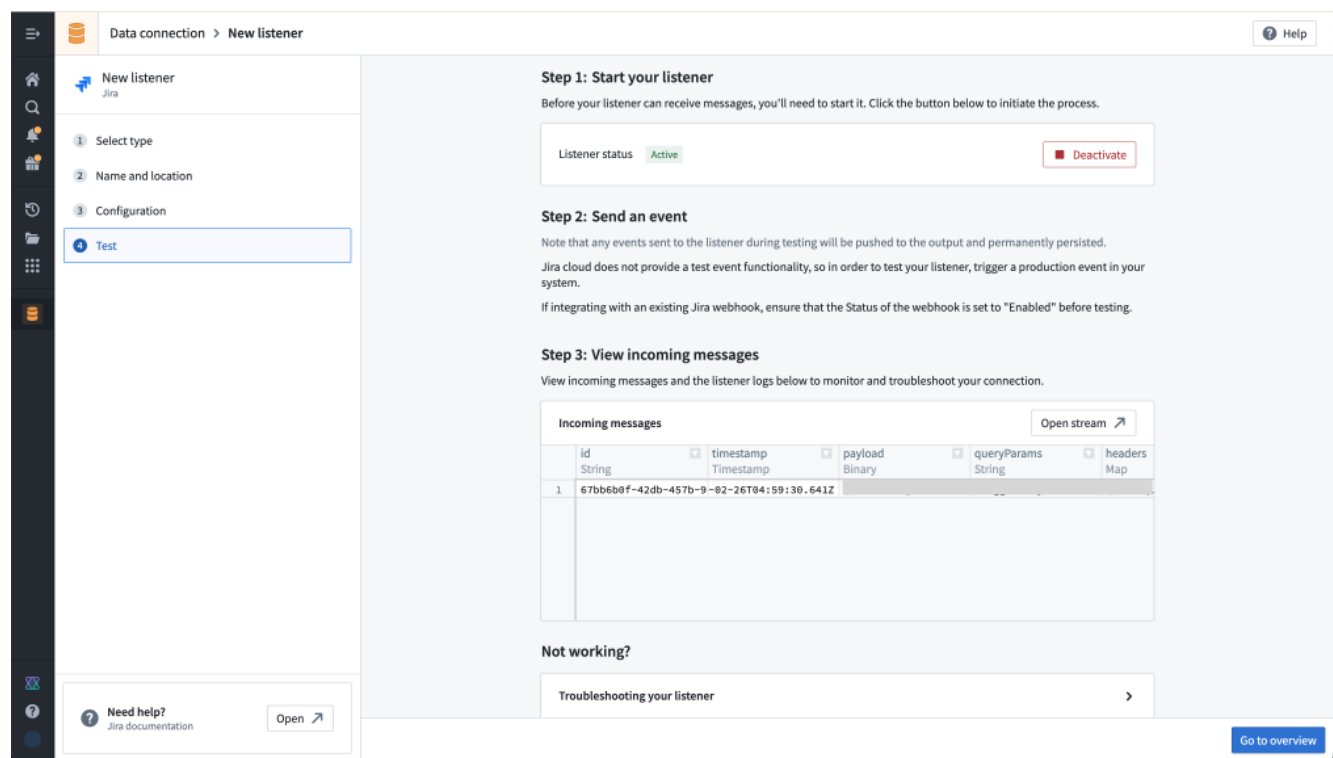
Transitions No linked transitions.

Edit Delete

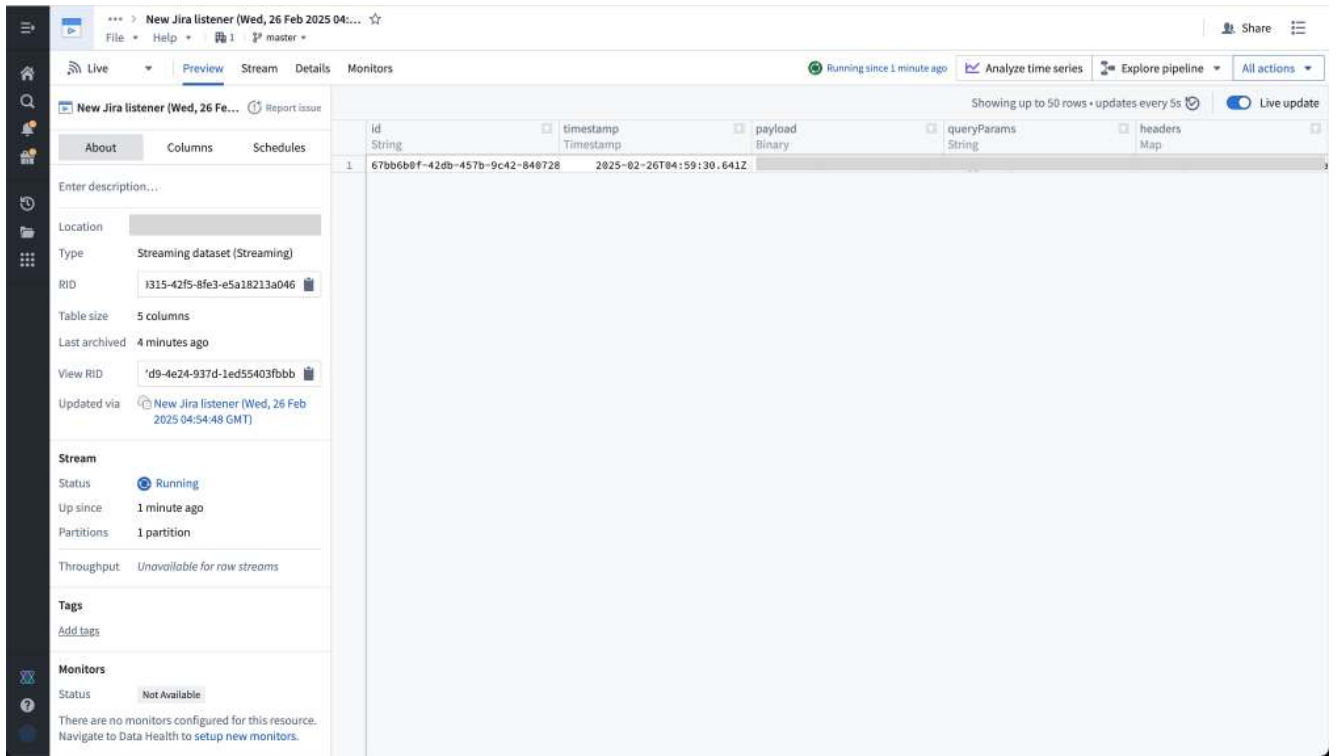
4. API Reference configuration in Foundry to proceed. Administrator approval is now required from the Information Security Officer. Review the toggle

Send feedback

5. Select **Start** in the listener test screen to start your listener.



6. You will need to allow inbound traffic from the system that is pushing data to your Foundry enrollment. This can be done in Control Panel by an Information Security Officer for your Foundry enrollment. [Documentation on managing ingress in the Palantir platform.](#)
7. Test the new configuration by making a change to trigger an event to your listener. You should see it appear in the incoming messages view in data connection, as well as in the underlying stream that the listener outputs to.



You can now use this streaming data in Foundry pipelines, to put data into the ontology, or monitor using Automate to trigger effects in real time.

All screenshots of Atlassian Jira® are provided for reference purposes only and are the property of Atlassian Corporation Plc.

Data connectivity & integration / Data Connection / HTTPS listeners / Set up a Slack listener

Set up a Slack listener

This guide shows step-by-step how to configure a listener for Slack, to get a real-time feed of events from Slack to a Foundry streaming dataset.

[Learn more about Slack.](#) ↗

Prerequisites

Prior to configuration, ensure:

- Your enrollment's ingress policy has been appropriately configured for the AWS region or country where your Slack instance is hosted. [Learn how to Configure ingress.](#)
- You must have your own instance of Slack with administrator access.

Instructions

1. Create a new Slack app for your integration. [Refer to external documentation.](#) ↗

API Reference ↗



Send feedback

Name app & choose workspace

App Name

My Custom App

Don't worry - you'll be able to change this later.

Pick a workspace to develop your app in:



Keep in mind that you can't change this app's workspace later. If you leave the workspace, you won't be able to manage any apps you've built for it. The workspace will control the app even if you leave the workspace.

[Sign into a different workspace](#)

By creating a Web API Application, you agree to the [Slack API Terms of Service](#).

Cancel

Create App

- Note down the **signing secret** value for your app from the **Basic Information** section. You will need this to set up your listener successfully.

Settings

Basic Information

Collaborators

Socket Mode

Install App

Manage Distribution

Features

App Home

Agents & AI Apps NEW

Workflow Steps NEW

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

Steps from Apps LEGACY

OAuth & Permissions

Event Subscriptions

User ID Translation

App Manifest

Beta Features

Submit to Slack Marketplace

Review & Submit

App Credentials

These credentials allow your app to access the Slack API. They are secret. Please don't share your app credentials with anyone, include them in public code repositories, or store them in insecure ways.

App ID

Date of App Creation

February 26, 2025

Client ID

Client Secret

.....

Show

Regenerate

You'll need to send this secret along with your client ID when making your `oauth.v2.access` request.

Signing Secret

.....

Show

Regenerate

Slack signs the requests we send you using this secret. Confirm that each request comes from Slack by verifying its unique signature.

Verification Token

Regenerate

This deprecated Verification Token can still be used to verify that requests come from Slack, but we strongly recommend using the above, more secure, signing secret instead.

- Create a Slack listener from the Data Connection application in Foundry.
 - In the field for **Message Signing Secret**, enter the **Signing Secret** value from the previous step.
 - Take note of the URL displayed in the **Endpoints** field on this page.
 - Select **save and continue**.

API Reference ↗

Send feedback

- b. Toggle to turn on event subscriptions, and then enter the listener URL from step 3 into the **Request URL** field.
- c. Slack will verify that the listener is correctly set up, and you will see a “Verified” message above your URL as shown below.
- d. You then need to add the events and associated scopes so that Slack will push events to your listener. As a simple example, you can add the `app_mention` event, which requires the `app_mentions:read` scope. This will send an event when the bot is explicitly mentioned.

Foundry Event ...

Settings

Basic Information

Collaborators

Socket Mode

Install App

Manage Distribution

Features

App Home

Agents & AI Apps NEW

Workflow Steps NEW

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

Steps from Apps LEGACY

OAuth & Permissions

Event Subscriptions

User ID Translation

App Manifest

Beta Features

Submit to Slack Marketplace

Review & Submit

Give feedback

Slack

Help

Event Subscriptions

Enable Events

On

Your app can subscribe to be notified of events in Slack (for example, when a user adds a reaction or creates a file) at a URL you choose. [Learn more.](#)

Request URL Verified

https://.com/inbound-message-listener/message/

Change

We'll send HTTP POST requests to this URL when events occur. As soon as you enter a URL, we'll send a request with a `challenge` parameter, and your endpoint must respond with the challenge value. [Learn more.](#)

Subscribe to bot events

Apps can subscribe to receive events the bot user has access to (like new messages in a channel). If you add an event here, we'll add the necessary [OAuth scope](#) for you.

Event Name	Description	Required Scope
app_mention	Subscribe to only the message events that mention your app or bot	app_mentions:read

Add Bot User Event

Subscribe to events on behalf of users

App unfurl domains

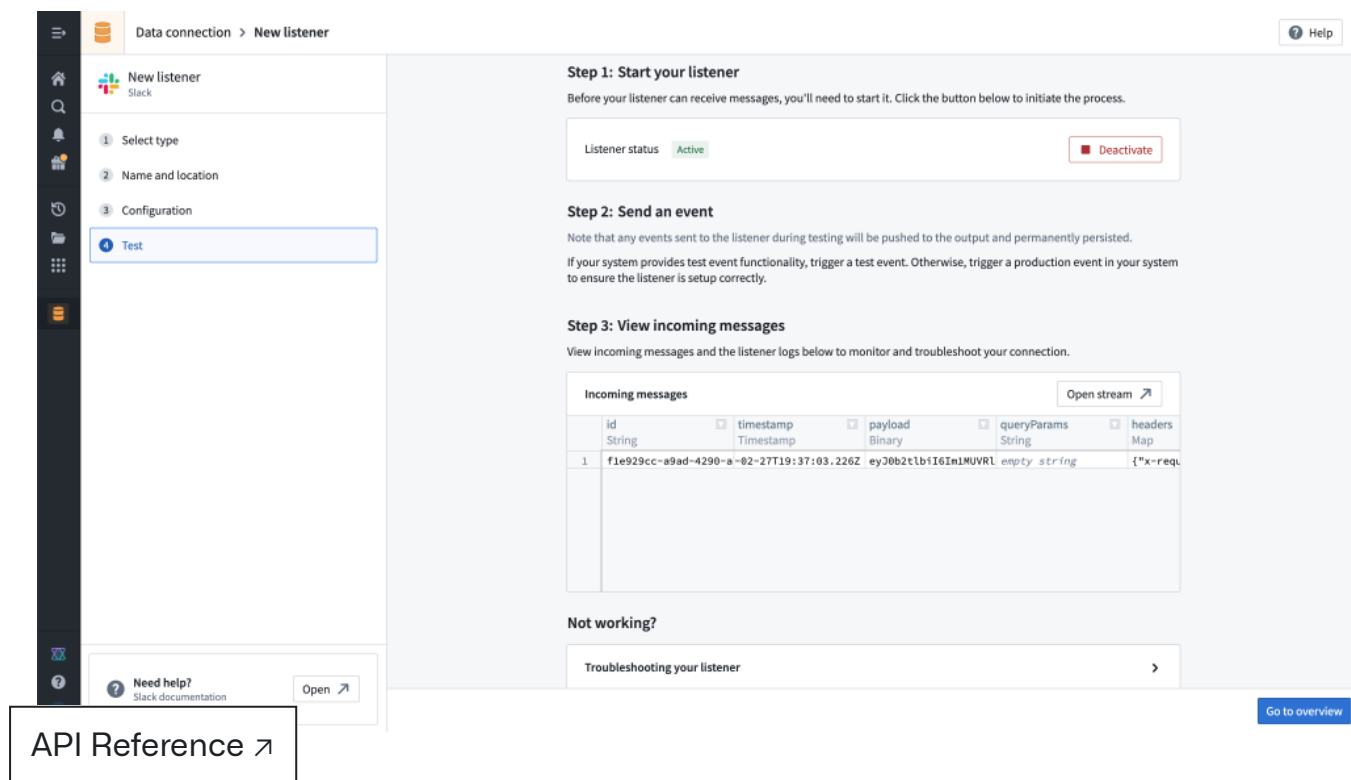
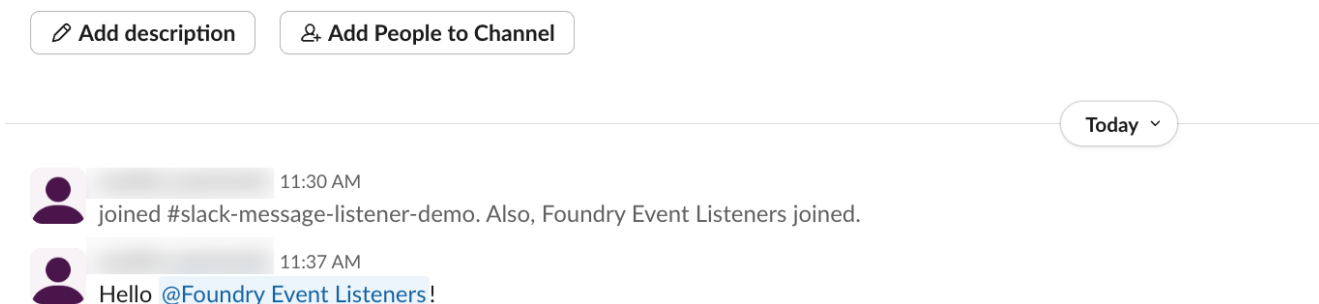
7. Remember to install your application into your Slack workspace. You should do this after specifying the desired events, since the application must be installed whenever the API Reference is updated.

Send feedback

8. Send a test message, and watch it appear in the listener's output stream.

slack-message-listener-demo

You created this channel today. This is the very beginning of the **# slack-message-listener-demo** channel.



Send feedback

All screenshots of Slack® are provided for reference purposes only and are the property of Slack Technologies, LLC.

PREVIOUS

← [Set up a Jira listener](#)

NEXT

[Create an AI-powered Slack bot](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / HTTPS listeners / Create an AI-powered Slack bot

Create an AI-powered Slack bot with listeners

You can use event listeners in Foundry to create AI-powered bots that can interact with chat workspaces outside of Foundry. This guide provides an example using Slack, but you can use similar methods to connect with other platforms, such as Microsoft Teams (via the Microsoft Bot Framework).

Overview

In this tutorial, you will build "Tennis Bot", an AI assistant that helps Slack users learn more about tennis.

This example Slack bot comprises the following components:

- A [Slack listener](#) to receive chat events.
- A [Slack source](#) to write back responses to the Slack API.
- An [AIP chatbot](#) to act as the "brain" of the bot.
- A [TypeScript function](#) to perform event processing.
- An [automation](#) to execute new events.

Step 1: Set up Slack listener

API Reference ↗

The first step is to create your Slack app, install it into your Slack workspace, and start it in your workspace. Then, you can use the [Slack listener setup guide](#) to create a Slack listener.

Send feedback

Since users will directly message the Tennis Bot with questions, you will need to add the relevant scopes, events, and configurations in the Slack app. For more information on configuring Slack apps, see the [official Slack documentation ↗](#).

Add the `im:history` and `chat:write` scopes for your app.

Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.

Bot Token Scopes

Scopes that govern what your app can access.

OAuth Scope	Description	
<code>im:history</code>	View messages and other content in direct messages that Tennis Bot has been added to	
<code>chat:write</code>	Send messages as Tennis Bot	

Add an OAuth Scope

Additionally, subscribe your app to the `message.im` event.

Tennis Bot

Settings

Basic Information

Collaborators

Socket Mode

Install App

Manage Distribution

Features

App Home

Agents & AI Apps NEW

Workflow Steps NEW

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

Steps from Apps LEGACY

OAuth & Permissions

Event Subscriptions

User ID Translation

App Manifest

Beta Features

Event Subscriptions

Enable Events

On

Your app can subscribe to be notified of events in Slack (for example, when a user adds a reaction or creates a file) at a URL you choose. [Learn more.](#)

Request URL Verified ✓

https://[REDACTED]/inbound-message-listener/message

Change

We'll send HTTP POST requests to this URL when events occur. As soon as you enter a URL, we'll send a request with a challenge parameter, and your endpoint must respond with the challenge value. [Learn more.](#)

Subscribe to bot events

▼

Apps can subscribe to receive events the bot user has access to (like new messages in a channel). If you add an event here, we'll add the necessary [OAuth scope](#) for you.

Event Name	Description	Required Scope
message.im	A message was posted in a direct message channel	im:history 🗑️

Add Bot User Event

You will also need to allow users to directly message the bot. This setting can be found in your Slack app settings under **Features > App Home**.

Messages Tab

Direct messages your app sends will show in this tab.

✓

☒ Allow users to send Slash commands and messages from the messages tab

After you have configured the listener, you will be able to verify that the listener is correctly configured in the **Test** tab of the listener setup wizard.

API Reference ↗

Send feedback

Data connection > Listeners > New listener

New listener
Slack

1 Select type
2 Name and location
3 Configuration
4 Administrator approval
5 Test

Help

Test your listener

Your message listener has been configured. Follow the steps below to test the connection and start receiving messages.

Your listener will not be able to receive messages from the external system unless the appropriate ingress is configured for your domain

Open control panel ↗

Step 1: Start your listener

Before your listener can receive messages, you'll need to start it. Click the button below to initiate the process.

Listener status

Stop

Step 2: Send an event

Note that any events sent to the listener during testing will be pushed to the output and permanently persisted.

If your system provides test event functionality, trigger a test event. Otherwise, trigger a production event in your system to ensure the listener is setup correctly.

Step 3: View incoming messages

View incoming messages and the listener logs below to monitor and troubleshoot your connection.

Incoming messages

Open stream ↗

	id	timestamp	payload	queryParams	headers
	String	Timestamp	Binary	String	Map
1	f9a5fb7c-fc12-423e-91df-620-05-18120:28:18.969Z	eyJsb2t1b1I6IjwSRUMQIiw	empty_string		{"content-length":"1639"
2	1456466a-c7d1-4138-9813-628-05-18720:28:43.1437	eyJ0b2t1b1I6IjwSRUMQIiw	empty_string		{"content-length":"9611"
3	38d24fc5-3a32-485c-8bd1-625-05-18720:22:14.783Z	eyJ0b2t1b1I6IjwSRUMQIiw	empty_string		{"content-length":"9611"

Need help?
Slack documentation

Open ↗

Go to overview

Step 2: Set up Slack source

Next, you will need to [create a new Slack source](#) with credentials to write back to your chat platform.

After installing your Slack app into your workspace, you can find the bearer token to configure your source within **Installed App Settings**.

API Reference ↗

Send feedback

Installed App Settings

OAuth Tokens

These tokens were automatically generated when you installed the app to **Foundry - Demo**. You can use these to authenticate your app. [Learn more](#).

Bot User OAuth Token

xoxb-

[REDACTED]

Copy

Access Level: Workspace

Reinstall to Foundry - Demo

You will need to enable exports and code repository imports for your source by navigating to **Connection settings**.

API Reference ↗

Send feedback

Export configuration

Enabling exports may be available even if the [point-and-click export workflows](#), such as Table Exports, are not available on this connector. All sources may be used in code, and to [use sources in code](#) alongside other Foundry inputs requires exports to be enabled for the source.

Enable exports

☒ Enable exports to this source

This will allow people to export datasets from Foundry to this source. By default, data without any markings will be exportable. If you wish to export data with additional [security markings](#), you'll be asked to configure what data is permissible to export to this source. Data marked with an organization will need that organization to be set as exportable.

☐ Enable exports to this source with webhooks

When using webhooks in interactive workflows, marking-based export controls are not enforced, and data passed to the webhook will be considered safe to export. This setting cannot be disabled, but it can be governed by controlling the set of users who have access to this source, and the webhooks provided by it.

☒ Enable exports to this source without markings validations

In certain interactive workflows the source can be used in, we are currently unable to validate the security markings of the data being exported. By enabling exports without markings validation, you acknowledge that you are responsible for ensuring that the data being exported is compliant with your organization's policies.

Code import configuration

Enable code imports

☒ Allow this source to be imported into code repositories

When a source is imported into a code repository, users with access to the code repository can write custom code using the connection details configured for that source, including accessing secret values.

Additionally, you must configure an API name for your source so that the source can be referenced. The API name configuration can be found under **Connection** **API Reference** [↗](#) **Export configuration**.

Send feedback

Step 3: Set up AIP Chatbot

In this step, you will [create a new chatbot in AIP Chatbot Studio](#) (formerly known as Agent Studio). You will need to configure [instructions](#) and [retrieval context](#) for the chatbot.

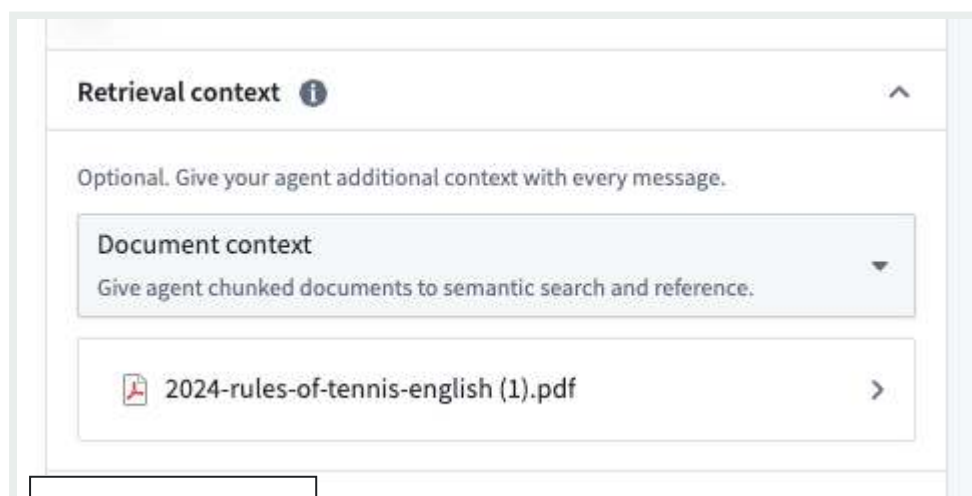
Specify chatbot instructions

To teach the chatbot how to respond as a tennis information bot, you can provide some task-specific [instructions](#) for the chatbot. An example prompt is:

```
You are an assistant for answering questions about tennis.  
  
If a query is not related to tennis, you should respond politely and explain the reason.  
  
Keep your responses concise and friendly.
```

Provide retrieval context

To give the chatbot a source of information about tennis, you can provide a PDF with the rules of tennis as [document context](#) to the chatbot; the chatbot will then be able to use this context to answer user queries.

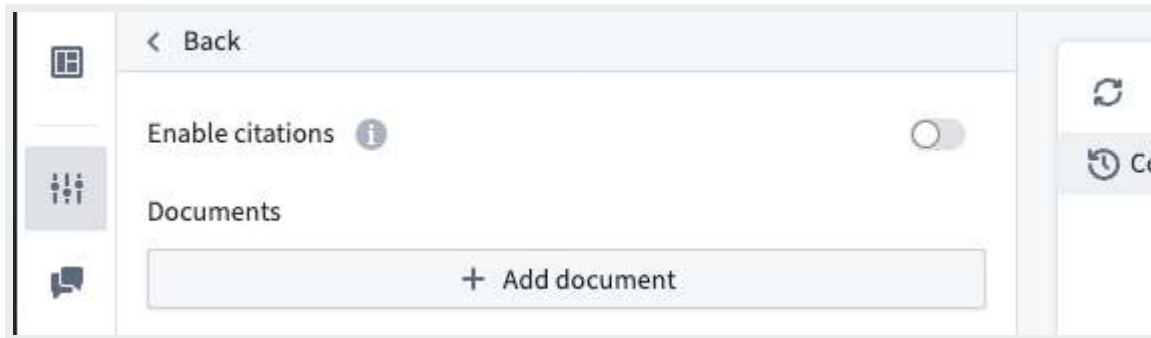


[API Reference](#) ↗

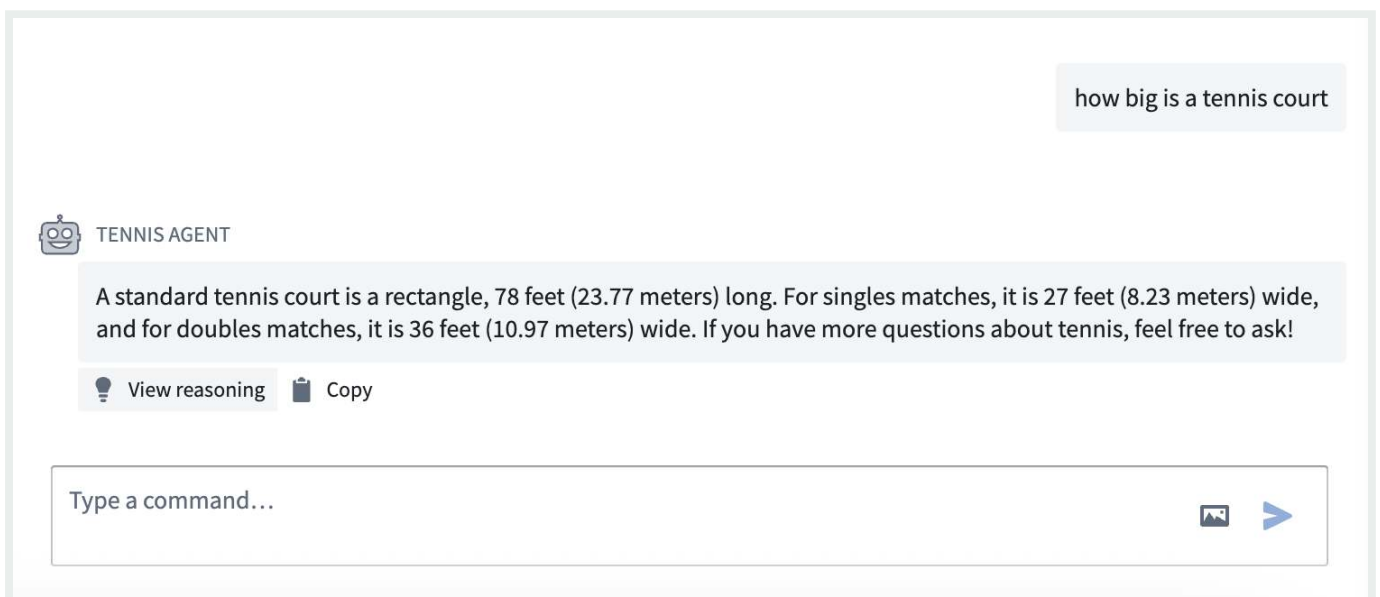
Ensure that you disable citations to avoid the chatbot including [XML](#) in its response; this will prevent the chatbot from including links back to the

[Send feedback](#)

Slack responses.



You can additionally test your chatbot to ensure it responds in an appropriate manner. Use Chatbot Studio for ad-hoc testing, or use [AIP Evals](#) to test against a full evaluation suite.

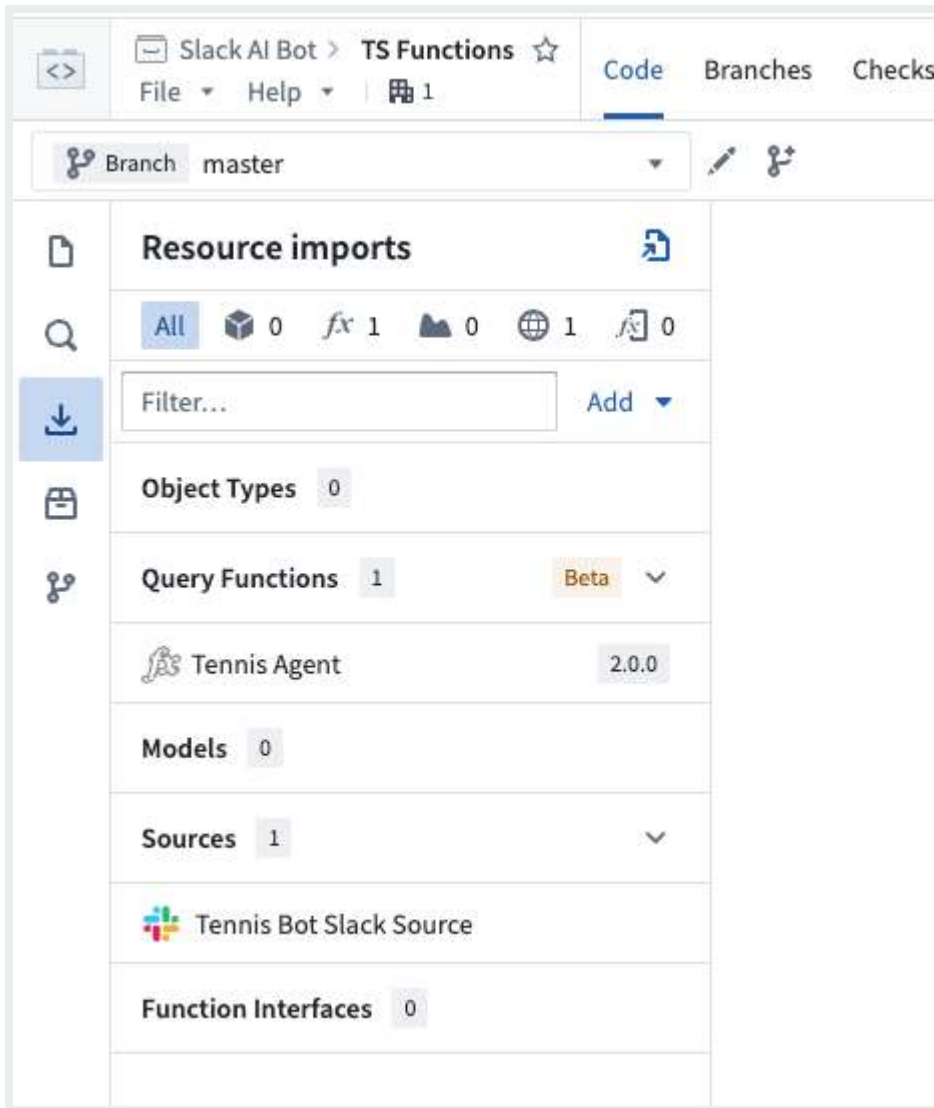


When you are satisfied with the chatbot's performance, [publish the chatbot as a function](#) so that you can use the chatbot in your event processing workflow.

Step 4: Import your source and chatbot function into a code repository

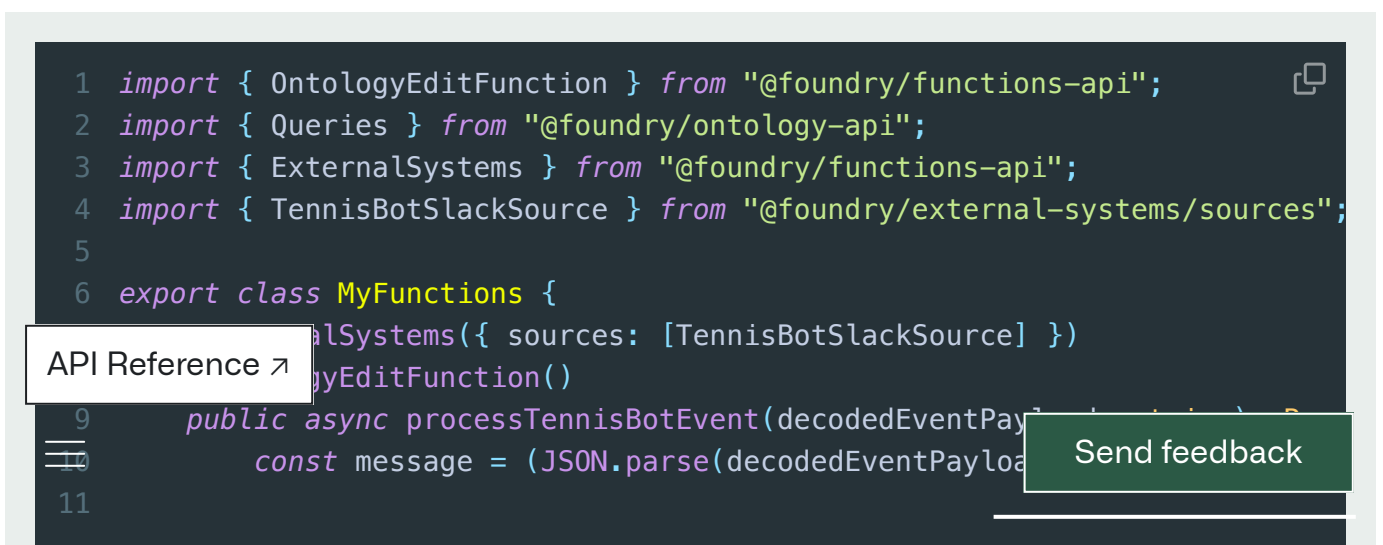
Once you have set up the chatbot, you will need to enable the chatbot to interact with Slack. First, [create a new code repository for writing TypeScript functions](#). Once created, import
b API Reference ↗ source and the published chatbot function into the repository.

Send feedback



Step 5: Set up listener event payload processing

To respond to messages, the bot will need to be able to process incoming listener event payloads. Create a TypeScript function to enable listener event payload processing.



```

12      // Only process non-bot messages (don't re-process messages that
13      if (message == null || message.bot_id != null) {
14          return;
15      }
16
17      const messageText = message.text;
18      const channel = message.channel;
19      if (messageText == null || channel == null) {
20          return;
21      }
22
23      // Pass the message to our agent
24      const agentResponse = await Queries.tennisAgentDocumentContextExa
25
26      // Forward the agent's response back to Slack
27      const postMessageUrl = `${TennisBotSlackSource.getHttpsConnection
28      const response = await TennisBotSlackSource.fetch(postMessageUrl,
29          method: "POST",
30          headers: {
31              ["Authorization"]: `Bearer ${TennisBotSlackSource.getSecr
32              ["Content-Type"]: "application/json"
33          },
34          body: JSON.stringify({
35              channel: channel,
36              text: agentResponse.markdownResponse,
37          })
38      })
39  }
40  }

```

For more information on using sources in TypeScript functions, see [the relevant documentation](#).

Commit and tag a version to [publish this function](#).

Step 6: Set up function-backed action

Navigate to [Ontology Manager](#) to create a [function-backed action](#) using your event

API Reference ↗ this action will be triggered for each incoming Slack message event.

Send feedback

Create a new action type

1 Action type

2 Metadata

3 Submission criteria

STEP 1

Select an action type you want to configure

Enable users to make changes to the ontology by configuring actions they can execute.

Object Link Function Webhook Interface Notification

Function

processTennisBotEvent

0.1.0

Required inputs

decodedEventPayload → Decoded Event Payload

Code Preview

Edit in Code Repositories

```
7 @ExternalSystems({ sources: [TennisBotSlackSource] })
8 @OntologyEditFunction()
9 public async processTennisBotEvent(decodedEventPayload: string):
10   Promise<void> {
11     const message = (JSON.parse(decodedEventPayload) as any).event;
```

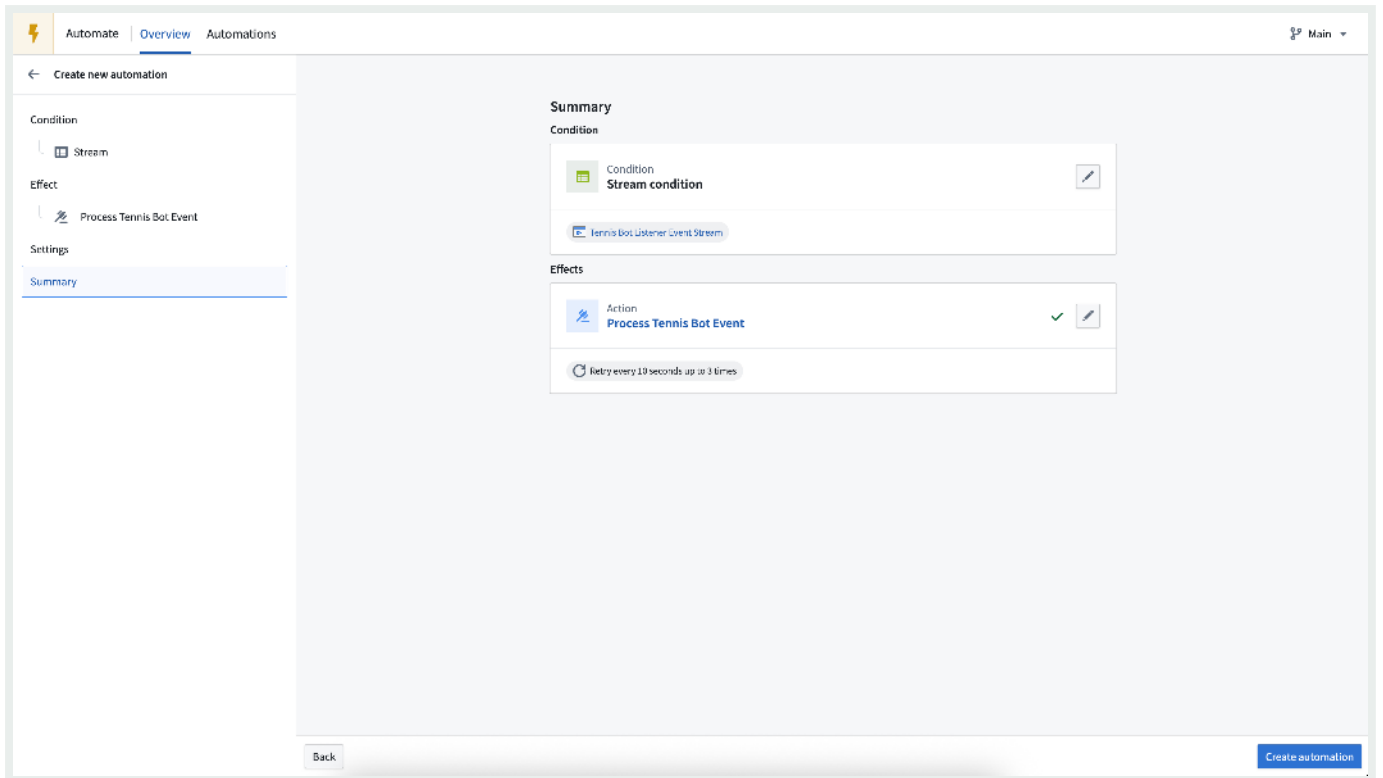
Skip

Next

Step 7: Set up automation to execute function-backed actions

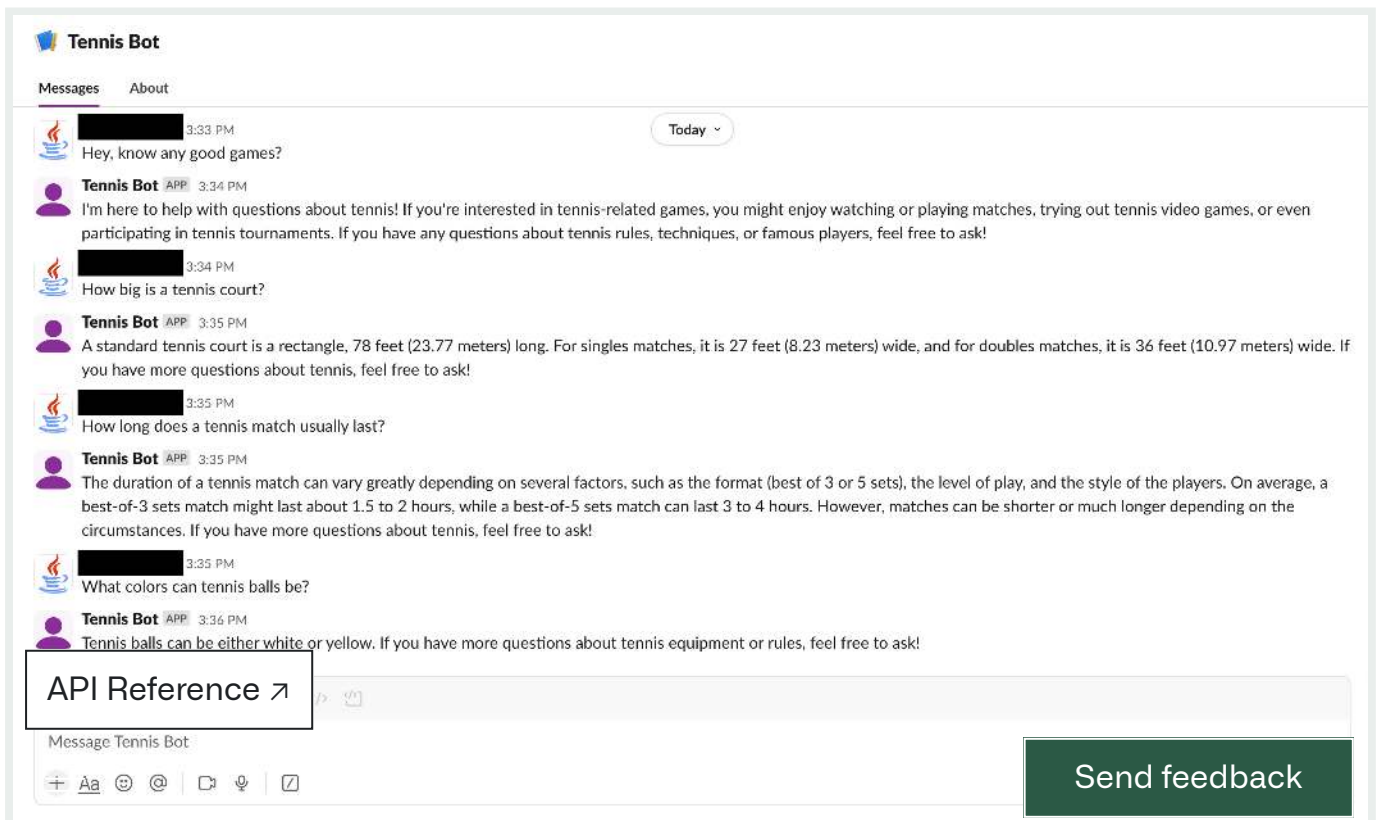
Next, create a new automation to execute the function-backed action for each new listener event. The new automation should have a "Stream" condition (it will execute when new rows are added to the stream). The input to this automation should be the output stream from the created Slack listener.

[API Reference ↗](#)[Send feedback](#)



Step 8: Test the bot

At this point, the Tennis Bot is ready for testing. Send a Slack direct message to your bot and wait for its response. The bot should respond with information about tennis.



Optional extensions and improvements

You have now created a basic AI-powered chatbot; from here, you can continue to add functionality to your bot. Example extensions and improvements include:

- Send an initial confirmation message when the event is first received (such as "Working on it!"), and then update the content of that message to the LLM response when the response is available. This reduces the time taken for the user to receive the first response.
- Maintain user conversation history in the chatbot conversation session; this enables conversations with past messages as context. You can use the [sessionRid](#) parameter of [the chatbot function](#) to enable conversational context.
 - For example, you can store the session RID from the [chatbot function output](#) (created on the first message in a thread) to an ontology object type, then look up the associated session RID for new messages on this object type to use in the [chatbot function input](#).
- To reduce processing latency, you can move the event processing implementation from an automation that calls an action to a streaming pipeline that calls an [external UDF](#). For more information on using streaming pipelines to process events, see the documentation on [event processing options](#).

All screenshots of Slack® are provided for reference purposes only and are the property of Slack Technologies, LLC.

PREVIOUS
← [Set up a Slack listener](#)

NEXT
[WebSocket listeners](#) / [Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Data connectivity & integration / Data Connection / WebSocket listeners / Overview

WebSocket listeners

⚠ Experimental

WebSocket listeners are an experimental capability that may not be available on your enrollment. To enable this capability, contact Palantir Support.

WebSocket listeners are primarily designed for real-time audio and telephony workflows, enabling bidirectional communication between external services and Foundry. Unlike [HTTPS listeners](#) that receive webhook requests and write to streams, WebSocket listeners maintain persistent connections for continuous data streaming into a custom WebSocket server hosted in a [compute module](#). For security considerations specific to HTTPS listeners, see [HTTPS listener security](#).

Because WebSocket connections are bidirectional, your compute module can also send responses back to the source system during an active connection. For example, in a telephony workflow, you can process incoming audio and send generated audio back to the caller in real time.

When to use WebSocket listeners

You can use WebSocket listeners in the following situations:

- The external system uses the WebSocket protocol for data delivery.

API Reference ↗

back during a call.



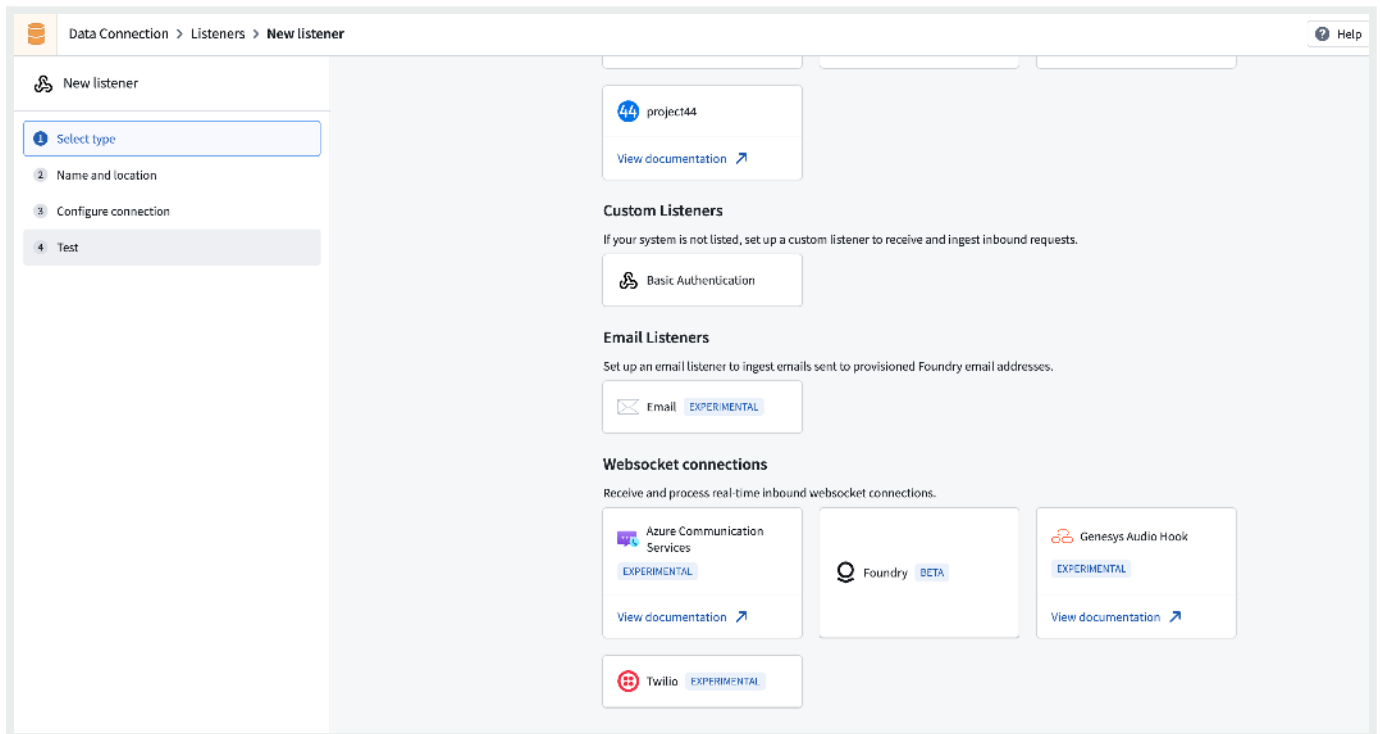
processing continuous audio streams from telephony system

Send feedback

Set up a WebSocket listener

1. Create a WebSocket listener

1. Navigate to **Data Connection > Listeners**.
2. Select **Create new listener** and choose the WebSocket connector you need.



i Connector availability

If the connector you require is not present, contact Palantir Support to enable it for your enrollment.

2. Configure an endpoint

All WebSocket listeners must be mounted on a dedicated subdomain. You must configure your Foundry instance to accept inbound connections from the external system's IP ranges.

[API Reference ↗](#)

1. Create a listener subdomain in **Control Panel > Domains & certificates**. [Learn more about listener subdomains.](#)

[Send feedback](#)

2. Configure ingress to allow connections from your external system. Refer to the [Configure ingress documentation](#).
3. Mount the subdomain to your listener. This requires approval before you can start receiving requests.

Set up endpoint 

Configure how the external system will connect to your listener

Subdomain Approved 

The listener will only be reachable by the external system over this subdomain

 Why do I need a subdomain? 

Network ingress

Ensure that you have an adequate range of IPs allowlisted on your subdomain to receive requests from the external system Manage ingress 

Endpoints 1 Rotate endpoints...

URL	Status	Actions
wss://	Active	 

3. Configure security

Each WebSocket listener type has specific authentication requirements. Configure the appropriate credentials in the **Configure security** section of your listener's **Configure connection** page.

 [API Reference](#)  **Configure server** page and specify which exportable markings your listener should allow. [Learn more about WebSocket listener security.](#)

Send feedback

Exportable markings

Only markings that you have permission to remove can be selected here. To update exportable markings, you must have permission to remove all markings being set as exportable, even if already set.

By default, data without any markings will be exportable. To export data with additional security markings, configure what data is permissible to export to this connection. Data marked with an organization will need that organization to be set as exportable.

Markings

Search for markings...

No markings are set as exportable for this connection

Organizations

Search for organizations...

Cancel

Save

4. Configure a compute module

WebSocket listeners route incoming data to a compute module for processing. When you create a listener, a compute module is automatically created for you—this is currently the only supported way to handle WebSocket connections in listeners.

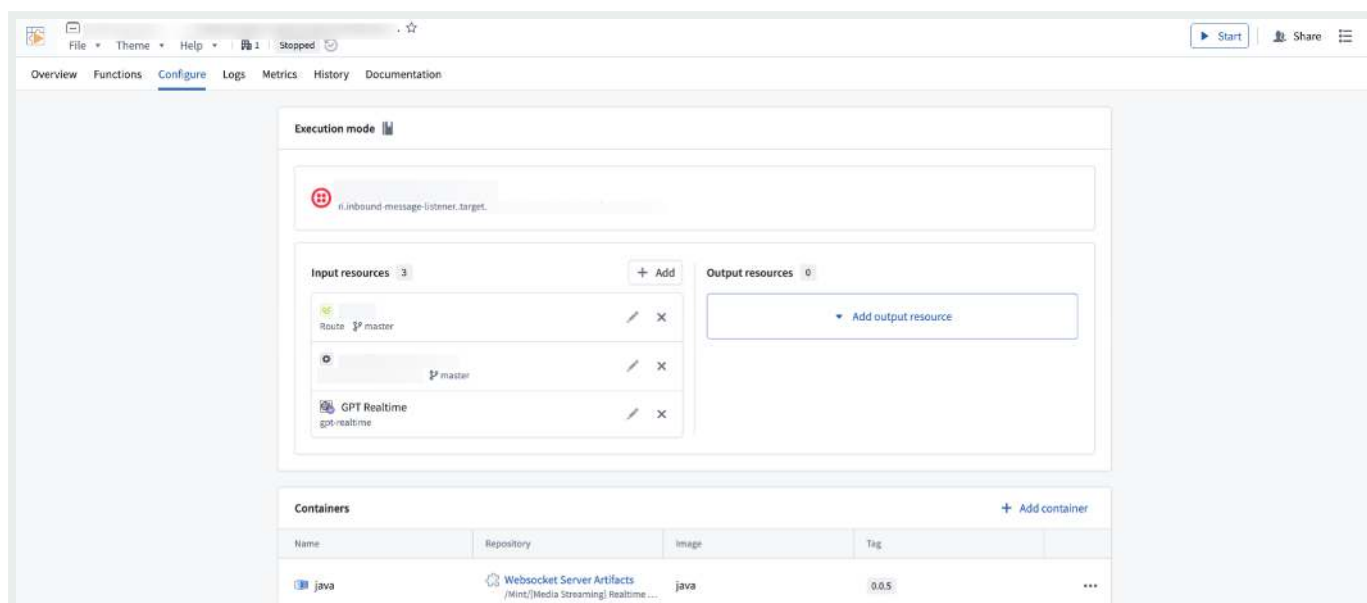
To configure the compute module, follow the steps below:

1. From the listener's **Configuration** page, navigate to the linked compute module.

resources your compute module needs as inputs and outputs. By default, your compute module has no permissions—you must explicitly grant access to each resource. [Learn more about configuring compute modules.](#)

Send feedback

3. Start your compute module and check the **Logs** tab to confirm everything is running as expected.



Supported WebSocket listeners

The following WebSocket listener types are available:

- [Twilio Media Streams](#): Streams real-time audio from Twilio voice calls over WebSocket connections.
- [Azure Communication Services](#): Streams audio from Azure Communication Services call automation workflows.
- [Genesys AudioHook](#): Streams real-time audio from Genesys Cloud voice interactions using the AudioHook protocol.

All product names, logos, and brands mentioned are trademarks of their respective owners. All company, product, and service names used in this document are for identification purposes only.

PREVIOUS



API Reference ↗

Create an AI-powered

NEXT

Security →

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / WebSocket listeners / Security

WebSocket listener security

WebSocket listeners differ from standard Foundry data ingestion, so ensure that you understand these security paradigms before enabling your connections.

Request authorization

Each WebSocket listener type has specific authentication requirements defined by the external system. Listeners implement the security protocols laid out by those external systems, which vary widely. Palantir makes no guarantees about the suitability or effectiveness of these external system protocols.

You are responsible for ensuring that you understand which guarantees each protocol does or does not provide for the incoming connections and data.

The specific protocols implemented for each listener can be found in the **Configure security** section of your listener's **Configure connection** page, as well as the external system's documentation.

Exportable marking validation

When WebSocket listeners process data, marking validation acts as a security control to prevent unauthorized data exfiltration. The system ensures that any data consumed by your output meets the listener's configured exportable marking requirements.

API Reference ↗

By default, only data without security markings can be read and incorporated into compute module inputs. If your compute module needs to process data with security markings, you must explicitly configure which markings are permitted for export in

Send feedback

the listener's settings. Only a user with the ability to declassify those markings can add them to the configuration.

Exportable markings

Only markings that you have permission to remove can be selected here.

To update exportable markings, you must have permission to remove all markings being set as exportable, even if already set.

By default, data without any markings will be exportable. To export data with additional security markings, configure what data is permissible to export to this connection. Data marked with an organization will need that organization to be set as exportable.

Markings

Search for markings...

No markings are set as exportable for this connection

Organizations

Search for organizations...

Cancel

Save

Subdomains

WebSocket listeners can be mounted to dedicated subdomains, allowing for granular ingress control, comprehensive governance workflows, and isolation of less secure endpoints from the environment's primary enrollment domains. [Learn more about listener subdomains.](#)

API Reference ↗

Endpoint rotation

Send feedback

If the listener's endpoint is compromised, you can rotate it to a new endpoint. [Learn more about endpoint rotation.](#)

PREVIOUS
← Overview

NEXT
Email listeners / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

Email listeners

Beta

Email listeners are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to request access to email listeners.

Email listeners receive inbound emails at dedicated, Foundry-managed email addresses. Incoming emails are validated, parsed, and forwarded to the platform for processing.

Unlike HTTPS and WebSocket listeners that receive connections from external systems over the network, email listeners receive standard email sent to a Foundry-managed address.

Configure a listener

Navigate to **Data Connection > Listeners** to set up an email listener.

For information on securely configuring email listeners and incoming messages, refer to the [email listener security](#) documentation.

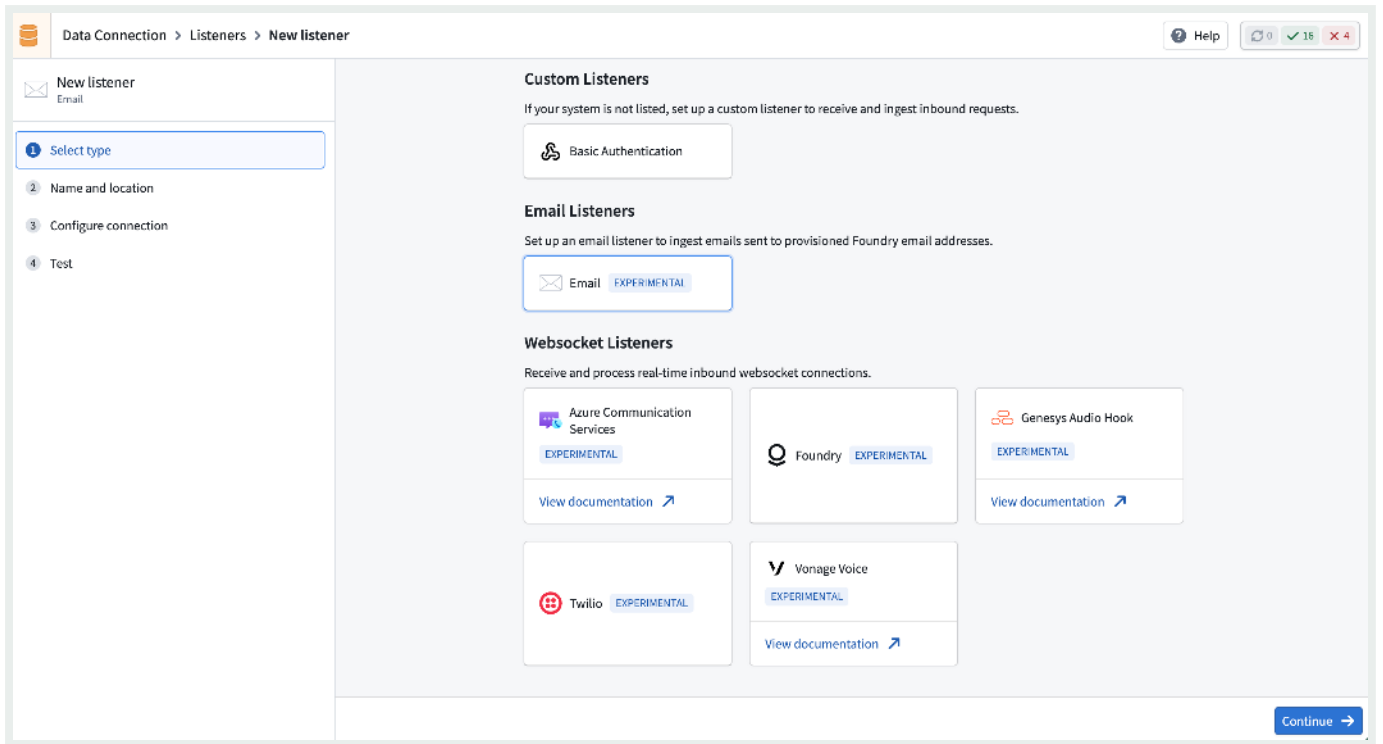
1. Create a new listener

Select **Create new listener** and choose the email listener type.

API Reference ↗

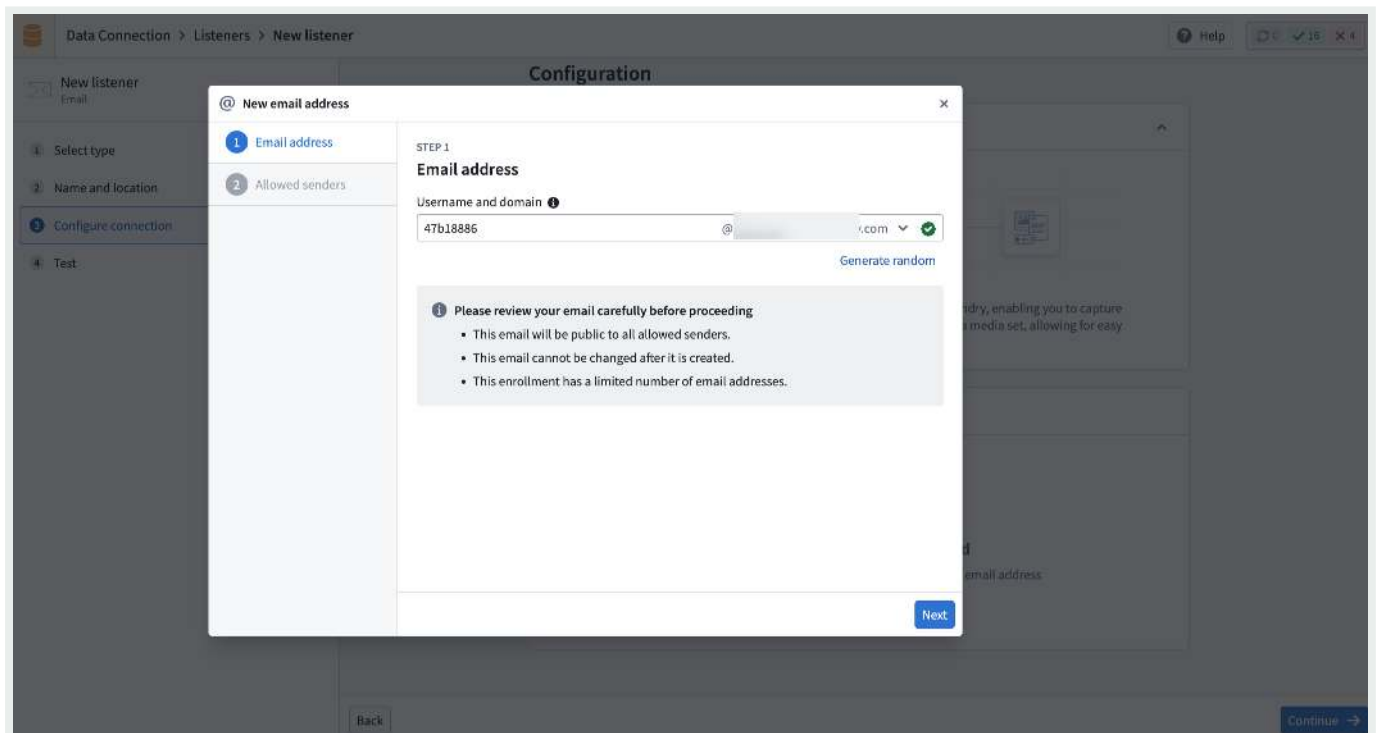


Send feedback



2. Configure connection

Configure a unique email address for the listener.



To configure the sender allowlist to control which senders can deliver email to the listener. You can allow specific email addresses or entire domains.

[API Reference](#)

[Send feedback](#)

Security

By default, email listeners do not accept email from any sender. You must explicitly configure which senders are permitted.

After creation, the listener enters a **Provisioning** state and requires approval before it can begin receiving emails.

[API Reference ↗](#)

[Send feedback](#)

Data Connection > Listeners > New listener

Help

0164

New listener

Email

1 Select type

2 Name and location

3 Configure connection

4 Test

Configuration

What is an email listener?

Email listener

47b18886@.com

Provisioning

Sender allowlist

External emailsNo external emails allowed

Email domains@example.com

Back

Continue

3. Activate the listener

Once the email address is approved and configured, activate the listener to begin receiving emails. Activated listeners process incoming emails and forward their content to a media set.

Data Connection > Listeners > New Email listener

Help

0164

Overview

Settings

Name and location

Configure connection

Test

Test your listener

Your listener has been configured. Follow the steps below to test the connection and start receiving messages.

Step 1: Start your listener

Before your listener can receive data, you'll need to start it. Click the button below to initiate the process.

Listener status

Stop

Step 2: Send an email

If your system provides test event functionality, trigger a test event. Otherwise, trigger a production event in your system to ensure the listener is setup correctly.

Send an email to your listener's email address to test the integration.

Step 3: View incoming messages

View incoming messages and the listener logs below to monitor and troubleshoot your connection.

Incoming messages

Open media set

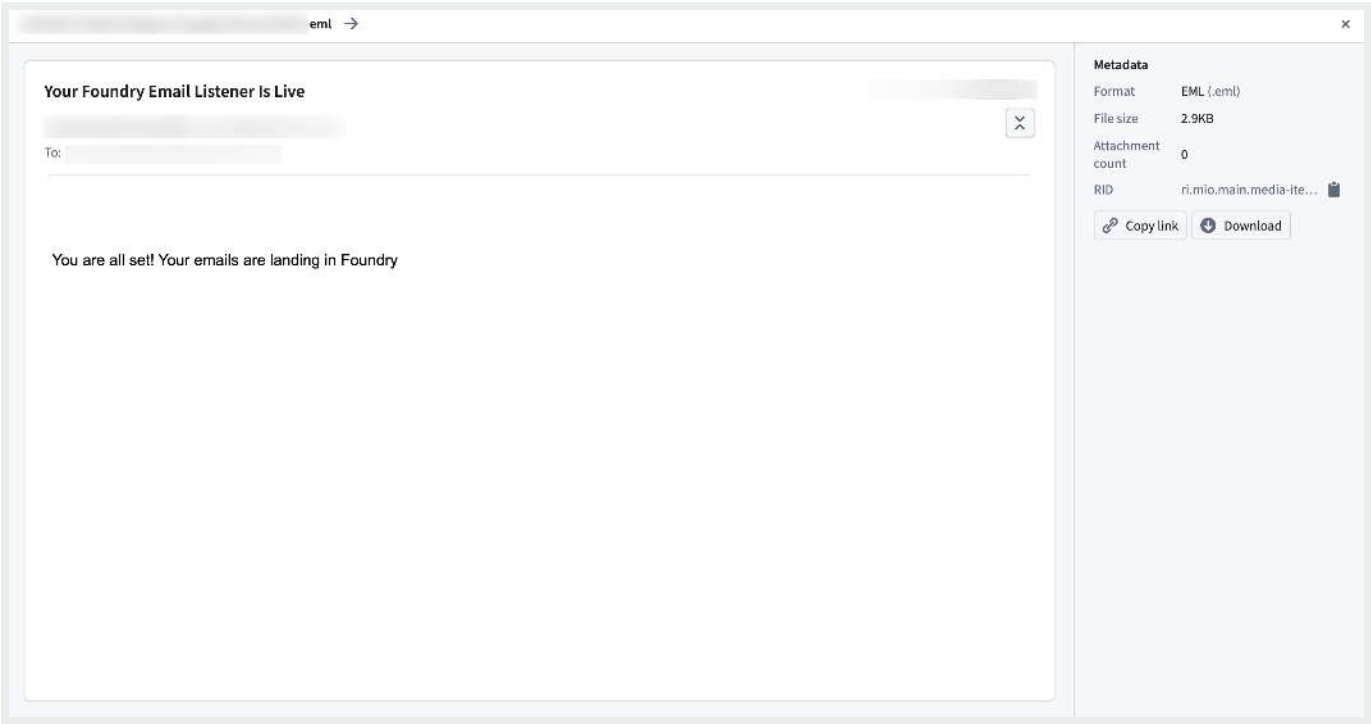
Empty media set

API Reference

Send feedback

4. Test the listener

Send a test email to the listener address to verify that it is receiving and processing messages correctly.



Listener states

Email listeners follow a lifecycle with the following states.

State	Description
Provisioning	The listener is being set up and is not yet processing emails.
Active	The listener is receiving and processing inbound emails.
Inactive	The listener is paused and can be reactivated.
Rejected	The approval request was denied. The listener can only be deleted.

You can deactivate an active listener at any time to temporarily stop processing emails, and reactivate it when needed.

API Reference ↗

- Individual email messages are limited to 40 MB in size, including attachments.

Send feedback

- Certain executable and script attachment types are blocked for security purposes (for example, `.exe`, `.bat`, and `.js` files).
- Each enrollment is limited to a maximum of 10 email listeners. Contact Palantir Support if you require more.

PREVIOUS

← WebSocket listeners / Security

NEXT

Security →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Email listener security

Email listeners apply multiple layers of security validation to incoming emails before processing them.

Email authentication

All incoming emails must pass the following AWS SES authentication checks before they are accepted.

Check	Description
SPF (Sender policy framework)	Verifies that the sending server is authorized to send on behalf of the sender's domain.
DKIM (DomainKeys identified mail)	Verifies that the email has not been tampered with in transit using cryptographic signatures.
DMARC (Domain-based message authentication)	Verifies that the email aligns with the sender domain's published authentication policy.
Spam detection	Scans the email for known spam indicators.
Virus scanning	Scans email content and attachments for malware.

If any of these checks fail, the email is rejected and is not forwarded for processing.

Sender allowlist

API Reference ↗

Each email listener must be configured with a sender allowlist that controls which senders can email to the listener. You can restrict access to the following

Send feedback

- **Specific email addresses:** Only emails from the listed addresses are accepted.
- **Entire domains:** All emails from a given domain are accepted.

Security

By default, email listeners do not accept email from any sender. You must explicitly configure which senders are permitted before the listener will process any incoming email.

Attachment restrictions

To prevent the delivery of potentially malicious content, email listeners block certain MIME types by default. Blocked types include executable and script formats, such as:

- Application executables (for example, `.exe`, `.msi`)
- Shell scripts (for example, `.sh`, `.bat`)
- JavaScript files

Size limits

Individual email messages, including all attachments, are limited to 40 MB. Emails exceeding this limit are rejected.

Data connectivity & integration / Data Connection / External connections from code / External transforms

External transforms

i [Data Connection sources](#) can now be directly imported into code repositories and are the preferred method to interact with external systems, superseding [credentials-based legacy external transforms](#)

External transforms allow connections to external systems from [Python transforms](#) repositories.

! External transforms do not support sources using [outbound applications](#) for authentication.

External transforms are primarily used to perform [batch sync](#), [export](#), and [media sync](#) workflows when one of the following is true:

- An existing Data Connection source type is not available.
- The desired capability is not available for the target source type.
- The capability offered through the Data Connection user interface does not have the desired features.

Solutions to these situations may include the following:

- Connecting to REST APIs, both over the Internet and within a private network.
- [API Reference](#) ↗ databases to arrange customized query logic not currently possible in the Data Connection user interface.
- Streaming data as needed during sync or export. This could include sending data together before writing to Foundry, handling custom encryption/decryption of data

Send feedback

during transfer, and more.

Any transforms that use [virtual tables](#) are also considered to be external transforms, since the transforms job must be able to reach out to the external system that contains the virtualized data. To use [virtual tables](#) in Python transforms, follow the instructions below for details on how to set up the source.

Setup guide

In this setup guide, we will walk through creating a Python transforms repository that connects to the [free public dictionary API ↗](#). The examples then use this API to explain various features of external transforms and how they can be used with the API.

i The dictionary API used in this setup guide is unaffiliated with Palantir and may change at any time. This tutorial is not an endorsement, recommendation, or suggestion to use this API for production use cases.

Prerequisite: Create a Python transforms repository

Before following this guide, be sure to first create a Python transforms repository and review how to author Python transforms as described in [our tutorial](#). All features of Python transforms are compatible with external transforms.

Prerequisite: Create a Data Connection source

Before you can connect to an external system from your Python repository, you must create a Data Connection source that you can import into code. For this tutorial, we will create a REST API source that connects to the dictionary API mentioned above.

Option 1: Create source in the external systems sidebar

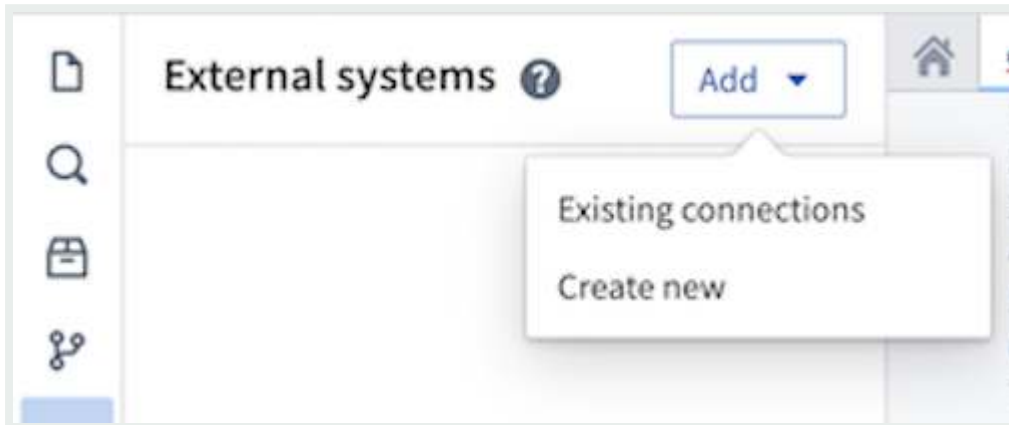
API Reference ↗

The quickest way to create a source for use in external transforms is from a Python transforms code repository. Once you have initialized a repository, complete the following steps to set up a generic source:

Send feedback

i If you are working in a Python transforms repository in a [VS Code workspace](#), the **External systems** settings are located in the **Settings** tab of the top navigation bar.

1. From the left side panel, open the **External systems** tab.
2. Select **Add > Create new**.



3. Choose a name for your source and a Project in which to store it. Upon creation, the newly created source will show up in the left side panel. Any egress policies, secrets and exportable markings can be directly configured from this panel.

[API Reference ↗](#)

[Send feedback](#)

[← Back](#)



My Generic Connection
Generic

Egress policies (0)[Manage](#)

 No egress policies are configured for this source. Network requests made using this source will fail.

Secrets (0)[Manage](#)

No secrets are configured for this source

Export configuration

Enable exports

- For this tutorial, you should add an [egress policy](#) for the dictionary API: `api.dictionaryapi.dev`. You will not need any secrets since this API does not require authentication, and export controls may be skipped for now. However, they will be required to [use Foundry data inputs with this source](#).
- Since this connection is to a REST API, you will be automatically prompted to convert your generic connector to a REST API source so that you can use the built-in Python requests client.

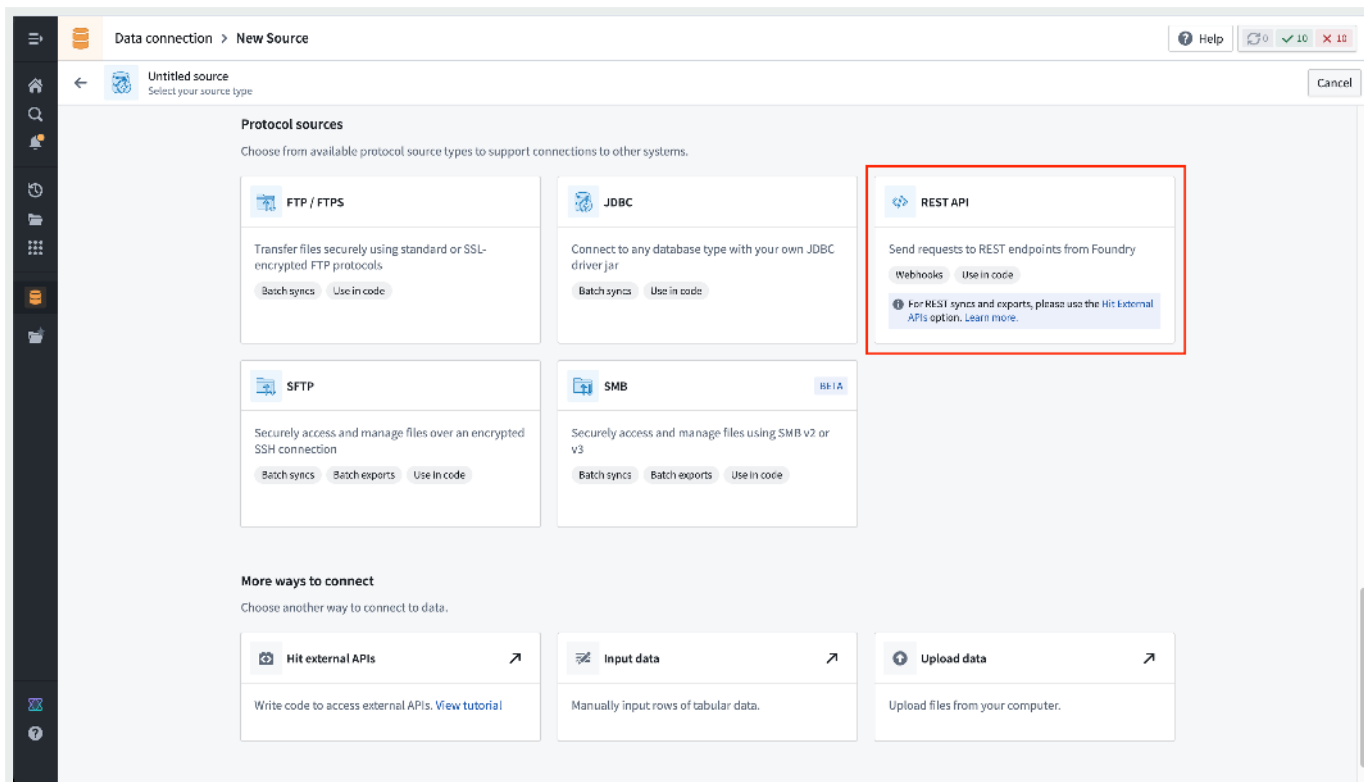
Option 2: Create a source in Data Connection

You may also create a source from the Data Connection application or use an existing source you have already configured. To use this option, follow the steps below:

- API Reference ↗

 Data Connection application within Foundry and choose **New Source**. From the list of options, select **REST API**.

[Send feedback](#)



2. Review the **Overview** page, then select **Continue** in the bottom right. You will be prompted to choose the connection worker: pick a [Foundry worker](#) to connect to the dictionary API, because [agent worker](#) connections are not supported for external transforms.
3. Choose a name for your source, and select a Project to which it should be saved.
4. Fill out the **Domains** section with the connection information of the API source. The configuration for the dictionary API example is shown below:

API Reference ↗

Send feedback

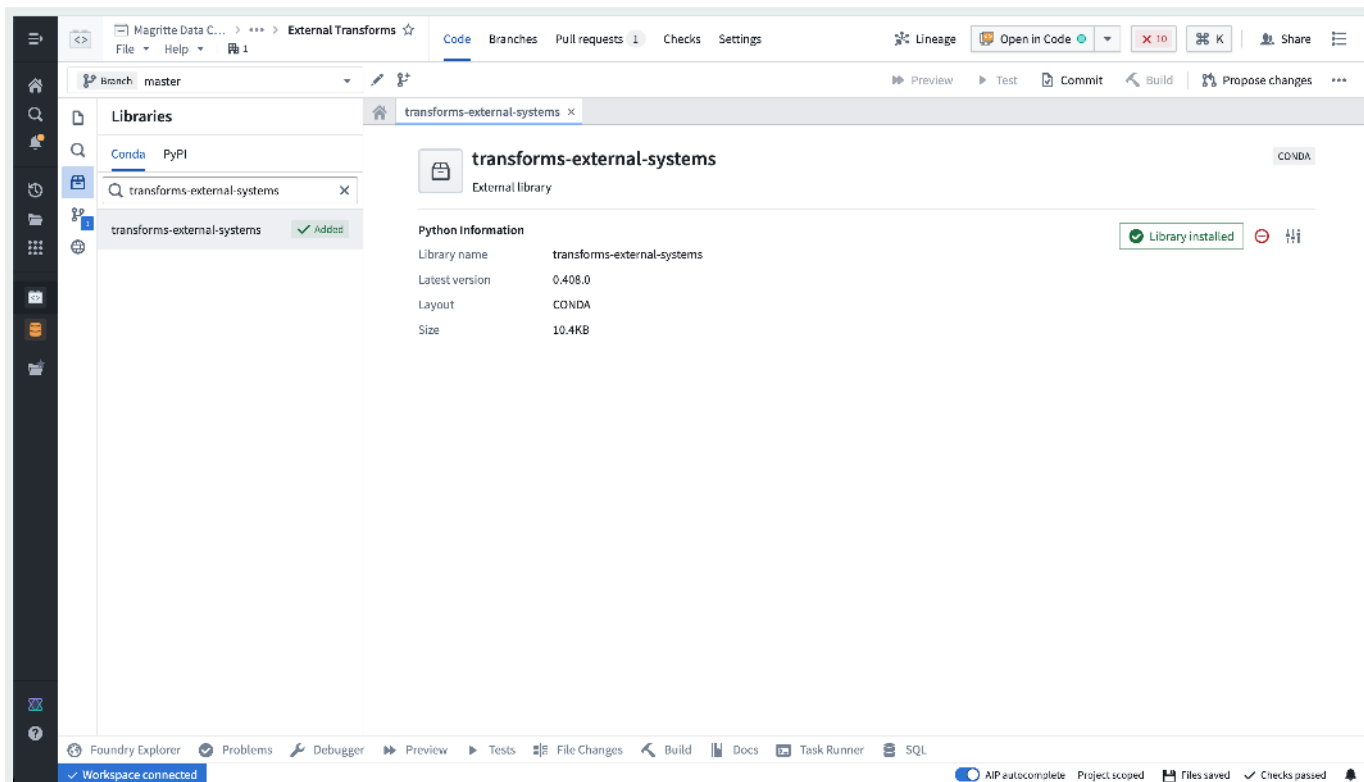
5. For this example, we also need to create the necessary egress policy. The policy will be automatically suggested in the **Network Connectivity** section if you completed the previous step:

6. Select **Save**, then **Save and continue** to complete the source setup.

Prerequisite: Import the `transforms-external-systems` library in your repository

To use external transforms, you must first import the `transforms-external-systems` library in your repository. Libraries are installed using the **Libraries** tab in the left side panel, searching for the desired library, then selecting **Install**.

Send feedback



[Learn more about installing and managing libraries..](#)

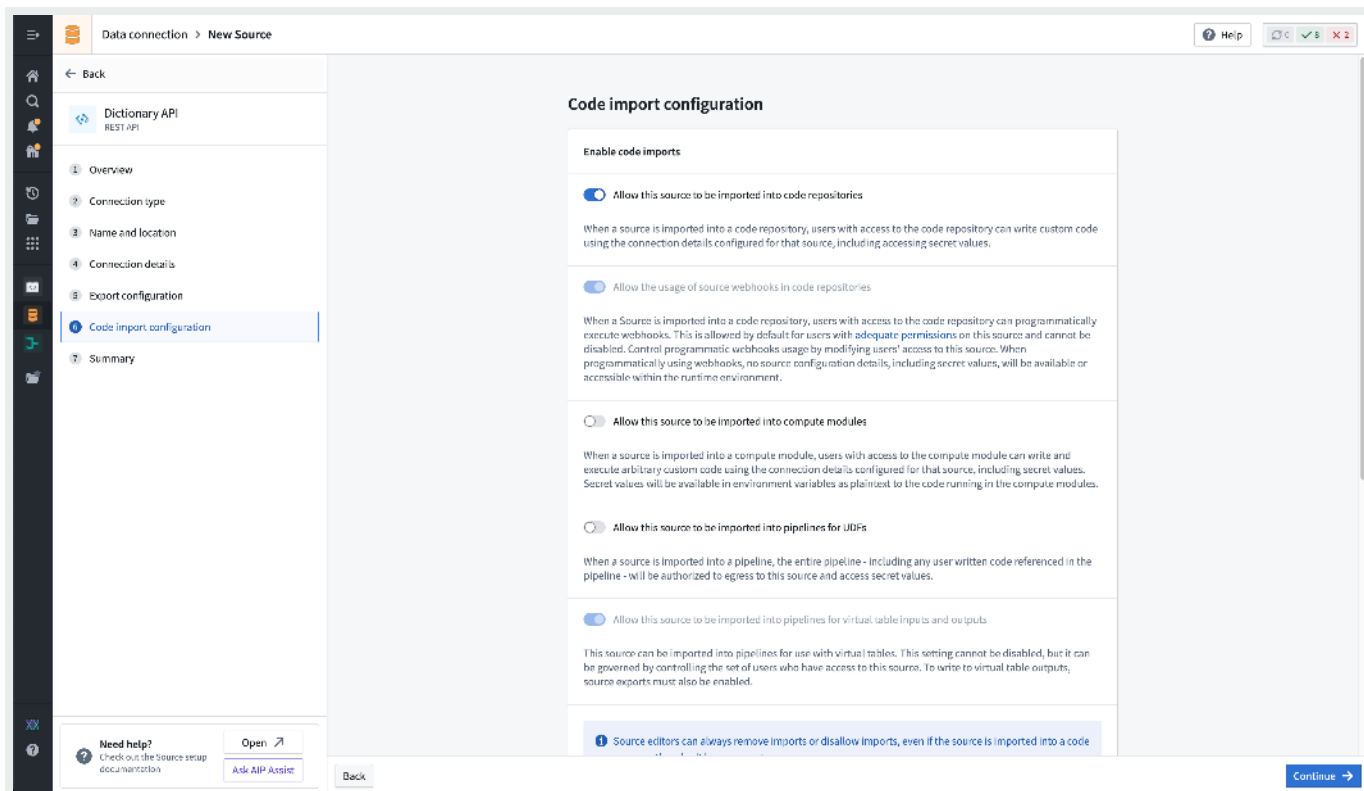
Prerequisite: Import a source into code

- i** REST API sources with multiple domains may not be imported. Instead, you should create a separate REST API source per domain if multiple domains are required in the same external transform.

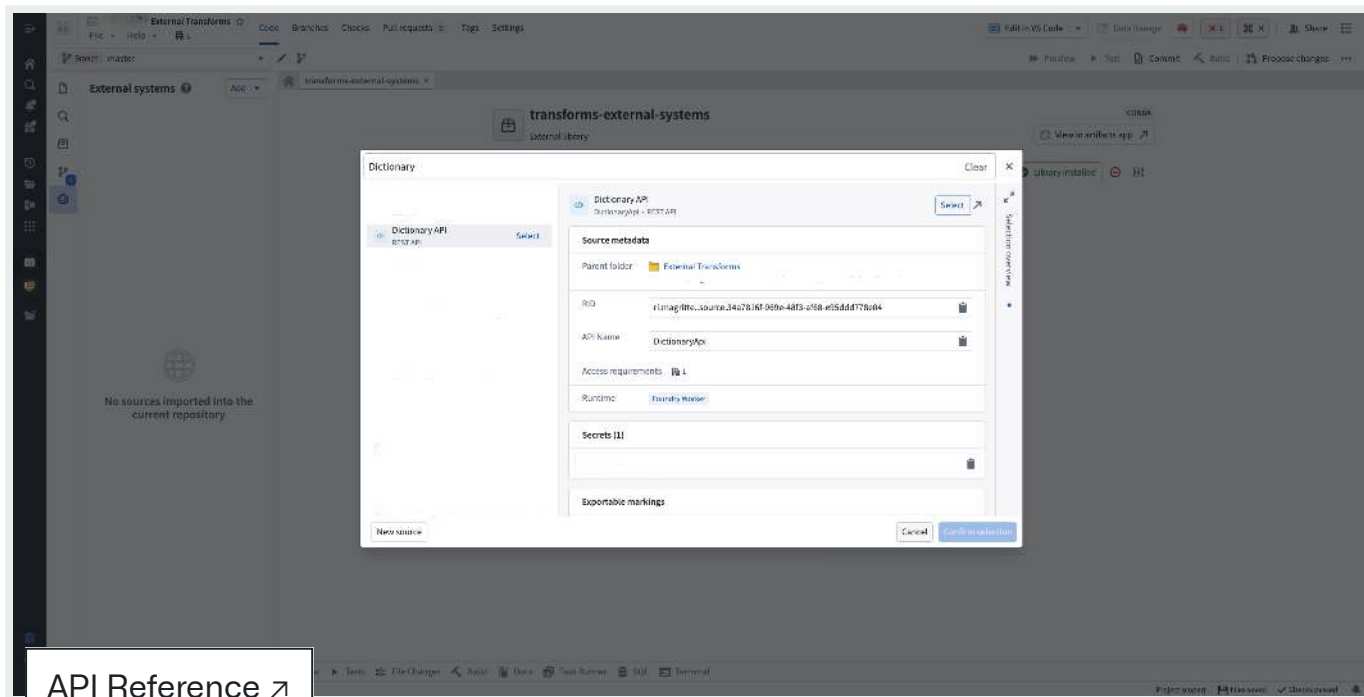
1. First, you must allow the REST API source to import into code. To configure this setting, navigate to the source in Data Connection, then to the **Connection settings > Code import configuration** tab.
2. Toggle on the option to **Allow this source to be imported into code repositories**. Any code repositories that import this source will be displayed on this page.

[API Reference ↗](#)

[Send feedback](#)



3. You are now ready to return to your code repository and import the source. In the repository, navigate to the left side panel and select the **External Systems** tab represented by the globe icon. Within the side panel, select **Add**, then search for the Dictionary API source that you previously created. Select this source, then **Confirm selection** to import.



Send feedback

i You must have at least `Editor` access to the source to be able to import it in the repository. Read more about [permissions](#)

Write external transforms

Once you set up a Python transforms repository that imports your Dictionary API source, you are ready to start writing Python transforms code that uses the source to connect externally.

i Review our [external transforms examples](#) to find fully configured examples of typical read or write workflows on top of common systems.

Import and configure the `@external_systems` decorator

To use external transforms, you must import `external_systems` decorator and `Source` object from the `transforms.external.systems` library:

```
1 from transforms.external.systems import external_systems, Source
```

You should then specify the sources that should be included in a transform by using the `external_systems` decorator:

```
1 @external_systems(  
2     dictionary_api_source=Source("ri.magritte..source.e301d738-b532-431a-8  
3 )
```

Sources will automatically be rendered as links to open in Data Connection and will display the `API Reference` instead of the resource identifier.

API Reference ↗

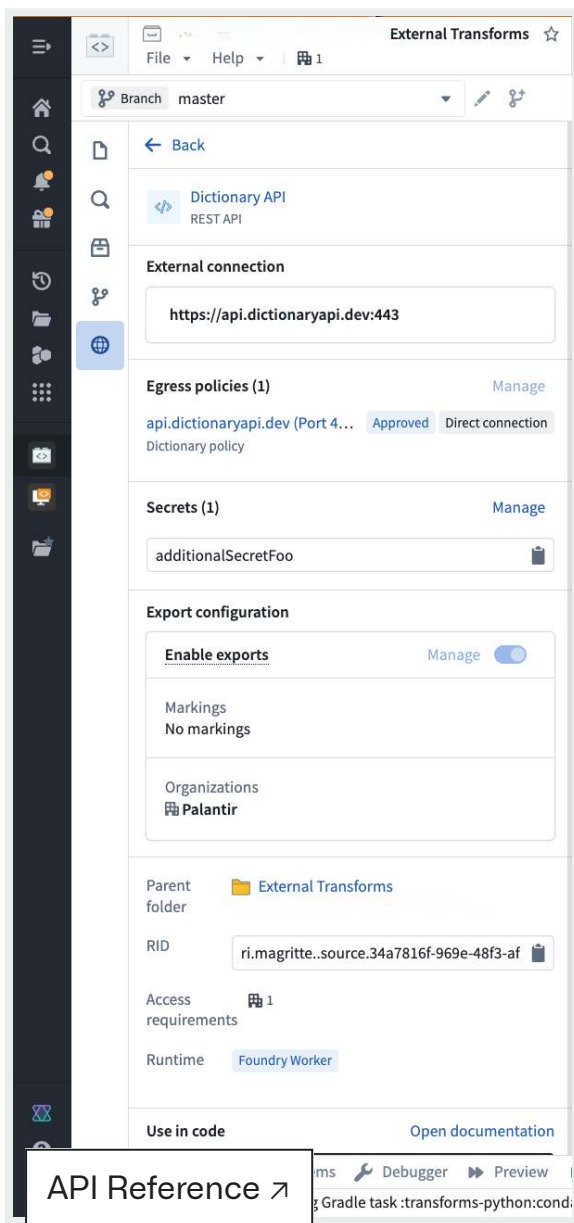
Access source attributes and credentials

Send feedback

Once a source is imported into your transform, you can access attributes of the source using the built-in connection object using the `get_https_connection()` method. The example below shows how we can grab the base URL of the Dictionary API source we configured in the previous step.

```
1 dictionary_api_url = dictionary_api_source.get_https_connection().url
```

Additional secrets or credentials stored on the source can also be accessed from the source. To identify the secret names that can be accessed, navigate to the left panel in your transform.



Use the following syntax to access secrets in code:

Send feedback

```
1 dictionary_api_source.get_secret("additionalSecretFoo")
```



- i** Currently, it is not possible to access source attributes that are not credentials unless the source provides an HTTPS client. For example, on a [PostgreSQL source](#) you will not be able to access the `hostname` or other non-secret attributes.

Use the built-in HTTP client

For sources that provide a RESTful API, the source object allows you to interact with a built-in HTTPS client. This client will be pre-configured with all of the details specified on the source, including any server or client certificates, and you can simply start making requests to the external system.

```
1 dictionary_api_url = dictionary_api_source.get_https_connection().url
2 dictionary_api_client = dictionary_api_source.get_https_connection().get_client()
3
4 # dictionary_api_client is a pre-configured Session object from Python `requests`
5 # Example of GET request:
6 response = dictionary_api_client.get(dictionary_api_url + "/api/v2/entries")
```

Alternatively, you can use your own client or source-specific Python libraries and use the source object to [retrieve attributes and credentials](#).

- ⚠** Changing the working directory (for example, using `os.chdir()`) in your transforms or UDFs may break references to environment variables necessary for establishing secure connections.

API Reference ↗

ing to an on-premise system using an [agent proxy runtime \(sunset\)](#), the built-in client since that will be automatically configured with the necessary agent proxy configuration.

Send feedback

Example: Import data from the Dictionary API

The below example illustrates a complete transform that runs through a list of words and retrieves their phonetic transcription from the Dictionary API.

```
1 from pandas import DataFrame
2 from transforms.api import transform
3 from transforms.api import Output, Input, TransformContext, transform_pandas
4 from transforms.external.systems import external_systems, Source
5 import pandas as pd
6 import logging
7
8 logger = logging.getLogger(__name__)
9
10
11 @external_systems(
12     dictionary_api_source=Source(
13         "<source_rid>"
14     )
15 )
16 @transform_pandas(Output("<output_dataset_rid>"))
17 def compute(dictionary_api_source) -> DataFrame:
18     dictionary_api_url = dictionary_api_source.get_https_connection().url
19     dictionary_api_client = dictionary_api_source.get_https_connection().client
20
21     words = ["apple", "dog", "cat"]
22
23     phonetics = []
24
25     for word in words:
26         logger.info("Fetching word from api.dictionaryapi.dev : " + word)
27
28         response = dictionary_api_client.get(
29             dictionary_api_url + "/api/v2/entries/en/" + word
30         ).json()
31
32         phonetics += [{"word": word, "phonetic": response[0]["phonetic"]}
33
34     return pd.DataFrame(phonetics)
```

[API Reference ↗](#)

Use Foundry inputs in external transform

[Send feedback](#)

External transforms often need to use Foundry input data. For example, you might want to query an API to gather additional metadata for each row in a tabular dataset. Alternatively, you might have a workflow where you need to export Foundry data into an external software system.

Such cases are considered *export-controlled workflows*, as they open the possibility of exporting secure Foundry data into another system with unknown security guarantees and severed data provenance. When configuring a source connection, the source owner must specify whether or not data from Foundry may be exported, and provide the set of security markings and organizations may be exported. Foundry provides governance controls to ensure developers can clearly encode security intent, and Information Security Officers can audit the scope and intent of workflows interacting with external systems.

Configure export controls on the source

Exports are controlled using [security markings](#). When configuring a source, the export configuration is used to specify which security markings and organizations are safe to export to the external system. This is done by navigating to the source in the data connection application, and then navigating to the **Connection settings > Export configuration** tab. You should then toggle on the option to **Enable exports to this source** and select the set of markings and organizations that may potentially be exported.

Doing this requires permission to remove markings on the relevant data and Organizations, since exporting is considered equivalent to removing markings on data within Foundry.

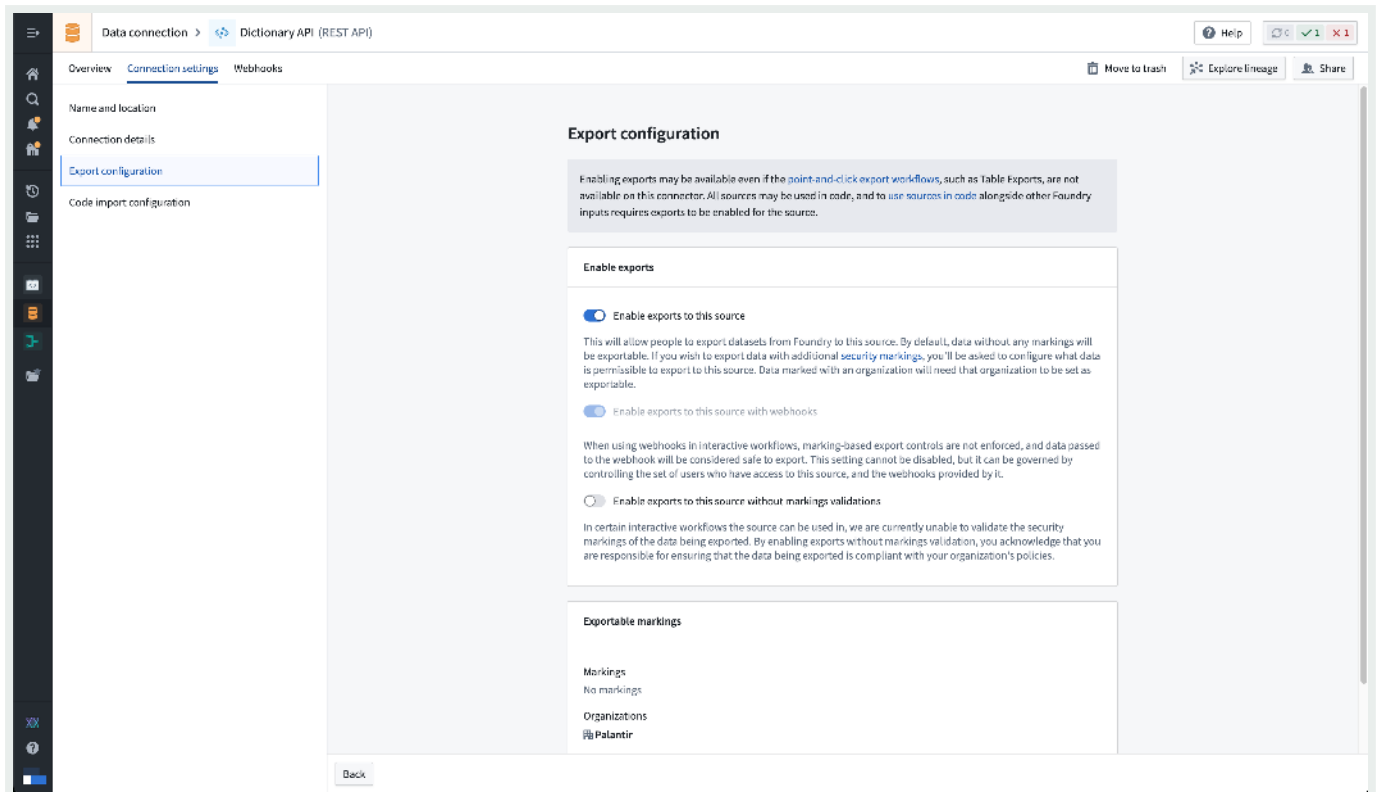
The setting to **Enable exports to this source** must be toggled on to allow the following:

- Use datasets, media sets, and streams as an input to Python transforms code importing this source.
- Use virtual tables registered on this source in Python transforms.

Below you can see an example export configuration for the Dictionary API source, allowing data from the `Palantir` organization with no additional security markings to be exported to the Dictionary API:

API Reference ↗

Send feedback



i Note that **Enable exports to this source** must be toggled on *even if you are not actually exporting data to this system*, since allowing Foundry data inputs into the same compute job with an open connection to this system means that data *could* be exported.

Example: Use Foundry imports alongside data from the Dictionary API

In this example, we use an input dataset of words instead of a static hard-coded list. It also illustrates basic error handling based on the status code of the response.

```
1 from pandas import DataFrame
2 from transforms.api import transform
3 from transforms.api import Output, Input, TransformContext, transform_parallel
4 from transforms.external.systems import external_systems, Source, Resolve
```

API Reference ↗

```
7
8 logger = logging.getLogger(__name__)
9
```

Send feedback

```

10
11 @external_systems(
12     dictionary_api_source=Source(
13         "<source_rid>"
14     )
15 )
16 @transform_pandas(
17     Output("<output_dataset_rid>"),
18     words_df=Input("<input_dataset_rid>"),
19 )
20 def compute(dictionary_api_source: ResolvedSource, words_df: DataFrame) -
21     dictionary_api_url = dictionary_api_source.get_https_connection().url
22     dictionary_api_client = dictionary_api_source.get_https_connection().
23
24     words = words_df["word"].tolist()
25
26     phonetics= []
27
28     for word in words:
29         logger.info("Fetching word from api.dictionaryapi.dev: " + word)
30
31         response = dictionary_api_client.get(
32             dictionary_api_url + "/api/v2/entries/en/" + word
33         )
34
35         if response.status_code == 200:
36             data = response.json()[0]
37
38             if "phonetic" in data:
39                 phonetic_transcription = data["phonetic"]
40             else:
41                 logger.warning(f"No phonetic transcription found for {word}")
42                 phonetic_transcription = None
43         else:
44             logger.warning(f"Request for {words} failed with status code")
45             phonetic_transcription = None
46
47         phonetics += [{"word": word, "phonetic": phonetic_transcription}]
48
49     return pd.DataFrame(phonetics)

```

Review the documentation below the find complex end-to-end examples for common systems:

- REST API sources
 - [OAuth Client Credentials grant](#)
 - [Self-signed server certificates](#)
- Foundry API
 - [Call the Foundry API from code](#)
- File-based systems
 - [Amazon S3](#)
 - [SMB](#)
- Custom Python SDKs
 - [BigQuery](#)
 - [Snowflake](#)
 - [SharePoint](#)
 - [SFTP](#)
 - [Kafka](#)
- Agent host file access
 - [Directory \(SSH\)](#)
- JDBC-based systems
 - [PostgreSQL](#)
 - [Microsoft SQL Server](#)

Permissions

Before using external transforms, make sure to familiarize yourself with the [Data Connection - Permissions reference](#) page.

Comparison of external transforms and legacy external transforms

API Reference ↗ The key workflow differences between external transforms and legacy external transforms:

Send feedback

- The tab for importing sources will always automatically show for external transforms. Previously, tabs for adding egress policy and credentials would only show after an Information Security Officer had toggled on the ability to use external systems in repository settings.
- Settings to allow external connections and the use of inputs are no longer located in repository settings. Instead, these are controlled on each individual source.
- Credentials, egress policies, and exportable markings are no longer specified in code. Instead, these settings are taken from the sources that are imported into the transform and applied automatically to the job.
 - If this configuration is changed at the source level, it will automatically be picked up by transforms that import the source without any code change or version bump required. This allows centralized governance of credentials, egress, and exportable Markings which will propagate immediately to downstream workflows.
 - Changes will take effect as of the start of a build and will not affect running builds.
- The decorator has changed from `@use_external_systems()` to `@external_systems()`.

Key advantages of external transforms include the following:

- Support for connecting to systems not accessible from the Internet
- Support for rotating/updating credentials without requiring code changes
- Support for sharing connection configuration across multiple repositories
- Out-of-the-box Python clients for selected source types
- Improved and simplified governance workflows for enabling and managing external transform repositories
- Visualization of external transforms connected to external sources in [Data Lineage](#)

Migrate to external transforms

There is currently no automatic migration path to update external transforms to external transforms. However, the manual action required is expected to be minimal for most workflows.

The following are the main steps to manually migrate to external transforms:

API Reference ↗

1. Migrate the code for credentials, egress policies, and export control Markings used in your existing legacy external transforms code.

Send feedback

2. Identify or configure Data Connection sources that connect to the systems you wish to connect to from your external transforms. Ensure these sources are configured to allow imports into code.
3. Import the relevant sources from step 2 into your existing Python transforms repository.
4. Change your code to import and use the new `@external_systems()` decorator with source references, then remove any instances of the `@use_external_systems()` decorator. This will likely involve updating any references to credentials in your transforms logic to instead reference credentials retrieved from the sources you are now importing.
5. Test your changes on a branch to ensure that your transforms continue to build successfully.
6. After merging your updated transforms code, you can now un-toggle the repository settings.

Transforms cannot contain both external transforms and their legacy version. To remedy this, you can migrate all legacy external transforms to use source-based external transforms instead (preferred), or split your transform into multiple transforms. Transforms can be split into one that uses the `use_external_systems` decorator and another that uses the `external_systems` decorator.

Capabilities

lightweight external transforms

External transforms are also compatible with the single-node (lightweight) [compute engines](#). Using lightweight compute can dramatically increase the execution speed for transforms operating on small and medium-sized data.

The below example shows how the `@lightweight` decorator can be added to a transform along with the `@external_systems` decorator. For more information on the options for configuring lightweight transforms, see the [Python transforms documentation](#).

API Reference ↗

```
2 @external_systems(  
  ==
```



Send feedback

```
3 dictionary_api_source=Source("<source_rid>")
4 )
```

For more in-depth examples, refer to [sources in Python](#).

Sources using agent proxy policies

External transforms support connecting to on-premise and privately hosted systems through Data Connection agents using [agent proxy policies](#).

The example below demonstrates how to connect to a PostgreSQL database using a Spark sidecar transform. For more information, see the [Spark sidecar transforms documentation](#).

Note that Spark sidecar transforms may not be needed for simpler use cases than the example below; other examples that do not need Spark sidecar transforms can be found [elsewhere on this page](#).

Create a source using an Agent network policy

To configure external transforms with private network access:

1. **Create a [PostgreSQL source](#) with direct connection runtime:** When creating your Data Connection source, select **Direct connection** as the runtime option.
2. **Configure network policy:** In the source's network connectivity settings, create a network policy that defines the routing through your Data Connection agent. Select **Agent proxy** and then choose the agents that should be used to proxy.

API Reference ↗

Send feedback

Request egress policy

×

1 Configure egress

2 Configure permissions

Policies must be applied to sources in Data Connection to take effect.

How to reach an address

Agent proxy

Description

Describe the type of traffic and/or destination that this policy will allow.

Export to cloud storage provider

Address

Specify the address that this policy will allow.

DNSmydomain.com

Port number

Specify the port number or range that this policy will allow.

Single port443

Advanced settings

Agent(s)

Search agents

Postgres Agent

9.2453.0HEALTHY

3. **Enable code imports:** In the code import configuration panel, toggle on **Allow this source to be imported into code repositories**.

Data connection > New Source

Help

← Back

Dictionary API
REST API

1 Overview

2 Connection type

3 Name and location

4 Connection details

5 Export configuration

6 Code import configuration

7 Summary

API Reference ↗

Need help?
Check out the Source setup documentation

Open ↗
Ask AIP Assisat

Back

Code import configuration

Enable code imports

☒ Allow this source to be imported into code repositories

When a source is imported into a code repository, users with access to the code repository can write custom code using the connection details configured for that source, including accessing secret values.

☒ Allow the usage of source webhooks in code repositories

When a Source is imported into a code repository, users with access to the code repository can programmatically execute webhooks. This is allowed by default for users with **adequate permissions** on this source and cannot be disabled. Control programmatic webhooks usage by modifying users' access to this source. When programmatically using webhooks, no source configuration details, including secret values, will be available or accessible within the runtime environment.

☐ Allow this source to be imported into compute modules

When a source is imported into a compute module, users with access to the compute module can write and execute arbitrary custom code using the connection details configured for that source, including secret values. Secret values will be available in environment variables as plaintext to the code running in the compute modules.

☐ Allow this source to be imported into pipelines for UDFs

When a source is imported into a pipeline, the entire pipeline - including any user written code referenced in the pipeline - will be authorized to egress to this source and access secret values.

☒ Allow this source to be imported into pipelines for virtual table inputs and outputs

This source can be imported into pipelines for use with virtual tables. This setting cannot be disabled, but it can be governed by controlling the set of users who have access to this source. To write to virtual table outputs, source exports must also be enabled.

Source editors can always remove imports or disallow imports, even if the source is imported

Send feedback

4. Bootstrap a transform code repository and import the source.

Example: PostgreSQL connection via Spark sidecar

This example shows how to query a PostgreSQL database using a containerized Flask API within a sidecar container.

Dockerfile

```
1 FROM python:3.8-slim
2 WORKDIR /usr/src/app
3 COPY requirements.txt ./
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 1234
7 USER 5001
8 CMD ["python", "app.py"]
```

requirements.txt

```
flask
psycopg2-binary
```

Flask application (app.py)

```
1 from flask import Flask, jsonify
2 import psycopg2
3
4 app = Flask(__name__)
5
6 @app.route('/tables', methods=['POST'])
7 def tables():
8     data = request.get_json()
9     password = data.get('password')
10
11     connection_params = {
12         "name": "<databasename>",
13         "user": "<user>",
14         "password": password,
15         "host": "postgres.com",
```

API Reference ↗

Send feedback

```

16         "port": "5432",
17     }
18
19
20     data = {"schema": [], "table": []}
21     with psycopg2.connect(**connection_params) as conn:
22         with conn.cursor() as cur:
23             cur.execute(
24                 "SELECT table_schema, table_name FROM information_schema."
25             )
26             tables = cur.fetchall()
27             for schema, table in tables:
28                 data["schema"].append(schema)
29                 data["table"].append(table)
30
31     return jsonify(data)

```

Transform implementation

```

1  import requests
2  from transforms.sidecar import sidecar
3  from transforms.external.systems import external_systems, Source
4  from transforms.api import transform_df, Output
5
6  @external_systems(
7      my_source=Source("<source_rid>")
8  )
9  @sidecar(image="simple", tag="0.0.1")
10 @transform_df(
11     Output("<output_dataset_rid>"),
12 )
13 def compute(my_source, ctx):
14     password = my_source.get_secret("PASSWORD")
15     response = requests.post("http://localhost:1234/tables", json={"passw
16     data = [(response.text,)]
17     columns = ["table_schema_json"]
18     return ctx.spark_session.createDataFrame(data, columns)

```

API Reference ↗

Advanced patterns

Send feedback

Write data to Parquet files with memory-aware buffering

When an external transform fetches many records from an external source, holding all records in memory before writing can cause out-of-memory errors. The

`BufferedParquetWriter` class below periodically flushes accumulated rows to Parquet files once a configurable memory threshold is exceeded. The pattern works with any external data source and uses `filesystem().open()` to write raw files and `put_metadata()` to finalize schema inference.

```
1 import gc
2 import logging
3 from datetime import datetime
4
5 import pyarrow as pa
6 import pyarrow.parquet as pq
7 from pympler import asizeof
8
9 from transforms.api import Output, transform, LightweightOutput
10 from transforms.external.systems import external_systems, Source, Resolve
11
12 logger = logging.getLogger(__name__)
13
14
15 class BufferedParquetWriter:
16     """Buffers rows in memory and flushes them to Parquet files when the
17     exceeds a configurable size threshold."""
18
19     def __init__(self, output: LightweightOutput, schema: pa.Schema, flush
20         self.output = output
21         self.schema = schema
22         self.flush_threshold_bytes = flush_threshold_mb * 1024 * 1024
23         self.buffer = []
24         self.buffer_size_bytes = 0
25         self.file_index = 0
26         self.row_size_estimate = None
27
28     def append(self, row: dict):
29         """Add a row to the buffer. If the buffer exceeds the threshold,
30         self.buffer.append(row)
31
32         estimate row size from the first row, then use that estimate go
33         if self.row_size_estimate is None:
34             self.row_size_estimate = asizeof.asizeof(r
35             self.buffer_size_bytes += self.row_size_estimate
```

API Reference ↗

Send feedback

```

36
37     if self.buffer_size_bytes >= self.flush_threshold_bytes:
38         self.flush()
39
40     def flush(self):
41         """Write buffered rows to a Parquet file and clear the buffer."""
42         if not self.buffer:
43             return
44         self.file_index += 1
45         table = pa.Table.from_pylist(self.buffer, schema=self.schema)
46         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
47         filename = f"output_{timestamp}_{self.file_index}.parquet"
48         with self.output.filesystem().open(filename, "wb") as f:
49             pq.write_table(table, f)
50         logger.info(f"Flushed {len(self.buffer)} rows to {filename}")
51         self.buffer = []
52         self.buffer_size_bytes = 0
53         gc.collect()
54
55     def finalize(self):
56         """Flush any remaining rows and trigger schema inference."""
57         self.flush()
58         if self.file_index > 0:
59             self.output.put_metadata()
60
61
62 # Define the schema for your output records
63 OUTPUT_SCHEMA = pa.schema([
64     ("id", pa.string()),
65     ("name", pa.string()),
66     ("value", pa.float64()),
67 ])
68
69
70 @external_systems(
71     my_source=Source("<source_rid>")
72 )
73 @transform.using(
74     output=Output("<output_dataset_rid>"),
75 ).with_resources(
76     memory_gb=4,
77 )
78 def compute(my_source: ResolvedSource, output: LightweightOutput):
79     writer = BufferedParquetWriter(output, OUTPUT_SCHEMA, flush_threshold=1000)
80     writer.set_mode("replace")
81
82     # Replace the loop below with your data fetching logic
83     # For example, iterate over pages from an API, rows from a database

```

API Reference ↗

Send feedback

```

84     # or messages from a message queue.
85     # Each call to append() adds data to the writer, which automatically
86     # flushes to a Parquet file when the buffer exceeds the threshold.
87     for record in fetch_records_from_source(my_source):
88         writer.append({
89             "id": record["id"],
90             "name": record["name"],
91             "value": record["value"],
92         })
93
94     # Write any remaining buffered rows and finalize schema inference.
95     writer.finalize()

```

Export a dataset as CSV

When you need to export data from Foundry as CSV, for example to deliver files to an external system that expects CSV format, you can use an external transform that reads an input dataset and writes a CSV file to the output using `filesystem().open()`. The output dataset can then be exported through [Data Connection](#).

```

1  import csv
2
3  from transforms.api import Input, Output, transform, LightweightInput, Li
4
5
6  @transform.using(
7      source_data=Input("<input_dataset_rid>"),
8      csv_output=Output("<output_dataset_rid>"),
9  )
10 def compute(source_data: LightweightInput, csv_output: LightweightOutput)
11     OUTPUT_FILENAME = "export.csv"
12
13     # Read the input dataset as a Polars DataFrame.
14     df = source_data.polars()
15     fieldnames = df.columns
16
17     # Write the data as a CSV file.
18     csv_output.filesystem().open(OUTPUT_FILENAME, "w") as f:
19         writer = csv.DictWriter(f, fieldnames=fieldnames)
20         writer.writeheader()

```

API Reference ↗

Send feedback

```
21         for row in df.iter_rows(named=True):
22             writer.writerow(row)
```

PREVIOUS

← Email listeners / Security

NEXT

External functions →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / External connections from code / External functions

External Functions

External Functions allow you to call [webhooks](#) from a Function and use them to interact with external systems. You can call these systems from applications built using [Workshop](#), [Actions](#), and [Functions](#).

To use a webhook in a Function, you must first configure a [Data Connection source](#) that supports the webhooks capability. Normally this will be a [REST API source](#). Once you have a source with webhooks configured, you can import the source into your [Functions repository](#) and create Functions that call webhooks and other logic.

You can apply external Functions for a variety of use cases, including the following:

- Use a single function to make an HTTP request to an external server and create a record, then write a matching record to the Ontology.
- Wrap a webhook call with custom pre- and post-processing logic. This is particularly useful if custom logic is required to go from Function inputs to the desired webhook request input parameters, or if post-processing on the webhook response is required before making Ontology edits.
- Chain together multiple external webhook requests and Ontology edits with intermediate processing logic. A single webhook cannot perform a dynamic number of external requests, but this can be accomplished using external Functions.
- Query an external system using a read-only request to render data ephemerally in a Workshop application.

API Reference ↗

ons may not currently be used to make arbitrary API calls from
e without first defining the request as a webhook in Data
ction.

Send feedback

Concepts

- A [Data Connection source](#) represents a connection to an external system, including any configuration for how to reach that system (such as networking details), and securely stored credentials.
- [Webhooks](#) are a capability supported by some source types that allow you to build structured requests to run interactively and send requests to that system.
- An [Ontology edit Function](#) is a [Function](#) that can be later configured as a [Function-backed Action](#) to write back to an object. It is annotated with the `@OntologyEditFunction()` decorator.
- A [query Function](#) is a read-only Function that cannot have any side effects, like altering an external system. It is annotated with the `@Query()` decorator.

Setup guide

In the following setup guide, we will walk through creating a webhook that calls a [free public dictionary API ↗](#). If you already have an existing source with webhooks configured, proceed to [external sources in functions](#) for calling the webhooks from functions.

i The dictionary API illustrated here is unaffiliated with Palantir and may change at any time. This tutorial is not an endorsement, recommendation, or suggestion to use this API for production use cases.

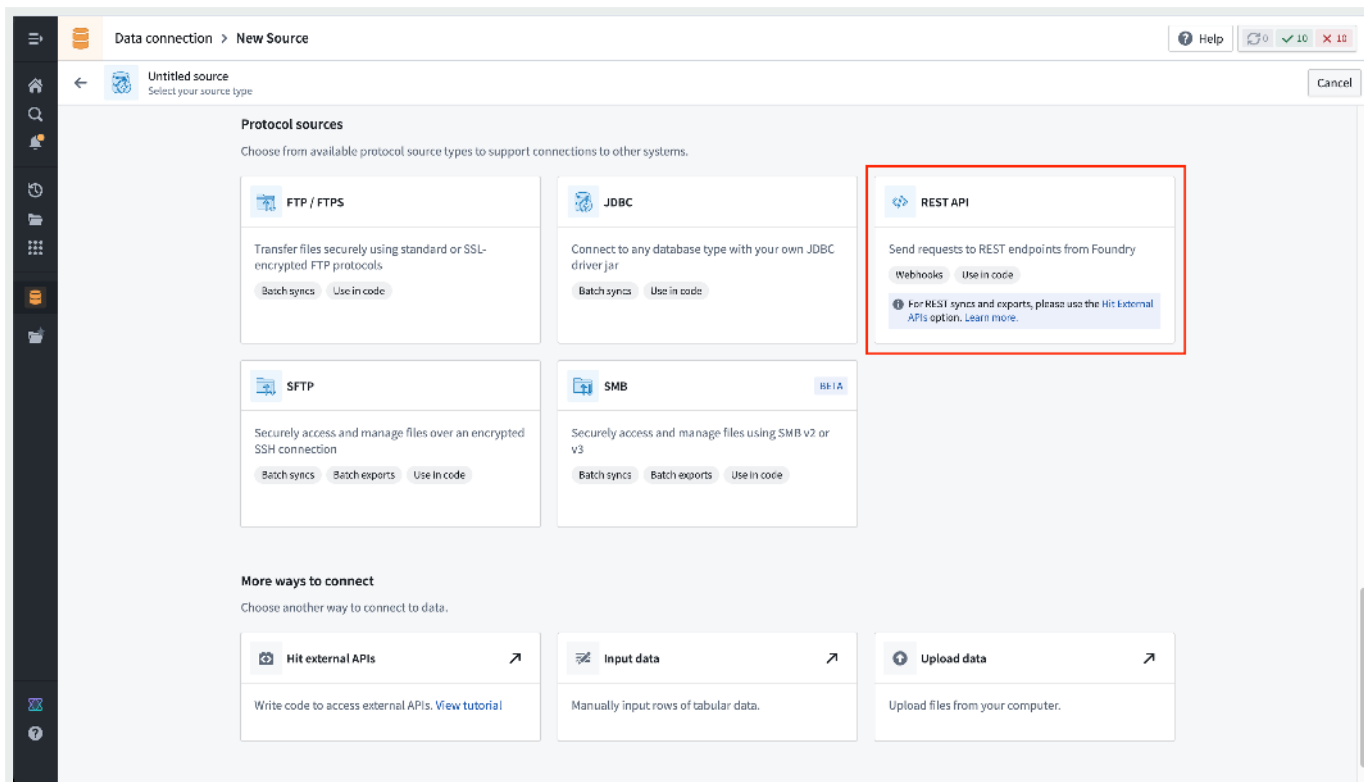
Create a Data Connection source

To connect to an external system from Functions, you must have a REST API source that can connect to the desired external API. You can follow instructions below to configure a new REST API source.

1. Navigate to the Data Connection application within Foundry and choose **New Source**.

API Reference ↗ options, select **REST API**.

Send feedback



2. Review the **Overview** page, then select **Continue** in the bottom right. You will be prompted to choose the connection runtime: a [Foundry worker](#), an [agent worker](#), or an [agent proxy runtime \(sunset\)](#). A Foundry worker is the preferred method when interacting with anything reachable from Foundry's network directly (via [direct connection egress policies](#) or anything hosted on a network different from Foundry's (via [agent proxy egress policies](#)). We will use a Foundry worker with a direct connection policy to connect to our free dictionary API.
3. Choose a name for your source, and select a project to which it should be saved.
4. Fill out the **Domains** section with the connection information of the API source. The configuration for our free dictionary API example is shown below:

API Reference ↗

Send feedback

The screenshot shows the 'Source Setup' page for a connection named 'MyDictionarySource'. The left sidebar contains navigation links: Overview, Connection settings (selected), and Webhooks. Under 'Connection settings', there are sub-links: Name and location, Connection details (selected), Export configuration, and Code import configuration. The main content area is titled 'Source Setup' and contains a 'Domains' section. This section has a table with one entry: 'https://api.dictionaryapi.dev' with 'No auth' and 'Port 443'. Below the table are fields for 'Domain base URL' (set to 'HTTPS' and 'api.dictionaryapi.dev'), 'Authentication' (set to 'None'), and 'Port (optional)' (set to '443'). A message states: 'Based on the selected authentication mode, you may need different inputs. To learn more about HTTP authentication, please visit the MDN documentation.' At the bottom of the 'Domains' section is a button 'Add new domain'. Below this is an 'Additional secrets' section with a description and an 'Add' button. At the very bottom are 'Back' and 'Save' buttons.

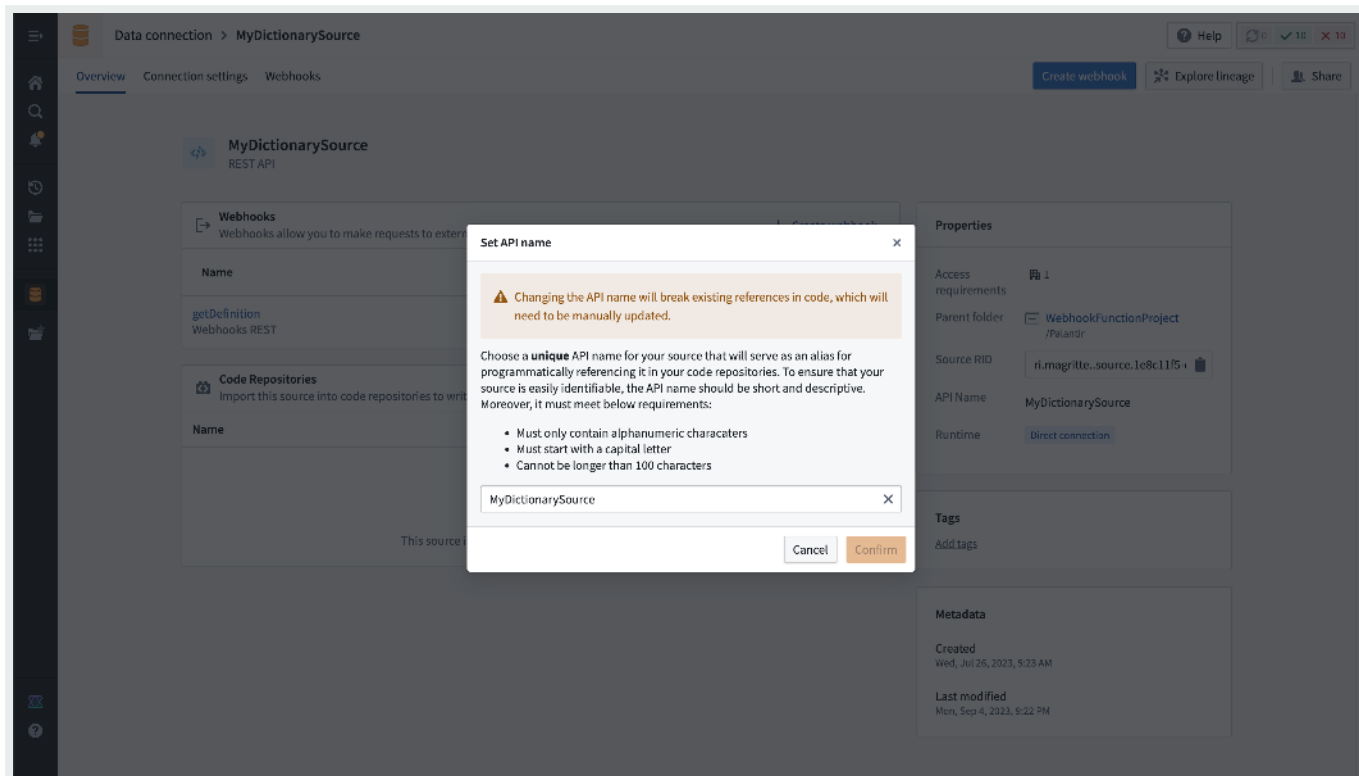
5. For this example, we also need to create the necessary egress policy. The policy will be automatically suggested in the **Network Connectivity** section if you completed the previous step:

The screenshot shows a section titled 'Suggested network egress policies' with the text: 'These suggested egress policies are required for Foundry to connect to your source.' Below this is a card for a policy named 'api.dictionaryapi.dev (Port 443)' with a description: 'A policy that allows a direct connection to a public dictionary API'. To the right of the card is a blue button labeled 'Import existing policy'.

6. Select **Save**, then choose **Save and continue** to complete the source setup. Before we configure a webhook to use on this source, navigate back to the source **Overview** page and ensure that an API name is set. This name is required to reference the source in code.

API Reference ↗

Send feedback



Create a webhook on a Data Connection source

Now, you must set up the webhook on the REST API source you configured in the previous step. Then, before you can call a webhook from a Function, you must configure the webhook and assign it an API name. A Data Connection source may have multiple webhooks associated with it. Note that your source API name should be unique within a namespace, and the webhook API name must be unique within the source.

Follow the steps below to configure a webhook that makes a request to the dictionary API to fetch the definition of a single word.

1. On the **Overview** page for the source, choose **Create webhook**. Give the webhook a name, description, and API name. As with the API source, we will reference the webhook in code.

API Reference ↗

Send feedback

Data connection > MyDictionarySource > getDefinition

Overview Settings History

getDefinition Webhook

Webhook name and connection

Request and Response Version 4

Request configuration

Responses

Storage and retention

Execution policy

Need help? Check out the Webhook setup documentation. Open Ask AIP Assist

We recommend naming the Webhook such that the name describe request to the external system. For example, "Change Flight record in Flights database" or "Create Invoice in ERP system A".

Name: getDefinition

Description: Accepts a single English word as a parameter and returns a list of definitions.

API name: GetDefinition

Changing the API name will break existing references in code, which will need to be manually updated.

Choose a unique API name for your webhook that will serve as an alias for programmatically referencing it in your code repositories. Keep in mind that the API name must be **unique** for all webhooks associated with the same external system. To ensure that your webhook is easily identifiable, the API name should be short, descriptive and meet below requirements:

- Must only contain alphanumeric characters
- Must start with a capital letter
- Cannot be longer than 100 characters

2. Define a parameter to pass in when executing the webhook. In our example, we will use a single string input parameter, `wordToDefine`.

Data connection > MyDictionarySource > getDefinition

Overview Settings History

getDefinition Webhook

Webhook name and connection

Request and Response Version 4

Request configuration

Responses

Storage and retention

Execution policy

Need help? Check out the Webhook setup documentation. Open Ask AIP Assist

GET https://api.dictionaryapi.../api/v2/entries/en/ wordToDefine

Query Params Authorization Headers Body Read API

+ Add new query param

+ Add new call Import call from cURL

Input Parameters (1)

Input parameters represent the inputs that can be passed in when a user executes the Webhook. Typically, a Webhook is configured for use in Action types, where you can specify a mapping between Action parameters and Webhook input parameters.

PARAMETER ID: wordToDefine Add description...

String

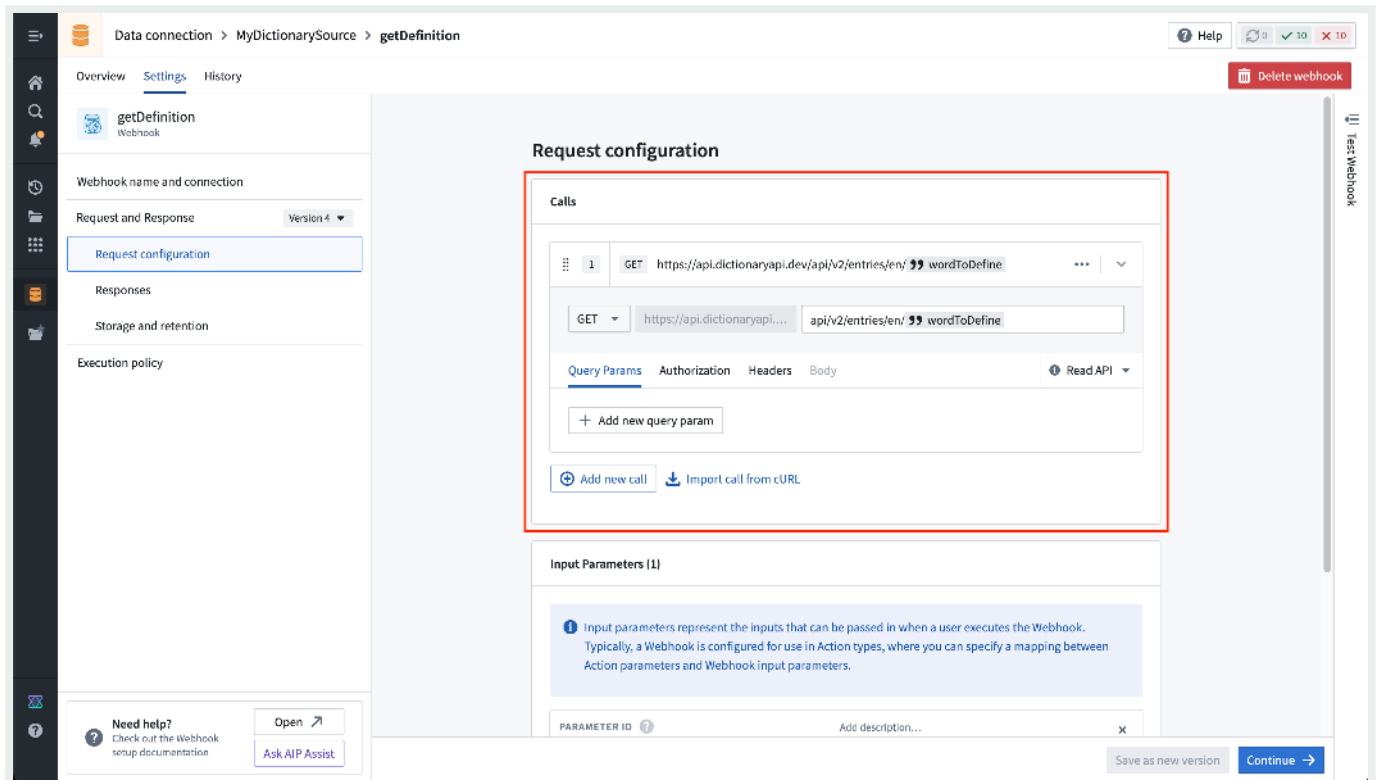
Only allow specific values

+ Add parameter

Save as new version Continue

API Reference > dictionary resource path in the URL, with the input parameter referenced at the end as shown below:

Send feedback

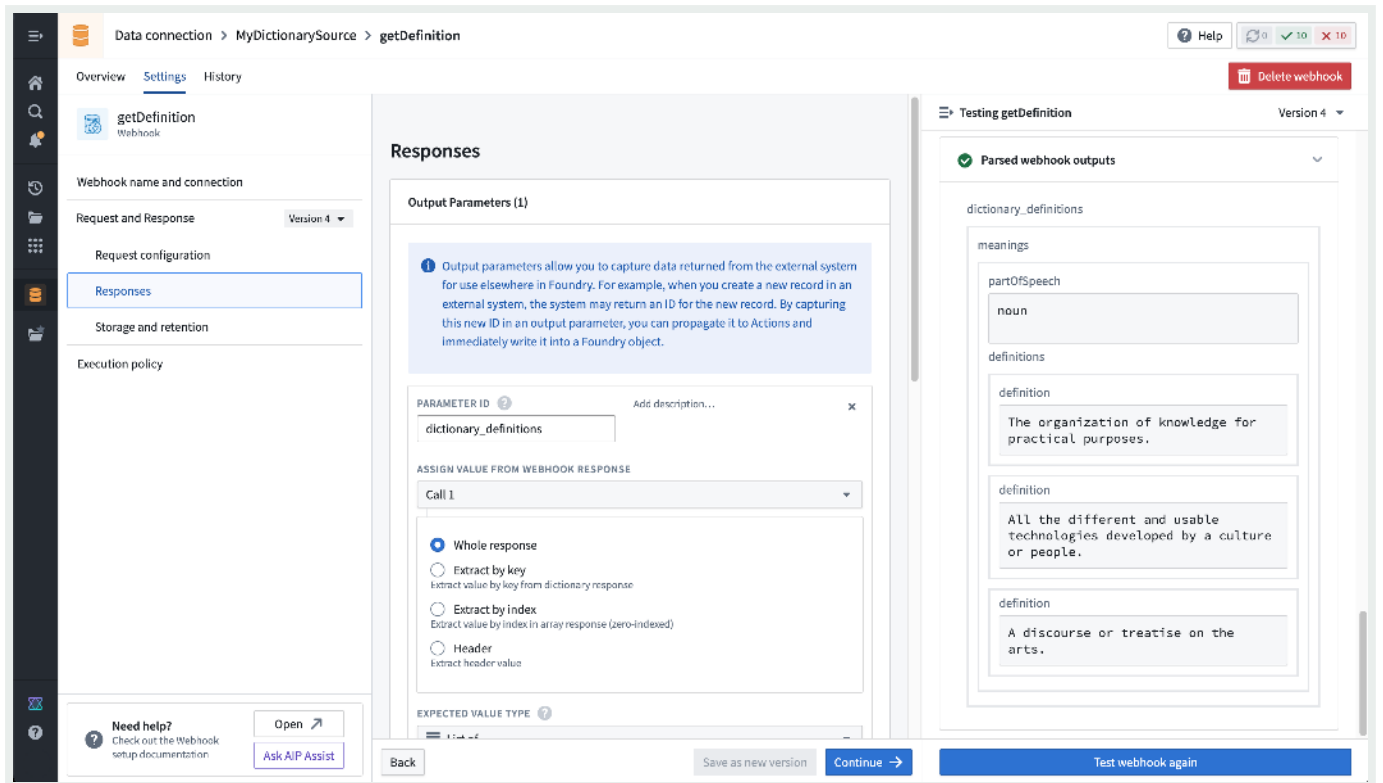


In our example, the API is a GET request that does not modify any data. Therefore, we will leave the default setting of `Read API`, allowing the webhook to be used in both types of Functions: `@Query()` and `@OntologyEditFunction()`. Webhooks marked with `Write API` may only be used in an `@OntologyEditFunction()`.

4. On the next page, you will see a panel that allows you to execute the webhook as currently configured. If you run the webhook, you will see an unparsed response:

[API Reference ↗](#)

[Send feedback](#)



5. Webhooks allow taking the response object returned from the external system and parsing fields according to a typed schema. For this example, we will extract a list of definitions for each returned part of speech. Run the webhook to ensure that some output is returned for a common word ("technology", for example).

Once the webhook is saved, it is usable across the platform.

[Learn more about webhook configuration options and usage.](#)

Next Steps

To begin using this source in a function, proceed to the guide on [external sources in functions](#).

← [PREVIOUS](#)
[External transforms](#)

[Sources in Python environments](#) → [NEXT](#)

[API Reference ↗](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Send feedback](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / External connections from code / Sources in Python environments

Sources in Python environments

Foundry provides the ability to connect to external systems in Python environments across the platform. These capabilities include [source-based external transforms](#), [external functions](#), and [compute modules](#) ↗. This page discusses common use cases and workflows for external systems in Python environments. For more information, visit Palantir's [external-systems](#) ↗ open source library.

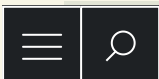
i Source initialization is not included in any of the examples below, as this will vary between environments. To learn how to obtain an initialized source object, refer to the environment's (for example, a transforms repository, functions repository, compute module, etc.) relevant documentation on usage of sources. You may additionally find snippets in the source information panel side bar with instructions for usage.

HTTPS client

For REST-based sources, Palantir provides a preconfigured HTTPS client built on top of the Python requests library.

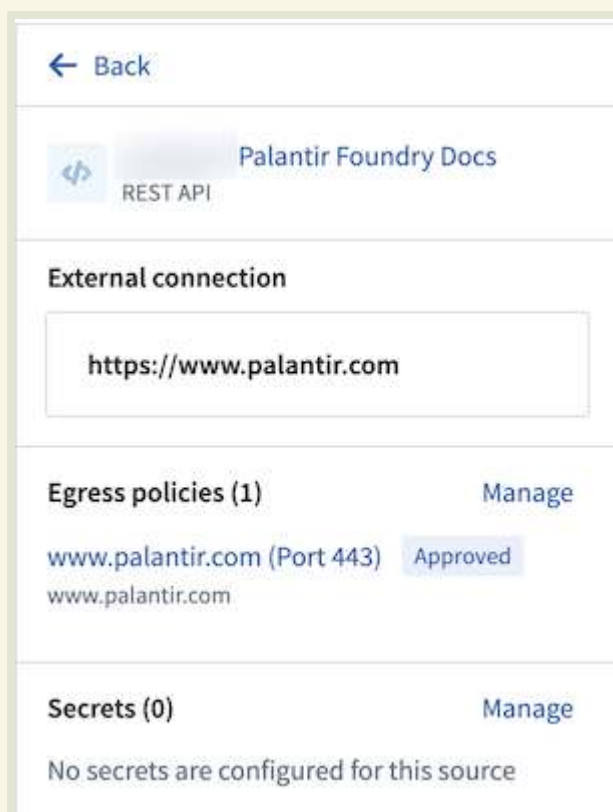
⚠ Note that sources must be initialized with only one HTTP connection; sources initialized with more or less than one HTTP connection are considered invalid and a preconfigured client will not be created. If you attempt to create a connection with multiple connection configuration, you will receive an error `Only single connection sources are supported.`

API Reference ↗



Send feedback

To find out how many connections your source has, refer to the source's sidebar panel in the **External connection** section of your given environment, as seen in the example below:



```
1 from external_systems.sources import Source, HttpsConnection
2 from requests import Session
3
4 my_source: Source = # Source is initialized differently based on the envi
5
6 https_connection: HttpsConnection = my_source.get_https_connection()
7
8 external_system_url: str = https_connection.url
9
10 http_client: Session = https_connection.get_client()
11
12 response = http_client.get(external_system_url + "/api/v1/example/", time
```

⚠ Changing the working directory (for example, using `os.chdir()`) before or during API Reference message may break references to environment variables necessary for establishing secure connections.

Send feedback

Secrets

Source secrets can be referenced using `get_secret("<secret_name>")` on the source.

```
1 from external_systems.sources import Source
2
3 my_source: Source = ...
4
5 my_secret: str = my_source.get_secret("SECRET_NAME")
```

Session credentials

A first-class method to retrieve and renew generated session credentials is available for some Foundry source types.

Supported source configurations

- **S3:** Cloud Identity, OIDC
- **BigQuery:** OIDC
- **Snowflake:** OIDC

Example: S3

```
1 import boto3
2
3 from external_systems.sources import AwsCredentials, Refreshable, Source,
4
5 S3_BUCKET_REGION = <aws_region>
6 S3_BUCKET_NAME = <bucket_name>
7
8 s3_source: Source = ...
9
10 refreshable_credentials: Refreshable[SourceCredentials] = s3_source.get_session_credentials()
11
12 session_credentials: SourceCredentials = refreshable_credentials.get_session_credentials()
13
14 if not isinstance(session_credentials, AwsCredentials):
```

API Reference ↗

Send feedback

```

15     raise ...
16
17 s3_client = boto3.client(
18     "s3",
19     region_name=S3_BUCKET_REGION,
20     aws_access_key_id=session_credentials.access_key_id,
21     aws_secret_access_key=session_credentials.secret_access_key,
22     aws_session_token=session_credentials.session_token,
23 )
24
25 s3_response = s3_client.list_objects_v2(Bucket=S3_BUCKET_NAME)

```

Example: OIDC credentials

For sources configured with [OIDC authentication](#), such as [Snowflake](#), session credentials are returned as `OAuthCredentials`. These contain a short-lived `access_token` and an `expiration` timestamp.

```

1 from external_systems.sources import OAuthCredentials, Refreshable, Source
2
3 my_source: Source = ...
4
5 refreshable_credentials: Refreshable[SourceCredentials] = my_source.get_session_credentials()
6
7 session_credentials: SourceCredentials = refreshable_credentials.get()
8
9 if not isinstance(session_credentials, OAuthCredentials):
10     raise ...
11
12 access_token: str = session_credentials.access_token

```

On-premises connectivity with agent-proxy egress policies

For [API Reference](#) [agent-proxy.policy](#) sources allow connections in code to be established to on-premise systems as if the connections were made over the open Internet. For more details on how this is configured, refer to the [agent-proxy doc](#)

Send feedback

Socket connections

For non-HTTPS connections to external systems that require connections through Foundry's agent proxy, a preconfigured socket is provided. Below is an example of using this socket with an on-premise SFTP server connection.

On-premise SFTP server example

This example uses the [Fabric 7](#) library.

```
1 import fabric
2
3 from external_systems.sources import Source
4 from socket import socket
5
6 SFTP_HOST = <sftp_host>
7 SFTP_PORT = <sftp_port>
8
9 on_prem_sftp_server_source: Source = ...
10
11 username: str = on_prem_sftp_server_source.get_secret("username")
12 password: str = on_prem_sftp_server_source.get_secret("password")
13
14 proxy_socket: socket = on_prem_sftp_server_source.create_socket(SFTP_HOST)
15
16 with fabric.Connection(
17     SFTP_HOST,
18     user=username,
19     port=SFTP_PORT,
20     connect_kwargs={
21         "password": password,
22         "sock": proxy_socket,
23     },
24 ) as sftp_conn:
25     sftp_client = sftp_conn.sftp()
26     file_list = sftp_client.listdir(".")
```

[API Reference 7](#) proxy URI

Send feedback

For more granular use cases, a pre-authenticated proxy URI is provided to allow connections to on-premises external systems.

Non-requests library example

In cases where the requests library client is not sufficient, you may need to use another HTTP client. This example uses the [HTTPX ↗](#) library.

```
1 import httpx
2
3 from external_systems.sources import Source
4 from typing import Optional
5
6 agent_proxy_source: Source = ...
7
8 authenticated_proxy_uri: Optional[str] = agent_proxy_source.get_https_proxy_uri()
9
10 source_url: str = agent_proxy_source.get_https_connection().url
11
12 with httpx.Client(proxy=authenticated_proxy_uri) as client:
13     response = client.get(source_url + "/api/v1/example/", timeout=10.0)
```

Source properties

For source types that are available via the [Foundry API](#), the configuration properties can also be directly accessed for use in code.

```
1 from external_systems.sources import Source
2
3 snowflake_source: Source = ...
4
5 account_id: str = snowflake_source.source_configuration.get("accountIdentifier")
```

API Reference ↗

PREVIOUS

← External functions

Legacy

Send feedback →

NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / External connections from code / Legacy external transforms

External transforms [Legacy]

⚠ Transforms with direct references to egress policies, credential values, and export control markings are in the [legacy](#) phase of development, and no additional development is expected. Full support remains available. We recommend using [source-based external transforms](#).

i [Source-based external transforms](#) is the supported way to connect to external systems with code.

Key advantages of using source-based external transforms over legacy external transforms include support for the following:

- Connecting to systems not accessible from the Internet.
- Rotating/updating credentials without requiring code changes.
- Sharing connection configuration across multiple repositories.
- Out-of-the-box Python clients for selected source types.
- Improved and simplified governance workflows for enabling and managing external transform repositories.
- The visualization of external transforms connected to external sources in [Data Lineage](#).
- Compatibility with [Virtual Tables](#).

You can use Python transforms in Code Repositories to connect to an external system

d API Reference ↗ could be useful if, for example, you require complex logic for interacting with an external system or you want to use a software development kit (SDK) to interact with the system. We recommend using external transforms to perform scheduled exports using REST APIs.

API Reference ↗



Send feedback

Before continuing, be sure to create a [Python transforms code repository](#) and configure it following the instructions below.

-  Transforms connecting to an external system can only be previewed using the Preview helper in Code Repositories if they do not use Foundry inputs.

Allow Code Repositories to set up external connections

This section covers the configurations required to use external transforms.

-  External transforms may only be used with repositories running in **SECURE** mode. Alternative security modes are only available by speaking with Palantir Support.

Enable external system interaction

Within the code repository you just created, a Foundry user with the **Information Security Officer** role should navigate to the [repository Settings](#) tab, then navigate to **Repository > External systems** to toggle on the option to **Allow access to external systems from this repository**.

The **Information Security Officer** is a default role in Foundry; users can be granted the **Information Security Officer** role in [Control Panel](#) under **Enrollment permissions**.

API Reference ↗

External systems

Allow this repository to interact with external systems and enable configuration of network egress and credentials.

Step 1: Allow access to external systems from this repository
☒ Allow access to external systems from this repository

Step 2: Install the library to use external transforms
The `transforms-external-systems` library is **installed** in this repository.
Manage configuration in the  **Egress and credentials** sidebar tab of the [Code Editor](#).

Step 3 (optional): Configure use of Foundry inputs with external systems
☐ Allow use of Foundry inputs with external systems in this repository

Send feedback

Enable dataset inputs

A Foundry user with the **Information Security Officer** role may optionally allow the use of inputs in external transforms.

When a dataset is used as an input to a transform that can communicate with an external system, any data in that dataset can potentially leave Foundry. A Foundry user with the **Information Security Officer** role should select the set of security markings and Organization markings for any data that should be allowed as an input to external transforms in this repository.

As an example, you may have a dataset with a **Sensitive** marking in the **Palantir** Organization. To use this dataset in an external transform, the Information Security Officer must add both the **Sensitive** marking and the **Palantir** Organization under step three, **Configure use of Foundry inputs with external systems**, in the code repository settings.

External systems

Allow this repository to interact with external systems and enable configuration of network egress and credentials.

Step 1: Allow access to external systems from this repository

☒ Allow access to external systems from this repository

Step 2: Install the library to use external transforms

The `transforms-external-systems` library is **installed** in this repository.

Manage configuration in the  **Egress and credentials** sidebar tab of the [Code Editor](#).

Step 3 (optional): Configure use of Foundry inputs with external systems

☒ Allow use of Foundry inputs with external systems in this repository

Select the markings and organizations for data that can be exported to external systems from this repository. Transforms within this repository can export data with these markings to external destinations with unknown security controls.

Markings

 Test Markings: Sensitive

Organizations

 Palantir

Edit

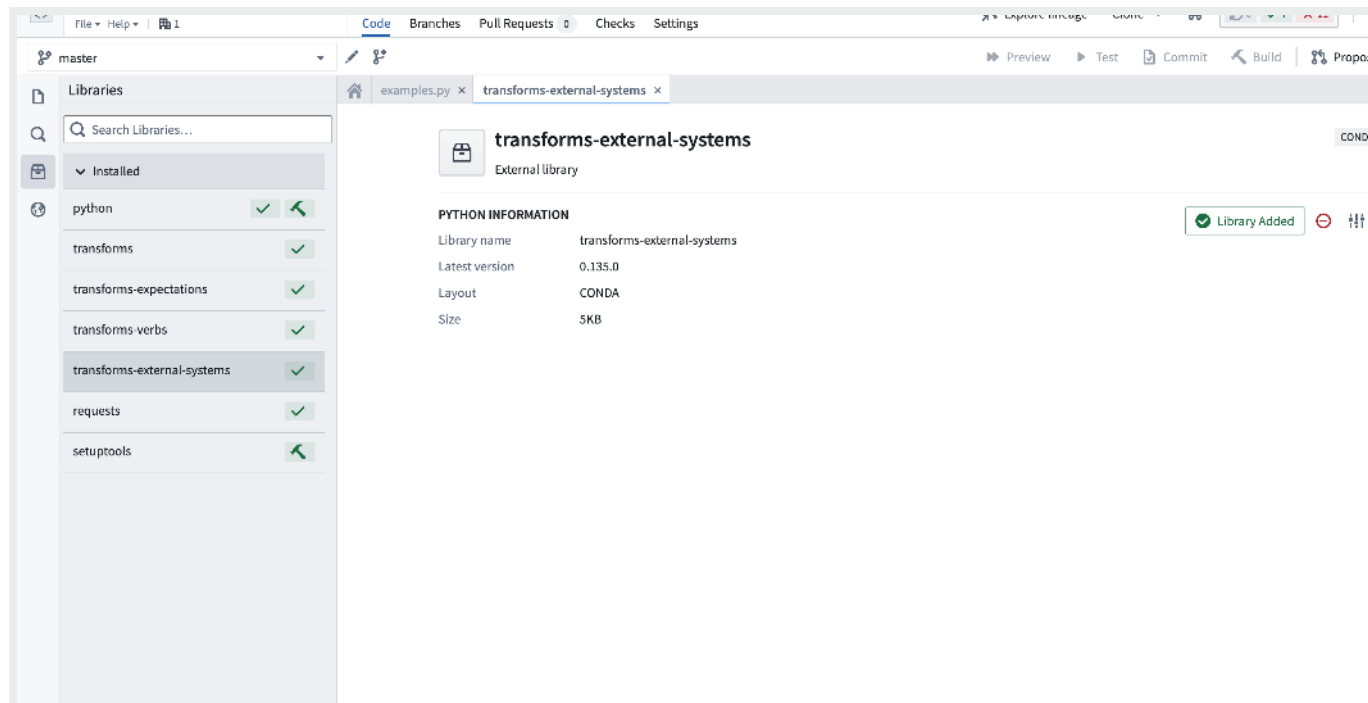
Add the `transforms-external-systems` library

API Reference ↗

Once external system interactions are enabled for the code repository, you must add the `transforms-external-systems` library from the **Libraries** sidebar tab interface.

Send feedback

i If you are working in a Python transforms repository in a [VS Code workspace](#), navigate to the **Settings** tab in the top navigation bar and select **External systems** to access the **Egress** and **Credentials** settings.



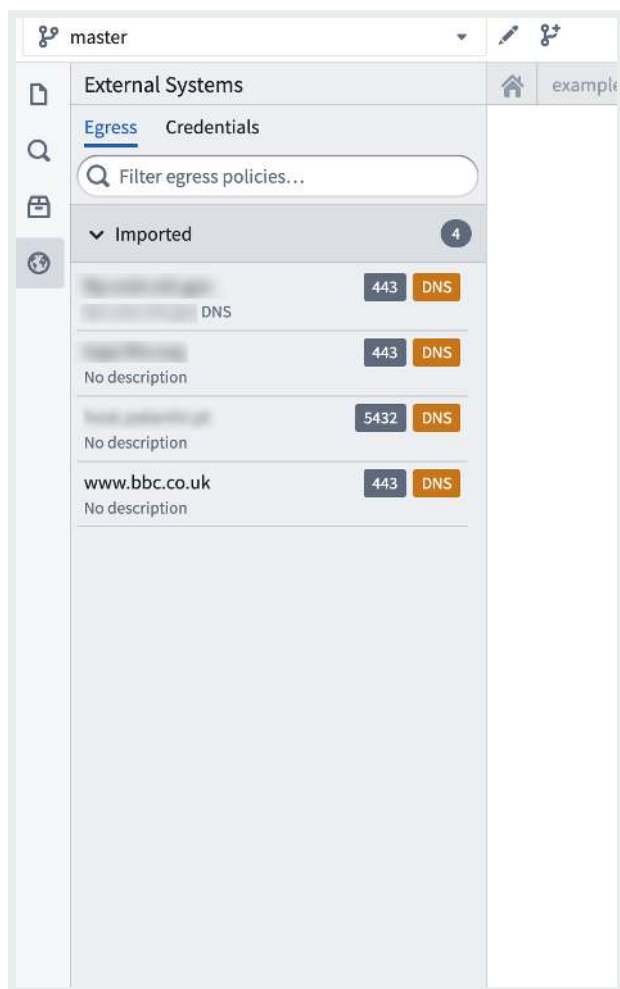
Configure egress and credentials

After adding the library, you can access a new **Egress and Credentials** tab to the left of the code interface.

In this tab, you can set up the network egress policies required for any endpoints called from your code, and add credentials (if necessary) for the repository to use. Note that not all endpoints require credentials.

[API Reference ↗](#)

[Send feedback](#)



Use network egress policies and credentials

Add egress policies

Foundry requires all connections initiated from user-authored code to be bound to a network egress policy when reaching out to external destinations. A network egress policy allows permitted connections that are initiated from user-authored code to reach destinations outside Foundry.

If self-service egress is enabled for your enrollment, you can [create and administer egress policies in Control Panel](#). If your connection requires you to add a new network route, contact your Connection Security Officer or Palantir Support with the details of your egress policy.

API Reference ↗

Send feedback

 If your Foundry instance does not have self-service egress enabled, contact Palantir Support to internally enable the endpoint and configure the policy.

Before using egress policies for external transforms (whether global or non-global), you must import them from the **Egress** tab in the code repository, found in the **Egress and Credentials** panel. The **Egress** tab exposes all policies to which you have permissions and classifies them into three categories:

- **Imported:** The network egress policies ready for use from your transform in the current code repository.
- **Importable:** Requires adding a Project reference to the network egress policy.
- **Access request required:** You have `Viewer` but not `Importer` permissions on the policy.

Within this tab, search for the desired network egress policy. If the policy is marked as `Importable`, select it and choose **Import egress policy to project**.

If the desired network egress policy is marked as `Access request required`, a Foundry user with the `Information Security Officer` role should grant you the `Importer` role in the **Network egress** page in Control Panel. To grant the `Importer` role, the Information Security Officer should first select the desired network policy, then select **Actions > Manage Sharing** to add you as an `Importer` of the policy.

Add credentials

If your external system requires authorized credentials (username and password) to gain access, you can add a set of credentials using the **Credentials** tab in the **Egress and Credentials** side panel and use them in your code.

 You must allowlist Foundry IP addresses if the source endpoint has an allowlisting process in place. Contact Palantir Support for the required IP addresses.

[API Reference ↗](#)

Pass certificates

[Send feedback](#)

If your external system requires an SSL/TLS certificate to be passed, you can store the certificate in a credential, then use the credential in the external transforms logic.

Write external transforms logic

You are now ready to write a Python data transform to an external system. We recommend reviewing the basic instructions on how to [write a Python data transformation](#) before writing an external transform.

From the `transforms.external.systems` package, import the required `use_external_systems` decorator along with the `EgressPolicy` and `Credential` (if needed) inputs. From the `transforms-api`, import the transform decorator and `Output`. If your external source does not require credentials, you do not need to add `Credential` inputs to your logic.

 You must use the `use_external_systems` decorator to successfully write external transforms logic. A transform cannot contain both the `use_external_systems` decorator and the `external_systems` decorator for source-based external transforms.

The following example shows how to use the `use_external_systems` decorator:

```
1 from transforms.api import transform, Output
2 from transforms.external.systems import use_external_systems, EgressPolicy
3
4 @use_external_systems(
5     egress=EgressPolicy('<policy RID>'),
6     creds=Credential('<credential RID>')
7 )
8 @transform(
9     output=Output('/path/to/output/dataset')
10 )
11 def compute(egress, creds, ...):
```

[API Reference ↗](#)

[Send feedback](#)

You can then set up a simple transform to reach out to an API:

```

1 from transforms.api import transform, Output
2 from transforms.external.systems import EgressPolicy, use_external_system
3 import requests
4
5
6 @use_external_systems(
7     egress=EgressPolicy('<policy RID>'),
8     creds=Credential('<credential RID>')
9 )
10 @transform(
11     output=Output('/path/to/output/dataset')
12 )
13 def compute(egress, output, creds):
14     username = creds.get('username')
15     password = creds.get('password')
16     response = requests.get('https://<API URL>', auth=(username, password))
17     with output.filesystem().open('response.json', 'w') as f:
18         f.write(response)

```

You can also specify multiple egress policies or credentials in a single transform, as shown in the example below.

```

1 from transforms.api import transform, Output
2 from transforms.external.systems import use_external_systems, EgressPolicy
3
4 @use_external_systems(
5     egress1=EgressPolicy('<first policy RID>'),
6     egress2=EgressPolicy('<second policy RID>'),
7     creds1=Credential('<first credential RID>'),
8     creds2=Credential('<second credential RID>')
9 )
10 @transform(
11     output=Output('/path/to/output/dataset')
12 )
13 def compute(egress1, egress2, creds1, creds2, ...):
14     # ...

```

[API Reference ↗](#)

Add secondary dataset inputs

[Send feedback](#)

In some cases, it may be useful to write external transforms that process Foundry input data. For example, you might want to query an API to gather additional metadata for each row in a tabular dataset. Alternatively, you might have a workflow where you need to mirror Foundry data into an external software system.

Such cases are considered *export-controlled workflows*, as they open the possibility of exporting secure Foundry data into another system with unknown security guarantees and severed data provenance. It is the transform developer's responsibility to assess the security of an export workflow and ensure that the data leaving Foundry is correctly secured in external systems. Foundry provides governance controls to ensure developers can clearly encode security intent, and Information Security Officers can audit the scope and intent of workflows interacting with external systems.

To opt into export controls, apply an `ExportControl` to your external transform. The `ExportControl` accepts a list of security Markings and Organization IDs that are intended for export. The transform job will then be guaranteed to fail if upstream data is marked with additional security Markings. To expand the set of exportable markings supported in a code repository, an Information Security Officer must adjust the repository settings. Navigate to the **Settings > Repository** tab. Then, in the **External Systems** section, choose to **Allow use of Foundry inputs with external systems in this repository** under Step 3.

 Security markings and Organizations are both implemented as markings and should be listed together in the export control configuration. An error message about missing markings may be referring to security markings or Organizations.

As an example, you could use an export external transform to download an image URL as specified in a Foundry dataset:

```
1 from transforms.api import transform, Input, Output
2 from transforms.external.systems import use_external_systems, ExportControl
3 from pyspark.sql.functions import udf
4 import shutil
5
6 @use_external_systems(
7     control=ExportControl(markings=['<marking ID>']),
8     egress=EgressPolicy(<policy RID>),
9 )
10 @transform(
```

[API Reference ↗](#)

[Send feedback](#)


```

11     images_output=Output('/path/to/output/dataset'),
12     image_urls=Input('/path/to/input/dataset'),
13 )
14 def compute(export_control, egress, images_output, image_urls):
15
16     @udf
17     def download(name, url):
18         response = requests.get(url, stream=True)
19         with images_output.filesystem().open(f'{name}.jpg', mode='wb') as out_file:
20             shutil.copyfileobj(response.raw, out_file)
21
22         return True
23
24     image_urls.dataframe().withColumn('downloaded', download(col('Name'),

```

Add Foundry media set outputs

When using external transforms to ingest media files like images, audio, or PDFs, you may want to write the files out to a [media set](#). To write a media item to an existing media set, programmatically put the item in the specified media set output:

```

1  from transforms.api import transform
2  from transforms.mediasets import MediaSetOutput
3  from transforms.external.systems import use_external_systems
4  import requests
5  import tempfile
6
7
8  @use_external_systems()
9  @transform(
10     media_set_output=MediaSetOutput('/path/to/output/media/set')
11 )
12 def compute(media_set_output):
13     response = requests.get('https://<API URL>')
14     fname = 'my_image.png'
15
16     with tempfile.NamedTemporaryFile() as tmp:
17         write(response.content)
18         flush()

```

API Reference ↗

Send feedback

```
20         with open(tmp.name, 'rb') as tmp_read:
21             media_set_output.put_media_item(tmp_read, path=fname)
```

More ways to connect to external systems

Interact with API services

You are able to programmatically access and use API services if they exist within the external system. As an example, you can use the [Mapbox Static Images API ↗](#) from an external transform to output a map image to a Foundry dataset:

```
1  from transforms.external.systems import EgressPolicy, use_external_system
2  from transforms.api import transform, Output
3  from mapbox import Static
4
5
6  @use_external_systems(
7      egress=EgressPolicy('<policy RID>'),
8      mapbox_creds=Credential('<credential RID>')
9  )
10 @transform(
11     output=Output('/Users/username/datasets/example_mapbox'),
12 )
13 def compute(output, egress, mapbox_creds):
14     mapbox_access_token = mapbox_creds.get('mapbox-token')
15     client = Static(access_token=mapbox_access_token)
16     london = client.image('mapbox.satellite', lat=51.5072, lon=0.0, z=12)
17     with output.filesystem().open('london.log', 'w') as f:
18         f.write(f'response code: {london.status_code}')
19     with open('london.png', 'wb') as f:
20         f.flush()
21     with output.filesystem().open('london.png', 'wb') as f:
22         f.write(london.content)
```

[API Reference ↗](#)

dataset files

[Send feedback](#)

You are able to programmatically open, stream, and interact with files as you would with a regular transform. Learn more about [reading and writing unstructured files](#).

Incremental processing

You are able to run external transforms incrementally as you would with a regular transform.

Maintaining state between execution is often needed in external incremental transforms. With regular incremental transforms, the input files are tracked and compared to the latest transaction processed. For external incremental transforms, the incremental state must be stored in an alternative way explicitly. One option is to save a state file which will hold the latest state of the incremental transform, as shown in the example below:

```
1 from transforms.api import transform, Input, Output, configure, incremental
2 from pyspark.sql import Row
3 from pyspark.sql import functions as F
4 from pyspark.sql import types as T
5 import logging
6 import time
7 import json
8 log = logging.getLogger(__name__)
9
10 @incremental()
11 @configure(profile=["KUBERNETES_NO_EXECUTORS"])
12 @use_external_systems(
13     egress=EgressPolicy('<policy RID>'),
14     creds=Credential('<credential RID>')
15 )
16 @transform(
17     out=Output("<output_dataset>"),
18 )
19 def compute(ctx, out, egress, creds):
20     # Get the filesystem of the output dataset to read and write files on
21     out_fs = out.filesystem()
22     state_filename = "_state.json" # Prepend with "_" to consider it a hidden file
23     state = {"last_seen" : 0} # Some arbitrary starting state
24
25     # Fetch the state from the output dataset:
26     with out_fs.open(state_filename, mode='r') as state_file:
27         data = json.load(state_file)
28         # Validate the fetched state:
```

API Reference ↗

Send feedback

```

30         state = data
31         logging.info(f"state file found, continuing from : {data}")
32
33     except Exception as e:
34         logging.warn("state file not found, starting over from default st
35
36
37     # Here, write logic to make API Calls, create custom processing, gene
38     # For example purposes, the following generates a dataframe and saves
39     out.write_dataframe(get_dataframe(ctx))
40     # Update the state for next iteration:
41     state["last_seen"] = 1 + state["last_seen"]
42
43     # Save the new state on the output dataset:
44     with out_fs.open(state_filename, "w") as state_file:
45         json.dump(state, state_file)
46
47
48 # This function generates a dataframe for example purposes. This step wou
49 def get_dataframe(ctx):
50     # Define the schema of the dataframe:
51     schema = T.StructType([
52         T.StructField("name", T.StringType(), True),
53         T.StructField("age", T.IntegerType(), True),
54         T.StructField("city", T.StringType(), True)
55     ])
56
57     # Create a list of rows:
58     data = [("Alice", 25, "New York"),
59            ("Bob", 30, "San Francisco"),
60            ("Charlie", 35, "London")]
61
62     # Create a PySpark dataframe:
63     df = ctx.spark_session.createDataFrame(data, schema=schema)

```

PREVIOUS

← Sources in Python environments

NEXT

Push data into a stream →

API Reference ↗ Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

Send feedback

[Terms of Use ↗](#)

[— Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Push data into a stream

Most data ingestion in Foundry focuses on pulling data from a source system and syncing it into a dataset or stream.

With streams, Foundry supports **push-based ingestion** to support event-based workflows.

Push-based record ingestion in Foundry follows the same principles as typical REST services. Through a series of REST endpoints, we expose a push-based API that can consume records and write them into streams and datasets. The following high-level information is required to push into a stream:

- The dataset resource identifier of the stream.
- The name of the branch.
- A token to authenticate the request.

i If you already have a Foundry source configured for your data, you may want to connect with that source instead. Learn how to [set up a streaming sync](#) with an existing source, or learn how to [set up a Kafka source](#).

If you need to receive inbound webhooks from systems that cannot properly authenticate with Foundry or conform to standard stream schemas, consider using [listeners](#).

Below, we will discuss the steps required to push data into a stream:

1. API Reference

2. [Records](#) into the stream.
3. [Listeners](#) stream.

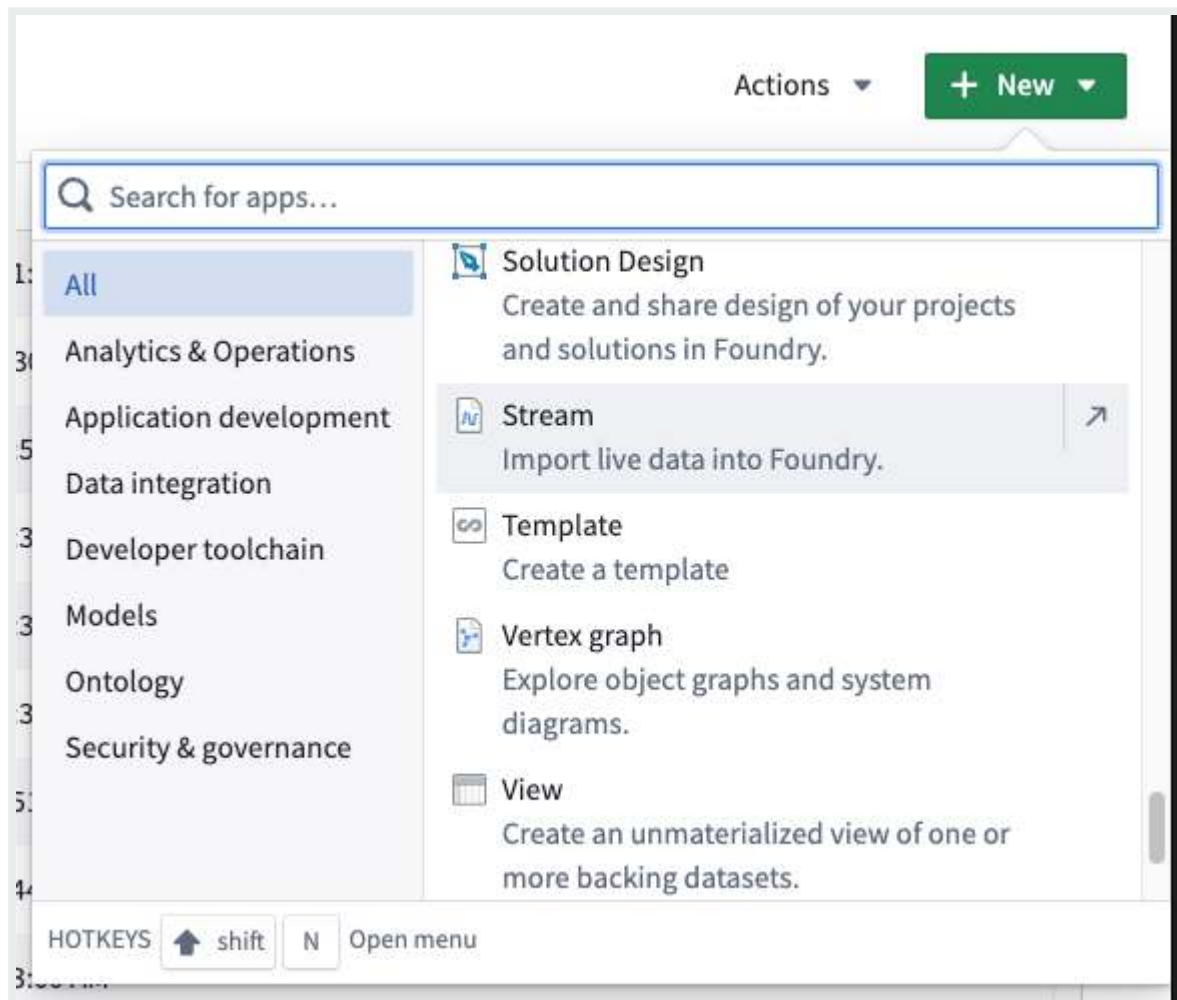
Send feedback

4. Test the stream.

Part 1. Initial setup

First, begin the stream creation workflow.

1. Log into Foundry.
2. Navigate to a Project.
3. Select **+ New** in the top right corner.
4. Scroll down and select **Stream**.



Next, click on the **API Reference** icon to view the schema, throughput, and keys for the stream.

API Reference ↗

- **Schema** defines the structure and types of your stream data.

Send feedback

- **Throughput** represents the data processing rate and will impact the number of partitions used in your stream. Learn more in our [throughput](#) and [partitions](#) documentation.
- **Keys** are used to guarantee ordering for unique IDs when using multiple partitions. Learn more in our [streaming keys](#) documentation.

For this tutorial, we will create a simple single partition stream.

- Set the schema to `sensor_id: String`, `temperature: Double`, and `created_at: Timestamp`.
- Set the throughput to `Normal`.

Then, configure your stream.

- Automatically generate schema from an existing JSON blob by selecting the `Generate from JSON sample...` button and pasting in your existing JSON blob.
- Set up [change data capture](#) if you are streaming real-time updates from a relational database.
- Consider the parallelism requirements of your stream to determine if you need higher throughput settings. Review our [partitions](#) documentation for more information.
- Consider setting a key for ordering guarantees when using multiple partitions.

API Reference ↗

Send feedback

Setup Stream Guide
/Polant/r/

Define Connect Push

Schema Generate from JSON sample...

Name	Type	Nullable
sensor_id	String	☐
temperature	Double	☐
created_at	Timestamp	☐
Add a column...	String	☐

Schema JSON

```
{
  "type": "STRING",
  "nullable": false,
  "name": "sensor_id",
  "customMetadata": {}
},
{
  "type": "DOUBLE",
  "nullable": false,
  "name": "temperature",
  "customMetadata": {}
}
```

Throughput Partitions: 3

Normal
Suitable for up to 5MB/s of throughput.

- Row order guaranteed on arrival
- Lowest compute cost
- Suitable for up to 5MB/s of throughput

Recommended

Very high
Best for data over 5MB/s of throughput.

- Order not guaranteed
- Higher compute cost
- Best for data over 5MB/s of throughput

Configuration

Keys
To guarantee ordering between unique IDs, select a set of key columns. Rows are not guaranteed to be stored in order unless they have equal key values or the stream is configured with a single partition.

Search...

Change data capture ☐

Change data capture enables you to stream updates from supported databases. Selection cannot be changed after stream creation.

Partitions: 3 3 columns 0 keys Destination is a project Create stream

i All validation errors must be addressed before selecting **Create stream**. Hover over the tooltips on the bottom of the page for more details about the error.

Select the **Create stream** button in the bottom right corner to navigate to the **Connect** page. Here, you can specify how to connect to the streaming data.

Part 2. Push records into the stream

We are now ready to connect our stream.

API Reference ↗

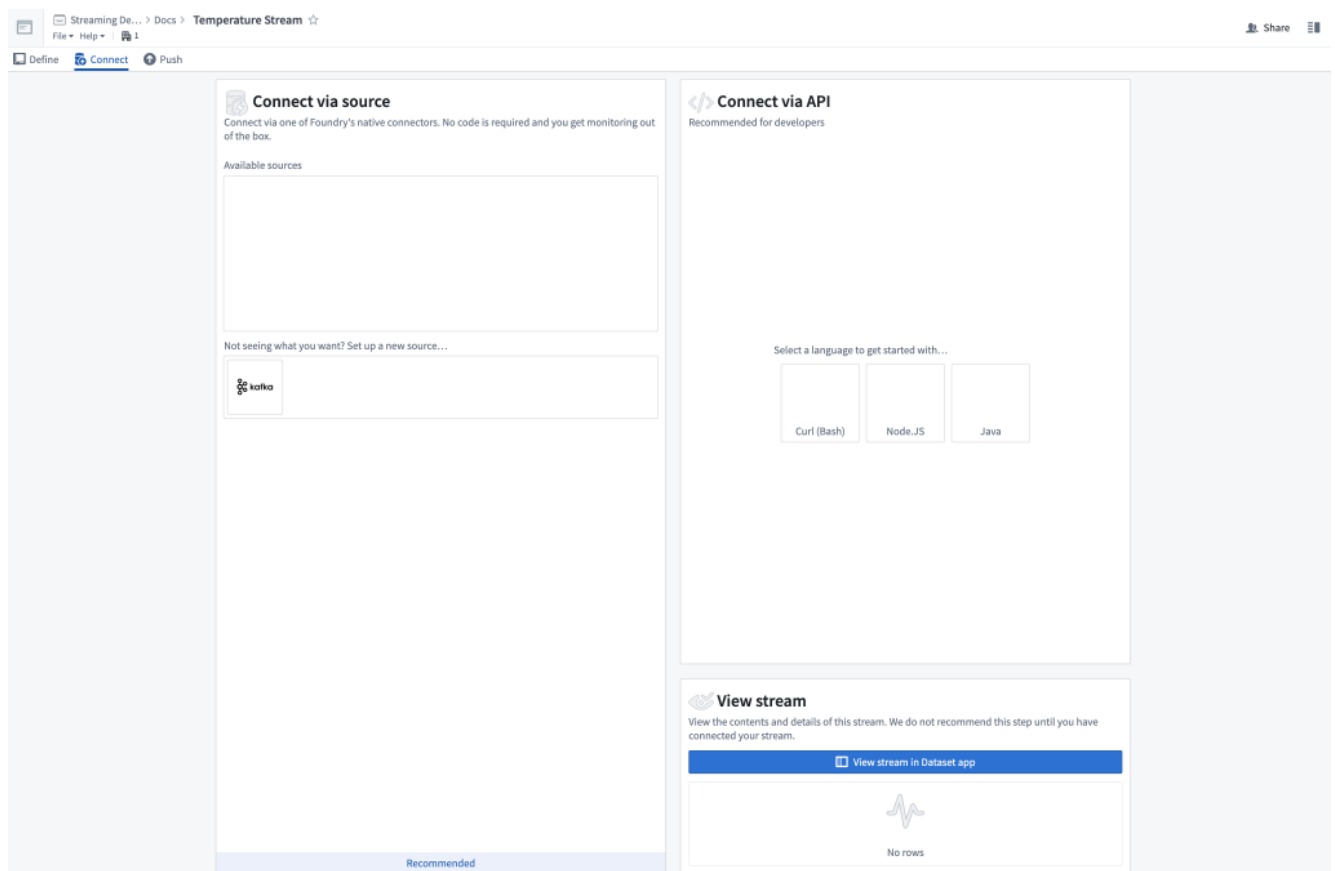
ingestion, select one of the options under the **Connect via API** section. Any programming technology that can make HTTP requests can be used to push records.

We provide examples for cURL, Python, JavaScript via Node, and

[Send feedback](#)

ingest data via a Foundry sync instead of pushing the data into Foundry, review how to [set up a streaming sync](#).

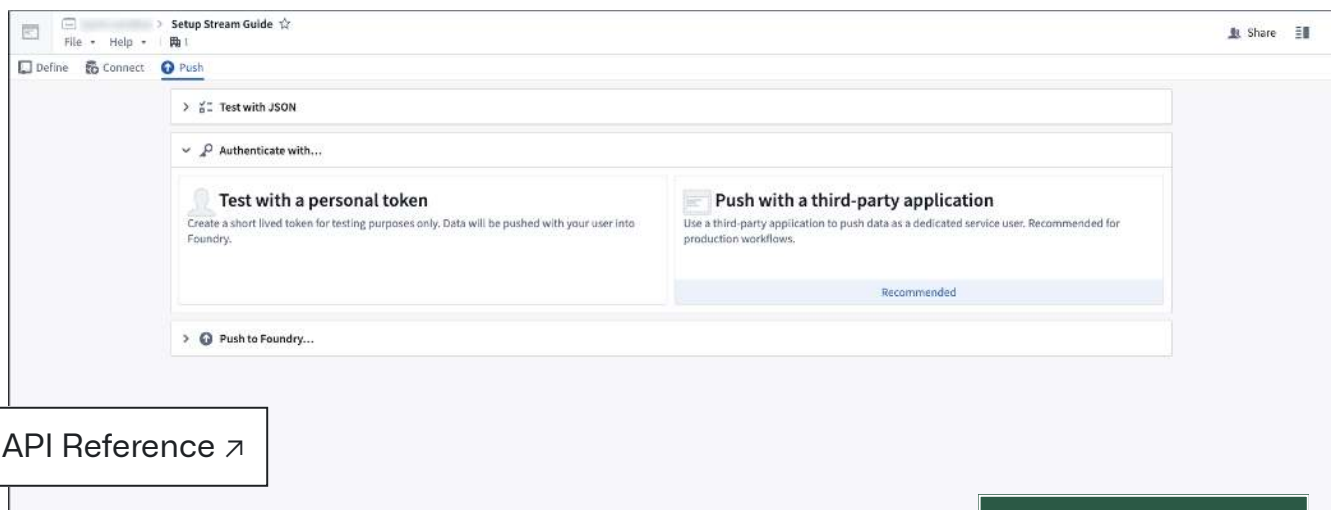
For this example, we will select **cURL**.



You will then land on the **Push** data workflow page.

2. Next, select an authentication mechanism. We support two ways to authenticate to the request:

- **Push with a third-party application (recommended):** This method uses an OAuth2 workflow to create a secure token that can be used to push records into your stream.
- **Push with a personal token:** This method uses a user-generated token for testing purposes only.



API Reference ↗

3. For this tutorial, choose the **Push with a third-party application**

Send feedback

steps on the screen to set up a third-party application and create your [client secret](#).

When configuring the application's operation restrictions, ensure the `api:use-streams-write` operation is included for the third-party application to push records into the stream.

Authenticate with... Third party application

**Test with a personal token**
Create a short lived token for testing purposes only. Data will be pushed with your user into Foundry.

**Push with a third party application**
Use a third party application to push data as a dedicated service user. Recommended for production workflows.
[Recommended](#)

CREATING A NEW APPLICATION TO AUTHENTICATE WITH

A third party application is a way of productionizing an API connection to Foundry. This example uses the OAuth2 Client Credentials grant flow to allow you to have a long lived application with the capability to refresh tokens when they expire. **If you have one created already, you can skip to the end of this step and paste in your client ID and client secret.**

- Jump to the Third-Party applications page in Foundry.
[Go to Third-party applications](#)
- In the top-right of the screen, click **Register new application...** and give the application a name.

Register new application

Application name

My Streaming Application

Client type

☒ **Server application** - Confidential client
Must use the client secret in token requests.

☐ **Native or single-page application** - Public client
Must use PKCE in token requests.

Cancel

Create

- When you have created your application, copy the client ID and client secret.

Application registered

Your application has been registered

In order to use this application, you need to enable it for your organization.

**My Streaming Application**
Managed by Palantir

Client ID

You will be able to view this ID under "Manage application" in the future.

Client secret

You will not be able to view this secret again.

Done

4. Now, select **Go to third-party applications** to open the third-party application management page in your Foundry platform settings.

5. Select **Register new application** at the top right of your screen.

USER SETTINGS

Profile

Account

Authorized applications

Third-party applications

Manage third party applications for Palantir.

Register new application...

Enabled applications

All applications

6. Choose a name, and set the client type to **Server application**.

API Reference ↗

Send feedback

Register new application



Application name

Test streaming app

Client type



Server application · Confidential client

Must use the client secret in token requests.



Native or single-page application · Public client

Must use PKCE in token requests.

Cancel

Create

7. Select **Create**, and you will be presented with your client ID and secret.



The client secret will not be accessible once you leave this page. Be sure to store it in a secure location.

8. Now, you can add the client ID and secret into the **Push** workflow page.

[API Reference ↗](#)

[Send feedback](#)

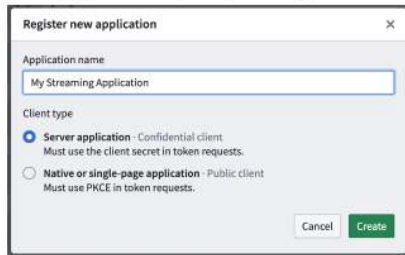
CREATING A NEW APPLICATION TO AUTHENTICATE WITH

A third party application is a way of productionizing an API connection to Foundry. This example uses the [OAuth2 Client Credentials grant flow](#) to allow you to have a long lived application with the capability to refresh tokens when they expire. **If you have one created already, you can skip to the end of this step and paste in your client ID and client secret.**

1. Jump to the Third-Party applications page in Foundry.

[Go to Third-party applications](#)

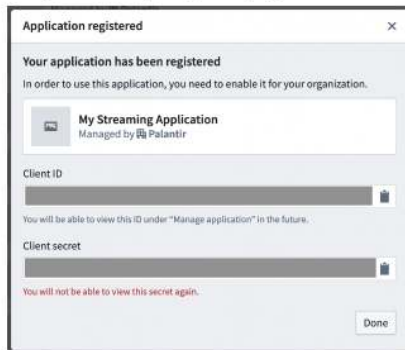
2. In the top-right of the screen, click **Register new application...** and give the application a name.



The dialog box titled "Register new application" contains the following fields and options:

- Application name:** A text input field with the value "My Streaming Application".
- Client type:** Two radio button options:
 - ☒ **Server application** - Confidential client
Must use the client secret in token requests.
 - ☐ **Native or single-page application** - Public client
Must use PKCE in token requests.
- Buttons:** "Cancel" and "Create".

3. When you have created your application, copy the client ID and client secret.



The dialog box titled "Application registered" contains the following information:

- Status:** "Your application has been registered. In order to use this application, you need to enable it for your organization."
- Application Card:** Shows "My Streaming Application" managed by Palantir.
- Client ID:** A text input field with a copy icon.
- Client secret:** A text input field with a copy icon and a warning message: "You will not be able to view this secret again."
- Button:** "Done".

4. Paste your client ID and client secret below



Two text input fields for credentials:

- Client ID:** 41c5cb213fa3db02a5e30337ef71e323
- Client secret:** dummy_secret

9. Scroll down to the **Configuring the Application** section of the workflow, then select **Manage application** to open the **Third-party applications** management page in your Foundry platform settings.

CONFIGURING THE APPLICATION

In order to use our application, we need to enable the client credentials flow. We can do this from the application management page.

1. Go to the "Third-party applications - Manage application" page for your application

[Manage application...](#)

2. Scroll to the **Authorization grant types** section and enable the **Client credentials grant**



A toggle switch for "Client credentials grant" with the description "Use this to support machine-to-machine authorization." and an "Enable" button.

3. Click **Save** before proceeding to the next step.

10. Next, enable the **Client credentials grant** setting.

[API Reference](#)

[Send feedback](#)

Third-party applications

Manage third party applications for

← Manage application

Cancel

Save

Application details

Details will be shown wherever the application appears, such as authorization screens and application tables.

Application name

Test Streaming app

Description (optional)

Add description...

Logo (optional)

Drag and drop an image or [choose from your files...](#)

Client details

Define the client type and provide redirect URLs for the application.

Client ID

Client type

☒ **Server application** · Confidential client
Must use the client secret in token requests.

☐ **Native or single-page application** · Public client
Must use PKCE in token requests.

Authorization grant types

Configure the authorization grant types that can be used when enabling this application.

Authorization code grant

Use this to support web applications, native applications, or single-page applications.

Enable

Client credentials grant

Use this to support machine-to-machine authorization.

Enable

11. Finally, click **Save** in the upper right corner.

Part 3: Share the stream

Now, we need to share the stream with the application we created.

1. First, return to the **Push** workflow page. Under the **Using Your New Application** section in the workflow, find the client ID that you generated.

USING YOUR NEW APPLICATION

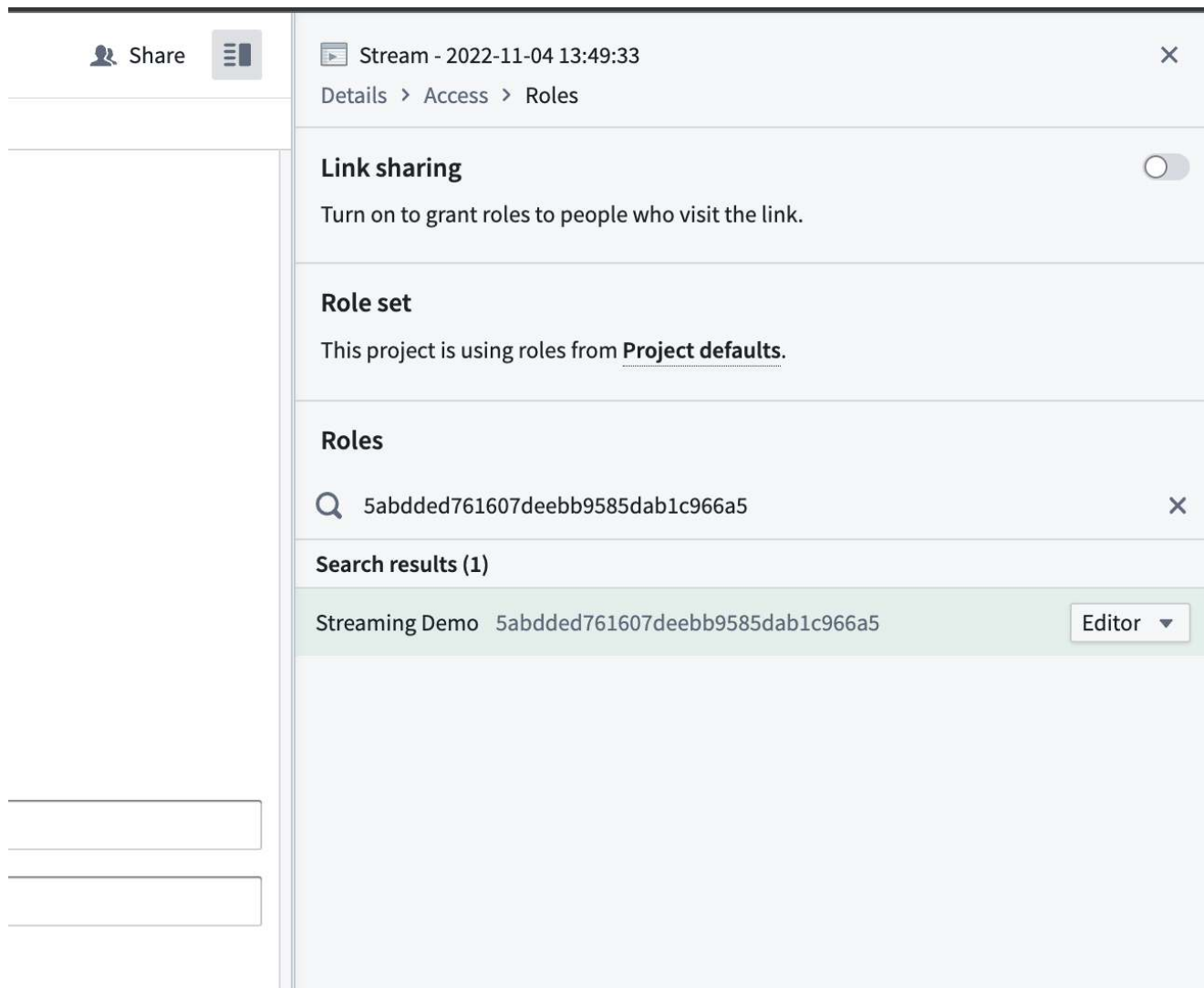
In order to use your third party application we will need to paste it into this dataset

application by clicking the **Share** button in the top right. Use the search in the **Roles** section and type in `41c5cb213fa3db02a5e30337ef71e323` to find your permissions. Afterwards, click the Save button.

API Reference ↗

Send feedback

2. Next, select **Share** on the top right to open the **Roles** tab of the stream **Details** sidebar to the right side of your screen.



3. Copy and paste the client ID into the **Roles** search field to find the application you created. Select the **+** to search for and choose the **Editor** role.
4. Choose **Save** at the bottom of the side panel to share the stream.
5. Return to the push workflow and select **Next step**.

Part 4: Test the stream

You will now be presented with code examples that can be used to push test records into the stream. The code examples differ depending on what language you selected in the previous steps. For this example, we are using cURL.

API Reference ↗

i Before running cURL commands, be sure to install [jq](#) for command-line JSON parsing.

Send feedback

1. First, copy the command from the first box. This will hit Foundry's OAuth2 endpoint, providing you with an access token you can use to push records.

Push to Foundry...

Curl (Bash)

Select a language to get started with...

Curl (Bash)

Node.JS

Java

1. In this sample, we will be using `jq`. Type in `jq --version` to check it is installed on your system. `jq` allows us to pull the access token from the JSON response to Foundry's OAuth endpoint.

2. We initially hit Foundry's OAuth endpoint to get an **access token**, access tokens can be reused for a limited amount of time specified in the response as `expiry_time`.

```
ACCESS_TOKEN=$(curl \
-H "Content-Type: application/x-www-form-urlencoded" \
-X POST "https://  
/multipass/api/oauth2/token?grant_ty
```

3. Now that we've stored the access token as a variable, we can use it in our curl command.

```
curl \
-H "Authorization: Bearer $ACCESS_TOKEN" \
-H "Content-Type: application/json" \
-X POST https://  
/stream-proxy/api/streams/r1.foundry.  
-d "[{ \"value\": { \"sensor_id\": \"value\", \"temperature\": 1.234 } }]"
```

4. If the request has been successful, your new rows will appear below.

No rows

2. Execute the command in a bash terminal on your Mac, Windows, or Linux machine. Once the command is executed, you will have an available variable called `$ACCESS_TOKEN`. You will use this variable in the next command to push records. The second command will use cURL to hit a Foundry endpoint with a post request that contains the records we want to insert into the stream. The command is prepopulated with a dummy record to push into the stream, but you could provide any data in the HTTP request that adheres to the schema of the stream.
3. Copy and paste the second command into your terminal and execute it. If the command is successful, you will see records appear in the stream.

[API Reference ↗](#)

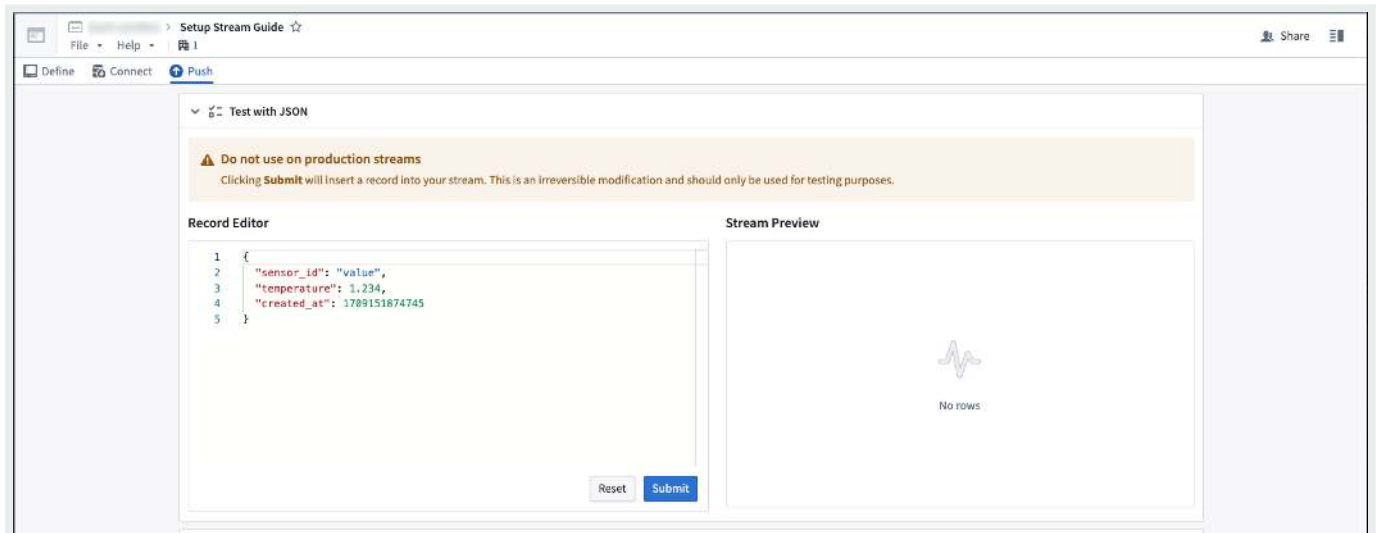
[Send feedback](#)

Start pipelining...		View this dataset...	
	sensor_id String	temperature Double	
1	sensor3	3.142	
2	sensor2	2.132	
3	sensor1	1.234	

To change the data pushed into the stream, modify the data parameter of the post request.

```
[{ \"value\": {\"sensor_id\": \"sensor4\", \"temperature\": 4.132} }]
```

You can also send test records into the stream from the user interface by expanding the **Test with JSON** card.



Next steps

[API Reference ↗](#)

Now that you have successfully pushed data into a stream, you are ready to start transforming your data. Select **Start pipelining** to navigate to the Pipelines page.

[Send feedback](#)

application where you will build your streaming pipeline. Learn how to [transform your data](#), and learn more about the [different transforms available in Pipeline Builder](#).

PREVIOUS

← [External connections from code / Legacy external transforms](#)

NEXT

[OpenID Connect \(OIDC\) authentication](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / OpenID Connect (OIDC) authentication

OpenID Connect (OIDC) authentication

[OpenID Connect \(OIDC\) ↗](#), also known as OIDC, is an open authentication protocol that allows you to authenticate to external source resources without the use of static credentials.

When using OIDC, you do not need to configure credentials for a source system in Foundry. Because of this, you can avoid duplicating source system credentials as long-lived secrets in Foundry. Instead, you will configure a trust relationship between Foundry and the source system. Foundry acts as the OIDC identity provider; every time a workflow in Foundry is required to authenticate with the source system (for example, a Data Connection sync), Foundry will issue an OIDC token with claims that identify the Data Connection source being used. The source system is able to validate those claims and provide a short-lived access token that can be used for subsequent interactions with the source system. The scope of that access token, such as the resources it is allowed to access, is managed entirely in the source system using the native authentication and authorization tools available. When configuring the trust relationship, you can add conditions to filter the incoming requests. Untrusted Foundry sources cannot request access tokens to resources in the source system they should not have access to.

Supported sources

The following sources support OIDC authentication. Refer to individual source documentation for more details on how to set up the trust relationship between OIDC and

API Reference ↗



and Azure Data Lake Storage Gen2 (Azure Blob Storage)

Send feedback

- [BigQuery](#)
- [Databricks](#)
- [Google Cloud Storage](#)
- [S3](#)
- [Snowflake](#)

OIDC identity token

The following is an example OIDC token generated by Foundry:

```
{
  "iss": "https://pltroidcpublicexample.blob.store.com/foundry",
  "sub": "ri.magritte..source.7f3b8e21-4d9a-6c2e-1b7d-8a5f3c9e0b4f",
  "aud": "your-source-system-audience",
  "iat": <issued-at>,
  "nbf": <not-before>,
  "exp": <expiry>,
  "jti": "<token-unique-identifier>",
  "scp": "<additional-scope>",
}
```

Claim	Claim type	Description
iss	issuer url	The URL that identifies Foundry as an OIDC identity provider.
sub	subject	The source RID of the Foundry source that is connecting to your source system.
aud	audience	The configured audience that identifies your source system.

The `source-rid` should be used to filter incoming requests so untrusted Foundry sources cannot access your resources.

Claims generated by Foundry expire after one hour.

[API Reference ↗](#)

[Send feedback](#)

[PREVIOUS](#)

[NEXT](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Add sync to a Marketplace product

Add sync or export to a Marketplace product

Use [Foundry DevOps](#) to include your Data Connection syncs and exports in [Marketplace products](#) for other users to install and reuse. [Learn how to create your first product.](#)

Supported features

Both standard [syncs](#) and [streaming syncs](#) can be included in Marketplace products. Additionally, Data Connection [exports](#) can be packaged alongside syncs in Marketplace products.

Adding Data Connection syncs and exports to products

To add a Data Connection sync or export to a product, first [create a product](#). Select **Add output**, then select a Data Connection source. The dialog will walk you through the process and allow you to include both syncs and exports in your Marketplace product.

[API Reference ↗](#)[Send feedback](#)

 Add data connection source

Select data connection source

 Outlook Sync

Change

☒ Syncs (1)

SYNC	DATASET	TYPE	TEMPLATED STATUS	TEMPLATED ACTIONS
☒ test_outlook	 test_outlook	Batch	Not templated	Create Delete

Cancel

Submit

Data connectivity & integration / Data Connection / Add virtual tables to a Marketplace product

Add virtual tables to a Marketplace product

Use [Foundry DevOps](#) to include your [virtual table](#) in a [Marketplace product](#) for other users to install and reuse. [Learn how to create your first product.](#)

Supported features

All virtual tables may be packaged and synced.

Currently, packaging an individual source is not supported, nor is packaging a source that has auto-registration of virtual tables enable.

Installers must ensure the destination source contains a table at the same location with the same schema as the original source to guarantee compatibility and functionality.

Adding virtual tables to products

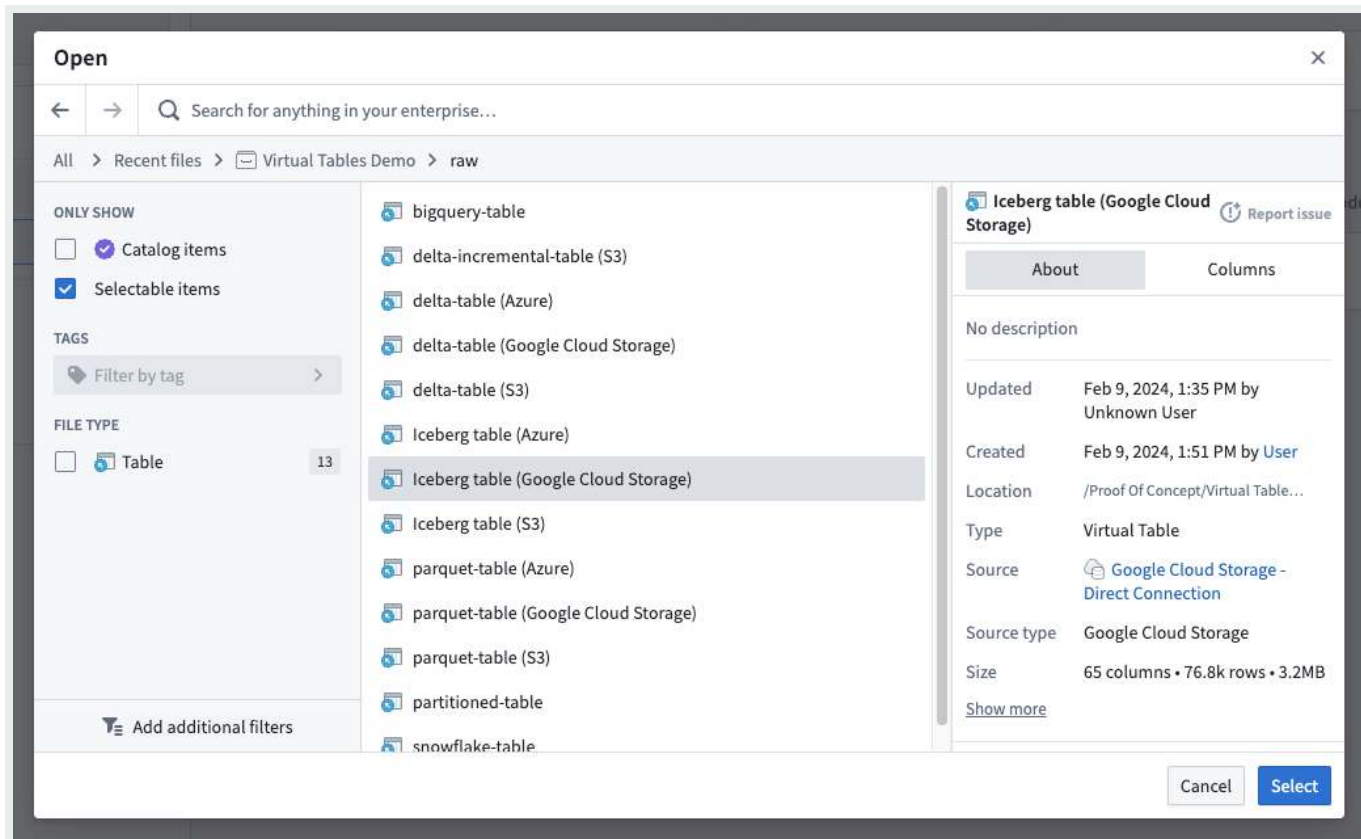
To add a virtual table to a product, first [create a product](#), then [add outputs](#). Choose the **Add files** option to navigate to the virtual table from within the [Compass](#) filesystem and add it to your product.

You can also select which virtual tables you would like to include in your product.

API Reference ↗



Send feedback



PREVIOUS
← [Add sync to a Marketplace product](#)

LEGACY DOCUMENTATION / SET UP A DIRECT CONNECTION
[Legacy documentation / Set up a direct connection](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Legacy documentation / Set up a direct connection

Set up a direct connection

⚠ This documentation is outdated and retained for historical reference only. For updated guidance on connection set up, refer to our documentation on [setting up a source](#).

⚠ Direct connections depend on Foundry's container infrastructure which is only available in Foundry's managed SaaS platform. As a result, cloud-based direct connections may not be available in your environment.

If you are trying to connect to a data source which is accessible over the Internet, such as a REST API, an SFTP server, or an Azure storage account, you can configure a direct connection to avoid needing to [set up an agent](#). Using a direct connection has a number of advantages:

- No need to provision, configure, and manage an agent and its host
- Avoids routing Internet-to-Foundry through your network
- Offers excellent uptime and performance as cloud-based Syncs do not depend on an agent software package or its host

If you are interested in configuring a cloud-based direct connection, follow these steps:

1. [Configure a network egress policy](#) for your enrollment.

2. [API Reference](#) [↗](#) [Tutorials](#) to connect to your data source.

3. [Create the Source in Data Connection](#).



Send feedback

Configure a network policy

⚠ You must have the *Information security officer* role on your [Enrollment](#) to configure network egress. If you do not have permissions to configure egress, contact your Palantir representative for help.

The *Information security officer* role can be found in the Enrollment permissions section of the [Control Panel](#). An administrator needs to have the *Enrollment administrator* role in order to see this section.

To configure a network policy, navigate to Control Panel using the **Other workspaces** link in the [Workspace sidebar](#). In Control Panel, select **Network egress** in the sidebar. If you can't see this option, contact your Palantir representative to go through the following steps.

Add network egress policy

1 [Configure egress](#)

2 [Configure permissions](#)

Policies must be applied to sources in Data Connection to take effect.

Description
Describe the type of traffic and/or destination that this policy will allow.

Export to cloud storage provider

Address
Specify the address that this policy will allow.

CIDR 192.168.0.0/16

Port number
Specify the port number or range that this policy will allow.

Single port 443

☐ Make this a global policy **Not recommended**

Global policies are applied to all user workflows, including code workbooks, code transforms, and modeling live deployments. This should only be enabled for highly trusted destinations.

[Next](#)

Add a network policy by selecting **Add network policy**. Add a description and connection details you provided when contacting Palantir:

[API Reference ↗](#)

- If you are connecting via HTTP(S), add the DNS hostname of your
- If you need to use a non-HTTP protocol, add a CIDR address and port

[Send feedback](#)

Keep the default **Optional** policy type selection, and select **Add network policy**.

Provision credentials

In the majority of cases, Foundry will require authorized credentials (such as a username and password) to access Sources. It is best practice to use a service account specifically for Foundry.

Provision a service account for the Source following any internal guidelines and processes that your organization has for establishing service accounts. Note the credentials before proceeding to the next step.

Create the Source in Data Connection

Once the above steps are done, you can proceed with creating the Source in Data Connection:

- After logging in, navigate to **Data Connection** using the [sidebar](#).
- Select the **Sources** tab.
- Select **New source** in the top-right.
- Select the [source type](#) corresponding to your data source.
- Select **Direct connection**, then select **Continue** in the bottom right.

Choose a way to connect to your data source

☒

Direct connection

Access your data source by connecting directly over the internet.

Method of connecting to sources networks.

☐

Via an agent

Access your data source via an Agent as an intermediary.

Can connect to a source that lives on a public network.

[API Reference ↗](#)

[Send feedback](#)

Save the Source in a Project

Next, name your Source and choose a [Project](#) to place it in. We generally recommend creating a new Project for each Source, as this provides the cleanest way to permission datasets derived from this Source. Consult the [Source permission best practices](#) for more information. Full guidance for how to structure data pipelines end-to-end in Foundry is available in the [recommended Project structure documentation](#).

Select **Create source and continue** in the bottom right.

Choose your network policy

On the next page, select the network policy you [configured earlier](#) by clicking **Use existing policy** and searching for the policy name.

API Reference ↗

Send feedback

Network Connectivity

↔ Switch to connect via Agent



Select a policy

A network egress policy is required to connect to sources outside Foundry's secure network. Select an existing policy if one already exists, otherwise create a new policy. [Learn more](#)

+ Use existing policy

 Create new policy

Add egress policies

Q mydomain

 mydomain.com (Port 443)

Select

 mydomain.com (Port 443)

Select

 mydomain.com (Port 44)

Select

 mydomain.com (Port 443)

No description

Select

Cancel

Select

API Reference ↗

Source and add drivers

Add details about how to connect to your source. These details will depend on the type you are using and typically consist of basic credentials such as connection URLs, cloud

Send feedback

provider regions, and so on.

[JDBC sources](#) may require adding and selecting **drivers** required to connect to your source. Although many drivers ship out-of-the-box with Foundry, you may need to upload and select a driver to proceed.

Add credentials

Add the credentials you [provisioned](#) previously to allow the direct connection to connect to your data.

Save and continue

Select **Save** in the bottom right to complete setting up your direct connection. Once your Source is fully set up, you can proceed to [set up a Sync](#) to bring data into Foundry.

PREVIOUS

← [Add virtual tables to a Marketplace product](#)

NEXT

[Agent worker runtime configuration reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Agents / Agent worker configuration reference

Agent worker configuration reference

⚠ Legacy

Agent worker is in the [legacy](#) phase of development, and no additional development is expected. Full support remains available. We recommend using [Foundry worker](#) connections when building a new source or updating existing workflows. See [Foundry worker vs. agent worker](#) for the operational reasons behind this recommendation.

This page describes the configuration options available when using an agent as a [worker](#) and assumes you are already familiar with [data connection agents](#) and their [generic configuration options](#).

To migrate an existing agent worker source to a Foundry worker, see [Switch source from agent worker to Foundry worker](#).

Set up an agent

You must [set up an agent](#) to use the agent worker. When configuring a source, you can select the **Agent worker** option and choose at least one available agent. Capabilities run over that ~~source connection~~ will be executed as jobs on the agent host.

API Reference ↗



 Capability execution

Send feedback

When using an agent worker, capabilities are executed by running Java code on the agent host directly. These Java processes may pull data from your systems and push data up to Foundry (as with [batch syncs](#)), or pull data from Foundry and push to your systems (as with [exports](#)), which comes with [known limitations](#).

Networking

All networking for agent worker sources is configured on the agent host. Foundry-side egress policies do not apply: the agent host must have firewalls, DNS, and proxy settings configured to reach the source systems directly. See [Configure agent network access](#) for the full reference.

Memory allocation and usage

Agent memory is one of the key factors determining the performance of capabilities executed using an agent worker.

The primary settings available for agent memory are:

- **JVM heap:** Configured on the agent settings page, and indicates how much memory the agent should allocate on startup. The allocated memory is reserved from the OS and must not exceed the available memory on the host machine. The default JVM heap value is 1 GB.
- **Host memory:** Based on the specification of the machine where the agent is installed. We recommend at least **16 GB** of memory.

Actual memory usage observed on the host will vary based on the workload currently being executed by the agent worker, including other processes running on the same host.

When observing and monitoring memory usage for an agent used as an agent worker, there are two primary metrics:

OS physical memory usage: The actual total memory usage on the agent host, including [API Reference ↗](#); but also any other processes running on the same host. This may go beyond the allocated JVM heap size for the agent process, and up to the total physical memory available.

Send feedback

- **Agent memory usage:** The memory usage by the agent, which will always be smaller than the configured JVM heap size.

For information on monitoring memory usage on agents, review [agent metrics and health monitoring](#).

Load balancing

When using an agent worker, multiple agents may be assigned to a single source connection. An agent is assigned jobs to execute specific [capabilities](#) configured on assigned source connections. Jobs are executed on one of the available agents at the time when the job is started.

Jobs are assigned randomly to one available agent amongst the assigned agents.

Maximum concurrent syncs defaults to `16`, and is configurable under **Agent settings**. The **Maximum concurrent syncs** quota is enforced across all [capabilities](#) and all assigned sources, meaning that any run of any capability on any source uses up one unit of the available concurrent sync quota. This includes [streaming syncs](#), [change data capture syncs](#), [exports](#), [webhooks](#) and legacy data connection [tasks](#). If the assigned agent already has the maximum number of concurrent syncs currently running, then new jobs will be queued until enough running capabilities finish.

In general, we do not recommend using multiple agents as a way to load balance a larger workload than could be successfully run on a single agent. The primary intended use of multiple agents is to allow for agents being taken offline for maintenance. For optimal performance and reliability, we recommend that each agent in a multi-agent setup should be able to handle the full set of [capabilities](#) configured on the assigned source connection(s).

Direct vs. data proxy upload strategy

Agent workers support two options to specify how data from batch syncs should be uploaded to the platform:

[API Reference](#) ↗

Send feedback

Data proxy mode

In data proxy mode, data is uploaded using the public Foundry API using the data proxy service. This leverages the same API gateway that handles Foundry API calls for reading and writing datasets.

Agents configured to use data proxy mode will contain the following in the agent configuration YAML:

```
1 destinations:
2   foundry:
3     uploadStrategy:
4       type: data-proxy
```

Direct mode [Sunset]

i Direct mode is not available on new agents or on enrollments set up after June 2024. Data proxy mode is the default and only option supported for new agents. Existing agents using direct mode will continue to work as long as the host IP addresses where the agent is installed do not change.

In direct mode, data is uploaded directly to the underlying storage buckets in the Foundry data catalog. While providing performance improvements, this is only possible with custom network configuration by Palantir Support, and is not available on our latest cloud infrastructure.

Agents configured to use direct mode will contain the following in the agent configuration YAML:

```
1 destinations:
2   foundry:
3     uploadStrategy:
4       type: direct
```

API Reference ↗

Send feedback

Custom JDBC drivers

See the [JDBC \(custom\) connector documentation](#) for instructions on adding custom JDBC drivers to an agent.

Drivers must be signed by Palantir Support and added directly to the agent to work with the agent worker.

 Contact Palantir Support for assistance signing custom JDBC drivers.

Credentials

One unique aspect of the agent worker is that in-memory credential decryption happens outside of Foundry. When entering credentials in the Data Connection user interface, rather than Foundry encrypting credentials, they are encrypted with the public key of each agent assigned to the source in the browser before being sent to Foundry. The encrypted credentials are then stored in Foundry and can only be decrypted by the respective agent using its private key when needed.

Note that all data in Foundry, including credentials, is encrypted at rest and in transit. Data is only ever decrypted in-memory where access is required.

This means that the following caveats and restrictions apply to credential configuration when using the agent worker:

- If the set of agents assigned to the source changes, credentials must be re-entered in Data Connection.
- If an agent is reprovisioned using a fresh download link, credentials will not be automatically transferred. Encryption keys from the existing agent must be recovered or else all credentials must be re-entered in Data Connection.

More information on moving agents between directories and hosts is covered in the [agent c](#) [API Reference ↗](#) [ce documentation](#), including instructions for retaining encrypted credentials when moving an existing agent directory.

Send feedback

Certificates

Agents communicate with both Foundry and your internal network. This means that agents need to have the correct certificates in their truststores for these connections to be established.

[Review the documentation on adding additional certificates for connecting to specific source systems.](#)

Add a private key

If the system you are connecting to requires mutual TLS (mTLS), you must manually add a private key to the agent.

The default bootstrapper keystore and truststore are regenerated any time the agent is restarted, and any changes made to the default keystore will be overridden on restart. The below instructions explain how to override the default keystore to point at a custom keystore in a different location on the agent host, and how to modify this custom keystore to add your private key.

1. Copy the default bootstrapper keystore and store it in a separate location on the agent host. Run the following commands with the same username that is running the agent on the host. You may choose to name the folder `security` or according to your preferences.

```
1 mkdir /home/<username>/security
2 cp <bootvisor_root>/var/data/processes/<bootstrapper_dir>/var/conf/keyS
```

2. Import the keys from the customer-provided keystore into the copied agent keystore using the Java `keytool` command line tool. If this tool is not already installed, find it in the `bin` directory of the JDK that is bundled with the agent.

```
1 $ keytool -importkeystore -srckeystore <CUSTOM_KEYSTORE.jks> -destkeystore
2 keyStore CUSTOM_KEYSTORE.jks to keyStore.jks...
3 Enter destination keystore password: keystore
4 Enter source keystore password:
```

API Reference ↗

Send feedback

- You can verify that the key/keys were added to the copied keystore using the `keytool -list` command:

```
1 $ keytool -list -keystore /home/<username>/security/keyStore.jks
2 Enter keystore password:
3 Keystore type: jks
4 Keystore provider: SUN
5
6 Your keystore contains 2 entries
7
8 <CUSTOM_KEY>, 15-Dec-2022, PrivateKeyEntry,
9 Certificate fingerprint (SHA-256): A5:B5:2F:1B:39:D3:DA:47:8B:6E:6A
10 server, 15-Dec-2022, PrivateKeyEntry,
11 Certificate fingerprint (SHA-256): DB:82:66:E8:09:43:30:9D:EF:0A:41
```

3. Use the `keytool -keypasswd` command to update the imported key password. The agent keystore requires that both the key and keystore passwords match.

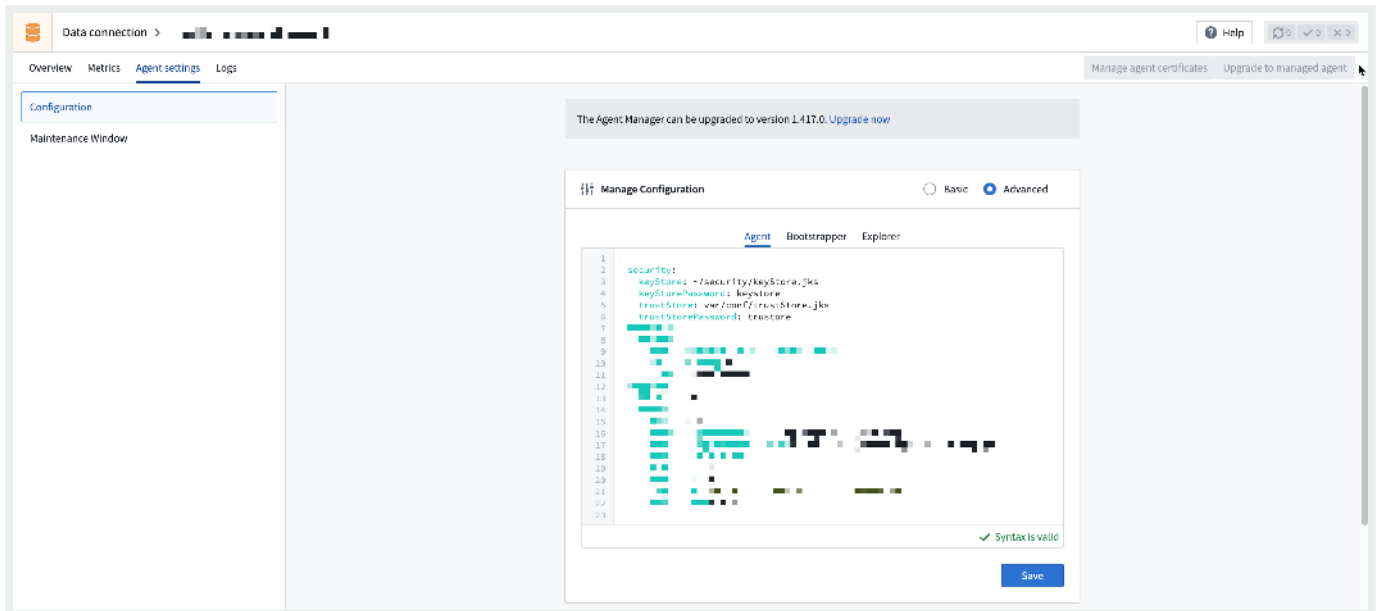
```
1 $ keytool -keypasswd -alias <CUSTOM_KEY> -new keystore -keystore /home/
2 Enter keystore password:
```

4. In [Data Connection](#), navigate to the agent, then open the **Agent settings** tab. In the **Manage Configuration** section, select **Advanced**, choose the **Agent** tab, and update the `keyStore` to point to the newly copied keystore. Then, add `keyStorePassword` and set it to the appropriate value (`keystore` , by default).

```
1 security:
2     keyStore: /home/<username>/security/keyStore.jks
3     keyStorePassword: keystore
4     trustStore: var/conf/trustStore.jks
5     ...
```

[API Reference ↗](#)

[Send feedback](#)



5. Finally, choose the **Explorer** tab and update both the `keyStorePath` and `keyStorePassword`. Save the new configuration.

```
1 security:
2   keyStorePath: /home/<username>/security/keyStore.jks
3   keyStorePassword: keystore
4   trustStorePath: var/conf/trustStore.jks
5   ...
```

6. Restart the agent.

i Note that the field is named `keyStore` when configuring in the **Agent** tab and `keyStorePath` in the **Explorer** tab. No changes are required to the **Bootstrapper** configuration.

Known limitations

Executing capabilities on the agent host directly comes with some downsides, such as:

API Reference [↗](#) **With conflicts on the agent:** This is particularly relevant for custom JDBC workflows, since custom JARs may conflict with dependencies that ship from Foundry and run on the agent classpath.

Send feedback

- **Contention between jobs running on the agent:** Jobs may arbitrarily use up to the entire memory and disk space allocated to the agent process and may cause other jobs to never start or crash. This is particularly problematic for webhooks run with an agent worker. In this scenario, we strongly recommend a dedicated agent for webhook executions so long-running syncs do not prevent short-running webhooks from executing.
- **Lack of support for some capabilities:** Some capabilities, such as [use in code](#), [virtual tables](#), and virtual media, are incompatible with the agent worker, since these require synchronous connections directly from Foundry and cannot run as jobs on the agent.

PREVIOUS

← [Agent configuration reference](#)

NEXT

[Agent proxy configuration reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Data Connection / Agents / Agent proxy configuration reference

Agent proxy configuration reference

This page describes the configuration options available when using an agent as a network proxy via [agent proxy egress policies](#) on [Foundry worker](#) connections and assumes you are already familiar with [data connection agents](#) and their [generic configuration options](#).

Agents used as a proxy provide a network path for services in Foundry to access systems within a separate network that are not directly accessible over the Foundry network. When Foundry is hosted in the cloud, a proxy is required to access systems which are not accessible over the Internet. When Foundry is hosted on a private network, a proxy is required to access systems in other networks, which may include the public Internet if otherwise not accessible.

Agent proxies allow applications in Foundry to operate as if they are connecting to a source directly over the Foundry network, and they can be configured with multiple agents to allow for alternating maintenance windows that prevent downtime. Follow the guide below to configure and manage an agent used as a proxy.

Use an agent as a proxy with a Data Connection source

You must first [set up an agent](#) before using it as a proxy. An existing agent already used for [agent worker](#) sources can be reused without additional configuration.

Once an agent has been set up, create an [agent proxy egress policy](#) using that agent. You can then create a data connection source and assign that policy to it.

API Reference ↗



Send feedback

Security controls specific to agent proxy

To ensure that connections over the proxy only have access to required resources, a number of security controls are available.

Security controls are either *mandatory* and cannot be disabled, or *optional* and are not enabled by default.

The following security control configurations are available:

Security control	Required?	Description
Source configuration enforcement	Yes	The agent proxy always restricts traffic to only the hostname(s) and port(s) configured on the assigned source.
Agent allowlist (Foundry)	Optional	Configure an allowlist in Foundry to filter traffic before it reaches the agent. Only agent resource owners can modify this setting.
Agent allowlist (local file)	Optional	Create an allowlist on the agent host for the local agent process to enforce within your network. Only users with SSH access to the agent host can modify this setting.
Agent host firewall	Optional	Set up host-level firewall controls to limit agents to communicating only with intended target systems (strongly recommended). These firewalls work independently of Foundry and add an extra security layer.

Source configuration enforcement

The hostname and port in the URL defined on the source restrict access to only that hostname and port when connecting to the agent proxy. Attempts to connect to any other hostname or port will result in an HTTP 403 (Unauthorized) response code from the proxy.

API Reference ↗

This prevents unauthorized connections when importing and using the connection in code.

Send feedback

If only this security control is used, ensure that users who are able to assign an agent to a particular source are also trusted to connect to any system that is reachable from the agent host.

Agent allowlist (Foundry)

Configure an allowlist in Foundry to restrict connections to specific IP addresses or CIDR blocks for a specific agent, regardless of assigned sources.

To configure this, navigate to **Agent settings**, then toggle the **Advanced** option in the **Manage Configuration** section. In the YAML file, add a block for `agentProxyConfiguration` at the same indentation level as the `security` block.

List the CIDR blocks and ports where you want to allow agent connections. For example:

```
1 agentProxyConfiguration:
2   allowListedCidrs:
3     - cidrBlock: '192.168.1.1/32'
4       port: 7000
5       endPort: 9000
6     - cidrBlock: '192.168.2.2/24'
7       port: 443
```

Agent allowlist (local file)

The same configuration used for an [agent allowlist in Foundry](#) can also be set in a file on the agent host itself, preventing users in Foundry from editing the configuration.

To configure an agent allowlist locally, follow the steps below:

1. SSH into the agent's host.
2. Create a folder named `agentProxyConfig` in the same directory that contains your `source-<version>` (the root folder).

[API Reference ↗](#) In the `agentProxyConfig` folder, create a file named `agentProxyConfig.yml`.

[Send feedback](#)

4. The `agentProxyConfig.yml` file and `agentProxyConfig` folder *must* be created with the root user, and the file permissions must be set to prevent the agent from writing to this file or folder. The agent will *not* run the agent proxy feature if the `agentProxyConfig.yml` file exists and is writeable, or if one of the parent directories is writeable by the agent.
5. The content of this file is the same as the configuration available in the agent settings interface.

For example:

```
1 allowListedCidrs:
2   - cidrBlock: '192.168.1.1/32'
3     port: 7000
4     endPoint: 9000
5   - cidrBlock: '192.168.2.2/24'
6     port: 443
```

Agent host firewall

As for any agent, [we recommend setting a firewall on the host to restrict connectivity to only what is necessary.](#)

Connection pool configuration

The agent proxy maintains a pool of connections for handling requests. You can configure the connection pool size to optimize performance based on your workload.

To configure the connection pool, navigate to **Agent settings**, then toggle the **Advanced** option in the **Manage Configuration** section. In the YAML file, add the connection pool settings within the `agentProxyConfiguration` block:

```
1 agentProxyConfiguration:
  connections: 250
  idleConnections: 100
```

API Reference ↗

Send feedback

Option	Default	Description
maxConnections	300	The maximum number of connections the agent proxy can open.
coreConnections	20	The minimum number of connections maintained in the pool.

i Some connectors, such as [Databricks](#), may open a large number of connections at the same time. If you observe connection pool exhaustion errors on your agent when using agent proxy policies, consider increasing the `maxConnections` and `coreConnections` values.

PREVIOUS
← [Agent worker configuration reference](#)

NEXT
[Troubleshooting reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)